

μ C/OS-III 开发指南

V1.5

本教程适用于正点原子所有 STM32 开发板

修订历史:

版本	日期	修改内容
V1.0	2022/06/18	第一次发布
V1.1	2023/02/02	优化文件截图，修改文中错别字
V1.2	2023/02/10	配合例程代码修改对 delay_init() 函数的相关描述
V1.3	2023/02/15	配合例程代码删除 CPU_IntDis() 的相关调用
V1.4	2023/02/25	添加精简工程的相关描述
V1.5	2023/03/03	添加初始化时间戳功能的相关描述



正点原子公司名称：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛：<http://www.openedv.com/forum.php>

正点原子官方网站：www.alientek.com

正点原子淘宝店铺：<https://openedv.taobao.com>

正点原子 B 站视频：<https://space.bilibili.com/394620890>

电话：020-38271790 传真：020-36773971

请下载原子哥 APP，数千讲视频免费学习，更快更流畅。

请关注正点原子公众号，资料发布更新我们会通知。



扫码下载“原子哥”APP



扫码关注正点原子公众号

第一章 μ C/OS-III 简介	13
1.1 初识 μ C/OS-III	13
1.1.1 什么是 μ C/OS-III?	13
1.1.2 为什么选择 μ C/OS-III?	13
1.1.3 μ C/OS-III 的特点	14
1.1.4 μ C/OS 各版本比较	14
1.2 磨刀不误砍柴工	15
1.2.1 查找资料	15
1.2.2 μ C/OS-III 官方文档	18
1.2.3 Cortex-M 架构资料	20
1.3 源码初探	20
1.3.1 源码下载	20
1.3.2 μ C/OS-III 文件预览	22
1.3.3 μ C/CPU 文件预览	25
1.3.4 μ C/LIB 文件预览	26
第二章 μ C/OS-III 移植	28
2.1 μ C/OS-III 移植	28
2.1.1 移植前准备	28
2.1.2 添加 μ C/OS-III 相关文件	28
2.1.3 修改 SYSTEM 文件	31
2.1.4 替换中断服务函数	36
2.1.5 配置 μ C/OS-III 及其相关组件	38
2.1.6 可选步骤（建议完成）	39
2.1.7 添加应用程序	42
2.1.8 使用 AC6 编译工程（扩展）	53
2.2 μ C/OS-III 移植实验	56
2.2.1 功能设计	56
2.2.2 软件设计	56
2.2.3 下载验证	58
第三章 μ C/OS-III 配置项	60
3.1 μ C/OS-III 及其相关组件的配置项	60

3.2 μC/OS-III 配置项.....	60
3.2.1 os_cfg.h 配置文件.....	60
3.2.2 os_cfg_app.h 配置文件.....	65
3.3 μC/CPU 配置项.....	66
3.3.1 cpu_cfg.h 配置文件.....	66
3.4 μC/LIB 配置项.....	68
3.4.1 lib_cfg.h 配置文件.....	68
第四章 μC/OS-III 中断管理.....	70
4.1 ARM Cortex-M 中断.....	70
4.1.1 ARM Cortex-M 中断简介.....	70
4.1.2 ARM Cortex-M 中断优先级管理.....	70
4.1.3 三个中断优先级配置寄存器.....	72
4.1.4 三个中断屏蔽寄存器.....	72
4.1.5 中断控制状态寄存器.....	74
4.2 μC/OS-III 中断配置项.....	75
4.3 μC/OS-III 中断管理详解.....	75
4.3.1 PendSV 和 SysTick 中断优先级配置.....	75
4.3.2 μC/OS-III 开关中断.....	77
4.4 μC/OS-III 临界区.....	78
4.4.1 函数 CPU_CRITICAL_ENTER().....	78
4.4.2 函数 CPU_CRITICAL_EXIT().....	79
4.4.3 函数 CPU_SR_ALLOC().....	79
4.4.4 μC/OS-III 临界区代码使用示例.....	79
4.5 μC/OS-III 中断嵌套计数器.....	80
第五章 μC/OS-III 任务基础知识.....	81
5.1 单任务系统和多任务系统.....	81
5.1.1 单任务系统.....	81
5.1.2 多任务系统.....	82
5.2 μC/OS-III 任务状态.....	82
5.3 μC/OS-III 任务优先级.....	84
5.4 μC/OS-III 任务调度方式.....	84
5.4.1 抢占式调度.....	85

5.4.2 时间片调度	85
5.5 μC/OS-III 任务控制块.....	85
5.6 μC/OS-III 任务栈.....	90
第六章 μC/OS-III 任务相关 API 函数	91
6.1 μC/OS-III 创建和删除任务相关 API 函数	91
6.1.1 函数 OSTaskCreate().....	91
6.1.2 函数 OSTaskDel()	93
6.2 μC/OS-III 任务创建与删除实验.....	95
6.2.1 功能设计	95
6.2.2 软件设计	95
6.2.3 下载验证	99
6.3 μC/OS-III 挂起和恢复任务相关 API 函数	100
6.3.1 函数 OSTaskSuspend().....	100
6.3.2 函数 OSTaskResume()	101
6.4 μC/OS-III 任务挂起与恢复实验.....	103
6.4.1 功能设计	103
6.4.2 软件设计	103
6.4.3 下载验证	108
第七章 μC/OS-III 启动流程	109
7.1 μC/OS-III 启动流程.....	109
7.2 μC/OS-III 屏蔽所有中断.....	110
7.3 μC/OS-III 内核初始化.....	110
7.4 μC/OS-III 创建第一个应用任务.....	118
7.5 μC/OS-III 开始任务调度.....	127
第八章 μC/OS-III 任务切换	132
8.1 PendSV 异常	132
8.2 μC/OS-III 何时触发任务切换.....	132
8.3 μC/OS-III 如何触发任务切换.....	133
8.3.1 函数 OSSched().....	133
8.3.2 函数 OSIntExit().....	137
8.3.3 挂起 PendSV 异常	140

8.4 μC/OS-III 如何进行任务切换.....	141
第九章 μC/OS-III 其他任务相关 API 函数	145
9.1 μC/OS-III 任务相关 API 函数	145
9.1.1 μC/OS-III 任务相关 API 函数预览	145
9.1.2 μC/OS-III 相关 API 函数详解	146
9.2 μC/OS-III 任务优先级修改实验.....	153
9.2.1 功能设计	153
9.2.2 软件设计	154
9.2.3 下载验证	158
9.3 μC/OS-III 时间片调度实验.....	160
9.3.1 功能设计	160
9.3.2 软件设计	160
9.3.3 下载验证	164
9.4 μC/OS-III 空闲任务钩子函数实验.....	165
9.4.1 功能设计	165
9.4.2 软件设计	165
9.4.3 下载验证	168
9.5 μC/OS-III 统计功能实验.....	168
9.5.1 功能设计	168
9.5.2 软件设计	169
9.5.3 下载验证	173
第十章 μC/OS-III 时间管理	175
10.1 μC/OS-III 系统时钟节拍.....	175
10.1.1 μC/OS-III 系统时钟节拍简介	175
10.1.2 μC/OS-III 系统时钟节拍来源.....	175
10.1.3 μC/OS-III 系统时钟节拍处理	177
10.2 μC/OS-III 任务延时相关函数.....	186
10.3 μC/OS-III 其他系统时钟节拍相关 API 函数	194
第十一章 μC/OS-III 信号量	196
11.1 μC/OS-III 信号量简介	196
11.2 μC/OS-III 二值信号量.....	196

11.2.1 μC/OS-III 二值信号量简介	196
11.2.2 μC/OS-III 二值信号量相关 API 函数.....	198
11.3 μC/OS-III 共享资源冲突访问实验.....	222
11.3.1 功能设计	222
11.3.2 软件设计	222
11.3.3 下载验证.....	225
11.4 μC/OS-III 二值信号量实验.....	226
11.4.1 功能设计	226
11.4.2 软件设计	226
11.4.3 下载验证.....	230
11.5 μC/OS-III 计数型信号量.....	231
11.6 μC/OS-III 计数型信号量实验.....	231
11.6.1 功能设计	231
11.6.2 软件设计	231
11.6.3 下载验证.....	235
11.7 μC/OS-III 优先级翻转	237
11.8 μC/OS-III 优先级翻转实验.....	238
11.8.1 功能设计	238
11.8.2 软件设计	238
11.8.3 下载验证.....	243
第十二章 μC/OS-III 互斥信号量.....	245
12.1 μC/OS-III 互斥信号量简介.....	245
12.2 μC/OS-III 互斥信号量相关 API 函数	247
12.3 μC/OS-III 互斥信号量实验.....	270
12.3.1 功能设计	270
12.3.2 软件设计	271
12.3.3 下载验证	275
12.4 μC/OS-III 互斥信号量优化优先级翻转实验.....	275
12.4.1 功能设计	275
12.4.2 软件设计	276
12.4.3 下载验证	281
第十三章 μC/OS-III 消息队列	283

13.1 μC/OS-III 消息队列简介	283
13.2 μC/OS-III 消息队列相关 API 函数	285
13.3 μC/OS-III 消息队列实验	307
13.3.1 功能设计	307
13.3.2 软件设计	308
13.3.3 下载验证	312
第十四章 μC/OS-III 任务内嵌信号量	314
14.1 μC/OS-III 任务内嵌信号量简介	314
14.2 μC/OS-III 任务内嵌信号量相关 API 函数	314
14.3 μC/OS-III 任务内嵌信号量实验	329
14.3.1 功能设计	329
14.3.2 软件设计	329
14.3.3 下载验证	333
第十五章 μC/OS-III 任务内嵌消息队列	334
15.1 μC/OS-III 任务内嵌消息队列简介	334
15.2 μC/OS-III 任务内嵌消息队列相关 API 函数	334
15.3 μC/OS-III 任务内嵌消息队列实验	348
15.3.1 功能设计	348
15.3.2 软件设计	349
15.3.3 下载验证	352
第十六章 μC/OS-III 事件标志	354
16.1 μC/OS-III 事件标志简介	354
16.1.1 事件标志	354
16.1.2 事件标志组	354
16.1.3 事件标志操作简介	355
16.2 μC/OS-III 事件标志相关 API 函数	356
16.3 μC/OS-III 事件标志实验	386
16.3.1 功能设计	386
16.3.2 软件设计	387
16.3.3 下载验证	392

第十七章 μC/OS-III 软件定时器	394
17.1 μC/OS-III 软件定时器简介	394
17.1.1 软件定时器	394
17.1.2 软件定时器超时回调函数	396
17.1.3 单次定时器和周期定时器	396
17.1.4 软件定时器的状态	396
17.1.5 软件定时器任务	396
17.1.6 软件定时器定时频率	404
17.1.7 软件定时器时序图	404
17.2 μC/OS-III 软件定时器相关 API 函数	406
17.3 μC/OS-III 软件定时器实验	430
17.3.1 功能设计	430
17.3.2 软件设计	431
17.3.3 下载验证	435
第十八章 μC/OS-III 时间戳	437
18.1 μC/OS-III 时间戳简介	437
18.1.1 μC/OS-III 时间戳功能时基定时器	437
18.1.2 μC/OS-III 时间戳功能配置项	437
18.1.3 μC/OS-III 时间戳功能初始化	438
18.2 μC/OS-III 时间戳相关 API 函数	440
18.3 μC/OS-III 时间戳实验	440
18.3.1 功能设计	440
18.3.2 软件设计	441
18.3.3 下载验证	443
第十九章 μC/OS-III 内存管理	445
19.1 μC/OS-III 内存管理简介	445
19.2 μC/OS-III 内存管理相关 API 函数	447
19.3 μC/OS-III 内存管理实验	456
19.3.1 功能设计	456
19.3.2 软件设计	456
19.3.3 下载验证	460

前言

1. 如何学习本书

本书是针对 μC/OS-III 进行系统性讲解的书籍，本书将从零开始，带领读者学习和使用 μC/OS-III。

本书共十九个章节，涵盖了 μC/OS-III 中各部分内容的讲解。最开始的 μC/OS-III 移植，将手把手带领读者领略 RTOS 的魅力，这是以往在使用裸机编程开发中体会不到的。接下来将会为读者介绍 ARM Cortex-M 中断部分的相关知识，这对于理解 μC/OS-III 有着至关重要的作用，特别是 μC/OS-III 中与任务管理的相关部分。接下来就是讲解 μC/OS-III 中一个任务相关的操作，例如创建任务、删除任务和任务切换等等，以及 μC/OS-III 中一些任务之同步和通讯的机制，例如信号量、消息队列、事件标志等等，最后还有 μC/OS-III 提供的一些功能，例如软件定时器、时间戳、内存管理等。

本书的内容循序渐进，将一步一步地带读者理解 μC/OS-III 的机制和原理。本书中也包含了大量的实验，并提供了配套的例程源码，让读者不仅知道机制和原理，还能够理解如何使用。

2. 本书的参考资料

- 1、μC/OS-III 3.08.01、μC/CPU 1.32.01、μC/LIB 1.39.01 官方源代码
- 2、μC/OS-III 官方在线文档《μC/OS-III Documentation》
- 3、μC/OS-III 针对 STM32 的硬件相关文档《μC/OS-III for the STMicroelectronics STM32》
- 4、ARM Cortex-M 内核知识相关书籍《The Definitive Guide to ARM® Cortex®-M3 and Cortex-M4 Processors》

3. 本书的配套硬件

本书基于正点原子 STM32 系列开发板编写，正点原子 STM32 系列开发板为本书的配套开发硬件，使用本书的配套开发硬件完成本书的相关实验，将免去因硬件差异带来的移植等问题。本书配套的正点原子 STM32 系列开发板如下表所示：

开发板型号	内核
MiniSTM32F103	ARM Cortex-M3
精英 STM32F103	ARM Cortex-M3
战舰 STM32F103	ARM Cortex-M3
探索者 STM32F407	ARM Cortex-M4
DMF407	ARM Cortex-M4
MiniPro STM32H750	ARM Cortex-M7

4. 本书的技术论坛

如果读者在阅读本书学习 μC/OS-III 的过程中遇到困难，可以到正点原子的官方论坛 <http://www.openedv.com/forum.php> 发帖交流。

第一章 μ C/OS-III 简介

从本章开始,我们就踏入了 μ C/OS-III 的大门, μ C/OS-III 是一个 RTOS 类的实时操作系统。在学习和使用 μ C/OS-III 之前,需要先了解什么是 μ C/OS-III? 为什么选择学习 μ C/OS-III? 以及 μ C/OS-III 的特点,这也是本章的目的。

本章分为如下几部分:

- 1.1 初始 μ C/OS-III
- 1.2 磨刀不误砍柴工
- 1.3 源码初探

1.1 初识 μ C/OS-III

1.1.1 什么是 μ C/OS-III?

μ C/OS-III 的发音为“Micro C O S Three”,这意味着 μ C/OS-III 是基于 C 语言编写的第三代小型操作系统,当然这里所说的第三代是相对于 μ C/OS 的前两个版本 μ C/OS 和 μ C/OS-II 而言的,后面也会介绍这三个版本的差别。 μ C/OS/III 是一个操作系统,准确地说应该是一个实时操作系统,也就是 RTOS (Real Time Operating System),与之类似的 RTOS 还有 FreeRTOS、RTX、RT-Thread 等,当然,本书的重点是 μ C/OS-III!

既然 μ C/OS-III 是一个实时操作系统,那么要学习 μ C/OS-III 就需要先了解一些操作系统的概念。操作系统最直观的特点就体现在,操作系统能够使得一个 CPU 核心“同时运行”多个任务,这个特性就被称为“多任务”。然而,实际上,一个 CPU 核心在某一时刻只能运行一个任务,而操作系统中任务调度器的责任就是决定在某一时刻 CPU 究竟要运行哪一个任务,任务调度器使得 CPU 在各个任务之间来回切换并处理任务,由于切换处理任务的速度非常快,因此给人造成了一种同一时刻有多个任务同时运行的错觉。

操作系统的分类方式可以由任务调度器的工作方式决定,比如有的操作系统给每个任务分配同样的运行时间,时间到了就切换到下一个任务,Unix 操作系统就是这样的。RTOS 的任务调度器被设计为可预测的,而这正是嵌入式实时操作系统所需要的。在实时环境中,要求操作系统必须实时地对某一个事件做出响应,因此任务调度器的行为必须是可预测的。像 μ C/OS-III 这种传统的 RTOS 类操作系统是由用户给每个任务分配一个任务优先级,任务调度器就可以根据此优先级来决定下一刻应该运行哪个任务。

1.1.2 为什么选择 μ C/OS-III?

前面说到,除了 μ C/OS-III 之外,还有多种 RTOS,那么为什么要选择 μ C/OS-III 呢? 因为对于初学者学习 RTOS,最重要的当然是相关的资料, μ C/OS-III 官方提供了十分完善的资料(后面会为大家介绍如何获取 μ C/OS-III 的官方资料)。其次, μ C/OS-III 支持多种不同架构的不同型号的处理器,例如 STM32 系列的 F1、F4、F7 和 H7 等型号的 MCU 都受 μ C/OS-III 的支持,读者只需要进行简单的移植操作,就能够在这些 MCU 上运行 μ C/OS-III 操作系统,极大地方便了我们学习和使用 μ C/OS-III。

1.1.3 μC/OS-III 的特点

μC/OS-III 是一个可裁剪、可固化到 ROM 的抢占式实时内核，并且可管理任务的数量不受限制。μC/OS-III 是第三代的 μC/OS 内核，具有以下几个重要的特性：

抢占式多任务管理：μC/OS-III 是一个支持多任务抢占的内核，因此总是优先执行任务优先级高的任务。

时间片调度：μC/OS-III 允许系统中有多个相同任务优先级的任务，如果系统中处于就绪状态的任务中，优先级最高的任务有多个，那么 μC/OS-III 将以时间片的方式调度任务，即根据用户指定的时间（时间片）轮流调度这些任务。

极短的中断禁用时间：μC/OS-III 通过锁定任务调度器代替禁用中断来保护一些关键区域（临界区），这确保了 μC/OS-III 能够快速响应中断。

任务数量不限：μC/OS-III 理论上支持不受限制的任务数量，但实际上，系统中任务的最大数量受处理器内存空间的限制。

任务优先级数量不限：μC/OS-III 支持的任务优先级数量不受限制，但对于大多数应用场景而言，使用 32~256 个任务优先级就绰绰有余了。

内核对象数量不限：μC/OS-III 提供了多种内核对象，如任务、信号量、事件标志、消息队列、软件定时器和内存区等，并且在不考虑处理器内存限制的情况下，用户可以无限制的创建这些内核对象。

时间戳：μC/OS-III 提供了时间戳功能，用户可以非常方便地测量系统在运行过程中，处理器处理某些事件所消耗的时间，以方便用户对系统进行针对性的优化。

自定义钩子函数：μC/OS-III 提供了一些在内核执行操作之前、之后或过程中的钩子函数，这样可以方便用户扩展 μC/OS-III 的功能。

防死锁：μC/OS-III 允许任务在等待某些内核对象前，设置一个等待的最大超时时间，这样可以有效地防止死锁的发生。

软件定时器：在 μC/OS-III 中，用户可以创建任意数量的“单次”和“周期”软件定时器，并且每个软件定时器都可以有独立的超时回调函数。

任务内嵌信号量：μC/OS-III 提供了任务的内嵌信号量功能，这使得任务可以直接获取来自其他任务或中断的信号，而不需要任何的中间内核对象，大大地提高了系统的运行效率。

任务内嵌消息队列：μC/OS-III 提供了任务的内嵌消息队列，这使得任务可以直接接收来自其他任务或中断的消息，而不需要任何的中间内核对象，大大地提高了系统的运行效率。

1.1.4 μC/OS 各版本比较

μC/OS 至今为止一共发展了三个版本，分别为 μC/OS、μC/OS-II 和 μC/OS-III，这三个版本的特征比较如下表所示：

特征	μC/OS	μC/OS-II	μC/OS-III
发布年份	1992	1998	2009
抢占式多任务	是	是	是
最大任务数	64	255	无限制
单个优先级的任务数	1	1	无限制
时间片调度	否	否	是
信号量	是	是	是
互斥信号量	否	是	是（支持嵌套）
事件标志	否	是	是

消息邮箱	是	是	否（不需要）
消息队列	是	是	是
固定大小的内存管理	否	是	是
直接向任务发送信号	否	否	是
无需调度的发送机制	否	否	是
直接向任务发送消息	否	否	是
软件定时器	否	是	是
任务挂起、恢复	否	是	是（支持嵌套）
防死锁	是	是	是
可裁剪	是	是	是
代码量	3K~8K	6K~26K	6K~24K
数据量	1K+	1K+	1K+
可固化	是	是	是
运行时配置	否	否	是
编译时配置	是	是	是
内核对象命名	否	是	是
任务寄存器	否	是	是
用户钩子函数	否	是	是

表 1.1.1 μC/OS 各版本特征比较

1.2 磨刀不误砍柴工

1.2.1 查找资料

不论学习什么，第一件事情就是查找资料，可能有的读者会觉得查找资料是一件很简单的事情。的确，查找资料是一件很简单的事情，但直接在搜索引擎中搜索绝不是一个高效的方法，因为搜索引擎给出的结果大多数的一些第三方的资料，这些资料有好有坏，资料的正确性参差不齐，需要一一辨别，并且第三方的资料也不够权威、更新不够即时。前面说到，μC/OS-III 官方为用户提供了大量的 μC/OS-III 相关的资料，那么要如何获取这些 μC/OS-III 相关的资料呢？那当然是 μC/OS-III 的官网网址了，μC/OS-III 的官网网址为 <https://weston-embedded.com/>，打开后如下图所示：

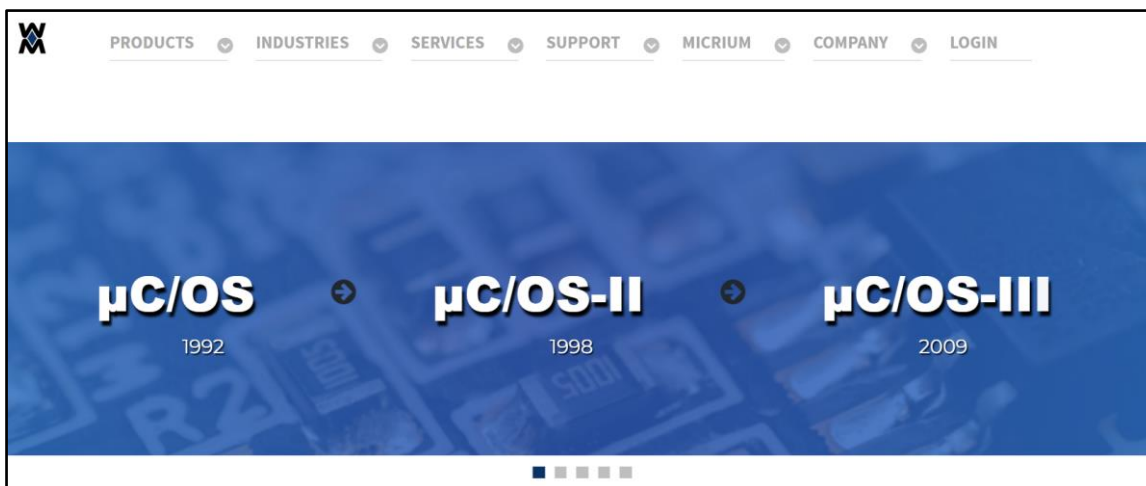


图 1.2.1 μC/OS-III 官网

打开 μC/OS-III 的官网网址就能看到 μC/OS 的发展历程，并且还能看到 μC/OS-III 的官网是全英文的，可见英语对一个嵌入式工程师的重要性。

接着打开顶部导航栏“MICRIUM”中的“Kernels”，就能看到 μC/OS 的详情页了，打开后如下图所示：

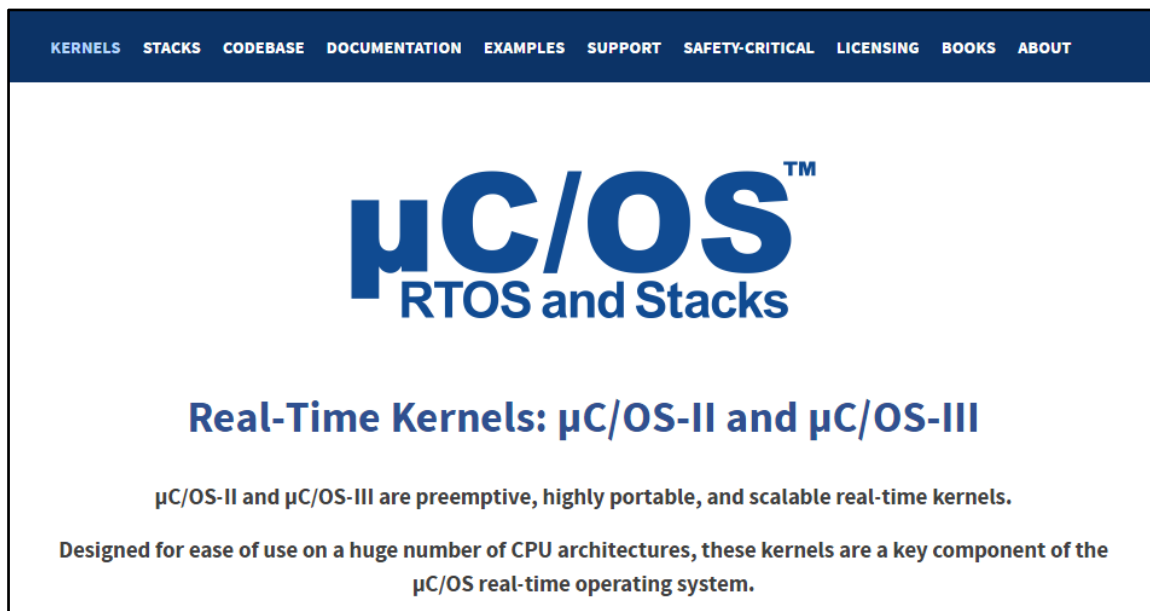


图 1.2.2 μC/OS 详情页

在 μC/OS 的详情页中可以看到，顶部导航栏中有一个“DOCUMENTATION”的标签按钮，这就是 μC/OS 官方资料的入口，顺带一提的是，μC/OS 的源代码获取入口为顶部导航栏中的“CODEBASE”标签按钮。

接着点击“DOCUMENTATION”，就能够打开 μC/OS 官方的文档中心，如下图所示：

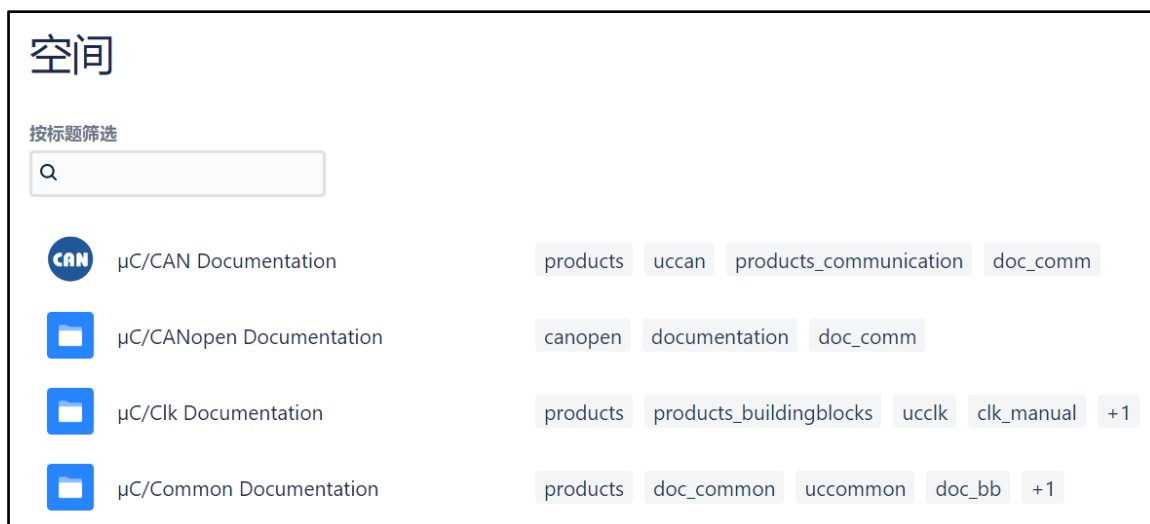


图 1.2.3 μC/OS 官方文档中心

在 μC/OS 官方的文档中心中可以看到 μC/OS 官方为自家的多种组件都提供了详细的文档资料，例如 μC/CAN、μC/CANopen、μC/Clock 等等。在文档中心中索引找到并打开 μC/OS-III 的在线文档资料，打开 μC/OS-III 的在线文档资料后，就能够在页面的左侧看到 μC/OS-III 在线文档的导航栏，如下图所示：

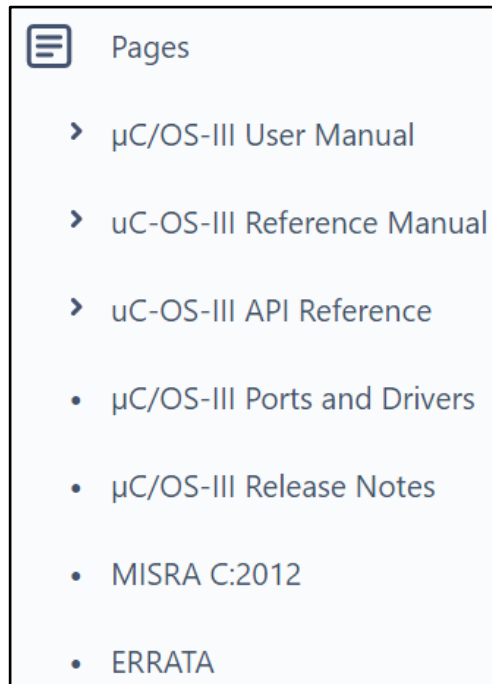


图 1.2.4 μC/OS-III 在线文档导航栏

从上图可以看到，μC/OS-III 官方提供了大量关于 μC/OS-III 的在线资料，其中包括了 μC/OS-III 的用户手册、μC/OS-III 的参考手册以及 μC/OS-III 的 API 参考手册等等。在 μC/OS-III 的 API 参考手册中详细地介绍了 μC/OS-III 中各个 API 的使用说明，包括 API 函数的参数说明、返回值说明以及 API 用法举例，任意打开一个创建任务的 API 函数，如下图所示：

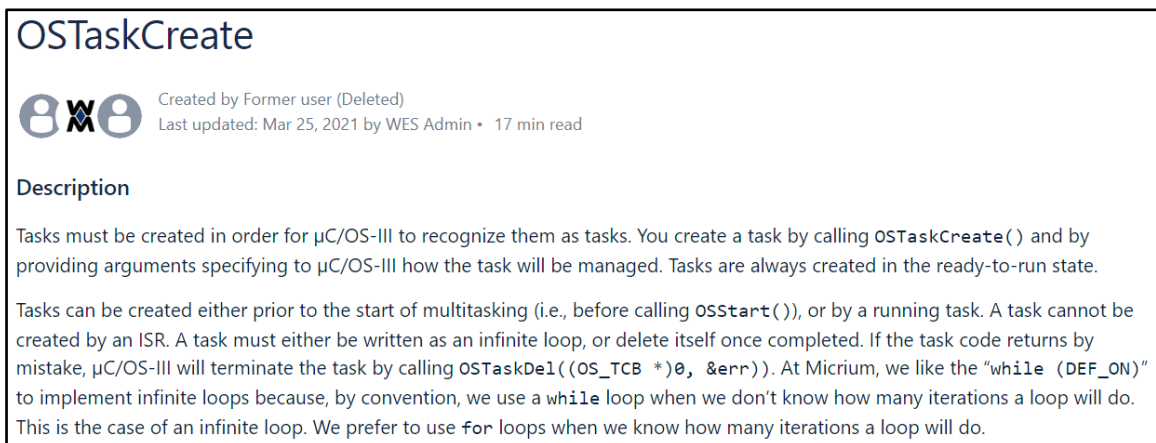


图 1.2.5 创建任务 API 函数

从上图中可以看出，文档对 μC/OS-III 中创建任务的 API 函数做了详细的描述，在这个页面下方，还提供了“OSTaskCreate”这个 API 函数的详细用法说明和用法举例，用法举例的代码如下图所示：

```
OS_TCB  MyTaskTCB;                                /* (1) 保存任务的任务控制块 */
CPU_STK  MyTaskStk[200];

void  MyTask (void *p_arg)                          /* (3) 任务函数的地址就是任务函数的函数名 */
{
    while (DEF_ON) {
```

```

        /* 仍待一个事件 */
        /* 任务体 */
    }
}

void SomeCode (void)
{
    OS_ERR err;
    :
    :
    OSTaskCreate(    &MyTaskTCB,          /* (1) 任务控制块          */
                    "My Task",           /* (2) 任务名              */
                    MyTask,              /* (3) 任务函数地址        */
                    (void *)0,          /* (4) 任务函数参数未用到  */
                    12,                 /* (5) 任务优先级          */
                    &MyTaskStk[0],      /* (6) 任务栈              */
                    10,                 /* (7) 栈生长的“水位”限制 */
                    200,                /* (8) 任务栈大小          */
                    5,                 /* (9) 任务消息队列的大小  */
                    10,                 /* (10) 任务时间片         */
                    (void *)0,          /* (11) 扩展指针           */
                    OS_OPT_TASK_STK_CHK + OS_OPT_TASK_STK_CLR, /* (12) 选项              */
                    &err);              /* (13) 错误代码          */
    /* 检查错误代码          (14)          */
    :
    :
}

```

1.2.2 μC/OS-III 官方文档

μC/OS-III 官方的文档和书籍做得非常好，并且大多数的书籍都有出售，具体读者可自行搜索有关在售书籍。

在从图 1.2.3 打开 μC/OS-III 的在线文档后，可以在页面的右侧看到，μC/OS-III 官方不仅提供了完整的在线文档，还提供了 PDF 格式的特定于硬件的 μC/OS-III 书籍，如下图所示：

 In addition to the full documentation at left, hardware-specific μC/OS-III books are also available for download in PDF format:

- [μC/OS-III for the Freescale Kinetis](#)
- [μC/OS-III for the Infineon XMC4500](#)
- [μC/OS-III for the Renesas SH7216](#)
- [μC/OS-III for the STMicroelectronics STM32](#)
- [μC/OS-III for the Renesas RX62N](#)
- [μC/OS-III for the NXP LPC1768](#)
- [μC/OS-III for the Texas Instruments LM3S9B92](#)

图 1.2.6 μC/OS-III 书籍

读者可以根据自己的硬件下载相关的 PDF 书籍，这里以意法半导体的 STM32 为例，点击上图中的“μC/OS-III for the STMicroelectronics STM32”，下载与 STM32 硬件相关的 μC/OS-III PDF 书籍，如下图所示：

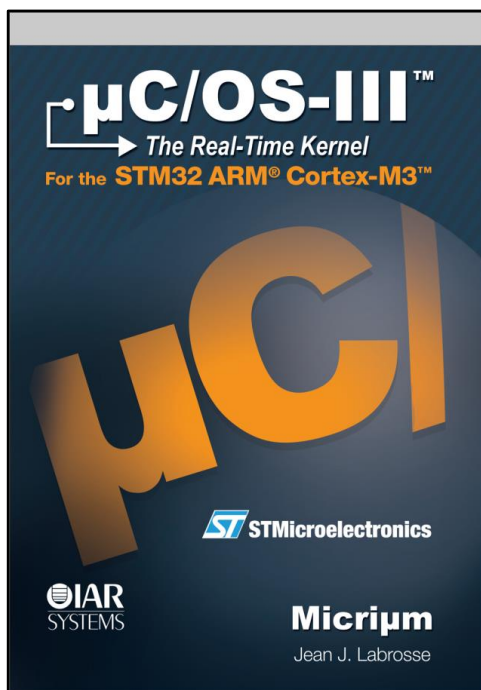


图 1.2.7 与 STM32 硬件相关的 μC/OS-III 书籍
书籍的目录如下图所示：

- 📖 Cover
- 📖 Table of Contents
- 📖 Part I
- 📖 Part II
 - 📖 Introduction
 - 📖 The ARM Cortex-M3 and the STM32
 - 📖 Setup
 - 📖 μC/OS-III Example #1
 - 📖 μC/OS-III Example #2
 - 📖 μC/OS-III Example #3
 - 📖 μC/OS-III Example #4
 - 📖 μC/OS-III port for the Cortex-M3
 - 📖 μC/CPU port for the Cortex-M3
 - 📖 IAR Systems IAR Embedded Workbench for ARM
 - 📖 Micrium's μC/Probe
 - 📖 μC/Eval-STM32F107 User's Guide
 - 📖 Bibliography
 - 📖 Licensing Policy
 - 📖 Index

图 1.2.8 μC/OS-III 书籍目录

其中目录中的“Part I”与在线文档类似，而“Part II”部分则是与 STM32 硬件相关的，可以看到 μC/OS-III 官方是以开发板“μC/Eval-STM32F107”为例进行介绍的。

1.2.3 Cortex-M 架构资料

本书是以正点原子的 STM32 系列板卡为例，讲解 μC/OS-III，在 μC/OS-III 的移植，任务切换的原理中会涉及芯片结构的相关知识，因此需要了解 ARM Cortex-M 架构的相关知识。有本详细地介绍了 ARM Cortex-M3 和 ARM Cortex-M4 的书籍叫做《The Definitive Guide to ARM Cortex M3 and Cortex M4 Processors, 3rd Edition》，这本书的中文翻译版本为《ARM Cortex-M3 与 Cortex-M4 权威指南（第 3 版）》，这本书对 ARM Cortex-M3 和 ARM Cortex-M4 作了非常详细的介绍，强烈建议想深入了解 ARM Cortex 的读者阅读此书，英文原版可以在 ARM 官方免费下载。此书的中文版本如下图所示：

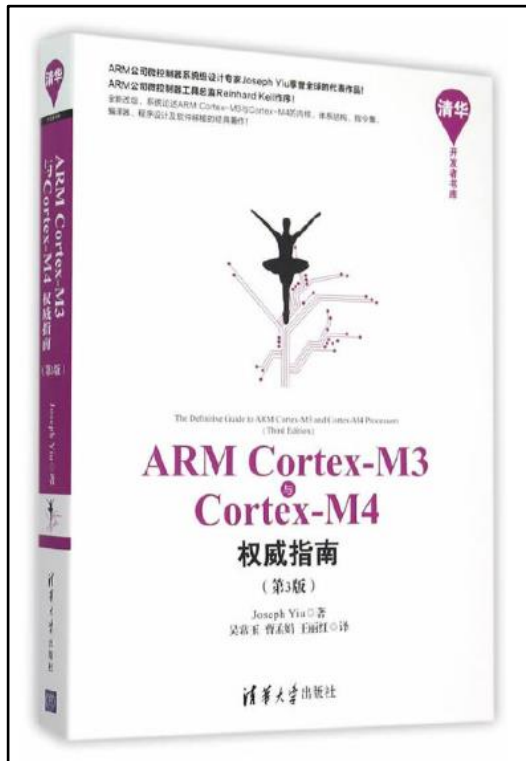


图 1.2.9 ARM Cortex-M3 与 Cortex-M4 权威指南（第 3 版）

后面的学习中涉及到 ARM Cortex-M 架构的知识均参考自这本书。

1.3 源码初探

1.3.1 源码下载

在上述的几小节中大致地介绍了 μC/OS-III，知道了什么是 μC/OS-III 以及如何获取 μC/OS-III 的官方资料。接下来就来看一下如何获取 μC/OS-III 的源码，因为在移植 μC/OS-III 的时候还需要用到 μC/OS-III 官方提供的两个额外的库，分别为 μC/CPU 和 μC/LIB，在下文会分别介绍这几个东西是干嘛的。

因此现在的目标就是要获取 μC/OS-III、μC/CPU 和 μC/LIB 这三个组件的源代码。值得一提的是，笔者在编写本教程时，以上三个组件的已经正式发布的最新版本对应的版本号如下表所示：

组件	版本号
μC/OS-III	v3.08.01

μC/CPU	v1.32.01
μC/LIB	v1.39.01

表 1.3.1 本教程所使用的 μC 组件版本号

因此本教程，是以以上版本号对应的组件进行编写的，如果后续再有更新，建议读者也按照此版本号进行下载、学习，在经过初学阶段后，再使用最新版本。

那么，到底如何获取 μC/OS-III、μC/CPU 和 μC/LIB 这三个组件的源代码呢？在图 1.2.2 μC/OS 详情页中的顶部导航栏中，可以看到“CODEBASE”，点击“CODEBASE”就能跳转到 μC/OS-III 官方的 GitHub 仓库了，如下图所示：

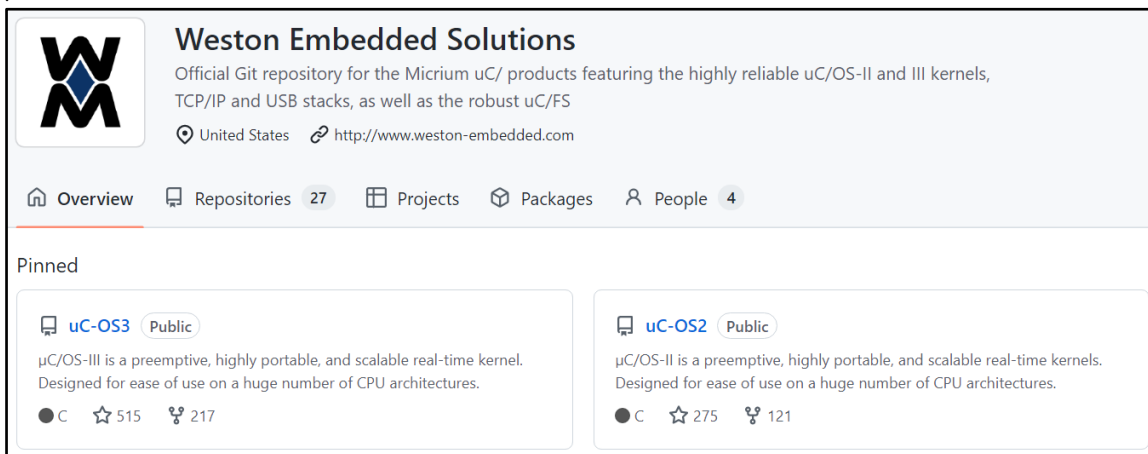


图 1.3.1 μC/OS-III 官方 GitHub 仓库

μC/OS-III 官方开源的源代码都放在了 GitHub 上使用 Git 进行管理，对 GitHub 和 Git 不熟悉的读者也不用担心，本教程会介绍如何在 GitHub 上获取仓库的源代码，这里以 μC/OS-III 为例进行演示，μC/CPU 和 μC/LIB 是类似的。

在图 1.3.1 中找到 uC-OS3（这就是 μC/OS-III，因为在路径中不能出现符号“/”，因此仓库的名称命名为 uC-OS3，μC/CPU 和 μC/LIB 也是类似的），如果在该页面中找不到对应的仓库，则点击图 1.3.1 中的“Repositories”，在全部仓库中进行检索。打开 μC/OS-III 的 GitHub 仓库后，如下图所示：

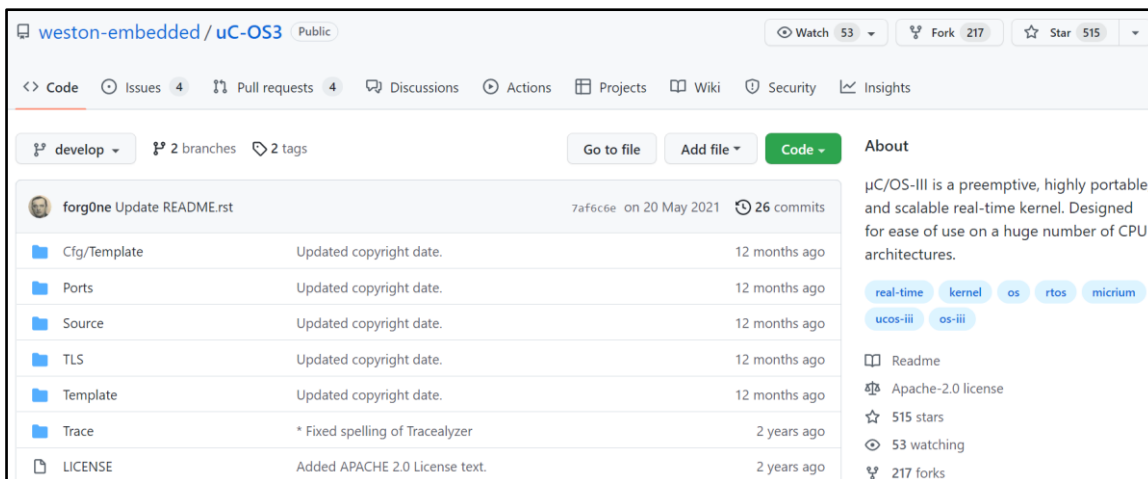


图 1.3.2 μC/OS-III 的 GitHub 仓库

此时还不能直接下载，因为该仓库默认展示的是一个正在开发中的分支，接下来需要切换到该仓库已经发布的 v3.08.01 版本的 Tag，然后才能下载，如下图所示：

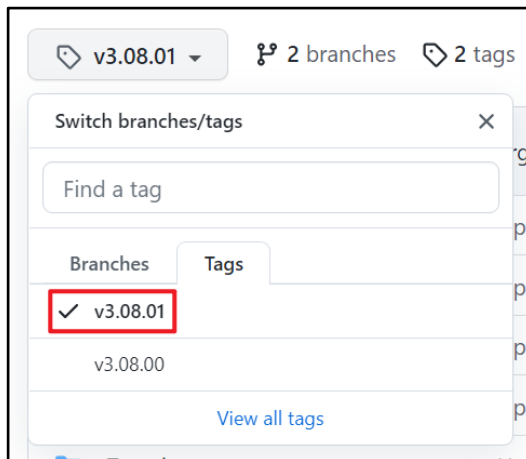


图 1.3.3 切换到指定 Tag

切换到指定的 Tag 后，就能够点击图 1.3.2 中的绿的图标“Code”，然后点击“Download ZIP”就能后下载到指定版本号为 v3.08.01 的 μC/OS-III 的源代码压缩包了，接着再对其进行解压，就能够得到 μC/OS-III 的源代码了。

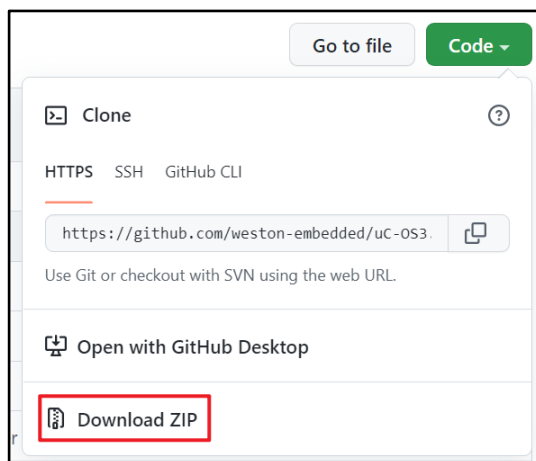


图 1.3.4 GitHub 下载按钮

按照以上步骤，再分别获取 μC/CPU 和 μC/LIB 的源代码，如下图所示：

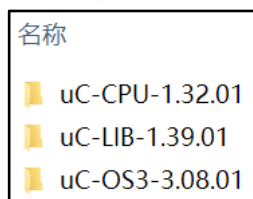


图 1.3.5 获取到的源代码

现在已经获取到了移植 μC/OS-III 所需的全部源代码了，在下面几个小节中，将分别对获取到的 μC/OS-III、μC/CPU 和 μC/LIB 源代码文件进行介绍。

1.3.2 μC/OS-III 文件预览

打开 uC-OS3-3.08.01 后可以看到如下图所示的文件和文件夹：

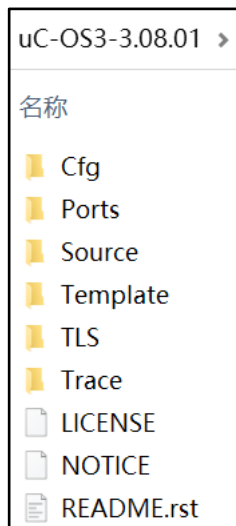


图 1.3.6 uC-OS3-3.08.01 文件夹

接下来介绍一些这些文件夹中都包含了那些东西。

1. Cfg 文件夹

Cfg 文件夹中包含的是 μC/OS-III 配置文件的模板文件，打开后如下图所示：

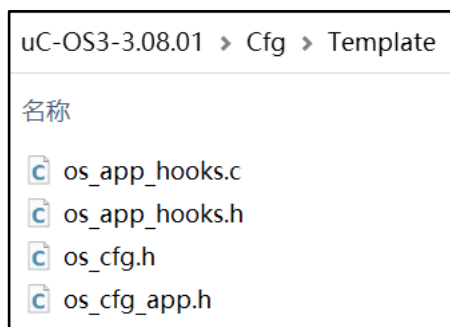


图 1.3.7 Cfg 文件夹

2. Ports 文件夹

Ports 文件夹包含的是与硬件相关的移植文件，打开后如下图所示：

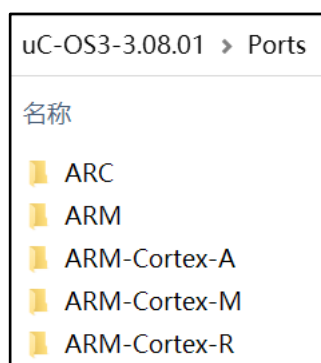


图 1.3.8 Ports 文件夹（部分）

限于篇幅，上图仅展示了 Ports 文件夹中的部分文件，从图中可以看出，μC/OS-III 提供了针对不同的芯片架构的移植文件，因为本教程是基于 STM32，因此在后续的移植中，将使用到 Ports 文件夹中 ARM-Cortex-M 文件夹中的文件。

3. Source 文件夹

通过文件夹的名称，就能猜出这是 μC/OS-III 的源码文件了，这也是 μC/OS-III 的核心文件，

打开后如下图所示：

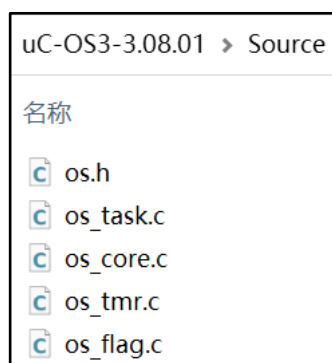


图 1.3.9 Source 文件夹（部分）

限于篇幅，上图仅展示了 Source 文件夹中的部分文件。可以看出，Source 文件夹中包含了与 μC/OS-III 内核、任务、软件定时器和事件标志相关的源码文件。

4. Template 文件夹

打开 Template 文件夹后，如下图所示：

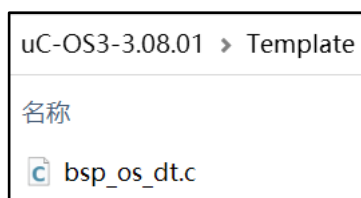


图 1.3.10 Template 文件夹

可以看出，Template 文件夹中仅包含了一个文件，这个文件是与动态 Tick 管理相关的文件。

5. TLS 文件夹

TLS 是 Thread Local Storage（线程本地存储）的缩写，打开后如下图所示：

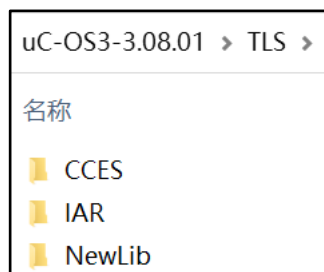


图 1.3.11 TLS 文件夹

6. Trace 文件夹

Trace 文件夹中是用来存放一些调试工具的，调试工具需要用户另外自行下载，μC/OS-III 的源代码文件中本身并不包含这一部分，调试工具主要有 SEGGER SystemView for uC/OS-III 和 Percepio Tracealyzer for uC/OS-III，感兴趣的读者可以自行下载体验。

7. 其他文件

文件	描述
LICENSE	μC/OS-III 的许可文件
NOTICE	μC/OS-III 的注意事项
README.rst	μC/OS-III 的 README 文件

表 1.3.2 其他文件

这几个文件不是很重要，但是建议做产品的读者，花点时间看一下。

1.3.3 μC/CPU 文件预览

打开 uC-CPU-1.32.01 后可以看到如下图所示的文件和文件夹：

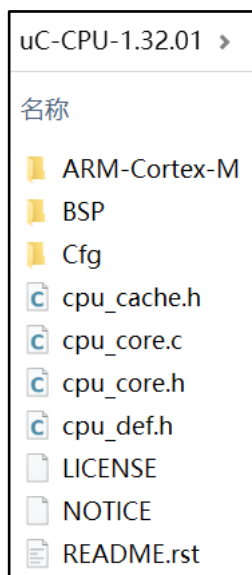


图 1.3.12 uC-CPU-1.32.01 文件夹（部分）

限于篇幅，这里仅展示了部分文件和文件夹，接下来介绍一些这些文件夹中都包含了那些东西。

1. ARM-Cortex-M 文件夹

此文件夹中包含了与 ARM Cortex-M 内核的 CPU 相关的移植文件。

2. BSP 文件夹

此文件夹中包含了与 BSP（Board Support Package，板级支持包）相关的文件，主要包含了用于配置和使用为 μC/OS-III 中时间戳功能提供支持的硬件定时器。

3. Cfg 文件夹

此文件夹中包含了 μC/CPU 这一组件的配置文件，打开后如下图所示：

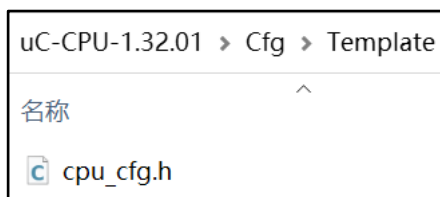


图 1.3.13 Cfg 文件夹

4. 其他文件

文件	描述
cpu_core.c	μC/CPU 的源码文件
cpu_core.h	
cpu_def.h	
cpu_cache.h	
LICENSE	μC/CPU 的许可文件
NOTICE	μC/CPU 的注意事项
README.rst	μC/CPU 的 README 文件

表 1.3.3 其他文件

1.3.4 μC/LIB 文件预览

打开 uC-LIB-1.39.01 后可以看到如下图所示的文件和文件夹：

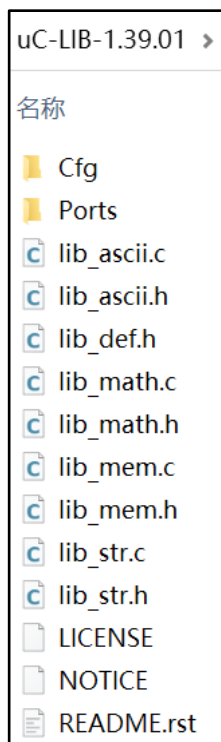


图 1.3.14 uC-LIB-1.39.01 文件夹

接下来介绍一些这些文件夹中都包含了那些东西。

1. Cfg 文件夹

此文件夹中包含了 μC/LIB 这一组件的配置文件，打开后如下图所示：

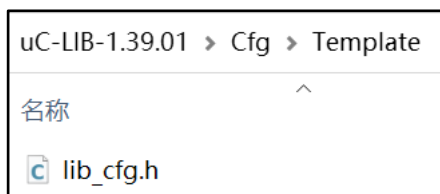


图 1.3.15 Cfg 文件夹

2. Ports 文件夹

此文件夹中包含了 μC/LIB 与硬件相关的移植文件，打开后如下图所示：

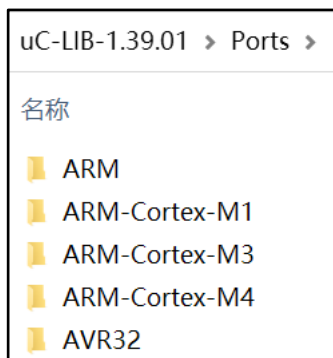


图 1.3.16 Ports 文件夹

从上图中可以看到，μC/LIB 组件仅支持了 ARM 和 AVR 部分架构的处理器，因此 μC/LIB

组件是移植 μ C/OS-III 时非必须的， μ C/LIB 主要是提供了 ASCII 字符操作、数学、内存管理、字符串操作的库，方便开发。

3. 其他文件

文件	描述
lib_def.h	μ C/LIB 的源码文件
lib_ascii.c	
lib_ascii.h	
lib_math.c	
lib_math.h	
lib_mem.c	
lib_mem.h	
lib_str.c	
lib_str.h	
LICENSE	μ C/LIB 的许可文件
NOTICE	μ C/LIB 的注意事项
README.rst	μ C/LIB 的 README 文件

表 1.3.4 其他文件

第二章 μC/OS-III 移植

在上一章中介绍了 μC/OS-III 并获取了 μC/OS-III 及其相关组件的源代码，接着还介绍了 μC/OS-III 及其相关组件的目录结构，本章就正式踏上 μC/OS-III 的学习之旅。在学习 μC/OS-III 之前，首先需要做好 μC/OS-III 的移植，本章就以正点原子的 STM32 系列开发板为例，进行 μC/OS-III 移植的介绍。

本章分为如下几部分：

- 2.1 μC/OS-III 移植
- 2.2 μC/OS-III 移植实验

2.1 μC/OS-III 移植

2.1.1 移植前准备

在开始移植 μC/OS-III 之前，需要提前准备好一个用于移植 μC/OS-III 的基础工程和 μC/OS-III 移植所需的相关源代码。

1. 基础工程

由于本教程后续的一些实验当中需要使用到 LED、LCD、串口、内存管理等外设及功能，因此就以正点原子标准例程-HAL 库版本的内存管理实验工程为基础工程进行 μC/OS-III 的移植。

2. μC/OS-III 移植所需源代码

移植 μC/OS-III 首先肯定需要 μC/OS-III 的源代码，其次还需要 μC/CPU 和 μC/LIB 这两个组件的源代码。在第一章中已经详细地介绍了如何获取 μC/OS-III、μC/CPU 和 μC/LIB 这三个组件的源代码，并且也说明了本教程所使用的软件版本，同样的，在开发板资料盘中也提供了本教程所使用的 μC/OS-III 及其相关组件的源代码，路径为：软件资料→μC/OS-III 学习资料→μC/OS-III 及其相关组件源代码.zip。

2.1.2 添加 μC/OS-III 相关文件

在准备好基础工程和 μC/OS-III 及其相关组件的源代码后，接下来就可以开始进行 μC/OS-III 的移植了。

1. 添加 μC/OS-III 及其相关组件源代码

在基础工程的 Middlewares 文件夹中新建一个 uC-OS3 子文件夹，如下图所示：



图 2.1.1 新建 uC-OS3 系文件夹

上图中 uC-OS3 文件夹就是新建的子文件夹，这里要说明的是，上图中其他文件及文件夹

是内存管理实验工程中原本就存在的。对于正点原子的不同 STM32 开发板，上图中所显示的文件和文件夹可能会有所差异，但只需要新建一个 uC-OS3 子文件夹即可。

接着就可以将 μC/OS-III 及其相关组件的源代码添加到刚刚新建的 uC-OS3 子文件夹中了。将 μC/OS-III、μC/CPU 和 μC/LIB 源代码添加到 uC-OS3 文件夹中，如下图所示：

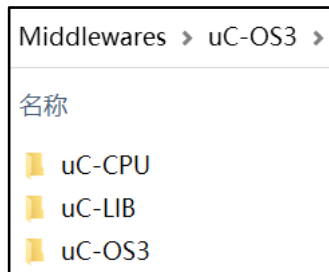


图 2.1.2 添加 μC/OS-III 及其相关组件源代码

这里要注意的是，从 GitHub 下载下来的 ZIP 源代码压缩包解压后的文件夹名称是带版本号后缀的，上图中是将文件夹进行重命名后的结果。

2. 将文件添加到工程

打开基础工程，新建四个分组，分别为 Middlewares/uC-OS3/BSP、Middlewares/uC-OS3/CPU、Middlewares/uC-OS3/LIB 和 Middlewares/uC-OS3/OS3，如下图所示：

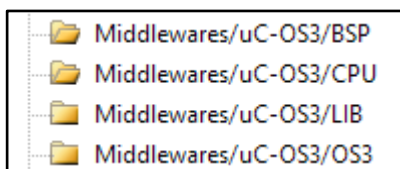


图 2.1.3 添加分组

Middlewares/uC-OS3/BSP 分组用于存放 μC/OS-III 等组件中与板级相关的文件，将 uC-CPU/BSP/Template/bsp_cpu.c 和 uC-OS3/Template/bsp_os_dt.c 这两个文件添加到 Middlewares/uC-OS3/BSP 分组中，如下图所示。

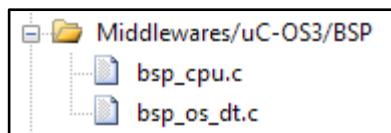


图 2.1.4 BSP 分组

Middlewares/uC-OS3/CPU 分组用于存放 μC/CPU 组件相关的文件，一共需要添加三个文件，分别为 uC-CPU/ARM-Cortex-M/ARMv7-M/ARM/cpu_a.asm、uC-CPU\ARM-Cortex-M\ARMv7-M\cpu_c.c 和 uC-CPU/cpu_core.c，将以上三个文件添加到 Middlewares/uC-OS3/CPU 分组中，如下图所示：

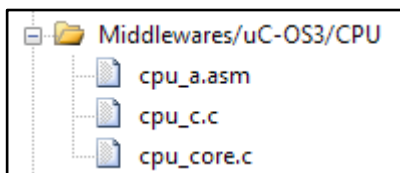


图 2.1.5 CPU 分组

Middlewares/uC-OS3/LIB 分组用于存放 μC/LIB 组件相关的文件，一共需要添加四个文件，分别为 uC-LIB/lib_ascii.c、uC-LIB/lib_math.c、uC-LIB/lib_mem.c 和 uC-LIB/lib_str.c，将以上四

个文件添加到 Middlewares/uC-OS3/LIB 分组中，如下图所示：

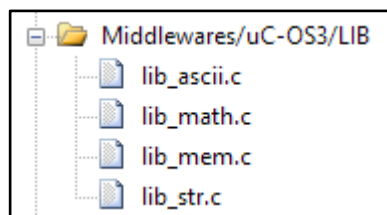


图 2.1.6 LIB 分组

Middlewares/uC-OS3/OS3 分组用于存放 μC/OS-III 组件相关的文件，一共需要添加十九个文件，分别为 uC-OS3/Cfg/Template/os_app_hooks.c、uC-OS3/Ports/ARM-Cortex-M/ARMv7-M/ARM/os_cpu_a.asm、uC-OS3/Ports/ARM-Cortex-M/ARMv7-M/os_cpu_c.c 以及 uC-OS3/Source 目录下除 __dbg_uCOS-III.c 外的所有 C 源文件，将以上列出的共十九个文件添加到 Middlewares/uC-OS3/OS3 分组中，如下图所示：

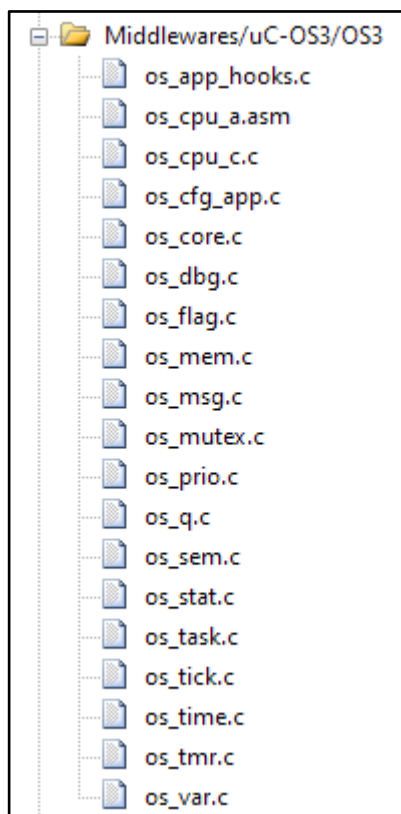


图 2.1.7 OS3 分组

3. 添加头文件路径

接下来添加 μC/OS-III 及其相关组件源代码的头文件路径，一共需要添加八个头文件路径，如下图红框所示：

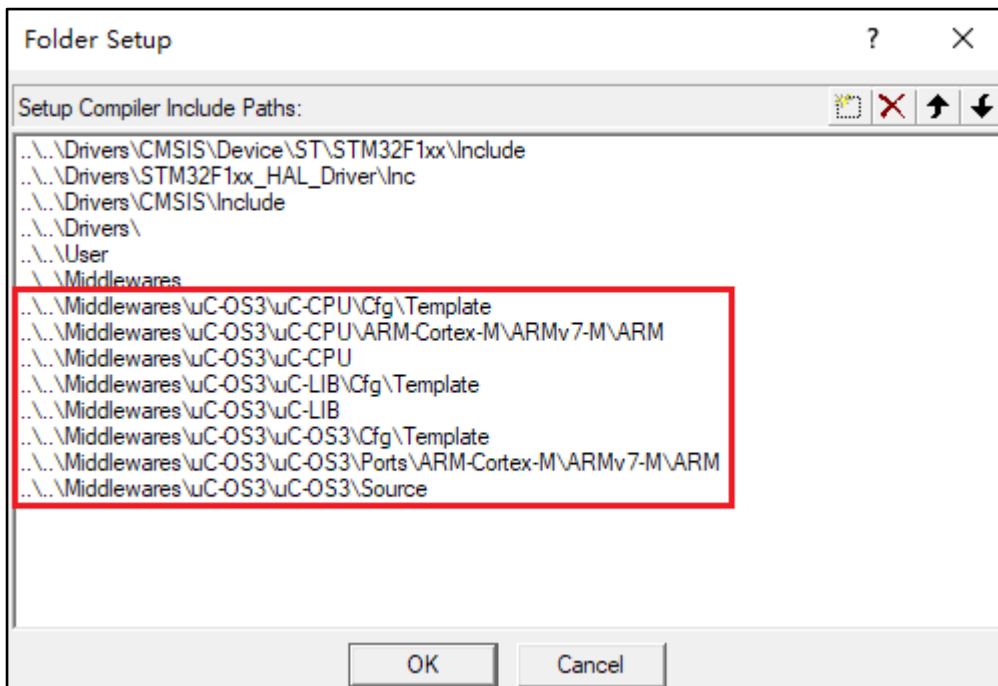


图 2.1.8 添加头文件路径

要注意的是，上图展示的是正点原子战舰 F103 开发板的头文件路径配置界面，对于正点原子 STM32 系列的其他开发板而言，只需添加上图中红框中的头文件路径，红框外的头文件路径为基础工程原本的配置，无需改动。

2.1.3 修改 SYSTEM 文件

SYSTEM 文件夹中一共需要修改三个文件，分别为 sys.h、usart.c 和 delay.c。

1. sys.h 文件

sys.h 文件的修改很简单，在 sys.h 文件中使用了宏 SYS_SUPPORT_OS 来定义是否支持 OS，因为要使用 μC/OS-III，因此应当将宏 SYS_SUPPORT_OS 定义为 1，具体的修改如下所示：

```
/**
 * SYS_SUPPORT_OS 用于定义系统文件夹是否支持 OS
 * 0, 不支持 OS
 * 1, 支持 OS
 */
#define SYS_SUPPORT_OS 1
```

2. usart.c 文件

usart.c 文件的修改也很简单，只需修改一个地方。在 usart.c 文件中包含的与 OS 相关的头文件，原本包含的与 OS 相关的头文件为“includes.h”，代码如下所示：

```
/* 如果使用 os, 则包括下面的头文件即可。 */
#if SYS_SUPPORT_OS
#include "includes.h" /* os 使用 */
#endif
```

现在要将其改为与 μC/OS-III 相关的头文件，因为在 usart.c 文件中要调用 μC/OS-III 的函数，修改后，如下所示：

```
/* 如果使用 os, 则包括下面的头文件即可。 */
```

```
#if SYS_SUPPORT_OS
#include "os.h"
#endif
```

3. delay.c 文件

delay.c 文件的改动比较大，大致分为六个步骤，如下所示：

第一步，修改 delay.c 文件中包含的与 OS 相关的头文件，原本包含的与 OS 相关的头文件为“includes.h”，代码如下所示：

```
/* 添加公共头文件（ucos 需要用到）*/
#include "includes.h"
```

现在要将其改为与 μC/OS-III 相关的头文件，因为在 delay.c 文件中要调用 μC/OS-III 的函数，修改后，如下所示：

```
/* 添加公共头文件（ucos 需要用到）*/
#include "os.h"
```

第二步，删除用于毫秒级延时的变量定义以及原本为了同时兼容 μC/OS-II 和 μC/OS-III 的宏定义和代码，删除的内容如下所示：

```
/* 定义 g_fac_ms 变量，表示 ms 延时的倍乘数，
 * 代表每个节拍的 ms 数，（仅在使能 os 的时候，需要用到）
 */
static uint16_t g_fac_ms = 0;

/*
 * 当 delay_us/delay_ms 需要支持 OS 的时候需要三个与 OS 相关的宏定义和函数来支持
 * 首先是 3 个宏定义：
 *      delay_osrunning          : 用于表示 OS 当前是否正在运行，以决定是否可以使用相关函数
 *      delay_ostickspersec      : 用于表示 OS 设定的时钟节拍，
 *                               delay_init 将根据这个参数来初始化 systick
 *      delay_osintnesting       : 用于表示 OS 中断嵌套级别，因为中断里面不可以调度，
 *                               delay_ms 使用该参数来决定如何运行
 * 然后是 3 个函数：
 *      delay_osschedlock        : 用于锁定 OS 任务调度，禁止调度
 *      delay_osschedunlock      : 用于解锁 OS 任务调度，重新开启调度
 *      delay_ostimedly          : 用于 OS 延时，可以引起任务调度。
 *
 * 本例程仅作 UCOSII 和 UCOSIII 的支持，其他 OS，请自行参考着移植
 */

/* 支持 UCOSII */
#ifdef OS_CRITICAL_METHOD /* OS_CRITICAL_METHOD 定义了
 * 说明要支持 UCOSII
 */

#define delay_osrunning OSRunning /* OS 是否运行标记，0，不运行；1，在运行 */
#define delay_ostickspersec OS_TICKS_PER_SEC /* OS 时钟节拍，即每秒调度次数 */
#define delay_osintnesting OSIntNesting /* 中断嵌套级别，即中断嵌套次数 */
```

```
#endif

/* 支持 UCOSIII */
#ifdef CPU_CFG_CRITICAL_METHOD /* CPU_CFG_CRITICAL_METHOD 定义了
    * 说明要支持 UCOSIII
    */

#define delay_osrunning OSRunning /* OS 是否运行标记,0,不运行;1,在运行 */
#define delay_ostickspersec OSCfg_TickRate_Hz /* OS 时钟节拍,即每秒调度次数 */
#define delay_osintnesting OSIntNestingCtr /* 中断嵌套级别,即中断嵌套次数 */
#endif

/**
 * @brief us 级延时,关闭任务调度(防止打断 us 级延迟)
 * @param 无
 * @retval 无
 */
static void delay_osschedlock(void)
{
#ifdef CPU_CFG_CRITICAL_METHOD /* 使用 UCOSIII */
    OS_ERR err;
    OSSchedLock(&err); /* UCOSIII 的方式,禁止调度,防止打断 us 延时 */
#else /* 否则 UCOSII */
    OSSchedLock(); /* UCOSII 的方式,禁止调度,防止打断 us 延时 */
#endif
}

/**
 * @brief us 级延时,恢复任务调度
 * @param 无
 * @retval 无
 */
static void delay_osschedunlock(void)
{
#ifdef CPU_CFG_CRITICAL_METHOD /* 使用 UCOSIII */
    OS_ERR err;
    OSSchedUnlock(&err); /* UCOSIII 的方式,恢复调度 */
#else /* 否则 UCOSII */
    OSSchedUnlock(); /* UCOSII 的方式,恢复调度 */
#endif
}

/**
 * @brief us 级延时,恢复任务调度
```

```

* @param   ticks: 延时的节拍数
* @retval  无
*/
static void delay_ostimedly(uint32_t ticks)
{
#ifdef CPU_CFG_CRITICAL_METHOD
    OS_ERR err;
    OSTimeDly(ticks, OS_OPT_TIME_PERIODIC, &err); /* UCOSIII 延时采用周期模式 */
#else
    OSTimeDly(ticks);                             /* UCOSII 延时 */
#endif
}

```

第三步，修改 SysTick 的中断服务函数，因为要使用 SysTick 为 μC/OS-III 内核提供时钟节拍，修改后的 SysTick 中断服务函数如下所示：

```

/**
* @brief   systick 中断服务函数,使用 OS 时用到
* @param   ticks: 延时的节拍数
* @retval  无
*/
void SysTick_Handler(void)
{
    /* OS 开始跑了,才执行正常的调度处理 */
    if (OSRunning == OS_STATE_OS_RUNNING)
    {
        /* 调用 uC/OS-III 的 SysTick 中断服务函数 */
        OS_CPU_SysTickHandler();
    }
    HAL_IncTick();
}

```

第四步，修改函数 delay_init()，该函数主要用于配置 SysTick 以及初始化用于 CPU 忙延时延时的变量，修改后的函数 delay_init()如下所示：

```

/**
* @brief   初始化延迟函数
* @param   sysclk: 系统时钟频率，即 CPU 频率(HCLK)
* @retval  无
*/
void delay_init(uint16_t sysclk)
{
#ifdef SYS_SUPPORT_OS
    uint32_t reload;
#endif

    SysTick->CTRL = 0;
}

```

```
HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
g_fac_us = sysclk;

#if SYS_SUPPORT_OS
    reload = sysclk;
    reload *= 1000000 / OSCfg_TickRate_Hz;

    SysTick->CTRL |= 1 << 1;
    SysTick->LOAD = reload;
    SysTick->CTRL |= 1 << 0;
#endif
}
```

第五步，修改函数 `delay_us()`，该函数用于微秒级的 CPU 忙延迟，修改后的函数 `delay_us()` 如下所示：

```
void delay_us(uint32_t nus)
{
    uint32_t ticks;
    uint32_t told, tnow, tcnt = 0;
    uint32_t reload;
    /* 定义用于接收错误代码的变量 */
    OS_ERR err;

    reload = SysTick->LOAD;
    ticks = nus * g_fac_us;
    /* 锁定 uC/OS-III 的任务调度器 */
    OSSchedLock(&err);
    told = SysTick->VAL;

    while (1)
    {
        tnow = SysTick->VAL;

        if (tnow != told)
        {
            if (tnow < told)
            {
                tcnt += told - tnow;
            }
            else
            {
                tcnt += reload - tnow + told;
            }
        }
    }
}
```

```

        told = tnow;

        if (tcnt >= ticks) break;
    }
}

/* 恢复 uC/OS-III 的任务调度器 */
OSSchedUnlock(&err);
}

```

第六步，修改函数 `delay_ms()`，该函数用于毫秒级的 CPU 忙延时，修改后的函数 `delay_ms()` 如下所示：

```

void delay_ms(uint16_t nms)
{
    uint32_t i;

    for (i=0; i<nms; i++)
    {
        delay_us(1000);
    }
}

```

至此，SYSTEM 文件夹针对 μC/OS-III 的修改就完成了。

2.1.4 替换中断服务函数

在 μC/OS-III 的移植过程中会遇到两个重要的中断，分别为 SysTick 中断和 PendSV 中断，这两个中断分别用于为 μC/OS-III 内核提供时基和任务切换，在上一小节修改 `delay.c` 文件的时候，已经涉及到了 SysTick 的中断服务函数。HAL 库会默认定义 SysTick 和 PendSV 的中断服务函数，因此在移植 μC/OS-III 的时候，需要将 HAL 库默认定义的 SysTick 和 PendSV 中断服务函数，替换为 μC/OS-III 提供了中断服务函数。

首先，删除 HAL 库默认定义的 SysTick 和 PendSV 的中断服务函数，根据不同型号的主控芯片，HAL 库将中断服务函数定义在不同的文件中，具体的对应关系如下：

正点原子的 STM32 系列开发板类型	中断服务函数所在文件
STM32F1	stm32f1xx_it.c 和 stm32f1xx_it.h
STM32F4	stm32f4xx_it.c 和 stm32f4xx_it.h
STM32F7	stm32f7xx_it.c 和 stm32f7xx_it.h
STM32H7	stm32h7xx_it.c 和 stm32h7xx_it.h

表 2.1.1 不同类型开发板与中断函数所在文件的对应关系

打开上表中的 C 源文件，并删除 SysTick 和 PendSV 的中断服务函数，被删除的代码如下所示：

```

/**
 * @brief This function handles PendSVC exception.
 * @param None
 * @retval None
 */

```



```
void PendSV_Handler(void)
{
}

/**
 * @brief This function handles SysTick Handler.
 * @param None
 * @retval None
 */
void SysTick_Handler(void)
{
    HAL_IncTick();
}
```

同时删除这两个中断服务函数在 C 源文件对应的 H 头文件中的声明，被删除的代码如下所示：

```
void PendSV_Handler(void);
void SysTick_Handler(void);
```

这么一来，就已经将 HAL 默认定义的 SysTick 和 PendSV 的中断服务函数删除了，接下来添加 μC/OS-III 中已经定义好的 SysTick 和 PendSV 中断服务函数。其中 SysTick 的中断服务函数已经在上一小节中修改 delay.c 文件的时候调用了，那么接下来只要添加 PendSV 中断服务函数即可，打开工程中的启动文件并找到向量表的定义位置，如下所示：

__Vectors	DCD	__initial_sp	; Top of Stack
	DCD	Reset_Handler	; Reset Handler
	DCD	NMI_Handler	; NMI Handler
	DCD	HardFault_Handler	; Hard Fault Handler
	DCD	MemManage_Handler	; MPU Fault Handler
	DCD	BusFault_Handler	; Bus Fault Handler
	DCD	UsageFault_Handler	; Usage Fault Handler
	DCD	0	; Reserved
	DCD	0	; Reserved
	DCD	0	; Reserved
	DCD	0	; Reserved
	DCD	SVC_Handler	; SVCcall Handler
	DCD	DebugMon_Handler	; Debug Monitor Handler
	DCD	0	; Reserved
	DCD	PendSV_Handler	; PendSV Handler
	DCD	SysTick_Handler	; SysTick Handler

将向量表中 PendSV 的中断服务函数修改为 μC/OS-III 中定义的 PendSV 中断服务函数，修改后的向量表如下所示：

__Vectors	DCD	__initial_sp	; Top of Stack
	DCD	Reset_Handler	; Reset Handler
	DCD	NMI_Handler	; NMI Handler
	DCD	HardFault_Handler	; Hard Fault Handler

```
DCD MemManage_Handler      ; MPU Fault Handler
DCD BusFault_Handler       ; Bus Fault Handler
DCD UsageFault_Handler     ; Usage Fault Handler
DCD 0                      ; Reserved
DCD 0                      ; Reserved
DCD 0                      ; Reserved
DCD 0                      ; Reserved
DCD SVC_Handler            ; SVCcall Handler
DCD DebugMon_Handler       ; Debug Monitor Handler
DCD 0                      ; Reserved
; 替换为 μC/OS-III 中提供的 PendSV 中断服务函数
DCD OS_CPU_PendSVHandler    ; PendSV Handler
DCD SysTick_Handler        ; SysTick Handler
```

最后修改该文件中原本对 PendSV 中断服务函数的弱定义代码，修改前的代码如下所示：

```
PendSV_Handler      PROC
                    EXPORT PendSV_Handler      [WEAK]
                    B       .
                    ENDP
```

修改后的代码如下所示：

```
OS_CPU_PendSVHandler PROC
                    EXPORT OS_CPU_PendSVHandler [WEAK]
                    B       .
                    ENDP
```

2.1.5 配置 μC/OS-III 及其相关组件

在第一章介绍 μC/OS-III 的时候就说到，μC/OS-III 是可裁剪的，那么移植 μC/OS-III 就需要涉及到对 μC/OS-III 的配置，当然，同时 μC/CPU 和 μC/LIB 也是需要配置的，但是鉴于移植 μC/OS-III，这里并不对 μC/OS-III 及其相关组件做过多的配置，本小节的目的是让 μC/OS-III 的正点原子的 STM32 系列开发板上“跑起来”，具体对 μC/OS-III 及其相关组件的配置，在后面的章节中会有详细的介绍。

为了让 μC/OS-III 在正点原子的 STM32 系列开发板上“跑起来”只需要修改 μC/CPU 的配置文件即可，而 μC/OS-III 和 μC/LIB 的配置不用改动。μC/CPU 的配置文件的路径在 uC-CPU/Cfg/Template/cpu_cfg.h，打开该文件，并将配置文件中用于配置配置 CPU 中断优先级寄存器使用位数的宏开关置 1 即可，修改前的代码如下所示：

```
#if 0
#define CPU_CFG_NVIC_PRIO_BITS    4u
#endif
```

修改后的代码如下所示：

```
/* 将此宏开关置 1 */
#if 1
#define CPU_CFG_NVIC_PRIO_BITS    4u
#endif
```

虽然这样的配置能够让 μC/OS-III “跑起来”，但是这样的配置过于“简单粗暴”，可是现在

还未对 μC/OS-III 及其相关组件做详细的介绍，因此建议新手读者直接复制使用本教程文档的配套例程源码中《UCOS-III 实验例程 2 UCOS-III 移植实验》实验的配置文件。对 μC/OS-III 及其相关组件的配置，一共涉及到四个配置文件，这四个配置文件的文件名和对应的路径如下表所示：

配置文件文件名	配置文件所在路径
cpu_cfg.h	uC-CPU/Cfg/Template/cpu_cfg.h
lib_cfg.h	uC-OS3/uC-LIB/Cfg/lib_cfg.h
os_cfg.h	uC-OS3/Cfg/Template/os_cfg.h
os_cfg_app.h	uC-OS3/Cfg/Template/os_cfg_app.h

表 2.1.2 配置文件

新手读者在初次移植 μC/OS-III 的时候，直接按照上表配置文件的路径，将本教程文档的配套例程源码中《UCOS-III 实验例程 2 UCOS-III 移植实验》实验的配置文件复制到自己的移植工程中使用即可，下文会对配置项做详细的介绍。

至此，μC/OS-III 的移植就完毕了，整体来说难度并不高，但是作为 μC/OS-III 的初学者，有些读者可能对移植过程中涉及的一些修改存在疑惑，对于这个问题，笔者建议读者耐心跟着本教程的步骤完成，在后续的讲解中，会一一为读者解决这些疑惑，学习本来就是一个循序渐进的过程，不能想着要一口吃成大胖子。

2.1.6 可选步骤（建议完成）

这个步骤是可选的，但是笔者强烈建议读者完成，因为这是为了方面后续的实验以及规范工程和精简工程文件。本小节可分为四个小部分，分别为配置时间戳功能修改工程目标名、移除 USMART 调试组件和删除无用 μC/OS-III 相关文件。

1. 配置时间戳功能

在后续的实验章节或读者的实际开发中，通常需要使用到 μC/OS-III 的时间戳功能，该功能会再后续相应的章节中进行介绍，但 μC/OS-III 中许多功能都依赖于时间戳功能，因此建议读者在移植的过程中，先对时间戳功能做好相应的配置。

由于还未对 μC/OS-III 的时间戳功能进行相关的介绍，因此建议新手读者直接复制本教程文档的配套例程源码中《UCOS-III 实验例程 2 UCOS-III 移植实验》对 μC/OS-III 中时间戳功能配置的相关配置文件，仅涉及到一个配置文件，该文件的文件名和对应的路径如下表所示：

配置文件文件名	配置文件所在路径
bsp_cpu.c	uC-CPU/BSP/Template/bsp_cpu.c

表 2.1.3 时间戳功能配置文件

新手读者在初次配置时间戳功能的时候，直接按照上表中时间戳功能相关配置文件的路径，讲本教程文档的配套例程源码中《UCOS-III 实验例程 2 UCOS-III 移植实验》实验的相应配置文件复制到自己的工程中即可，下文会对 μC/OS-III 的时间戳功能做详细的介绍。

2. 修改工程目标名称

本教程是以标准例程-HAL 库版本的内存管理实验工程为基础工程，内存管理实验工程的工程名为“MALLOC”，为了规范工程，笔者建议将工程的目标名修改为“uC-OS3”或根据读者的实际场景进行修改，修改如下图所示：

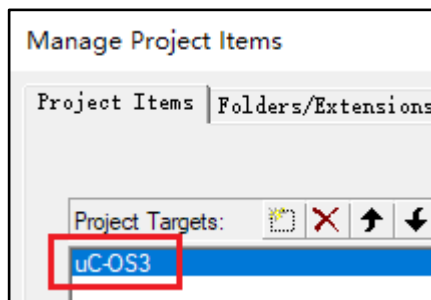


图 2.1.9 修改工程目标名

3. 移除 USMART 调试组件

由于本教程并未使用到 USMART 调试组件，因此建议将 USMART 调试组件从工程中移除，如果读者需要使用 USMART 调试组件的话，也可以选择保留。

移除 USMART 调试组件，首先删除工程中 USMART 调试组件的分组，被删除的分组如下图所示：

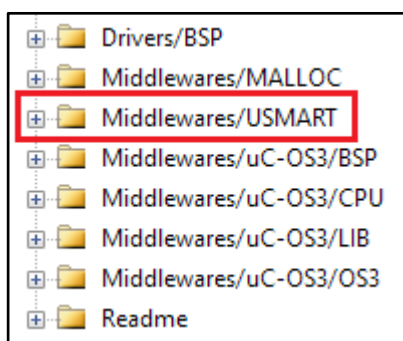


图 2.1.10 删除 USMART 调试组件分组

接着将工程文件夹中 USMART 调试组件的文件夹删除即可，被删除的文件夹如下图所示：



图 2.1.11 删除 USMART 调试组件文件夹

4. 删除无用 μC/OS-III 相关文件

在 μC/OS-III 的移植过程中，用到了 μC/OS-III 官方提供的许多文件，但也还有大量架构相关等的文件没有用上，为了精简工程，方便工程的保存和传输，笔者建议删除无用的 μC/OS-III 相关文件或文件夹。

精简后 uC-CPU 文件夹中的文件如下图所示：

```

uC-CPU/
|-- ARM-Cortex-M
|   |-- ARMv7-M
|   |   |-- ARM
|   |   |   |-- cpu.h
|   |   |   |-- cpu_a.asm
|   |   |-- cpu_c.c
|   |-- BSP
|   |   |-- Template
|   |   |-- bsp_cpu.c
|   |-- Cfg
|   |   |-- Template
|   |   |-- cpu_cfg.h
|   |-- cpu_cache.h
|   |-- cpu_core.c
|   |-- cpu_core.h
|   |-- cpu_def.h

```

图 2.1.12 精简后的 uC-CPU 文件夹
精简后的 uC-LIB 文件夹中的文件如下图所示：

```

uC-LIB/
|-- Cfg
|   |-- Template
|   |-- lib_cfg.h
|-- lib_ascii.c
|-- lib_ascii.h
|-- lib_def.h
|-- lib_math.c
|-- lib_math.h
|-- lib_mem.c
|-- lib_mem.h
|-- lib_str.c
|-- lib_str.h

```

图 2.1.13 精简后的 uC-LIB 文件夹
精简后的 uC-OS3 文件夹中的文件如下图所示：

```
uC-OS3/
|-- Cfg
|   |-- Template
|       |-- os_app_hooks.c
|       |-- os_app_hooks.h
|       |-- os_cfg.h
|       |-- os_cfg_app.h
|-- Ports
|   |-- ARM-Cortex-M
|       |-- ARMv7-M
|           |-- ARM
|               |-- os_cpu.h
|               |-- os_cpu_a.asm
|               |-- os_cpu_c.c
|-- Source
|   |-- os.h
|   |-- os_cfg_app.c
|   |-- os_core.c
|   |-- os_dbg.c
|   |-- os_flag.c
|   |-- os_mem.c
|   |-- os_msg.c
|   |-- os_mutex.c
|   |-- os_prio.c
|   |-- os_q.c
|   |-- os_sem.c
|   |-- os_stat.c
|   |-- os_task.c
|   |-- os_tick.c
|   |-- os_time.c
|   |-- os_tmr.c
|   |-- os_trace.h
|   |-- os_type.h
|   |-- os_var.c
|-- Template
|   |-- bsp_os_dt.c
```

当然，读者也可以根据自己的实际需求保留有用的文件。

当然，读者也可以根据自己的实际需求保留有用的文件。

2.1.7 添加应用程序

移植好 $\mu\text{C}/\text{OS-III}$ 之后,当然要测试一下移植是否成功。在本小节的步骤中,一共需要修改一个文件并添加两个文件,要修改的文件为 `main.c` 文件,添加的两个文件分别为 `uc-os3_demo.c` 和 `uc-os3_demo.h`。对于 `main.c` 文件,主要是函数 `main()` 中完成一些硬件的初始化,最后调用文件 `us-os3_demo.c` 文件中的函数 `uc_os3_demo()`,为添加的两个文件中的 C 源文件用于编写 $\mu\text{C}/\text{OS-III}$ 的应用代码, H 头文件为该 C 源文件的头文件,用于声明 $\mu\text{C}/\text{OS-III}$ 应用程序的入口函数 `us os3 demo()`。

1. main.c 文件

由于正点原子的 STM32 系列开发板众多，这次这里仅以正点原子战舰开发板为例为读者进行演示，其他开发板类似。读者可以根据自己所移植的目标开发板在本教程的配套例程源码的《UCOS-III 移植实验》实验中查看对应的 main.c 文件。正点原子战舰开发板《UCOS-III 移植实验》中的 main.c 文件如下所示：

```
/**
*****
* @file      main.c
* @author    正点原子团队 (ALIENTEK)
* @version   V1.0
* @date      2022-04-13
* @brief     uC/OS-III 实验
* @license   Copyright (c) 2020-2032, 广州市星翼电子科技有限公司
*****
* @attention
*
* 实验平台: 正点原子 战舰 F103 开发板
* 在线视频: www.yuanzige.com
* 技术论坛: www.openedv.com
* 公司网址: www.alientek.com
* 购买地址: openedv.taobao.com
*
*****
*/

#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/usart/usart.h"
#include "./SYSTEM/delay/delay.h"
#include "./BSP/LED/led.h"
#include "./BSP/LCD/lcd.h"
#include "./BSP/KEY/key.h"
#include "./BSP/SRAM/sram.h"
#include "./MALLOC/malloc.h"
#include "uc-os3_demo.h"

int main(void)
{
    HAL_Init();                /* 初始化 HAL 库 */
    sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟, 72Mhz */
    delay_init(72);            /* 延时初始化 */
    usart_init(115200);         /* 串口初始化为 115200 */
    led_init();                /* 初始化 LED */
    lcd_init();                /* 初始化 LCD */
    key_init();                /* 初始化按键 */
    sram_init();               /* SRAM 初始化 */
    my_mem_init(SRAMIN);       /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMEX);       /* 初始化外部 SRAM 内存池 */
}
```

```
uc_os3_demo(); /* 运行 uC/OS-III 例程 */
}
```

可以看到，在 main.c 文件中只包含了一个 main() 函数，main() 函数主要就是完成了一些外设的初始化，如串口、LED、LCD、按键等，并在最后调用了函数 uc_os3_demo()。

2. uc-os3_demo.c 文件

```
/**
 * 正点原子团队 (ALIENTEK)
 * 2022-04-13
 * uC/OS-III 移植实验
 * Copyright (c) 2020-2032, 广州市星翼电子科技有限公司
 *
 * 实验平台: 正点原子 战舰 F103 开发板
 * 在线视频: www.yuanzige.com
 * 技术论坛: www.openedv.com
 * 公司网址: www.alientek.com
 * 购买地址: openedv.taobao.com
 */

#include "uc-os3_demo.h"
#include "../SYSTEM/usart/usart.h"
#include "../BSP/LED/led.h"
#include "../BSP/LCD/lcd.h"
#include "../MALLOC/malloc.h"

/*uC/OS-III 配置*/

/* START_TASK 任务 配置
 * 包括: 任务优先级 任务栈大小 任务控制块 任务栈 任务函数
 */

#define START_TASK_PRIO    2          /* 任务优先级 */
#define START_STK_SIZE     512        /* 任务栈大小 */
OS_TCB                     StartTask_TCB; /* 任务控制块 */
```



```

CPU_STK                StartTask_STK[START_STK_SIZE]; /* 任务栈 */
void start_task(void *p_arg);                          /* 任务函数 */

/* TASK1 任务 配置
 * 包括：任务优先级 任务栈大小 任务控制块 任务栈 任务函数
 */
#define TASK1_PRIO      (OS_CFG_PRIO_MAX - 4)          /* 任务优先级 */
#define TASK1_STK_SIZE  1024                          /* 任务栈大小 */
OS_TCB                 Task1Task_TCB;                  /* 任务控制块 */
CPU_STK                *Task1Task_STK;                 /* 任务栈 */
void task1(void *p_arg);                               /* 任务函数 */

/* TASK2 任务 配置
 * 包括：任务优先级 任务栈大小 任务控制块 任务栈 任务函数
 */
#define TASK2_PRIO      (OS_CFG_PRIO_MAX - 4)          /* 任务优先级 */
#define TASK2_STK_SIZE  1024                          /* 任务栈大小 */
OS_TCB                 Task2Task_TCB;                  /* 任务控制块 */
CPU_STK                *Task2Task_STK;                 /* 任务栈 */
void task2(void *p_arg);                               /* 任务函数 */

/*****

/* LCD 刷屏时使用的颜色 */
uint16_t lcd_discolor[11] = { WHITE, BLACK, BLUE, RED,
                               MAGENTA, GREEN, CYAN, YELLOW,
                               BROWN, BRRED, GRAY};

/**
 * @brief   uC/OS-III 例程入口函数
 * @param   无
 * @retval  无
 */
void uc_os3_demo(void)
{
    OS_ERR err;

    lcd_show_string(10, 10, 220, 32, 32, "STM32", RED);
    lcd_show_string(10, 47, 220, 24, 24, "uC/OS-III Porting", RED);
    lcd_show_string(10, 76, 220, 16, 16, "ATOM@ALIENTEK", RED);

    /* 初始化 uC/OS-III */
    OSInit(&err);

```

```

/* 创建 Start Task */
OSTaskCreate(    (OS_TCB*      ) &StartTask_TCB,
                  (CPU_CHAR*    ) "start_task",
                  (OS_TASK_PTR  ) start_task,
                  (void*        ) 0,
                  (OS_PRIO      ) START_TASK_PRIO,
                  (CPU_STK*     ) StartTask_STK,
                  (CPU_STK_SIZE ) START_STK_SIZE / 10,
                  (CPU_STK_SIZE ) START_STK_SIZE,
                  (OS_MSG_QTY   ) 0,
                  (OS_TICK      ) 0,
                  (void*        ) 0,
                  (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                  (OS_ERR*      ) &err);

/* 开始任务调度 */
OSStart(&err);

for (;;)
{
    /* 不会进入这里 */
}

/**
 * @brief   start_task
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度, 时间片设为默认值 */

```

```

OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

/* 创建 Task1 */
Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK1_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                  (CPU_CHAR*    ) "task1",
                  (OS_TASK_PTR  ) task1,
                  (void*        ) 0,
                  (OS_PRIO      ) TASK1_PRIO,
                  (CPU_STK*     ) Task1Task_STK,
                  (CPU_STK_SIZE ) TASK1_STK_SIZE / 10,
                  (CPU_STK_SIZE ) TASK1_STK_SIZE,
                  (OS_MSG_QTY   ) 0,
                  (OS_TICK      ) 0,
                  (void*        ) 0,
                  (OS_OPT        ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                  (OS_ERR*      ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      ) &Task2Task_TCB,
                  (CPU_CHAR*    ) "task2",
                  (OS_TASK_PTR  ) task2,
                  (void*        ) 0,
                  (OS_PRIO      ) TASK2_PRIO,
                  (CPU_STK*     ) Task2Task_STK,
                  (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,
                  (CPU_STK_SIZE ) TASK2_STK_SIZE,
                  (OS_MSG_QTY   ) 0,
                  (OS_TICK      ) 0,
                  (void*        ) 0,
                  (OS_OPT        ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                  (OS_ERR*      ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

/**
 * @brief   task1
 * @param   p_arg : 传入参数(未用到)

```

```

* @retval 无
*/
void task1(void *p_arg)
{
    uint32_t task1_num = 0;
    OS_ERR err;

    while(1)
    {
        lcd_clear lcd_discolor[++task1_num % 14]); /* 刷新屏幕 */
        lcd_show_string(10, 10, 220, 32, 32, "STM32", RED);
        lcd_show_string(10, 47, 220, 24, 24, "uC/OS-III Porting", RED);
        lcd_show_string(10, 76, 220, 16, 16, "ATOM@ALIENTEK", RED);
        LED0_TOGGLE(); /* LED0 闪烁 */
        OSTimeDly(1000, OS_OPT_TIME_DLY, &err); /* 延时 1000ticks */
    }
}

/**
* @brief task2
* @param p_arg : 传入参数(未用到)
* @retval 无
*/
void task2(void *p_arg)
{
    float float_num = 0.0;
    OS_ERR err;

    while(1)
    {
        float_num += 0.01f; /* 更新数值 */
        printf("float_num: %0.4f\r\n", float_num); /* 打印数值 */
        OSTimeDly(1000, OS_OPT_TIME_DLY, &err); /* 延时 1000ticks */
    }
}

```

对于 uc-os3_demo.c 文件, 这里先简单介绍以下这个文件的代码结构, 这个文件的代码结构可分为六个部分, 分别是包含头文件、μC/OS-III 相关配置、全局变量及自定义函数、应用程序入口函数、开始任务入口函数、其他任务入口函数, 接下来分别地介绍以上一个部分的代码。

(1) 包含头文件

包含的头文件分成两个部分, 分别为 μC/OS-III 相关的头文件和其他头文件, 如下所示:

```

#include "uc-os3_demo.h"
#include "../SYSTEM/usart/usart.h"
#include "../BSP/LED/led.h"

```

```
#include "./BSP/LCD/lcd.h"
#include "./MALLOC/malloc.h"

/*uC/OS-III*****
#include "os.h"
#include "cpu.h"

/*****
```

(2) μC/OS-III 相关配置

μC/OS-III 的配置主要包括所创建 μC/OS-III 任务的相关定义（任务优先级、任务栈大小、任务控制块、任务栈、任务函数）以及定义应用于全局的 μC/OS-III 相关变量（信号量、事件标志、消息队列、软件定时器等），如下所示：

```
/*uC/OS-III 配置*/

/* START_TASK 任务 配置
 * 包括：任务优先级 任务栈大小 任务控制块 任务栈 任务函数
 */
#define START_TASK_PRIO    2                /* 任务优先级 */
#define START_STK_SIZE     512              /* 任务栈大小 */
OS_TCB                     StartTask_TCB;   /* 任务控制块 */
CPU_STK                    StartTask_STK[START_STK_SIZE]; /* 任务栈 */
void start_task(void *p_arg);               /* 任务函数 */

/* TASK1 任务 配置
 * 包括：任务优先级 任务栈大小 任务控制块 任务栈 任务函数
 */
#define TASK1_PRIO         (OS_CFG_PRIO_MAX - 4) /* 任务优先级 */
#define TASK1_STK_SIZE     1024                  /* 任务栈大小 */
OS_TCB                     Task1Task_TCB;        /* 任务控制块 */
CPU_STK                    *Task1Task_STK;        /* 任务栈 */
void task1(void *p_arg);    /* 任务函数 */

/* TASK2 任务 配置
 * 包括：任务优先级 任务栈大小 任务控制块 任务栈 任务函数
 */
#define TASK2_PRIO         (OS_CFG_PRIO_MAX - 4) /* 任务优先级 */
#define TASK2_STK_SIZE     1024                  /* 任务栈大小 */
OS_TCB                     Task2Task_TCB;        /* 任务控制块 */
CPU_STK                    *Task2Task_STK;        /* 任务栈 */
void task2(void *p_arg);    /* 任务函数 */

/*****
```

(3) 全局变量及自定义函数

这部分主要用来定义全局变量及自定义的函数，如下所示：

```
/* LCD 刷屏时使用的颜色 */
uint16_t lcd_discolor[11] = { WHITE, BLACK, BLUE, RED,
                              MAGENTA, GREEN, CYAN, YELLOW,
                              BROWN, BRRED, GRAY};
```

(4) 应用程序入口函数

这部分就是函数 `uc_os3_demo()`，函数 `uc_os3_demo()` 一开始就是在 LCD 上显示一些具体实验相关的信息，然后对 μC/OS-III 内核进行初始化，接着创建一个开始任务，最后开启 μC/OS-III 内核的任务调度，如下所示：

```
/**
 * @brief   uC/OS-III 例程入口函数
 * @param   无
 * @retval  无
 */
void uc_os3_demo(void)
{
    OS_ERR err;

    lcd_show_string(10, 10, 220, 32, 32, "STM32", RED);
    lcd_show_string(10, 47, 220, 24, 24, "uC/OS-III Porting", RED);
    lcd_show_string(10, 76, 220, 16, 16, "ATOM@ALIENTEK", RED);

    /* 初始化 uC/OS-III */
    OSInit(&err);

    /* 创建 Start Task */
    OSTaskCreate(    (OS_TCB*      ) &StartTask_TCB,
                    (CPU_CHAR*    ) "start_task",
                    (OS_TASK_PTR  ) start_task,
                    (void*        ) 0,
                    (OS_PRIO     ) START_TASK_PRIO,
                    (CPU_STK*     ) StartTask_STK,
                    (CPU_STK_SIZE) START_STK_SIZE / 10,
                    (CPU_STK_SIZE) START_STK_SIZE,
                    (OS_MSG_QTY   ) 0,
                    (OS_TICK      ) 0,
                    (void*        ) 0,
                    (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                    (OS_ERR*      ) &err);

    /* 开始任务调度 */
    OSStart(&err);

    for (;;)
    {
    }
```

```
{
    /* 不会进入这里 */
}
}
```

(5) 开始任务入口函数

这部分就是开始任务的入口函数，开始任务主要用于初始化 μC/CPU 组件、配置 μC/OS-III 内核节拍的时基定时器、初始化 μC/OS-III 的内存管理功能、开启 μC/OS-III 内核的时间片调度，并且还会创建或初始化一些特定于实验的和 μC/OS-III 相关的功能（信号量、事件标志、消息队列、软件定时器等），最后还会创建其他用于实验演示的任务，如下所示：

```
/**
 * @brief   start_task
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                    (CPU_CHAR*     ) "task1",
                    (OS_TASK_PTR   ) task1,
                    (void*         ) 0,
                    (OS_PRIO       ) TASK1_PRIO,
                    (CPU_STK*      ) Task1Task_STK,
                    (CPU_STK_SIZE  ) TASK1_STK_SIZE / 10,
                    (CPU_STK_SIZE  ) TASK1_STK_SIZE,
                    (OS_MSG_QTY    ) 0,
                    (OS_TICK       ) 0,
                    (void*         ) 0,
```

```

(OS_OPT      ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
(OS_ERR*     ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                       TASK2_STK_SIZE * sizeof(CPU_STK));
OSTaskCreate(  (OS_TCB*      ) &Task2Task_TCB,
               (CPU_CHAR*    ) "task2",
               (OS_TASK_PTR  ) task2,
               (void*        ) 0,
               (OS_PRIO      ) TASK2_PRIO,
               (CPU_STK*     ) Task2Task_STK,
               (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,
               (CPU_STK_SIZE ) TASK2_STK_SIZE,
               (OS_MSG_QTY   ) 0,
               (OS_TICK      ) 0,
               (void*        ) 0,
               (OS_OPT      ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
               (OS_ERR*     ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

(6) 其他任务入口函数

这部分包含了实验中用于演示的任务入口函数，如下所示：

```

/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    uint32_t task1_num = 0;
    OS_ERR err;

    while(1)
    {
        lcd_clear lcd_discolor[++task1_num % 14]); /* 刷新屏幕 */
        lcd_show_string(10, 10, 220, 32, 32, "STM32", RED);
        lcd_show_string(10, 47, 220, 24, 24, "uC/OS-III Porting", RED);
        lcd_show_string(10, 76, 220, 16, 16, "ATOM@ALIENTEK", RED);
        LED0_TOGGLE(); /* LED0 闪烁 */
        OSTimeDly(1000, OS_OPT_TIME_DLY, &err); /* 延时 1000ticks */
    }
}

```



```

    }
}

/**
 * @brief   task2
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task2(void *p_arg)
{
    float float_num = 0.0;
    OS_ERR err;

    while(1)
    {
        float_num += 0.01f;           /* 更新数值 */
        printf("float_num: %0.4f\r\n", float_num); /* 打印数值 */
        OSTimeDly(1000, OS_OPT_TIME_DLY, &err); /* 延时 1000ticks */
    }
}

```

以上就是 uc_os3_demo.c 文件的代码结构，本教程配套的实验例程都会按照这个代码结构进行实验代码的编写，建议读者先熟悉以下这个代码结构。

3. uc-os3_demo.h 文件

```

#ifndef __UC_OS3_DEMO_H
#define __UC_OS3_DEMO_H

void uc_os3_demo(void);

#endif

```

uc-os3_demo.h 这个文件很简单，就是将函数 uc_os3_demo()进行声明，方便被其他 C 源文件调用。

2.1.8 使用 AC6 编译工程（扩展）

首先说明，本教程配套的实验工程全部采用 AC5 进行开发。

AC 是“ARM Compiler”的简称，是用于编译 ARM 处理器代码的编译工具链，AC 后面的 5 和 6 表示的是 AC 的版本号，在 Keil MDK 中集成了 AC5 和 AC6 编译工具链，这里简单地对比一下 AC5 和 AC6 的差异，如下表所示：

对比项	AC5	AC6	说明
中文支持	较好	较差	AC6 对中文支持极差，goto definition 无法使用，报错等
代码兼容性	较好	较差	AC6 对某些代码优化可能导致运行异常，需慢慢调试
编译速度	较慢	较快	AC6 编译速度比 AC5 快
语法检查	一般	严格	AC6 语法检查非常严格，代码严谨性较好

表 2.1.3 AC5&AC6 简单对比

虽然 AC6 的编译速度比较快，但是 AC5 在各方面的兼容性都比 AC6 要好，因此笔者推荐新手读者使用 AC5。当然读者可以自由地选择使用 AC5 或 AC6，正点原子的源码都是支持 AC6 的，只不过在 Keil MDK 的选项配置方面有些差异，具体差异如下表所示：

选项	AC5	AC6	说明
Target -> ARM Compiler	V5.xx	V6.xx	选择对应的编译工具

表 2.1.4 AC5&AC6 配置差异

通过以上配置正点原子的裸机程序已经能够正常使用 AC6 编译工具进行编译了，但是对于 μC/OS-III 还需要稍作修改，需要修改两个地方，分别是 μC/OS-III 的 os_cpu_a.asm 文件和头文件路径。

1. μC/OS-III 的 os_cpu_a.asm 文件

在 2.1.2 小节中，往工程添加 os_cpu_a.asm 文件的路径为 uC-OS3/Ports/ARM-Cortex-M/ARMv7-M/ARM/os_cpu_a.asm，这是在使用 AC5 编译工具时，如果使用 AC6 编译工具，则需将 os_cpu_a.asm 文件应替换成 uC-OS3/Ports/ARM-Cortex-M/ARMv7-M/GNU/os_cpu_a.S 文件，如下图所示：

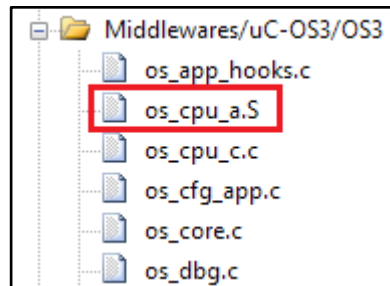


图 2.1.12 替换 os_cpu_a.asm 文件

因为 os_cpu_a.S 文件是按照 GNU 风格编写的汇编文件，直接编译的话会报错，这是还需要对该文件进行单独配置，让 Keil MDK 知道，该文件是使用 GNU 风格编写的汇编文件。配置的方法也很简单，首先在工程分组目录中对 os_cpu_a.S 文件右键并点击“Options for File ‘os_cpu_a.S’...”打开该文件的配置页面，如下图所示：

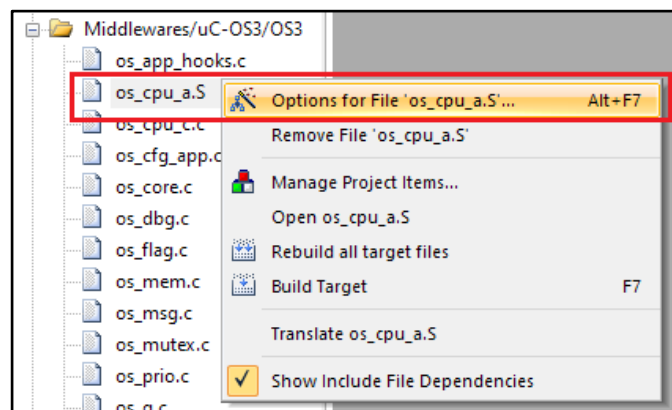


图 2.1.13 文件右键菜单图

接着在弹出的配置页面中，点击顶部菜单栏中的“Asm”选项卡，最后将“Assembler Option”（这个选项要求先将 Keil MDK 的编译工具改为 AC6 后，才会出现）中的选项改为“armclang (GNU Syntax)”即可，如下如所示：

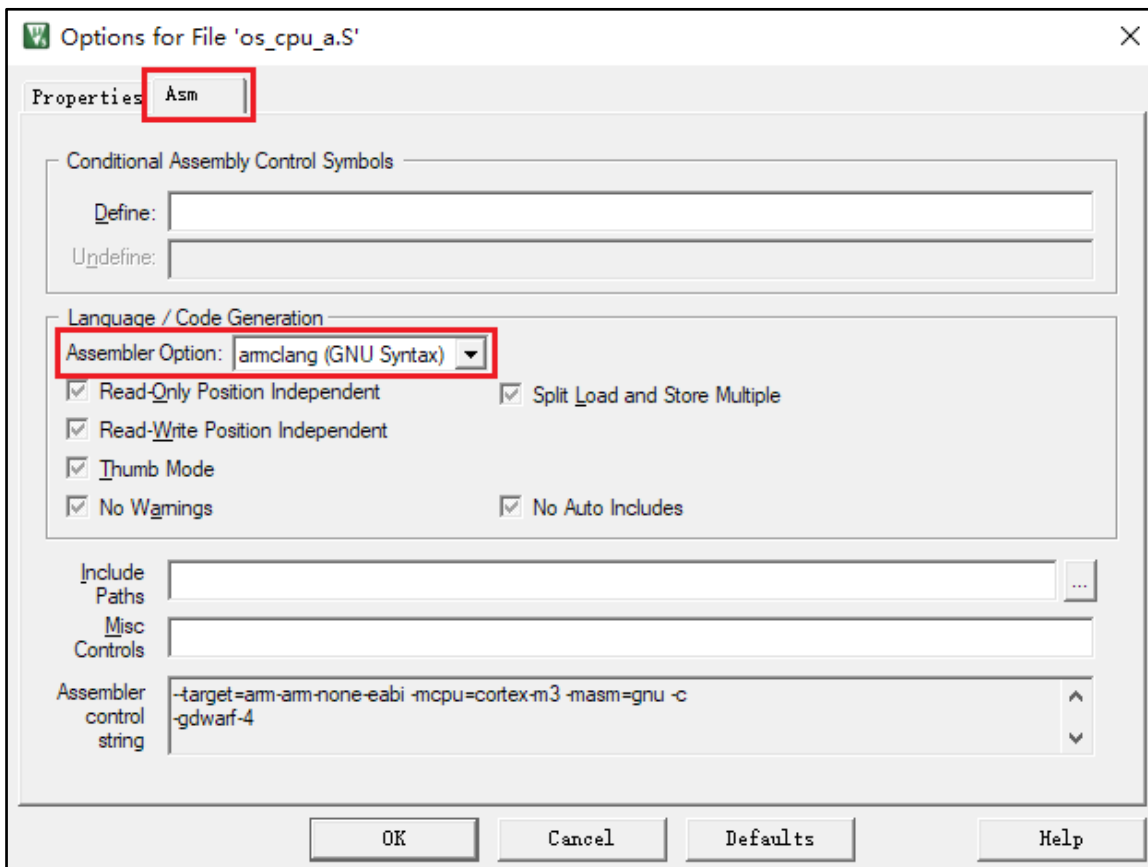


图 2.1.14 修改配置选项

当然，别忘了点击“OK”保存并退出。

2. 头文件路径

在 2.1.2 小节中，添加了一个以 os_cpu_a.asm 文件路径为路径的头文件路径，由于使用 AC6 编译器，使用了 os_cpu_a.S 文件替换了 os_cpu_a.asm 文件，因此需要将原来以 os_cpu_a.asm 文件路径为路径的头文件路径的路径，修改为 os_cpu_a.S 文件的路径，修改后如下图所示：

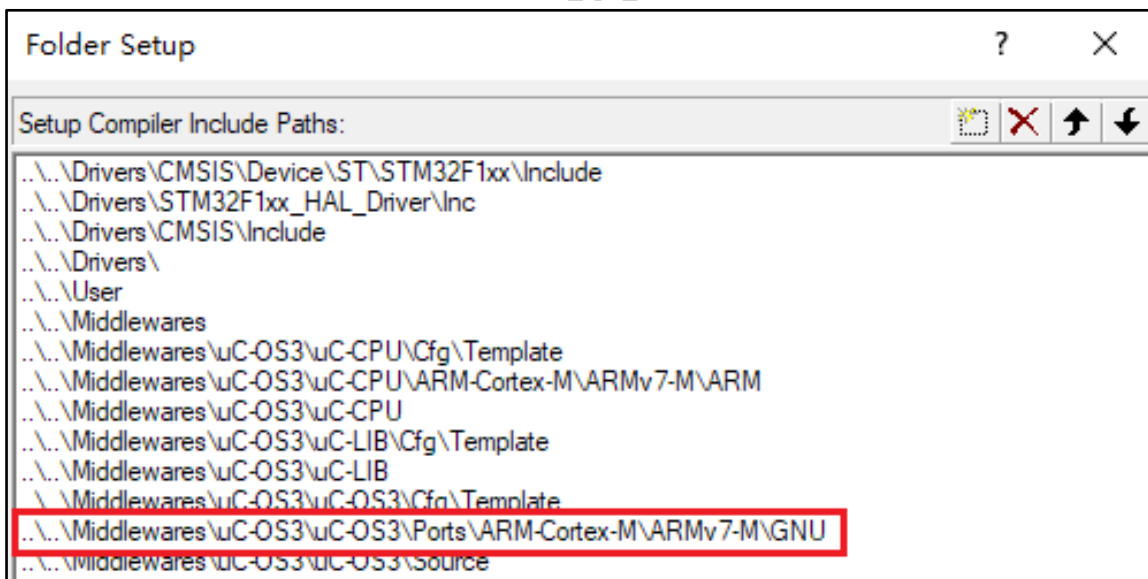


图 2.1.15 修改头文件路径

这里要注意的是，在切换了 Keil MDK 编译工具后，最好完成地重新编译一遍工程，即

“Rebuild all target files”而不是“Build Target”，以免出现一些莫名的问题。

2.2 μC/OS-III 移植实验

2.2.1 功能设计

1. 例程功能

本实验主要用于验证 μC/OS-III 移植是否成功，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建其他任务
task1	用于刷新 LCD 背景颜色和控制 LED0 闪烁
task2	用于通过串口打印浮点计算结果

表 2.2.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 2 UCOS-III 移植实验》。

2.2.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

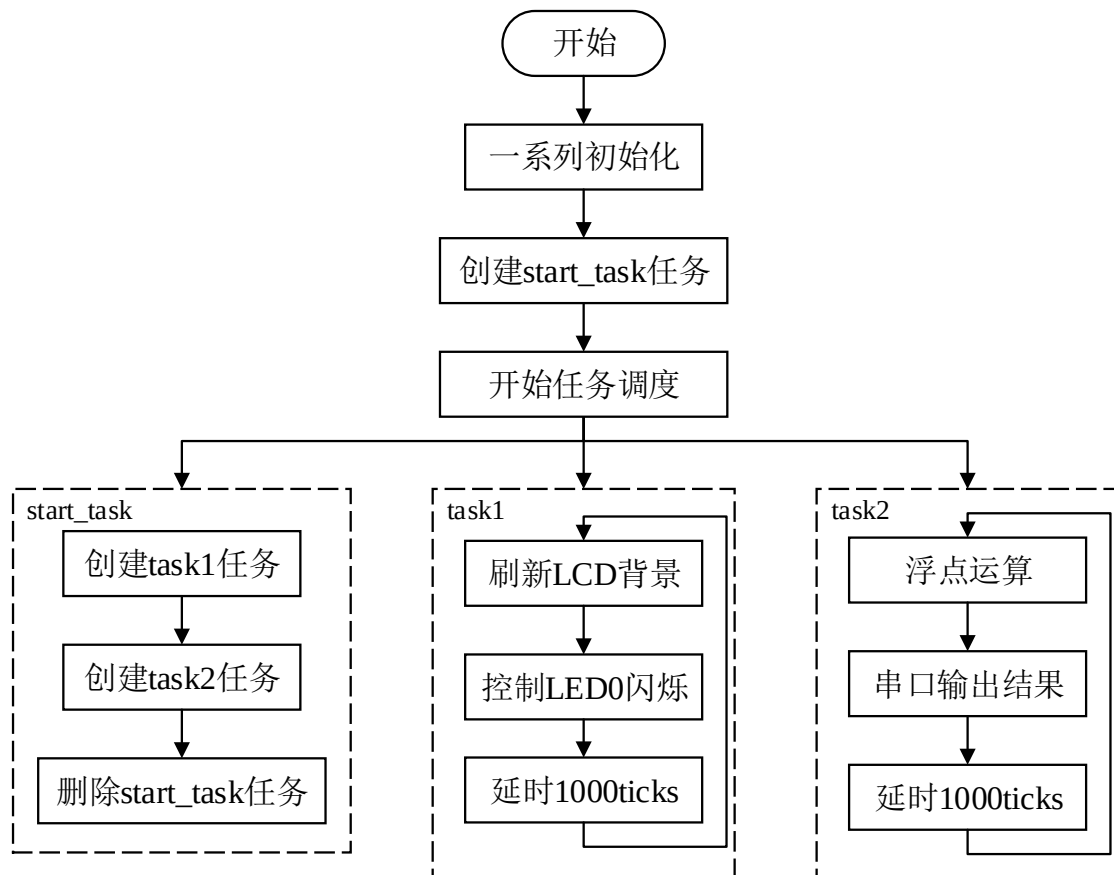


图 2.2.1 程序流程图

2. 程序解析

在 2.1.7 小节中已经对整体的代码结构作了说明，这里主要解析 task1 任务和 task2 任务，而具体的配置、任务创建、任务调度等 μC/OS-III 的相关知识，在下文会具体分析。

(1) task1 任务

task1 任务的入口函数代码如下所示：

```
/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    uint32_t task1_num = 0;
    OS_ERR err;

    while(1)
    {
        lcd_clear lcd_discolor[++task1_num % 14]); /* 刷新屏幕 */
        lcd_show_string(10, 10, 220, 32, 32, "STM32", RED);
        lcd_show_string(10, 47, 220, 24, 24, "uC/OS-III Porting", RED);
        lcd_show_string(10, 76, 220, 16, 16, "ATOM@ALIENTEK", RED);
        LED0_TOGGLE(); /* LED0 闪烁 */
        OSTimeDly(1000, OS_OPT_TIME_DLY, &err); /* 延时 1000ticks */
    }
}
```

可以看到，task1 任务比较简单，在一个 while(1) 循环中，每间隔 1000 个 ticks 就刷新一次屏幕背景，并控制 LED0 闪烁。

(2) task2 任务

```
/**
 * @brief  task2
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task2(void *p_arg)
{
    float float_num = 0.0;
    OS_ERR err;

    while(1)
    {
        float_num += 0.01f; /* 更新数值 */
        printf("float_num: %0.4f\r\n", float_num); /* 打印数值 */
        OSTimeDly(1000, OS_OPT_TIME_DLY, &err); /* 延时 1000ticks */
    }
}
```

```
}  
}
```

可以看到，task2 任务也很简单，在一个 while(1) 循环中，每间隔 1000 个 ticks 就进行一次浮点运算，并将运算结果通过串口输出。

本实验所设计的任务都比较简单，并没有具体实际的意义，只是为了验证 μC/OS-III 移植的成功与否，因此本实验所涉及的一些函数包括函数的用法，读者暂时无需深究，后面的内容中会有详细的讲解。

2.2.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示，并且 LCD 屏幕的背景颜色和板载的 LED0 每间隔 1000 个 ticks 就切换一次状态（根据 os_cfg_app.h 文件的相关配置，1000 个 ticks 大致相当于 1000ms 的时间，os_cfg_app.h 文件是 μC/OS-III 的配置文件之一，具体配置和 ticks-to-ms 换算的方法，在下文讲解到 μC/OS-III 时间管理的时候会具体分析）。



图 2.2.2 LCD 显示内容

同时通过串口调试助手可以看到，串口每间隔 1000 个 ticks 就输出一浮点计算的结果，如下图所示：



图 2.2.3 串口调试助手显示内容

第三章 μC/OS-III 配置项

μC/OS-III 作为一个完善的操作系统，提供了丰富的功能，但在通常实际的应用场景中，需要考虑到各方面的因素，例如芯片上 Flash 和 RAM 的大小。为了使得 μC/OS-III 能够适用于各种各样的场景，μC/OS-III 被设计成可配置和裁剪的，在前面的章节中，也提及了 μC/OS-III 的配置项，本章就来详细地讲解如何配置和裁剪 μC/OS-III。

本章分为如下几部分：

- 3.1 μC/OS-III 及其相关组件的配置项
- 3.2 μC/OS-III 配置项
- 3.3 μC/CPU 配置项
- 3.4 μC/LIB 配置项

3.1 μC/OS-III 及其相关组件的配置项

在前面章节移植 μC/OS-III 的时候，提到了 μC/OS-III 及其相关组件的配置项，μC/OS-III、μC/CPU、μC/LIB 各自对应的配置项如表 2.1.2 所示，并且，μC/OS-III 官方提供了这些配置项的模板，用户仅需在模板的基础上稍作修改就能够应用到自己的工程中了。下面就分别来看一下 μC/OS-III、μC/CPU 和 μC/LIB 配置项中的配置项，了解如何裁剪和配置 μC/OS-III 使之适用于实际的应用开发当中。

3.2 μC/OS-III 配置项

μC/OS-III 一共有两个配置项，分别为 `os_cfg.h` 和 `os_cfg_app.h`，在 μC/OS-III 的官方在线文档中，对这两个文件都做了详细的解释，其中针对 `os_cfg.h` 配置项的在线文档网址为 <https://micrium.atlassian.net/wiki/spaces/osiiiidoc/pages/132335/>，针对 `os_cfg_app.h` 配置项的在线文档网址为 <https://micrium.atlassian.net/wiki/spaces/osiiiidoc/pages/132378/>。

3.2.1 os_cfg.h 配置项

`os_cfg.h` 文件主要用于配置 μC/OS-III 内核的一些功能，例如使能或禁用钩子函数、参数检查、信号量功能、消息队列功能等等，具体包含的配置项，如下表所示：

杂项配置项
OS_CFG_APP_HOOKS_EN
OS_CFG_ARG_CHK_EN
OS_CFG_CALLED_FROM_ISR_CHK_EN
OS_CFG_DBG_EN
OS_CFG_TICK_EN
OS_CFG_DYN_TICK_EN
OS_CFG_INVALID_OS_CALLS_CHK_EN
OS_CFG_OBJ_TYPE_CHK_EN
OS_CFG_OBJ_CREATED_CHK_EN
OS_CFG_TS_EN
OS_CFG_PRIO_MAX

OS_CFG_SCHED_LOCK_TIME_MEAS_EN
OS_CFG_SCHED_ROUND_ROBIN_EN
OS_CFG_STK_SIZE_MIN
事件标志配置项
OS_CFG_FLAG_EN
OS_CFG_FLAG_DEL_EN
OS_CFG_FLAG_MODE_CLR_EN
OS_CFG_FLAG PEND_ABORT_EN
内存管理配置项
OS_CFG_MEM_EN
互斥信号量配置项
OS_CFG_MUTEX_EN
OS_CFG_MUTEX_DEL_EN
OS_CFG_MUTEX PEND_ABORT_EN
消息队列配置项
OS_CFG_Q_EN
OS_CFG_Q_DEL_EN
OS_CFG_Q_FLUSH_EN
OS_CFG_Q PEND_ABORT_EN
信号量配置项
OS_CFG_SEM_EN
OS_CFG_SEM_DEL_EN
OS_CFG_SEM PEND_ABORT_EN
OS_CFG_SEM_SET_EN
任务管理配置项
OS_CFG_STAT_TASK_EN
OS_CFG_STAT_TASK_STK_CHK_EN
OS_CFG_TASK_CHANGE_PRIO_EN
OS_CFG_TASK_DEL_EN
OS_CFG_TASK_IDLE_EN
OS_CFG_TASK_PROFILE_EN
OS_CFG_TASK_Q_EN
OS_CFG_TASK_Q PEND_ABORT_EN
OS_CFG_TASK_REG_TBL_SIZE
OS_CFG_TASK_STK_REDZONE_EN
OS_CFG_TASK_STK_REDZONE_DEPTH
OS_CFG_TASK_SEM PEND_ABORT_EN
OS_CFG_TASK_SUSPEND_EN
任务本地存储配置项
OS_CFG_TLS_TBL_SIZE
时间管理配置项
OS_CFG_TIME_DLY_HMSM_EN

OS_CFG_TIME_DLY_RESUME_EN
软件定时器管理配置项
OS_CFG_TMR_EN
OS_CFG_TMR_DEL_EN
跟踪记录功能配置项
OS_CFG_TRACE_EN
OS_CFG_TRACE_API_ENTER_EN
OS_CFG_TRACE_API_EXIT_EN

表 3.2.1 os_cfg.h 配置文件配置项

1. OS_CFG_APP_HOOKS_EN

此宏用于配置使能或禁用应用程序指定的钩子函数。当此宏配置为 1 时，使能应用程序指定的钩子函数，通过这个功能，用户可以很方便地在应用程序中扩展 μC/OS-III 的功能。

2. OS_CFG_ARG_CHK_EN

此宏用于配置使能或禁用 μC/OS-III 函数的参数检查功能。当此宏配置为 1 时，μC/OS-III 会检查函数的传入参数是否合法，例如传递给函数的指针是否非空、传递的参数是否在允许的范围值内、参数对应的选项是否有效等。启用此功能将会增加 μC/OS-III 占用的 Flash 空间和 RAM 空间。

3. OS_CFG_CALLED_FROM_ISR_CHK_EN

此宏用于配置使能或禁用检查是否在中断中非法调用任务级函数。当此宏配置为 1 时，μC/OS-III 会在执行函数前，判断该函数是否为中断中被非法调用。

4. OS_CFG_DBG_EN

此宏用于配置使能或禁用 μC/OS-III 的代码调试功能。当此宏配置为 1 时，会编译 os_dbg.c 文件中的内容，以帮助进行代码调试。

5. OS_CFG_TICK_EN

此宏用于配置使能或禁用 μC/OS-III 的内核时钟节拍。当此宏配置为 1 时，则使能 μC/OS-III 内核的时钟节拍功能。

6. OS_CFG_DYN_TICK_EN

此宏用于配置使能或禁用 μC/OS-III 的动态时钟节拍管理功能。当此宏配置为 1 时，则使能 μC/OS-III 的动态时钟节拍管理功能。

7. OS_CFG_INVALID_OS_CALLS_CHK_EN

此宏用于配置使能或禁用检查是否在 μC/OS-III 系统开始运行前非法调用系统函数。当此宏配置为 1 时，则使能该检查功能。

8. OS_CFG_OBJ_TYPE_CHK_EN

此宏用于配置使能或禁用检查 μC/OS-III 对象操作函数传入的对象参数类型是否合法。当此宏配置为 1 时，则使能该检查功能。

9. OS_CFG_OBJ_CREATED_CHK_EN

此宏用于配置使能或禁用检查 μC/OS-III 对象是否被重复创建。当此宏配置为 1 时，则使能该检查功能。

10. OS_CFG_TS_EN

此宏用于配置使能或禁用时间戳功能。当此宏配置为 1 时，则使能时间戳功能。

11. OS_CFG_PRIO_MAX

此宏用于定义任务优先级的最大数值。系统中最高任务优先级的数值为 0，最低任务优先级的数值为 OS_CFG_PRIO_MAX-1。

12. OS_CFG_SCHED_ROUND_ROBIN_EN

此宏用于配置使能或禁用时间片调度功能。当此宏配置为 1 时，则使能时间片调度功能。

13. OS_CFG_STK_SIZE_MIN

此宏用于定义系统中允许的最小任务栈的大小，默认单位为字（由变量类型 CPU_STK 决定）。

14. OS_CFG_FLAG_EN

此宏用于使能或禁用事件标志功能。当此宏配置为 1 时，则使能事件标志功能。

15. OS_CFG_FLAG_DEL_EN

此宏用于使能或禁用事件标志的删除功能。当此宏配置为 1 时，会编译函数 OSFlagDel()，用于删除事件标志。

16. OS_CFG_FLAG_MODE_CLR_EN

此宏用于使能或禁用在等待或发布事件标志时，可以同时设置或清除事件标志功能。当此宏配置为 1 时，则使能该功能。

17. OS_CFG_FLAG_PEND_ABORT_EN

此宏用于使能或禁用终止等待事件标志功能。当此宏配置为 1 时，会编译函数 OSFlagPendAbort()，用于终止等待事件标志。

18. OS_CFG_MEM_EN

此宏用于使能或禁用 μC/OS-III 的内存管理功能。当此宏配置为 1 时，则使能 μC/OS-III 的内存管理功能。

19. OS_CFG_MUTEX_EN

此宏用于使能或禁用互斥信号量功能。当此宏配置为 1 时，则使能互斥信号量功能。

20. OS_CFG_MUTEX_DEL_EN

此宏用于使能或禁用互斥信号量的删除功能。当此宏配置为 1 时，会编译函数 OSMutexDel()，用于删除互斥信号量。

21. OS_CFG_MUTEX_PEND_ABORT_EN

此宏用于使能或禁用终止等待互斥信号量功能。当此宏配置为 1 时，会编译函数 OSMutexPendAbort()，用于终止等待互斥信号量。

22. OS_CFG_Q_EN

此宏用于使能或禁用消息队列功能。当此宏配置为 1 时，则使能消息队列功能。

23. OS_CFG_Q_DEL_EN

此宏用于使能或禁用消息队列删除功能。当此宏配置为 1 时，会编译函数 OSQDel()，用于删除消息队列。

24. OS_CFG_Q_FLUSH_EN

此宏用于使能或禁用清空消息队列功能。当此宏配置为 1 时，会编译函数 OSQFlush()，用于清空消息队列中的消息。

25. OS_CFG_Q_PEND_ABORT_EN

此宏用于使能或禁用终止等待消息队列功能。当此宏配置为 1 时，会编译函数 OSQPendAbort()，用于终止等待消息队列。

26. OS_CFG_SEM_EN

此宏用于使能或禁用信号量功能。当此宏配置为 1 时，则使能信号量功能。

27. OS_CFG_SEM_DEL_EN

此宏用于使能或禁用信号量删除功能。当此宏配置为 1 时，会编译函数 OSSemDel()，用于删除信号量。

28. OS_CFG_SEM_PEND_ABORT_EN

此宏用于使能或禁用终止等待信号量功能。当此宏配置为 1 时，会编译函数 OSSemPendAbort()，用于终止等待信号量。

29. OS_CFG_SEM_SET_EN

此宏用于使能或禁用强制设置信号量值功能。当此宏配置为 1 时，会编译函数 OSSemSet()，用于强制设置信号量的值。

30. OS_CFG_STAT_TASK_EN

此宏用于使能或禁用 μC/OS-III 的任务统计功能。当此宏配置为 1 时，则使能 μC/OS-III 的任务统计功能。

31. OS_CFG_STAT_TASK_STK_CHK_EN

此宏用于使能或禁用在进行任务统计时，检查任务栈的情况。当此宏配置为 1 时，则使能该功能。

32. OS_CFG_TASK_CHANGE_PRIO_EN

此宏用于使能或禁用更改任务优先级的功能。当此宏配置为 1 时，会编译函数 OSTaskChangePrio()，用于更改任务的任务优先级。

33. OS_CFG_TASK_DEL_EN

此宏用于使能或禁用任务删除功能。当此宏配置为 1 时，会编译函数 OSTaskDel()，用于删除任务。

34. OS_CFG_TASK_IDLE_EN

此宏用于使能或禁用空闲任务功能。当此宏配置为 1 时，则使能空闲任务功能。

35. OS_CFG_TASK_PROFILE_EN

此宏用于使能或禁用任务分析功能。当此宏配置为 1 时，则使能任务分析功能。

36. OS_CFG_TASK_Q_EN

此宏用于使能或禁用任务内嵌的消息队列功能。当此宏配置为 1 时，则使能任务内嵌的消息队列功能，此时会编译 OSTaskQXXXX() 系列函数，用于操作任务内嵌的消息队列。

37. OS_CFG_TASK_Q_PEND_ABORT_EN

此宏用于使能或禁用终止等待任务内嵌的消息队列功能。当此宏配置为 1 时，会编译函数 OSTaskQPendAbort()，用于终止等待任务内嵌的消息队列。

38. OS_CFG_TASK_REG_TBL_SIZE

此宏用于定义特定于任务的寄存器数量，在任务控制块中有一个拥有 OS_CFG_TASK_REG_TBL_SIZE 个元素的数组，应用程序可以在这个数组中保存一些特定于任务的值，例如错误信息、统计信息等。

39. OS_CFG_TASK_STK_REDZONE_EN

此宏用于使能或禁用 RedZone 栈检查功能，当此宏配置为 1 时，则使能 RedZone 栈检查功能，该功能用于检测栈越界问题。

40. OS_CFG_TASK_STK_REDZONE_DEPTH

此宏用于定义任务栈 RedZone 深度，默认单位为字（由变量类型 CPU_STK 决定）。

41. OS_CFG_TASK_SEM_PEND_ABORT_EN

此宏用于使能或禁用终止等待任务内嵌信号量功能。此宏配置为 1 时，会编译函数 OSTaskSemPendAbort()，用于终止等待任务内嵌信号量。

42. OS_CFG_TASK_SUSPEND_EN

此宏用于使能或禁用任务挂起和恢复功能。此宏配置为 1 时，会编译函数 OSTaskSuspend() 和函数 OSTaskResume()，分别用于挂起和恢复任务运行。

43. OS_CFG_TLS_TBL_SIZE

此宏用于定义本次存储寄存器的数量。

44. OS_CFG_TIME_DLY_HMSM_EN

此宏用于使能或禁用毫秒级任务延时功能。此宏配置为 1 时，会编译函数 OSTimeDlyHMSM()，用于进行毫秒级延时。

45. OS_CFG_TIME_DLY_RESUME_EN

此宏用于使能或禁用终止任务延时功能。此宏配置为 1 时，会编译函数 OSTimeDlyResume()，用于终止任务延时。

46. OS_CFG_TMR_EN

此宏用于使能或禁用软件定时器功能。当此宏配置为 1 时，则使能软件定时器功能。

47. OS_CFG_TMR_DEL_EN

此宏用于使能或禁用删除软件定时器功能。当此宏配置为 1 时，会编译函数 OSTmrDel()，用于删除软件定时器。

48. OS_CFG_TRACE_EN

此宏用于使能或禁用跟踪记录功能。当此宏配置为 1 时，则使能跟踪记录功能（此功能需配合第三方软件使用，如“SEGGER SystemView for uC/OS-III”、“Percepio Tracealyzer for uC/OS-III”等）。

49. OS_CFG_TRACE_API_ENTER_EN

此宏用于使能或禁用记录进入跟踪记录的 API 函数的调用。当此宏配置为 1 时，则使能该功能。

50. OS_CFG_TRACE_API_EXIT_EN

此宏用于使能或禁用记录退出跟踪记录的 API 函数的调用。当此宏配置为 1 时，则使能该功能。

3.2.2 os_cfg_app.h 配置文件

os_cfg_app.h 文件主要用于配置于应用程序相关的 μC/OS-III 内核配置，例如空闲任务、任务统计任务、软件定时器任务的任务参数配置等等，具体包含的配置项，如下表所示：

杂项配置项
OS_CFG_ISR_STK_SIZE
OS_CFG_MSG_POOL_SIZE
OS_CFG_TASK_STK_LIMIT_PCT_EMPTY
空闲任务配置项
OS_CFG_IDLE_TASK_STK_SIZE
任务统计任务配置项
OS_CFG_STAT_TASK_PRIO
OS_CFG_STAT_TASK_RATE_HZ
OS_CFG_STAT_TASK_STK_SIZE
系统时钟节拍配置项
OS_CFG_TICK_RATE_HZ
软件定时器任务配置项
OS_CFG_TMR_TASK_PRIO
OS_CFG_TMR_TASK_STK_SIZE

OS_CFG_TMR_TASK_RATE_HZ

表 3.2.2 os_cfg_app.h 配置文件配置项

1. OS_CFG_ISR_STK_SIZE

此宏用于定义用于异常的 Main Stack 栈的大小，默认单位为字（由变量类型 CPU_STK 决定）。

2. OS_CFG_MSG_POOL_SIZE

此宏用于定义消息队列的消息池大小。

3. OS_CFG_TASK_STK_LIMIT_PCT_EMPTY

此宏用于定义空闲任务、统计任务、软件定时器任务任务栈的剩余最小值，当任务栈剩余量小于这个值，说明任务栈溢出，单位：百分比。

4. OS_CFG_IDLE_TASK_STK_SIZE

此宏用于定义空闲任务任务栈的大小，默认单位为字（由变量类型 CPU_STK 决定）。

5. OS_CFG_STAT_TASK_PRIO

此宏用于定义任务统计任务的任务优先级。

6. OS_CFG_STAT_TASK_RATE_HZ

此宏用于定义任务统计任务的运行频率，单位：赫兹。

7. OS_CFG_STAT_TASK_STK_SIZE

此宏用于定义任务统计任务栈的大小，默认单位为字（由变量类型 CPU_STK 决定）。

8. OS_CFG_TICK_RATE_HZ

此宏用于定义系统时钟节拍的频率，单位：赫兹。

9. OS_CFG_TMR_TASK_PRIO

此宏用于定义软件定时器任务的任务优先级。

10. OS_CFG_TMR_TASK_STK_SIZE

此宏用于定义软件定时器任务的任务栈大小，默认单位为字（由变量类型 CPU_STK 决定）。

11. OS_CFG_TMR_TASK_RATE_HZ

此宏用于定义软件定时器任务的运行频率，单位：赫兹。

3.3 μC/CPU 配置项

μC/CPU 的配置文件为 cpu_cfg.h，在 μC/OS-III 的官方在线文档中，对这个文件做了详细的解释，在线文档的网址为 <https://micrium.atlassian.net/wiki/spaces/cpudoc/overview/>。

3.3.1 cpu_cfg.h 配置文件

cpu_cfg.h 文件主要用于配置 μC/CPU 组件，配置一些于 CPU 硬件相关的配置项，例如时间戳功能时基硬件定时器位数、CPU 是否支持前导零和后导零位数计算、CPU 为大端或小端格式等等，具体包含的配置项，如下表所示：

名称配置项
CPU_CFG_NAME_EN
CPU_CFG_NAME_SIZE
时间戳配置项
CPU_CFG_TS_32_EN
CPU_CFG_TS_64_EN
CPU_CFG_TS_TMR_SIZE

中断关闭耗时测量配置项
CPU_CFG_INT_DIS_MEAS_EN
CPU_CFG_INT_DIS_MEAS_OVRHD_NBR
零计数配置项
CPU_CFG_LEAD_ZEROS_ASM_PRESENT
CPU_CFG_TRAIL_ZEROS_ASM_PRESENT
CPU 端格式配置项
CPU_CFG_ENDIAN_TYPE
Cache 管理配置项
CPU_CFG_CACHE_MGMT_EN
中断管理配置项
CPU_CFG_KA_IPL_BOUNDARY
ARM Cortex-M 配置项
CPU_CFG_NVIC_PRIO_BITS

表 3.3.1 cpu_cfg.h 配置文件配置项

1. CPU_CFG_NAME_EN

此宏用于使能或禁止 CPU 主机名功能。当此宏配置为 DEF_ENABLE 时，则使能 CPU 主机名功能。

2. CPU_CFG_NAME_SIZE

此宏用于定义 CPU 主机名的最大长度。

3. CPU_CFG_TS_32_EN

此宏用于使能或禁用使用 32 位硬件定时器作为时间戳功能的时基定时器。此宏配置为 DEF_ENABLE 时，使能使用 32 位硬件定时器作为时间戳功能的时基定时器。

4. CPU_CFG_TS_64_EN

此宏用于使能或禁用使用 64 位硬件定时器作为时间戳功能的时基定时器。此宏配置为 DEF_ENABLE 时，使能使用 64 位硬件定时器作为时间戳功能的时基定时器。

5. CPU_CFG_TS_TMR_SIZE

此宏用于定义时间戳的实际使用位数，可定义为 CPU_WORD_SIZE_08（实际使用 8 位时间戳）、CPU_WORD_SIZE_16（实际使用 16 位时间戳）、CPU_WORD_SIZE_32（实际使用 32 位时间戳）、CPU_WORD_SIZE_64（实际使用 64 位时间戳）。

6. CPU_CFG_INT_DIS_MEAS_EN

此宏用于使能或禁用使用时间戳测量进入临界区和退出临界区期间的耗时时间。此宏配置为 DEF_ENABLE 时，使能使用时间戳测量进入临界区和退出临界区期间的耗时时间。

7. CPU_CFG_INT_DIS_MEAS_OVRHD_NBR

此宏用于定义执行多少次临界区进入和退出函数，以计算出平均的进入临界区和退出临界区期间的额外耗时时间。

8. CPU_CFG_LEAD_ZEROS_ASM_PRESENT

此宏用于指示 CPU 具有硬件计算前导零的指令，如 CLZ 指令。在 μC/OS-III 中，函数 CPU_CntLeadZeros() 用于计算前导零，如果定义了此宏，则函数 CPU_CntLeadZeros() 定义在文件 cpu_a.asm 中使用 CLZ 指令实现；反之，则函数 CPU_CntLeadZeros() 定义在文件 cpu_core.c 文件中使用软件算法实现。

9. CPU_CFG_TRAIL_ZEROS_ASM_PRESENT

此宏用于指示 CPU 具有硬件计算后导零的指令，如 RBIT 指令和 CLZ 指令配合使用。在

μC/OS-III 中，函数 CPU_CntTrailZeros() 用于计算后导零，如果定义了此宏，则函数 CPU_CntTrailZeros() 定义在 cpu_a.asm 文件中使用 RBIT 指令配置 CLZ 指令实现；反之，则函数 CPU_CntTrailZeros() 定义在 cpu_core.c 文件中使用软件算法实现。

10. CPU_CFG_ENDIAN_TYPE

此宏用于指示 CPU 的端格式，当此宏配置为 CPU_ENDIAN_TYPE_BIG 时，表明 CPU 为大端格式，当此宏配置为 CPU_ENDIAN_TYPE_LITTLE 时，表明 CPU 为小端格式。

11. CPU_CFG_CACHE_MGMT_EN

此宏用于使能或禁用 μC/OS-III 的 Cache 管理功能。当此宏配置为 DEF_ENABLE 时，则使能 μC/OS-III 的 Cache 管理功能。

12. CPU_CFG_KA_IPL_BOUNDARY

此宏用于定义受 μC/OS-III 管理的最高中断优先等级，中断优先级低于此宏定义值（中断优先级数值大于此宏定义值）的中断受 μC/OS-III 管理。

13. CPU_CFG_NVIC_PRIO_BITS

此宏用于定义中断优先级配置寄存器的实际使用位数。

3.4 μC/LIB 配置项

μC/LIB 的配置文件为 lib_cfg.h，在 μC/OS-III 的官方在线文档中，对这个文件做了详细的解释，在线文档的网址为 <https://micrium.atlassian.net/wiki/spaces/libdoc138/pages/175778/>。

3.4.1 lib_cfg.h 配置文件

lib_cfg.h 文件主要用于配置 μC/LIB 组件，具体包含的配置项，如下表所示：

参数检查配置项
LIB_MEM_CFG_ARG_CHK_EXT_EN
内存库组件优化配置项
LIB_MEM_CFG_OPTIMIZE_ASM_EN
内存申请配置项
LIB_MEM_CFG_DBG_INFO_EN
LIB_MEM_CFG_HEAP_SIZE
LIB_MEM_CFG_HEAP_PADDING_ALIGN
LIB_MEM_CFG_HEAP_BASE_ADDR
浮点数转字符串配置项
LIB_STR_CFG_FP_EN
LIB_STR_CFG_FP_MAX_NBR_DIG_SIG

表 3.4.1 lib_cfg.h 配置文件配置项

1. LIB_MEM_CFG_ARG_CHK_EXT_EN

此宏用于使能或禁用检查内存库（文件 lib_mem.c）中函数的传入参数是否合法。当此宏配置为 DEF_ENABLE 时，则使能检查内存库中函数的传入参数是否合法。

2. LIB_MEM_CFG_OPTIMIZE_ASM_EN

此宏用于使能或禁用使用内存库中优化的内存相关操作函数。当此宏配置为 DEF_ENABLE 时，则使能使用内存库中优化的内存相关操作函数，例如使用内存库中的函数 Mem_Copy() 代替标准 C 库中的函数 memcpy()。

3. LIB_MEM_CFG_DBG_INFO_EN

此宏用于使能或禁用为申请的每一个内存块分配一个用于跟踪调试的名称。当此宏配置为 DEF_ENABLE 时，则使能位申请的每一个内存块分配一个用于跟踪调试的名称。

4. LIB_MEM_CFG_HEAP_SIZE

此宏用于定义用于内存库管理的内存堆的大小，单位：字节。

5. LIB_MEM_CFG_HEAP_PADDING_ALIGN

此宏用于定义使用函数 Mem_HeapAlloc()申请内存时，所申请的内存大小需按多少字节向上对齐。

6. LIB_MEM_CFG_HEAP_BASE_ADDR

此宏用于定义用于内存库管理的内存堆的起始地址。当没有定义这个宏时，用于内存库管理的内存堆将由编译器自动分配，当定义了这个宏后，用于内存库管理的内存堆将被分配到该宏定义的地址上。

7. LIB_STR_CFG_FP_EN

此宏用于使能或禁用浮点数转字符串功能。当此宏配置为 DEF_ENABLE 时，将编译函数 Str_FmtNbr_32()，用于浮点数转字符串。

8. LIB_STR_CFG_FP_MAX_NBR_DIG_SIG

此宏用于配置浮点数转字符串函数 Str_FmtNbr_32()的转换精度。

第四章 μC/OS-III 中断管理

中断是学习 μC/OS-III 的过程中一个重要的部分，任务切换、系统时钟节拍等等，都是利用中断来完成的，并且，既然是 RTOS，那么对中断的响应也应该是十分迅速的。μC/OS-III 有一套中断管理的方法，并且在用户使用 μC/OS-III 的中断管理前，仅需进行少量的配置，使用起来非常方便。本章会结合 ARM Cortex-M 的 NVIC 来讲解 STM32 平台下 μC/OS-III 的中断管理。

本章分为如下几部分：

- 4.1 ARM Cortex-M 中断
- 4.2 μC/OS-III 中断配置项
- 4.3 μC/OS-III 中断管理详解
- 4.4 μC/OS-III 临界区
- 4.5 μC/OS-III 中断嵌套计数器

4.1 ARM Cortex-M 中断

4.1.1 ARM Cortex-M 中断简介

中断是 CPU 的一种常见特性，中断一般由硬件产生，当中断发生后，会中断 CPU 当前正在执行的程序而跳转到中断对应的服务程序中去执行，ARM Cortex-M 内核的 MCU 具有一个用于中断管理的嵌套向量中断控制器（NVIC，全称：Nested Vectored Interrupt Controller）。

ARM Cortex-M 的 NVIC 最大可支持 256 个中断源，其中包括 16 个系统中断和 240 个外部中断。然而芯片厂商一般情况下都用不完这些资源，以正点原子的战舰开发板为例，所使用的 STM32F103ZET6 芯片就只用到了 10 个系统中断和 60 个外部中断。

4.1.2 ARM Cortex-M 中断优先级管理

ARM Cortex-M 使用 NVIC 对不同优先级的中断进行管理，首先看一下 NVIC 在 CMSIS 中的结构体定义，如下所示：

```
typedef struct
{
    __IOM  uint32_t  ISER[8U]           /* 中断使能寄存器 */
        uint32_t RESERVED0[24U];
    __IOM  uint32_t  ICER[8U];          /* 中断除能寄存器 */
        uint32_t RESERVED1[24U];
    __IOM  uint32_t  ISPR[8U];          /* 中断使能挂起寄存器 */
        uint32_t RESERVED2[24U];
    __IOM  uint32_t  ICPR[8U];          /* 中断除能挂起寄存器 */
        uint32_t RESERVED3[24U];
    __IOM  uint32_t  IABR[8U];          /* 中断有效位寄存器 */
        uint32_t RESERVED4[56U];
    __IOM  uint8_t   IP[240U];          /* 中断优先级寄存器 */
        uint32_t RESERVED5[644U];
    __OM   uint32_t  STIR;              /* 软件触发中断寄存器 */
}
```

```
} NVIC_Type;
```

在 NVIC 的相关结构体中，成员变量 IP 用于配置外部中断的优先级，其定义如下所示：

```
__IOM uint8_t IP[240U]; /* 中断优先级寄存器 */
```

可以看到成员变量 IP 是一个 uint8_t 类型的数组，数组一共有 240 个元素，数组中每一个 8bit 的元组就用来配置一个对应的外部中断的优先级。

综上可知，ARM Cortex-M 使用了 8bit 位宽的寄存器来配置中断的优先级，这个寄存器就是中断优先级配置寄存器，因此中断优先级的配置范围就应该是 0~255。但是芯片厂商一般用不完这些资源，对于 STM32，只用到了中断优先级配置寄存器的高四位[7:4]，低四位[3:0]取零处理，因此 STM32 提供了最大 $2^4=16$ 级（0~15）的中断优先级等级，如下图所示：

Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
用于表达优先级				没有实现，读回零			

图 4.1.1 中断优先级配置寄存器

中断优先级配置寄存器的值与对应的优先等级成反比，即中断优先级配置寄存器的值越小，中断的优先等级就越高。

STM32 的中断优先级可分为抢占优先级和子优先级，抢占优先级和子优先级的区别如下：

抢占优先级：抢占优先级高的中断可以打断正在执行但抢占优先级低的中断，即中断嵌套。

子优先级：当抢占优先级相同时，子优先级高的中断不能打断正在执行但子优先级低的中断，即子优先级不支持中断嵌套。

STM32 中每个中断的优先级由抢占优先级和子优先级共同组成，使用中断优先级配置寄存器的高四位来配置抢占优先级个子优先级，那么中断优先级配置寄存器的高四位是如何分配设置抢占优先级和子优先级的呢？一共由五种分配方式，对应着中断优先级分组的五中分组情况，优先级分组的五中分组情况在 HAL 库中进行了定义，如下所示：

```
#define NVIC_PRIORITYGROUP_0 0x00000007U /* 优先级分组 0 */
#define NVIC_PRIORITYGROUP_1 0x00000006U /* 优先级分组 1 */
#define NVIC_PRIORITYGROUP_2 0x00000005U /* 优先级分组 2 */
#define NVIC_PRIORITYGROUP_3 0x00000004U /* 优先级分组 3 */
#define NVIC_PRIORITYGROUP_4 0x00000003U /* 优先级分组 4 */
```

优先级分组对应的抢占优先级和子优先级分配方式如下表所示：

优先级分组	抢占优先级	子优先级	优先级配置寄存器高 4 位
NVIC_PriorityGroup_0	0 级抢占优先级	0-15 级子优先级	0bit 用于抢占优先级 4bit 用于子优先级
NVIC_PriorityGroup_1	0-1 级抢占优先级	0-7 级子优先级	1bit 用于抢占优先级 3bit 用于子优先级
NVIC_PriorityGroup_2	0-3 级抢占优先级	0-3 级子优先级	2bit 用于抢占优先级 2bit 用于子优先级
NVIC_PriorityGroup_3	0-7 级抢占优先级	0-1 级子优先级	3bit 用于抢占优先级 1bit 用于子优先级
NVIC_PriorityGroup_4	0-15 级抢占优先级	0 级子优先级	4bit 用于抢占优先级 0bit 用于子优先级

表 4.1.1 优先级分组

STM32 的中断即要设置中断优先级分组又要设置抢占优先级和子优先级，看起来十分复杂。其实为了更好的使用 μC/OS-III 的中断管理，笔者强烈建议在 STM32 上使用 μC/OS-III 时，使用中断优先级分组 4（NVIC_PriorityGroup_4）即优先级配置寄存器的高四位全部用于抢占优先级，不使用子优先级，那么这么一来只需要设置中断的抢占优先级即可，本教程配套的例程源码也全部将中断优先级分组设置为中断优先级分组 4（NVIC_PriorityGroup_4），如下所示：

```
/* Set Interrupt Group Priority */
HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4);
```

4.1.3 三个中断优先级配置寄存器

除了外部中断，系统中断有独立的中断优先级配置寄存器，分别为 SHPR1、SHPR2、SHPR3，下面就分别来看一下这三个寄存器的作用。

1. SHPR1 寄存器

SHPR1 寄存器的地址为 0xE000ED18，用于配置 MemManage、BusFault、UsageFault 的中断优先级，各比特位的功能描述如下表所示：

比特位	名称	功能描述
[31:24]	PRI_7	保留
[23:16]	PRI_6	UsageFault 中断优先级
[15:8]	PRI_5	BusFault 中断优先级
[7:0]	PRI_4	MemManage 中断优先级

表 4.1.2 SHPR1 寄存器

2. SHPR2 寄存器

SHPR2 寄存器的地址为 0xE000ED1C，用于配置 SVCALL 的中断优先级，各比特位的功能描述如下表所示：

比特位	名称	功能描述
[31:24]	PRI_11	SVCALL 中断优先级
[23:0]	-	保留

表 4.1.3 SHPR2 寄存器

3. SHPR3 寄存器

SHPR3 寄存器的地址为 0xE000ED20，用于配置 PendSV、SysTick 的中断优先级，各比特位的功能描述如下表所示：

比特位	名称	功能描述
[31:24]	PRI_15	SysTick 中断优先级
[23:16]	PRI_14	PendSV 中断优先级
[15:0]	-	保留

表 4.1.4 SHPR3 寄存器

μC/OS-III 在配置 PendSV 和 SysTick 中断优先级的时候，就使用到了 SHPR3 寄存器，因此请读者多留意该寄存器。

4.1.4 三个中断屏蔽寄存器

ARM Cortex-M 有三个用于屏蔽中断的寄存器，分别为 PRIMASK、FAULTMASK 和 BASEPRI，下面就分别来看一下这三个寄存器的作用。

1. PRIMASK 寄存器

作用：PRIMASK 寄存器有 32bit，但只有 bit0 有效，是可读可写的，将 PRIMASK 寄存器设置为 1 用于屏蔽除 NMI 和 HardFault 外的所有异常和中断，将 PRIMASK 寄存器清 0 用于使能中断。

用法一：

```
CPSIE I          /* 清除 PRIMASK（使能中断） */

CPSID I          /* 设置 PRIMASK（屏蔽中断） */
```

用法二：

```
MRS R0, PRIMASK /* 读取 PRIMASK 值 */

MOV R0, #0
MSR PRIMASK, R0  /* 清除 PRIMASK（使能中断） */

MOV R0, #1
MSR PRIMASK, R0  /* 设置 PRIMASK（屏蔽中断） */
```

用法三：

```
__get_PRIMASK(); /* 读取 PRIMASK 值 */

__set_PRIMASK(0U); /* 清除 PRIMASK（使能中断） */

__set_PRIMASK(1U); /* 设置 PRIMASK（屏蔽中断） */
```

2. FAULTMASK 寄存器

作用：FAULTMASK 寄存器有 32bit，但只有 bit0 有效，也是可读可写的，将 FAULTMASK 寄存器设置为 1 用于屏蔽除 NMI 外的所有异常和中断，将 FAULTMASK 寄存器清零用于使能中断。

用法一：

```
CPSIE F          /* 清除 FAULTMASK（使能中断） */

CPSID F          /* 设置 FAULTMASK（屏蔽中断） */
```

用法二：

```
MRS R0, FAULTMASK /* 读取 FAULTMASK 值 */

MOV R0, #0
MSR FAULTMASK, R0 /* 清除 FAULTMASK（使能中断） */

MOV R0, #1
MSR FAULTMASK, R0 /* 设置 FAULTMASK（屏蔽中断） */
```

用法三：

```
__get_FAULTMASK(); /* 读取 FAULTMASK 值 */

__set_FAULTMASK(0U); /* 清除 FAULTMASK（使能中断） */

__set_FAULTMASK(1U); /* 设置 FAULTMASK（屏蔽中断） */
```

3. BASEPRI 寄存器

作用：BASEPRI 有 32bit，但只有低 8 位[7:0]有效，也是可读可写的。BASEPRI 寄存器比起 PRIMASK 和 FAULTMASK 寄存器直接屏蔽掉大部分中断的方式，BASEPRI 寄存器的功能显得更加细腻，BASEPRI 用于设置一个中断屏蔽的阈值，设置好 BASEPRI 后，中断优先级低于 BASEPRI 的中断就都会被屏蔽掉，μC/OS-III 就是使用 BASEPRI 寄存器来管理受 μC/OS-III 管理的中断的，而不受 μC/OS-III 管理的中断，则不受 μC/OS-III 的影响。

用法一：

```
MRS R0, BASEPRI          /* 读取 BASEPRI 值 */

MOV R0, #0

MSR BASEPRI, R0          /* 清除 BASEMASK（使能中断） */

MOV R0, #0x60            /* 举例 */

MSR BASEPRI, R0          /* 设置 BASEMASK（屏蔽优先级低于 0x60 的中断） */
```

用法二：

```
__get_BASEPRI();          /* 读取 BASEPRI 值 */

__set_BASEPRI(0);         /* 清除 BASEPRI（使能中断） */

__set_BASEPRI(0x60);      /* 设置 BASEPRI（屏蔽优先级小于 0x60 的中断） */
```

4.1.5 中断控制状态寄存器

中断控制状态寄存器（ICSR）的地址为 0xE000ED04，用于设置和清除异常的挂起状态，以及获取当前系统正在执行的异常编号，各比特位的功能描述如下表所示：

比特位	名称	功能描述
[31]	NMIPENDSET	设置 NMI 挂起
[30:29]	-	保留
[28]	PENDSVSET	设置 PendSV 挂起
[27]	PENDSVCLR	清除 PendSV 挂起
[26]	PENDSTSET	设置 SysTick 挂起
[25]	PENDSTCLR	清除 SysTick 挂起
[24]	-	保留
[23]	Reserved for Debug use	调试使用
[22]	ISRPENDING	设置外部中断挂起
[21]	-	保留
[20:12]	VECTPENDING	挂起的异常编号
[11]	RETTOBASE	指示是否存在抢占的异常
[10:9]	-	保留
[8:0]	VECTACTIVE	正在执行的异常编号

表 4.1.5 中断控制状态寄存器

这个寄存器主要关注 VECTACTIVE 段[8:0]，通过读取 VECTACTIVE 段就能够判断当前执行的代码是否在中断中。

4.2 μC/OS-III 中断配置项

在第 3.3 小节《μC/CPU 配置项》中提到了 μC/OS-III 中关于中断的两个配置项，并对这两个配置项做了相应的描述，本小节主要结合第 4.1 小节《ARM Cortex-M 中断》，为读者分析如何配置这两个 μC/OS-III 与中断相关的配置项。

1. CPU_CFG_NVIC_PRIO_BITS

此宏用于定义中断优先级配置寄存器的实际使用位数，中断优先级配置寄存器实际使用到多少比特位，这个宏就应该定义成多少，因为 STM32 的优先级配置寄存器都只使用到了高四位，因此对于 STM32 而言，这个宏应该配置为 4。

2. CPU_CFG_KA_IPL_BOUNDARY

此宏用于定义受 μC/OS-III 管理的最高中断优先等级，中断优先级低于此宏定义值（中断优先级数值大于此宏定义值）的中断受 μC/OS-III 管理。此宏定义的值可以根据用户的实际应用场景来决定，本教程的配套例程源码全部将此宏定义配置为 4，即中断优先级为 4~15 的中断由 μC/OS-III 管理，而中断优先级为 0~3 的中断不由 μC/OS-III 管理，如下图所示：

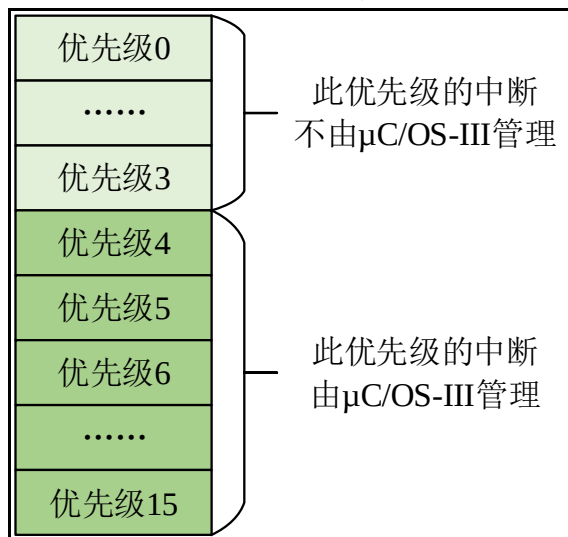


图 4.2.1 中断优先级配置示例图

4.3 μC/OS-III 中断管理详解

在了解了 ARM Cortex-M 中断和 μC/OS-III 中断配置项的相关内容后，接下来本小节将通过分析 μC/OS-III 源代码的方式来讲解 μC/OS-III 的如何管理中断的。

4.3.1 PendSV 和 SysTick 中断优先级配置

在 4.1.3 小节《三个中断优先级配置寄存器》中提到，μC/OS-III 使用 SHPR3 寄存器配置 PendSV 和 SysTick 的中断优先级，那么 μC/OS-III 具体是如何配置这两个中断的优先级的呢？

1. PendSV 中断优先级配置

PendSV 主要用于任务切换，因此在 μC/OS-III 内核开始进行多任务处理前，也就是在 μC/OS-III 内核启动之前，就需要配置好 PendSV。在文件 os_cpu_a.asm 中有标号（汇编中的标号，类似于 C 语言中的函数名）OSSStartHighRdy，OSSStartHighRdy 用于开启系统中第一个任务，在函数 OSSStart() 中被调用，具体的 μC/OS-III 启动流程会在下文中分析，这里只关注 PendSV 的配置部分。

首先看一下 OSSStartHighRdy 中的部分代码，如下所示：


```

; 关中断
CPSID    I
; 配置 PendSV 为最低中断优先级
MOV32    R0, NVIC_SYSPRI14
MOV32    R1, NVIC_PENDSV_PRI
STRB     R1, [R0]
    
```

其中 NVIC_SYSPRI14 和 NVIC_PENDSV_PRI 的定义如下所示：

```

NVIC_SYSPRI14    EQU    0xE000ED22
NVIC_PENDSV_PRI  EQU    0xFF
    
```

前面说配置 PendSV 中断优先级使用到的寄存器是 SHPR3 寄存器，SHPR3 寄存器的地址为 0xE000ED20，而 NVIC_SYSPRI14 的值为 0xE000ED22，就正好是 SHPR3[23:16]的起始地址，也就是 PendSV 中断优先级配置的起始地址。

结合上面的代码，可以看出，μC/OS-III 将 PendSV 的中断优先级配置为 0xFF，也就是最低优先级。

2. SysTick 中断优先级配置

SysTick 主要用于为 μC/OS-III 内核提供时钟节拍，在调用函数 OSStart()后，需要调用函数 OS_CPU_SysTickInit()对 SysTick 进行配置，在对 SysTick 的配置过程中，就包括对 SysTick 中断优先级的配置，函数 OS_CPU_SysTickInit()中对 SysTick 中断优先级配置的代码如下所示：

```

void OS_CPU_SysTickInit (CPU_INT32U cnts)
{
    CPU_INT32U prio;
    CPU_INT32U basepri;

    /* 计算受 μC/OS-III 管理的最高中断优先级在中断优先级配置寄存器中表示的值 */
    basepri=(CPU_INT32U) (CPU_CFG_KA_IPL_BOUNDARY<<(8u-CPU_CFG_NVIC_PRIO_BITS));
    /* 获取 SHPR3 寄存器的值 */
    prio = CPU_REG_SCB_SHPRI3;
    /* 将 SHPR3[31:24]清零 */
    prio &= 0x00FFFFFFu;
    /* 将 SHPR3[31:24]设置为受 μC/OS-III 管理的最高优先级 */
    prio |= (basepri << 24u);
    /* 写入 SHPR3 寄存器 */
    CPU_REG_SCB_SHPRI3 = prio;

    /* 仅列出该函数中用于配置 SysTick 中断优先级的代码
     * 省略其余代码
     */
}
    
```

以上函数 OS_CPU_SysTickInit()的代码仅列出了用于配置 SysTick 中断优先级的代码，其余的代码省略。

从上面的代码中可以看出，μC/OS-III 就是用过设置 SHPR3 寄存器，来配置 SysTick 的中断优先级，SysTick 的中断优先级被配置为受 μC/OS-III 管理的最高优先级，在 4.2 小节《μC/OS-III 中断配置项》中说明本教程配套的实验例程将宏 CPU_CFG_KA_IPL_BOUNDARY 配置为 4，

即表示受 μC/OS-III 管理的最高中断优先级为 4，因此在通过函数 OS_CPU_SysTickInit() 配置后的 SysTick 的中断优先级就为 4。

从上面的代码中就可以看出，函数 OS_CPU_SysTickInit() 在计算 SysTick 中断优先级的时候，就使用到了 4.2 小节《μC/OS-III 中断配置项》中提到的两个 μC/OS-III 中与中断相关的配置项。这两个配置项不仅仅用于计算 SysTick 的中断优先级，还用于在 μC/OS-III 开关中断时，设置 BASEPRI 寄存器的值。

4.3.2 μC/OS-III 开关中断

在文件 cpu_a.asm 中提供了两组同于开关中断的标号，如下所示：

```
CPU_IntDis      ; 关闭所有中断
CPU_IntEn       ; 打开所有中断

CPU_SR_Save     ; 保存中断状态，并关闭受 μC/OS-III 管理的中断
CPU_SR_Restore  ; 恢复中断状态
```

接下来分别来看一下这四个标号后的内容。

1. CPU_IntDis

标号 CPU_IntDis 后的内容如下所示：

```
CPU_IntDis
CPSID    I      ; 关闭中断
BX       LR     ; 返回
```

从上面的代码中可以看到，标号 CPU_IntDis 就是操作了 PRIMASK 寄存器，“简单粗暴”地屏蔽所有的中断。

2. CPU_IntEn

标号 CPU_IntEn 后的内容如下所示：

```
CPU_IntEn
CPSIE    I      ; 打开中断
BX       LR     ; 返回
```

从上面的代码中可以看出，标号 CPU_IntEn 同样的是操作了 PRIMASK 寄存器，“简单粗暴”地取消屏蔽所有的中断。

3. CPU_SR_Save

标号 CPU_SR_Save 后的内容如下所示：

```
CPU_SR_Save
CPSID    I      ; 关闭中断
PUSH     {R1}    ; 入栈 R1
MRS      R1, BASEPRI ; R1 = BASEPRI
MSR      BASEPRI, R0 ; BASEPRI = R0
DSB                      ; 数据同步
ISB                      ; 指令同步
MOV      R0, R1    ; R0 = R1
POP      {R1}    ; 出栈 R1
CPSIE    I      ; 打开中断
BX       LR     ; 返回
```

从上面的代码中可以看到，标号 CPU_SR_Save 后的内容可简单分为如下三个步骤：

1. 将 BASEPRI 寄存器的值读入到 R1 寄存器中，保存 BASEPRI 寄存器的原始值。
2. 将 R0 寄存器的值写入 BASEPRI 寄存器，R0 寄存器中保存的值就是程序在调用标号 CPU_SR_Save 时，传入的参数。
3. 将 R1 寄存器中的值赋给 R0 寄存器，在程序中调用标号 CPU_SR_Save 时，CPU_SR_Save 的返回值就保存在 R0 寄存器中。

因此标号 CPU_SR_Save 所做的事情就是，根据入参设置 BASEPRI 寄存器，并返回在设置 BASEPRI 寄存器前 BASEPRI 寄存器的值，以获取中断状态并关闭受 μC/OS-III 管理的中断。

4. CPU_SR_Restore

标号 CPU_SR_Restore 后的内容如下所示：

```
CPU_SR_Restore
CPSID    I           ; 关闭中断
MSR      BASEPRI, R0 ; BASEPRI = R0
DSB                      ; 数据同步
ISB                      ; 指令同步
CPSIE    I           ; 打开中断
BX       LR           ; 返回
```

从上面的代码中可以看出，标号 CPU_SR_Restore 就是用来给 BASEPRI 寄存器赋指定值的。配个标号 CPU_SR_Save 使用就能够在需要进行中断保护（临界区，后面章节中会介绍）的代码前后屏蔽并恢复受 μC/OS-III 管理的中断，以达到关键代码不受中断影响的目的。

4.4 μC/OS-III 临界区

临界区指的是代码中的一些关键部分，临界区中代码的运行要求不能被打断，而中断或任务切换都有可能打断临界区中正在运行的关键代码，那么就必须要通过屏蔽中断来保护临界区中代码的执行。

μC/OS-III 提供了两个用于在进入临界区前屏蔽中断和在退出临界区后恢复中断的函数，这两个函数为函数 CPU_CRITICAL_ENTER() 和函数 CPU_CRITICAL_EXIT()，接下来分别来看一下这两个函数。

4.4.1 函数 CPU_CRITICAL_ENTER()

函数 CPU_CRITICAL_ENTER() 实际上是一个宏，定义在 cpu.h 文件中，其代码如下所示：

```
#define CPU_CRITICAL_ENTER() \
do \
{ \
    CPU_INT_DIS(); \
} while (0)
```

从上面的代码中可以看出，宏 CPU_CRITICAL_ENTER() 实际上是调用了函数 CPU_INT_DIS()。函数 CPU_INT_DIS() 实际上也是一个宏，定义在 cpu.h 文件中，具体的代码如下所示：

```
#define CPU_INT_DIS() \
do \
{ \
    cpu_sr = CPU_SR_Save( CPU_CFG_KA_IPL_BOUNDARY << \
                          (8u - CPU_CFG_NVIC_PRIO_BITS)); \
}
```

```
} while (0)
```

结合上面的代码，可以看出，函数 CPU_CRITICAL_ENTER() 会调用函数 CPU_SR_Save() 设置 BASEPRI 寄存器，并将设置前 BASEPRI 寄存器的值保存在变量 cpu_sr 中。

使用函数 CPU_CRITICAL_ENTER() 进入临界区时，会将 BASEPRI 寄存器设置为 CPU_CFG_KA_IPL_BOUNDARY << (8u - CPU_CFG_NVIC_PRIO_BITS)，这个的结果就是屏蔽受 μC/OS-III 管理的中断优先级的中断，而不受 μC/OS-III 管理的中断，是不受影响的。

4.4.2 函数 CPU_CRITICAL_EXIT()

函数 CPU_CRITICAL_EXIT() 实际上是一个宏，定义在 cpu.h 文件中，具体的代码如下所示：

```
#define CPU_CRITICAL_EXIT() \
do \
{ \
    CPU_INT_EN(); \
} while (0)
```

从上面的代码中可以看出，宏 CPU_CRITICAL_EXIT() 实际上是调用了函数 CPU_INT_EN()。函数 CPU_INT_EN() 实际上也是一个宏，定义在 cpu.h 文件中，具体的代码如下所示：

```
#define CPU_INT_EN() \
do \
{ \
    CPU_SR_Restore(cpu_sr); \
} while (0)
```

结合上面的代码，可以看出，函数 CPU_CRITICAL_EXIT() 会调用函数 CPU_SR_Restore() 并传入在 CPU_CRITICAL_ENTER() 中获取到的变量 cpu_sr，将 BASEPRI 恢复到调用函数 CPU_CRITICAL_ENTER() 之前的状态。可见函数 CPU_CRITICAL_ENTER() 和函数 CPU_CRITICAL_EXIT() 是需要配合使用的。

4.4.3 函数 CPU_SR_ALLOC()

在前面讲解函数 CPU_CRITICAL_ENTER() 和函数 CPU_CRITICAL_EXIT() 时，可以看到变量 cpu_sr 作为中间变量来保存 BASEPRI 寄存器的值，但是并没有看到这个变量的定义，这是因为这个变量是需要用户在使用函数 CPU_CRITICAL_ENTER() 和函数 CPU_CRITICAL_EXIT() 之前手动定义的，并且必须使用 μC/OS-III 提供了宏来定义变量 cpu_sr，这个宏就是 CPU_SR_ALLOC()，这个宏定义在 cpu.h 文件中，具体的代码如下所示：

```
#define CPU_SR_ALLOC() \
CPU_SR cpu_sr = (CPU_SR) 0
```

从上面的代码中可以看到，宏 CPU_SR_ALLOC() 实际上就是定义了一个变量类型为 CPU_SR 的变量 cpu_sr，并将变量 cpu_sr 的值初始化为 0。

4.4.4 μC/OS-III 临界区代码使用示例

在 μC/OS-III 源码中使用大量的临界区代码，用户在编写应用程序的时候，如果有需要，也可以在应用程序中定义临界区代码，只需要在临界区代码前后加上函数 CPU_CRITICAL_ENTER() 和函数 CPU_CRITICAL_EXIT()，当然也不要忘了使用函数 CPU_SR_ALLOC() 来定义 cpu_sr 变量，下面是一个使用示例，如下所示：

```
void Function(void)
```

```
{
    uint32_t a;
    uint32_t b;
    CPU_SR_ALLOC();          /* 必须定义在所有局部变量之后 */

    /* 非临界区代码 */

    CPU_CRITICAL_ENTER();    /* 进入临界区 */

    /* 临界区代码 */

    CPU_CRITICAL_EXIT();     /* 退出临界区 */

    /* 非临界区代码 */
}
```

需要注意的是，CPU_SR_ALLOC()必须在所有局部变量定义后再定义。

4.5 μC/OS-III 中断嵌套计数器

在 μC/OS-III 中，会通过全局变量 OSIntNestingCtr 记录中断嵌套的次数，方便 μC/OS-III 判断当前是否处于中断状态。当全局变量 OSIntNestingCtr 大于 0 的时候，就表示当前处于中断状态，当全局变量 OSIntNestingCtr 等于 0 的时候，就表示当前不是处于中断状态。

全局变量 OSIntNestingCtr 是在中断服务函数中更新的，因此，μC/OS-III 提供了两个分别用于中断服务函数前后的函数，分别为 OSIntEnter()和函数 OSIntExit()。其中函数 OSIntEnter()只是简单地更新了全局变量 OSIntNestingCtr 的值，而函数 OSIntExit()除了更新全局变量 OSIntNestingCtr 的值，同时还会根据需要进行任务切换。

下面以 μC/OS-III 提供了 SysTick 的中断服务函数为例，展示函数 OSIntEnter()和函数 OSIntExit()的使用：

```
void OS_CPU_SysTickHandler(void)
{
    CPU_SR_ALLOC();

    CPU_CRITICAL_ENTER();
    /* 进入中断后，先调用函数 OSIntEnter() */
    OSIntEnter();
    CPU_CRITICAL_EXIT();

    /* 中断服务函数的内容 */
    OSTimeTick();

    /* 中断返回前，调用函数 OSIntExit() */
    OSIntExit();
}
```

第五章 μC/OS-III 任务基础知识

任务和任务管理是 RTOS 的核心，μC/OS-III 也不例外，并且，绝大多数项目使用 RTOS 的目的就是为了使用 RTOS 的多任务管理能力。对于初学者，特别是没有 RTOS 基础的读者，了解 μC/OS-III 的任务管理机制，是非常有必要的。为了帮助读者更好地理解 μC/OS-III 的任务管理机制，本章就先介绍 μC/OS-III 任务的一些基础知识。

本章分为如下几部分：

- 5.1 单任务系统和多任务系统
- 5.2 μC/OS-III 任务状态
- 5.3 μC/OS-III 任务优先级
- 5.4 μC/OS-III 任务调度方式
- 5.5 μC/OS-III 任务控制块
- 5.6 μC/OS-III 任务栈

5.1 单任务系统和多任务系统

5.1.1 单任务系统

单任务系统的编程方式，即裸机的编程方式，这种编程方式的框架一般都是在 main() 函数中使用一个大循环，在循环中顺序地调用相应地函数以处理相应的事务，这个大循环的部分可视为应用程序的后台，而应用程序的前台，则是各种中断的中断服务函数。因此单任务系统也叫做前后台系统，前后台系统的运行示意图，如下图所示：

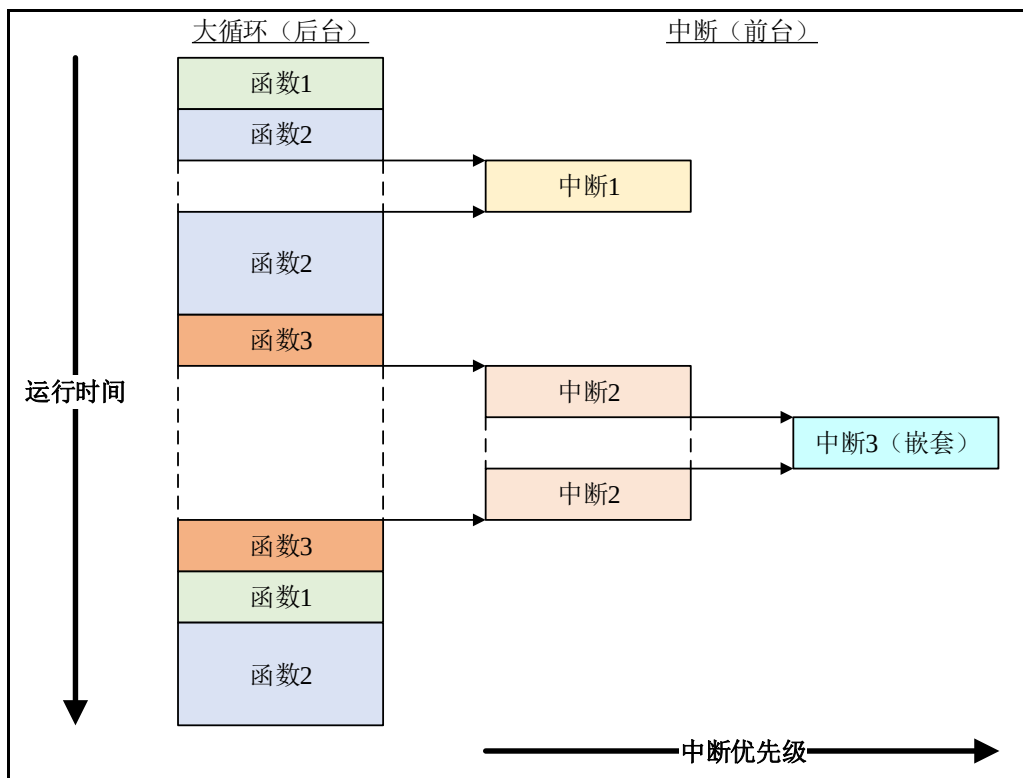


图 5.1.1 前后台系统运行示意图

从上图中可以看出，前后台系统的实时性很差，因为大循环中函数处理的事务没有优先级之分，因此在大循环中函数处理的事务需要顺序地被执行，不论待处理的事务紧急程度有多高，没轮到只能等着，虽然中断能后处理一些紧急的事务，但是在一些大型的嵌入式应用中，这样的单任务系统就会显得力不从心。

5.1.2 多任务系统

多任务系统在处理事务的实时性上比单任务系统要好得多，从宏观上来看，多任务系统的多个任务是可以“同时”运行的，因此紧急的事务就可以无需等待 CPU 处理完其他事务，在被处理。

要注意的是，上面提到的多任务系统中多个任务可以“同时”运行，是从宏观的角度而言的，对于单核的 CPU 而言，CPU 在同一时刻只能处理一个任务，但是多任务系统的任务调度器会根据相关的任务调度算法，将 CPU 的使用权分配给任务，在任务获取到 CPU 使用权之后的极短时间（宏观角度）后，任务调度器又会将 CPU 的使用权分配给其他任务，如此往复，在宏观角度看来，就像是多个任务在同时运行了一样。

多任务系统的运行示意图，如下图所示：

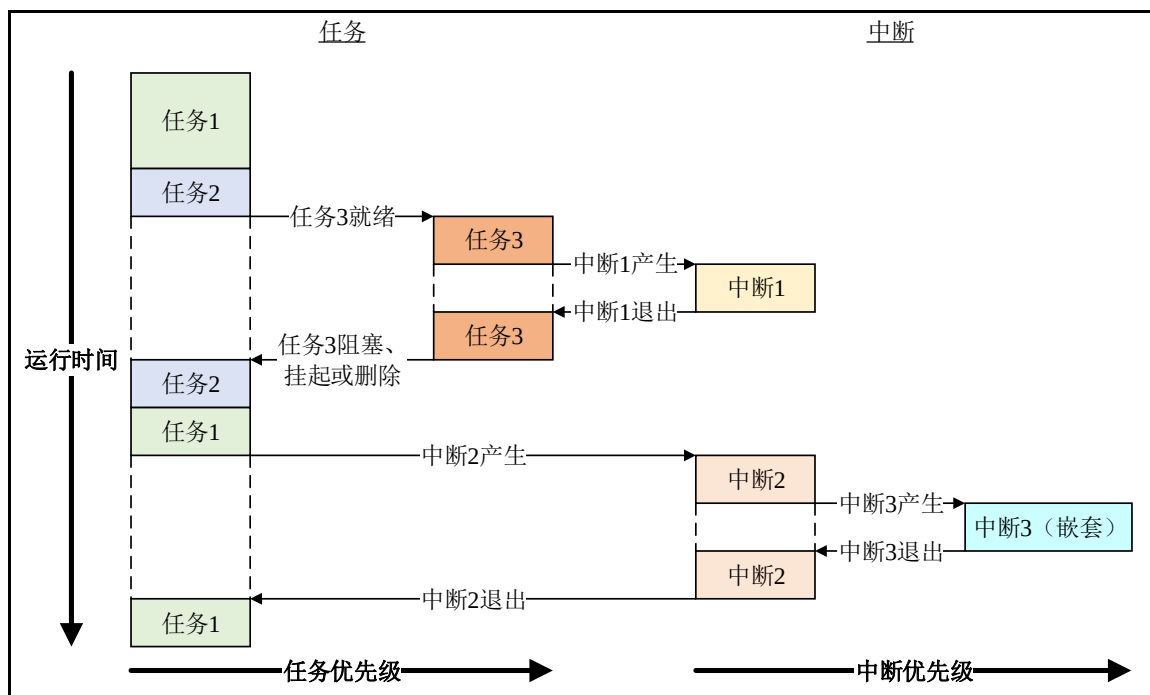


图 5.1.2 多任务系统运行示意图

从上图可以看出，相较于单任务系统而言，多任务系统的任务是具有优先级的，高优先级的任务可以像中断的抢占机制一样，抢占低优先级任务的运行，从而获得 CPU 的使用权；优先级相同的任务则各轮流运行一段极短的时间（宏观角度），从而产生“同时”运行的错觉。以上就是抢占式调度和时间片调度的基本原理。

在任务有了优先级的多任务系统中，用户就可以将紧急的事务放在优先级高的任务中进行处理，那么整个系统的实时性就会大大地提高。

5.2 μC/OS-III 任务状态

μC/OS-III 中任务存在五种状态，分别为休眠态、就绪态、运行态、挂起态和中断态，一个任务在某一时刻一定是处于这五种状态中的一种，μC/OS-III 中的五种任务状态之间的转换图如

下图所示：

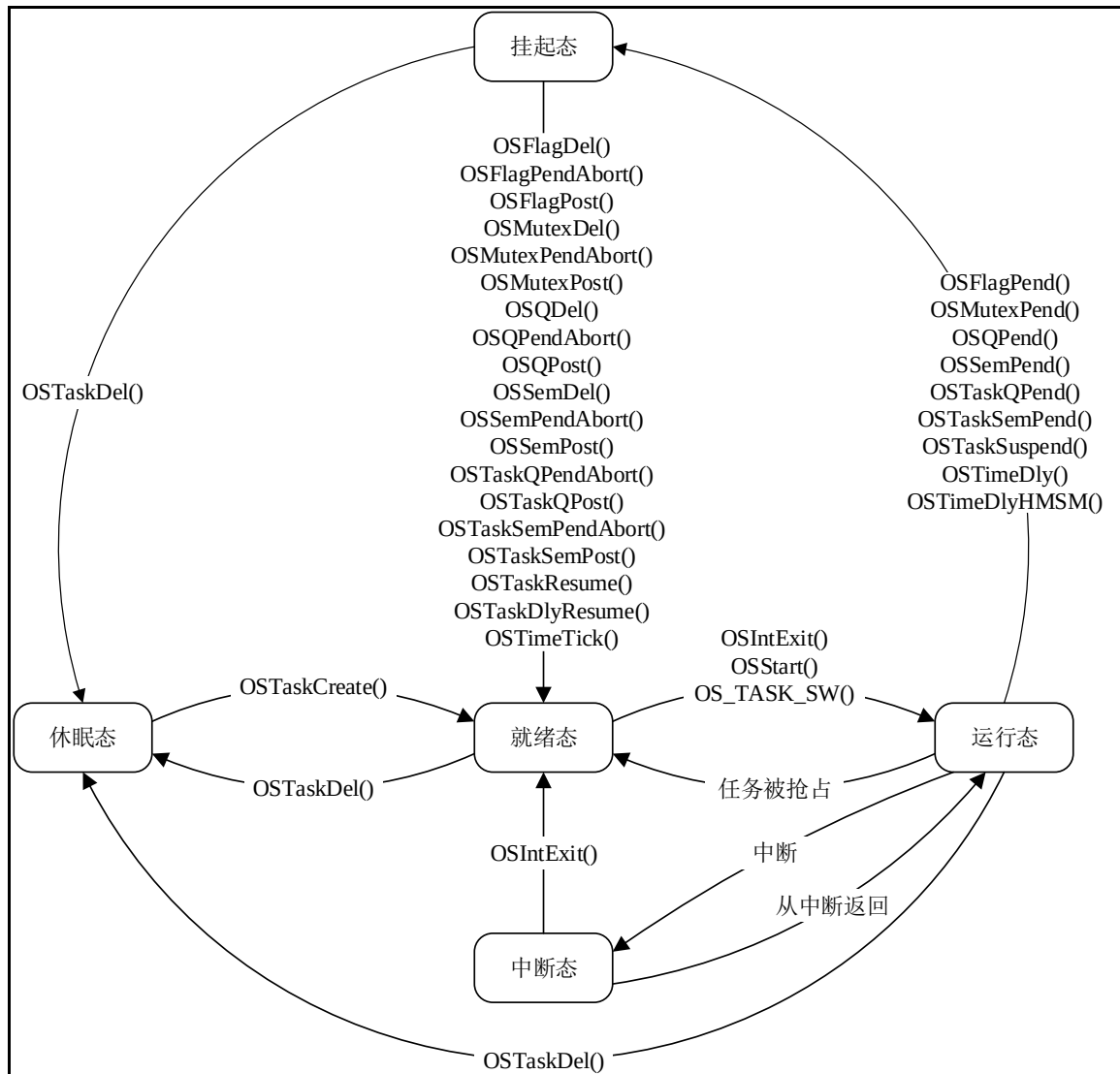


图 5.2.1 任务状态转换图

下面分别介绍了 μC/OS-III 中的这五种任务状态：

1. 休眠态

处于休眠态的任务对应的是那些存在于内存中，但是尚未提供给 μC/OS-III 内核的任务。应用程序可以通过调用函数 OSTaskCreate() 对一个处于休眠态的任务进行创建，休眠态任务的任务代码实际上已经存在于代码空间中，只是需要应用程序通知 μC/OS-III 内核。当不再需要某个任务时，可以通过函数 OSTaskDel() 删除这个任务，被删除的任务就会重新回到休眠态，这里所说的删除，并非将任务从代码空间中删除，而仅仅是将任务切换到休眠态，让 μC/OS-III 内核不再管理这个任务。

2. 就绪态

当任务被函数 OSTaskCreate() 创建后，准备好运行但还未运行时，任务就处于就绪态。μC/OS-III 中有一个就绪态任务链表，就绪态任务按照任务的优先级排序被链接到就绪态任务链表中。

3. 运行态

处于运行态的任务就是当前 CPU 正在执行的任务，对于只具有单核 CPU 的 MCU 而言，

在某一时刻只能执行一个任务，换言之，只具有单核 CPU 的 MCU，在某一时刻中，只存在一个运行态任务。

4. 挂起态

当处于运行态的任务需要等待某一事件时，例如等待信号量、消息队列或事件标志等，这时运行态任务就会让出 CPU 的使用权，切换到挂起态等待事件发生。当等待的事件发生后，处于挂起态的任务就会切换到就绪态，等待任务调度器调度运行。

5. 中断态

当处于运行态的任务被中断打断，CPU 跳转去执行中断的中断服务函数时，原本处于运行态的任务就会切换到中断态，直到中断服务函数处理结束后，在切换回运行态继续运行。

5.3 μC/OS-III 任务优先级

任务优先级是决定任务调度器如何分配 CPU 使用权的因素之一。每一个任务都被分配一个 0~(OS_CFG_PRIO_MAX-1)的任务优先级，并且 μC/OS-III 支持多了任务具有相同的任务优先级，宏 OS_CFG_PRIO_MAX 是在 μC/OS-III 的配置文 os_cfg.h 中定义配置的。

在 cpu_cfg.h 文件中有宏 CPU_CFG_LEAD_ZEROS_ASM_PRESENT，该宏用于配置 μC/OS-III 使用硬件指令的方法或是软件算法的方法计算前导零数量，μC/OS-III 使用位图的方式记录当前系统中存在的所有任务优先级，在 μC/OS-III 系统中存在的最高任务优先级时，就会使用到前导零计数。对于 STM32 而言，STM32 是具有硬件计算前导零的指令的，并且最大支持 32 比特位的数，因此宏 OS_CFG_PRIO_MAX 的最大值就是 32。当然，配置系统支持的任务优先级数量越多，系统消耗的资源也就越多，因此读者的实际的工程开发中，应当合理地根据实际需求配置宏 OS_CFG_PRIO_MAX。

μC/OS-III 的任务优先级高低与其对应的任务优先级数值是成反比的，也就是说，任务优先级数值为 0 的任务是最高优先级的任务，任务优先级数值为(OS_CFG_PRIO_MAX-1)的任务是优先级最低的任务。μC/OS-III 中任务优先级高低与其对一个数值大小的逻辑关系于 STM32 的中断优先级高低与其对应数值的逻辑关系是一样的，这也很容易理解，如下图所示：

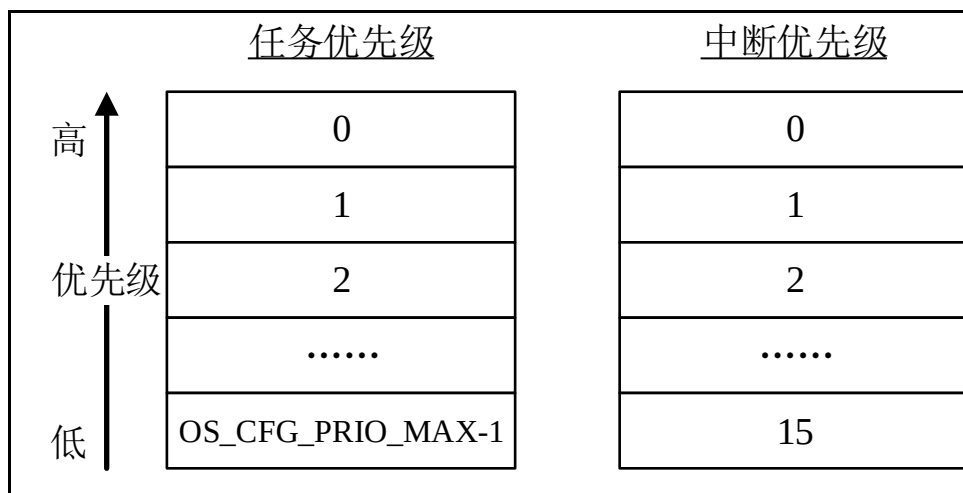


图 5.3.1 任务优先级与中断优先级

5.4 μC/OS-III 任务调度方式

任务调度是 μC/OS-III 的一部分，负责确定接下来应该运行哪一个任务。μC/OS-III 是一个基于任务优先级的抢占式内核，抢占式调度是 μC/OS-III 的主要任务调度方式，并且 μC/OS-III 在抢占式调度的基础上还支持时间片调度，接下来分别介绍这两种任务调度方式。

5.4.1 抢占式调度

在 μC/OS-III 中，每一个任务都会根据其重要性被分配一个任务优先级，重要性高的任务分配到的任务优先级高，任务优先级高的任务就能够抢占任务优先级低的任务，从而获得 CPU 的使用权，这也就是“抢占”一词的意思。

在抢占式调度中，如果一个具有高任务优先级的任务因等待某一事件而被挂起后，CPU 的使用权会交给任务优先级低的任务，此时，只要任务优先级高的任务等待的事件发生，那么 μC/OS-III 会立即挂起正在运行的低任务优先级的任务，而去处理任务优先级高的任务，这一过程就是抢占的过程。

同样的如果被挂起的高优先级的任务所等待的事件在中断中发生，那么中断服务函数退出后不会返回任务优先级低的任务运行，而是会直接返回任务优先级高的任务去运行。

抢占式调度主要是针对任务优先级不同的任务，每一个任务都有一个任务优先级，任务优先级高的任务可以抢占任务优先级低的任务运行，只有当任务优先级高的任务被挂起，低任务优先级的任务才能够运行。

5.4.2 时间片调度

时间片调度是针对任务优先级相同的任务而言的，当多个具有相同任务优先级的任务就绪时，任务调度器会根据用户设置的任务时间片轮流地运行这些任务，当然这些任务的运行依然会被任务优先级更高的任务抢占。时间片是以一次系统时钟节拍为单位的，例如 μC/OS-III 默认设置的任务时间片为 100，则 μC/OS-III 会在当前任务运行 100 次系统时钟节拍的时间后，切换到另一个相同任务优先级的任务中运行。

时间片调度涉及系统时钟节拍的内容，这部分内容会在下文的时间管理章节中进行介绍。

5.5 μC/OS-III 任务控制块

任务控制块（TCB，Task Control Block）是 μC/OS-III 内核用来存放任务信息的数据结构，因此每个任务都需要独自的任务控制块。任务控制块所需的内存空间需要由用户手动分配，并在调用任务相关的函数（例如函数 OSTaskCreate()）时，传入任务控制块在内存中的首地址，任务控制块结构体定义在 os.h 文件中，如下所示：

```
struct os_tcb {
    /* 指向任务栈栈顶的指针 */
    CPU_STK          *StkPtr;

    /* 指向用户自定义数据的指针 */
    void              *ExtPtr;

    /* 指向任务栈“水位”限制的指针 */
    CPU_STK          *StkLimitPtr;

#ifdef OS_CFG_DBG_EN > 0u
    /* 指向任务名的指针 */
    CPU_CHAR          *NamePtr;
#endif

    /* 指向任务链表中下一个任务控制块的指针 */
    OS_TCB            *NextPtr;
};
```

```

/* 指向任务链表中上一个任务控制块的指针 */
OS_TCB          *PrevPtr;

#if (OS_CFG_TICK_EN > 0u)
/* 指向 Tick 任务链表中下一个任务控制块的指针 */
OS_TCB          *TickNextPtr;
/* 指向 Tick 任务链表中上一个任务控制块的指针 */
OS_TCB          *TickPrevPtr;
#endif

#if ( (OS_CFG_DBG_EN > 0u) || \
      (OS_CFG_STAT_TASK_STK_CHK_EN > 0u) || \
      (OS_CFG_TASK_STK_REDZONE_EN > 0u) )
/* 指向任务栈起始地址的指针 */
CPU_STK          *StkBasePtr;
#endif

#if defined(OS_CFG_TLS_TBL_SIZE) && (OS_CFG_TLS_TBL_SIZE > 0u)
/* 任务本地存储数组 */
OS_TLS           TLS_Tbl[OS_CFG_TLS_TBL_SIZE];
#endif

#if (OS_CFG_DBG_EN > 0u)
/* 指向任务函数的指针 */
OS_TASK_PTR      TaskEntryAddr;
/* 指向任务函数参数的指针 */
void             *TaskEntryArg;
#endif

/* 指向挂起态任务链表中下一个任务控制块的指针 */
OS_TCB          *PendNextPtr;
/* 指向挂起态任务链表中上一个任务控制块的指针 */
OS_TCB          *PendPrevPtr;
/* 指向所等待内核对象的指针 */
OS_PEND_OBJ      *PendObjPtr;
/* 任务挂起等待内核对象的类型 */
OS_STATE         PendOn;
/* 任务挂起的结果 */
OS_STATUS        PendStatus;

/* 任务当前的状态 */
OS_STATE         TaskState;
/* 任务优先级 */

```

```
    OS_PRIO                Prio;
#if (OS_CFG_MUTEX_EN > 0u)
    /* 任务原始优先级 */
    OS_PRIO                BasePrio;
    /* 指向任务拥有的互斥信号量链表的指针 */
    OS_MUTEX                *MutexGrpHeadPtr;
#endif

#if (    (OS_CFG_DBG_EN > 0u) || \
        (OS_CFG_STAT_TASK_STK_CHK_EN > 0u) || \
        (OS_CFG_TASK_STK_REDZONE_EN > 0u))
    /* 任务栈大小 */
    CPU_STK_SIZE            StkSize;
#endif

    /* 任务操作选项 */
    OS_OPT                  Opt;

#if (OS_CFG_TS_EN > 0u)
    /* 任务时间戳 */
    CPU_TS                  TS;
#endif

#if (defined(OS_CFG_TRACE_EN) && (OS_CFG_TRACE_EN > 0u))
    /* 用于第三方调试工具 */
    CPU_INT16U              SemID;
#endif

    /* 用于任务接收信号的计数型信号量 */
    OS_SEM_CTR              SemCtr;

#if (OS_CFG_TICK_EN > 0u)
    /* 任务延时的剩余时钟节拍数 */
    OS_TICK                 TickRemain;
    /* 用于任务周期延时 */
    OS_TICK                 TickCtrPrev;
#endif

#if (OS_CFG_SCHED_ROUND_ROBIN_EN > 0u)
    /* 任务时间片 */
    OS_TICK                 TimeQuanta;
    /* 任务剩余时间片 */
    OS_TICK                 TimeQuantaCtr;
#endif

#if (OS_MSG_EN > 0u)
```

```
/* 指向任务接收到的消息的指针 */
void                *MsgPtr;

/* 任务接收到的消息的大小 */
OS_MSG_SIZE        MsgSize;
#endif

#if (OS_CFG_TASK_Q_EN > 0u)
/* 任务内嵌消息队列 */
OS_MSG_Q            MsgQ;
#endif
#if (OS_CFG_TASK_PROFILE_EN > 0u)
/* 消息从发送到接收花费的时间 */
CPU_TS              MsgQPendTime;
/* 消息从发送到接收花费的最长时间 */
CPU_TS              MsgQPendTimeMax;
#endif
#endif

#if (OS_CFG_TASK_REG_TBL_SIZE > 0u)
/* 特定于任务的寄存器表 */
OS_REG              RegTbl[OS_CFG_TASK_REG_TBL_SIZE];
#endif

#if (OS_CFG_FLAG_EN > 0u)
/* 任务挂起等待的事件标志 */
OS_FLAGS            FlagsPend;
/* 任务挂起等待但已发生的事件标志 */
OS_FLAGS            FlagsRdy;
/* 任务挂起等待事件标志的类型 */
OS_OPT              FlagsOpt;
#endif

#if (OS_CFG_TASK_SUSPEND_EN > 0u)
/* 记录任务被挂起的嵌套次数 */
OS_NESTING_CTR      SuspendCtr;
#endif

#if (OS_CFG_TASK_PROFILE_EN > 0u)
/* 任务的 CPU 使用率 */
OS_CPU_USAGE        CPUUsage;
/* 任务最大的 CPU 使用率 */
OS_CPU_USAGE        CPUUsageMax;
/* 任务执行次数 */
OS_CTX_SW_CTR       CtxSwCtr;
```

```

/* 任务在任务切换前的运行时间 */
CPU_TS          CyclesDelta;

/* 任务启动时的时间戳 */
CPU_TS          CyclesStart;

/* 任务的总运行时间 */
OS_CYCLES       CyclesTotal;

/* 任务上次的总运行时间 */
OS_CYCLES       CyclesTotalPrev;


/* 信号量发出信号所用时间 */
CPU_TS          SemPendTime;

/* 信号量发出信号所用的最大时间 */
CPU_TS          SemPendTimeMax;
#endif

#if (OS_CFG_STAT_TASK_STK_CHK_EN > 0u)
/* 任务栈已使用量 */
CPU_STK_SIZE    StkUsed;

/* 任务栈剩余量 */
CPU_STK_SIZE    StkFree;
#endif

#ifdef CPU_CFG_INT_DIS_MEAS_EN
/* 任务关闭中断的最长时间 */
CPU_TS          IntDisTimeMax;
#endif

#if (OS_CFG_SCHED_LOCK_TIME_MEAS_EN > 0u)
/* 任务锁定任务调度器的最长时间 */
CPU_TS          SchedLockTimeMax;
#endif

#if (OS_CFG_DBG_EN > 0u)
/* 指向用于代码调式的任务链表中上一个任务控制块的指针 */
OS_TCB          *DbgPrevPtr;

/* 指向用于代码调式的任务链表中下一个任务控制块的指针 */
OS_TCB          *DbgNextPtr;

/* 指向用于代码调式的任务名的指针 */
CPU_CHAR        *DbgNamePtr;
#endif

#if (defined(OS_CFG_TRACE_EN) && (OS_CFG_TRACE_EN > 0u))
/* 用于第三方调试工具 */
CPU_INT16U      TaskID;
#endif

```

```
};
```

从上面的代码中可以看出，μC/OS-III 的任务控制块结构体中包含了大量的成员变量，但是大多数成员变量都是可以通过配置文件中的配置项进行裁剪的。

5.6 μC/OS-III 任务栈

不论是逻辑编程还是 RTOS 编程，栈空间的使用都是非常重要的。函数中的局部变量、函数调用时的现场保护和现场恢复等都是要使用到栈空间的。

对于 μC/OS-III，在创建一个任务前，需要为任务准备好一块内存空间，这一内存空间将作为任务的栈空间进行使用，μC/OS-III 中创建任务函数的函数原型如下所示：

```
void OSTaskCreate( OS_TCB*      p_tcb,
                  CPU_CHAR*    p_name,
                  OS_TASK_PTR   p_task,
                  void*         p_arg,
                  OS_PRIO      prio,
                  CPU_STK*      p_stk_base,
                  CPU_STK_SIZE stk_limit,
                  CPU_STK_SIZE stk_size,
                  OS_MSG_QTY    q_size,
                  OS_TICK      time_quanta,
                  void*         p_ext,
                  OS_OPT        opt,
                  OS_ERR*       p_err)
```

其中参数 `p_stk_base` 就是任务栈空间的首地址，参数 `stk_size` 就是任务栈空间的大小，μC/OS-III 会根据这两个参数，设置好任务的栈空间。

参数 `stk_size` 用于表示任务栈空间的大小，其单位由参数 `p_stk_base` 的变量类型决定，从 μC/OS-III 中创建任务函数的函数原型中可以看到，参数 `p_stk_base` 的变量类型为 `CPU_STK*`，对于 STM32 所使用的 ARM Cortex-M 对应的 `CPU_STK` 定义如下所示：

```
typedef unsigned int  CPU_INT32U;
typedef CPU_INT32U    CPU_STK;
```

因此 `stk_size` 表示的任务栈大小的单位为字，例如，在文件 `os_cfg_app.h` 文件中的配置项 `OS_CFG_IDLE_TASK_STK_SIZE` 定义如下所示：

```
#define OS_CFG_IDLE_TASK_STK_SIZE 64u
```

该配置项用于指定空闲任务的任务栈大小，这里配置为 64，因为任务栈大小的单位为字，因此空闲任务的实际栈大小为 64×4 (1 字为 4 字节) = 256 字节。

第六章 μC/OS-III 任务相关 API 函数

在了解了 μC/OS-III 中任务的基础知识后，本章就开始学习 μC/OS-III 中任务相关的 API 函数。本章将着重讲解 μC/OS-III 中几个任务相关的 API 函数的用法，而不会涉及太多原理性的知识，相关的原理性知识会在后面的章节中进行详细地分析。由简入难，先知其然，然后再知其所以然，这也是学习的一种方法。

本章分为如下几部分：

- 6.1 μC/OS-III 创建和删除任务相关 API 函数
- 6.2 μC/OS-III 任务创建与删除实验
- 6.3 μC/OS-III 挂起和恢复任务相关 API 函数
- 6.4 μC/OS-III 任务挂起与恢复实验

6.1 μC/OS-III 创建和删除任务相关 API 函数

μC/OS-III 中用于创建和删除任务的 API 函数如下表所示：

函数	描述
OSTaskCreate()	创建任务
OSTaskDel()	删除任务

表 6.1.1 任务创建和删除 API 函数

6.1.1 函数 OSTaskCreate()

此函数用于创建任务，任务的任务控制块以及任务栈空间所需的内存，需要由用户手动分配并提供。当任务被创建好后，就会立马处于就绪态，此时只要任务调度器处于正常工作状态，那么创建好的任务就会由任务调度器调度。要注意的是，不能在中断服务函数中创建任务。

1. 函数原型

该函数的函数原型，如下所示：

```
void OSTaskCreate( OS_TCB*      p_tcb,
                  CPU_CHAR*    p_name,
                  OS_TASK_PTR  p_task,
                  void*        p_arg,
                  OS_PRIO      prio,
                  CPU_STK*     p_stk_base,
                  CPU_STK_SIZE stk_limit,
                  CPU_STK_SIZE stk_size,
                  OS_MSG_QTY    q_size,
                  OS_TICK      time_quanta,
                  void*        p_ext,
                  OS_OPT        opt,
                  OS_ERR*      p_err);
```

2. 函数形参

函数 OSTaskCreate() 的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
p_name	指向作为任务名的 ASCII 字符串的指针
p_task	指向任务函数的指针
p_arg	指向任务函数参数的指针
prio	任务的优先级
p_stk_base	指向任务栈起始地址的指针
stk_limit	任务的栈“水位”限制大小
stk_size	任务的栈大小
q_size	任务内嵌消息队列的大小
time_quanta	任务的时间片
p_ext	指向用户自定义数据的指针
opt	任务选项
p_err	指向接收错误代码变量的指针

表 6.1.2 函数 OSTaskCreate()形参相关描述

3. 函数返回值

函数 OSTaskCreate()的返回值描述，如下表所示：

返回值	描述
无	无

表 6.1.3 函数 OSTaskCreate()返回值相关描述

4. 函数错误代码

函数 OSTaskCreate()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	任务创建成功
OS_ERR_ILLEGAL_CREATE_RUN_TIME	定义了 OS_SAFETY_CRITICAL_IEC61508，且在 OSStart()之后非法地创建内核对象
OS_ERR_PRIO_INVALID	非法的任务优先级数值
OS_ERR_STAT_STK_SIZE_INVALID	任务栈在初始化期间溢出
OS_ERR_STK_INVALID	指向任务栈起始地址的指针为空
OS_ERR_STK_SIZE_INVALID	任务栈小于配置项 OS_CFG_STK_SIZE_MIN
OS_ERR_STK_LIMIT_INVALID	任务栈“水位”限制大小大于或等于任务栈大小
OS_ERR_TASK_CREATE_ISR	在中断中非法地创建任务
OS_ERR_TASK_INVALID	指向任务函数的指针为空
OS_ERR_TCB_INVALID	指向任务控制块的指针为空

表 6.1.4 函数 OSTaskCreate()错误代码相关描述

5. 函数注意事项

1. 任务栈必须定义为 CPU_STK 类型。
2. 在任务函数中必须调用 μC/OS-III 提供的用来让任务挂起的函数，例如延时函数、任务挂起函数或等待内核对象（等待消息队列、事件标志、信号量、互斥信号量等）的函数，这样才能够让其他任务获得 CPU 的使用权。
3. 应用程序中创建的任务不能够使用数值为 0、1、OS_CFG_PRIO_MAX-2 和 OS_CFG_PRIO_MAX-1 的任务优先级，因为这些任务优先级是保留给 μC/OS-III 使用的。

6. 函数用法举例

```

OS_TCB MyTaskTCB; /* (1) 保存任务的任务控制块 */
CPU_STK MyTaskStk[200];

void MyTask (void *p_arg) /* (3) 任务函数的地址就是任务函数的函数名 */
{
    while (DEF_ON) {
        /* 仍待一个事件 */
        /* 任务体 */
    }
}

void SomeCode (void)
{
    OS_ERR err;
    :
    :
    OSTaskCreate(    &MyTaskTCB, /* (1) 任务控制块 */
                    "My Task", /* (2) 任务名 */
                    MyTask, /* (3) 任务函数地址 */
                    (void *)0, /* (4) 任务函数参数未用到 */
                    12, /* (5) 任务优先级 */
                    &MyTaskStk[0], /* (6) 任务栈 */
                    10, /* (7) 栈生长的“水位”限制 */
                    200, /* (8) 任务栈大小 */
                    5, /* (9) 任务消息队列的大小 */
                    10, /* (10) 任务时间片 */
                    (void *)0, /* (11) 扩展指针 */
                    OS_OPT_TASK_STK_CHK + OS_OPT_TASK_STK_CLR, /* (12) 选项 */
                    &err); /* (13) 错误代码 */
    /* 检查错误代码 (14) */
    :
    :
}
    
```

6.1.2 函数 OSTaskDel()

此函数用于删除任务。当不再需要某一任务时，可以使用此函数来删除任务。删除任务并不会删除任务的代码或释放任务栈，删除任务指示意味着，任务的代码和任务栈都不再由 μC/OS-III 内核管理，因此被删除任务的任务代码和任务栈依然可以在创建其他任务时使用。

1. 函数原型

该函数的函数原型，如下所示：

```

void OSTaskDel( OS_TCB* p_tcb,
                OS_ERR* p_err)
    
```

2. 函数形参

函数 OSTaskDel() 的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
p_err	指向接收错误代码变量的指针

表 6.1.5 函数 OSTaskDel() 形参相关描述

3. 函数返回值

函数 OSTaskDel() 的返回值描述，如下表所示：

返回值	描述
无	无

表 6.1.6 函数 OSTaskDel() 返回值相关描述

4. 函数错误代码

函数 OSTaskDel() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	任务删除成功
OS_ERR_ILLEGAL_DEL_RUN_TIME	定义了 OS_SAFETY_CRITICAL_IEC61508，且在 OSStart() 之后非法地删除内核对象
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_STATE_INVALID	任务处于无效状态
OS_ERR_TASK_DEL_IDLE	非法删除空闲任务
OS_ERR_TASK_DEL_INVALID	非法删除 μC/OS-III 的中断服务任务
OS_ERR_TASK_DEL_ISR	在中断中非法地删除任务

表 6.1.7 函数 OSTaskDel() 错误代码相关描述

5. 函数注意事项

1. 不能删除空闲任务和 μC/OS-III 的中断服务任务。
2. 在删除拥有资源的任务时，要特别小心。

6. 函数用法举例

```
OS_TCB MyTaskTCB;

void TaskX (void *p_arg)
{
    OS_ERR err;

    while (DEF_ON) {
        :
        :
        OSTaskDel (&MyTaskTCB,
                   &err);

        /* 检查错误代码 */
        :
        :
    }
}
```

6.2 μC/OS-III 任务创建与删除实验

6.2.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数进行任务的创建与删除，本实验设计了四个任务，这四个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建其他任务
task1	用于任务创建和删除测试
task2	用于任务创建和删除测试
task3	用于删除测试任务

表 6.2.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 6-1 UCOS-III 任务创建与删除实验》。

6.2.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

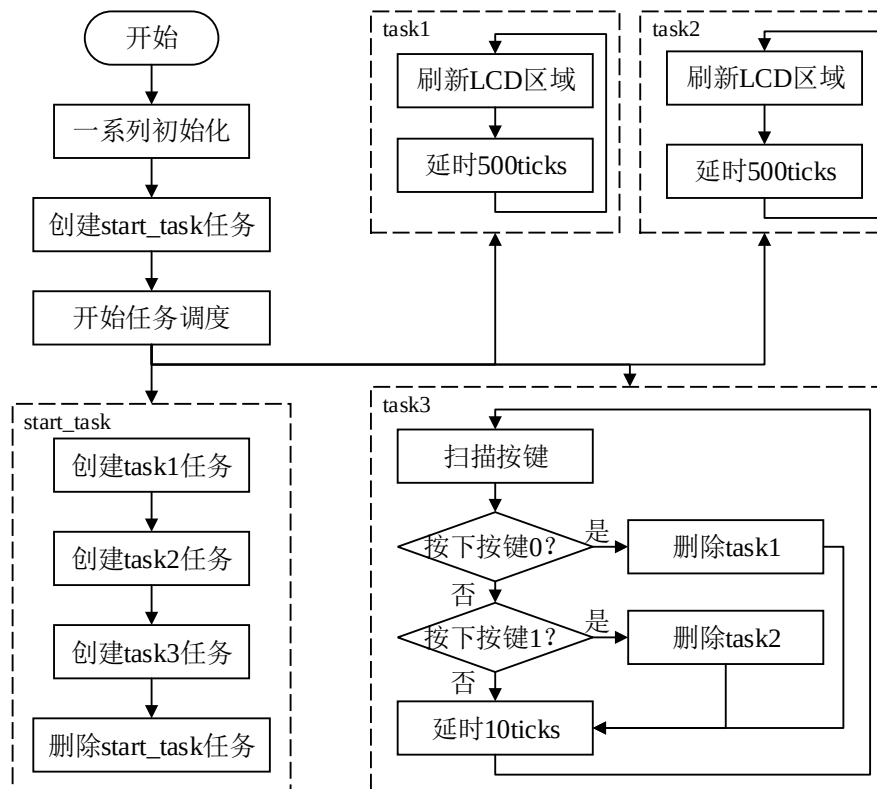


图 6.2.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSTaskCreate()

此函数用于创建任务，请参考第 6.1.1 小节《函数 OSTaskCreate()》。

(2) 函数 OSTaskDel()

此函数用于删除任务，请参考第 6.1.2 小节《函数 OSTaskDel()》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_TASK_DEL_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                    (CPU_CHAR*    ) "task1",
                    (OS_TASK_PTR  ) task1,
                    (void*        ) 0,
                    (OS_PRIO      ) TASK1_PRIO,
                    (CPU_STK*     ) Task1Task_STK,
                    (CPU_STK_SIZE ) TASK1_STK_SIZE / 10,
                    (CPU_STK_SIZE ) TASK1_STK_SIZE,
                    (OS_MSG_QTY   ) 0,
                    (OS_TICK      ) 0,
                    (void*        ) 0,
```

```

        (OS_OPT          ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*         ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK2_STK_SIZE * sizeof(CPU_STK));
OSTaskCreate(   (OS_TCB*      ) &Task2Task_TCB,
                (CPU_CHAR*    ) "task2",
                (OS_TASK_PTR  ) task2,
                (void*        ) 0,
                (OS_PRIO      ) TASK2_PRIO,
                (CPU_STK*     ) Task2Task_STK,
                (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,
                (CPU_STK_SIZE ) TASK2_STK_SIZE,
                (OS_MSG_QTY   ) 0,
                (OS_TICK      ) 0,
                (void*        ) 0,
                (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                (OS_ERR*      ) &err);

/* 创建 Task3 */
Task3Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK3_STK_SIZE * sizeof(CPU_STK));
OSTaskCreate(   (OS_TCB*      ) &Task3Task_TCB,
                (CPU_CHAR*    ) "task3",
                (OS_TASK_PTR  ) task3,
                (void*        ) 0,
                (OS_PRIO      ) TASK3_PRIO,
                (CPU_STK*     ) Task3Task_STK,
                (CPU_STK_SIZE ) TASK3_STK_SIZE / 10,
                (CPU_STK_SIZE ) TASK3_STK_SIZE,
                (OS_MSG_QTY   ) 0,
                (OS_TICK      ) 0,
                (void*        ) 0,
                (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                (OS_ERR*      ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务、task2 任务和 task3 任务，并且 task1 任务、task2 任务和 task3 任务的任务栈都是动态分配的，当然，也可以静态地定任务栈，就像定义一个数组一样。

(2) task1 和 task2 任务

```

/**
 * @brief   task1
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task1(void *p_arg)
{
    uint32_t task1_num = 0;
    OS_ERR err;

    while (1)
    {
        lcd_fill(6, 131, 114, 313, lcd_discolor[++task1_num % 11]);
        lcd_show_xnum(71, 111, task1_num, 3, 16, 0x80, BLUE);

        OSTimeDly(500, OS_OPT_TIME_DLY, &err);
    }
}

/**
 * @brief   task2
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task2(void *p_arg)
{
    uint32_t task2_num = 0;
    OS_ERR err;

    while (1)
    {
        lcd_fill(126, 131, 233, 313, lcd_discolor[11 - (++task2_num % 11)]);
        lcd_show_xnum(191, 111, task2_num, 3, 16, 0x80, BLUE);

        OSTimeDly(500, OS_OPT_TIME_DLY, &err);
    }
}
    
```

从以上代码中可以看到，task1 和 task2 任务分别每间隔 500ticks 就刷新一次屏幕，task1 和 task2 任务主要是用于测试任务的创建与删除，当 task1 和 task2 任务被创建后，就能够看到屏幕每间隔 500ticks 进行一次刷新；当 task1 和 task2 任务被删除后，屏幕上的内容就不再变化。

(4) task3 任务

```

/**
    
```

```

* @brief   task3
* @param   p_arg : 传入参数(未用到)
* @retval  无
*/
void task3(void *p_arg)
{
    uint8_t key = 0;
    OS_ERR err;

    while (1)
    {
        key = key_scan(0);

        switch (key)
        {
            case KEY0_PRES:                /* 删除任务 1 */
            {
                OSTaskDel(&Task1Task_TCB, &err);
                myfree(SRAMIN, Task1Task_STK);
                break;
            }
            case KEY1_PRES:                /* 删除任务 2 */
            {
                OSTaskDel(&Task2Task_TCB, &err);
                myfree(SRAMIN, Task2Task_STK);
                break;
            }
            default:
            {
                break;
            }
        }

        OSTimeDly(10, OS_OPT_TIME_DLY, &err);
    }
}

```

从上面的代码中可以看到，task3 任务负责扫描按键，当检测到 KEY0 按键被按下时候，调用函数 OSTaskDel()删除 task1 任务；当检测到 KEY1 按键被按下时，调用函数 OSTaskDel()删除 task2 任务。并且在删除任务后，由于任务栈占用的内存是在任务创建前使用动态内存管理进行分配的，因此任务删除后可以很方便的将任务栈占用的内存释放掉。

6.2.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 6.2.2 LCD 显示内容

其中，每间隔 500ticks，Task1 和 Task2 的屏幕区域就刷新一次，并且 Task1 和 Task2 后方数字也随之加一，此时表示 task1 和 task2 任务均被创建，并且正在运行中。

当按下 KEY0 按键后，task1 任务被删除，此时，Task1 的屏幕区域不再刷新，并且 Task1 后方的数字也不再改变，表明 task1 任务已被删除，不再运行。当按下 KEY1 按键后，task2 任务被删除，此时，Task2 的屏幕区域不再刷新，并且 Task2 后方的数字也不再改变，表明 task2 任务已经被删除，不再运行。

6.3 μC/OS-III 挂起和恢复任务相关 API 函数

μC/OS-III 中用于挂起和恢复任务的 API 函数如下表所示：

函数	描述
OSTaskSuspend()	挂起任务
OSTaskResume()	恢复任务

表 6.3.1 任务挂起和恢复 API 函数

6.3.1 函数 OSTaskSuspend()

此函数用于无条件地挂起任务，被挂起的任务不会参与任务调度，如果被挂起的任务是当前正在运行的任务，那么就会发起任务调度，将 CPU 的使用权交给另一个任务。被函数 OSTaskSuspend()挂起的任务只能通过函数 OSTaskResume()恢复。如果任务挂起等待其他内核对象，那么即使任务被该函数恢复后，也将继续挂起等待内核对象。

1. 函数原型

该函数的函数原型，如下所示：

```
void OSTaskSuspend(OS_TCB* p_tcb,
                  OS_ERR* p_err)
```

2. 函数形参

函数 OSTaskSuspend()的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
p_err	指向接收错误代码变量的指针

表 6.3.2 函数 OSTaskSuspend()形参相关描述

3. 函数返回值

函数 OSTaskSuspend()的返回值描述，如下表所示

返回值	描述
无	无

表 6.3.3 函数 OSTaskSuspend()返回值相关描述

4. 函数错误代码

函数 OSTaskSuspend()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	任务挂起成功
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_SCHED_LOCKED	任务调度器被锁定
OS_ERR_STATE_INVALID	任务处于无效状态
OS_ERR_TASK_SUSPEND_CTR_OVF	任务挂起嵌套次数溢出
OS_ERR_TASK_SUSPEND_ISR	在中断中非法挂起任务
OS_ERR_TASK_SUSPEND_IDLE	非法挂起空闲任务
OS_ERR_TASK_SUSPEND_INT_HANDLER	非法挂起 μC/OS-III 的中断服务任务

表 6.3.4 函数 OSTaskSuspend()错误代码相关描述

5. 注意事项

1. 函数 OSTaskSuspend()与函数 OSTaskResume()必须成对出现。
2. 被函数 OSTaskSuspend()挂起的任务只能通过调用函数 OSTaskResume()恢复。

6. 函数用法举例

```
void TaskX (void *p_arg)
{
    OS_ERR err;

    (void)&p_arg;
    while (DEF_ON) {
        :
        :
        OSTaskSuspend( (OS_TCB *)0,
                       &err);      /* 挂起当前任务 */
        /* 检查错误代码 */
        :
        :
    }
}
```

6.3.2 函数 OSTaskResume()

此函数用于恢复被函数 OSTaskSuspend()挂起的任务，并且只有该函数才能恢复被函数

OSTaskSuspend()挂起的任务，由此可见，被函数 OSTaskSuspend()挂起的任务只能由另一个任务通过函数 OSTaskResume()进行恢复。

1. 函数原型

该函数的函数原型，如下所示：

```
void OSTaskResume ( OS_TCB* p_tcb,
                    OS_ERR* p_err)
```

2. 函数形参

函数 OSTaskResume()的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
p_err	指向接收错误代码变量的指针

表 6.3.5 函数 OSTaskResume()形参相关描述

3. 函数返回值

函数 OSTaskResume()的返回值描述，如下表所示：

返回值	描述
无	无

表 6.3.6 函数 OSTaskResume()返回值相关描述

4. 函数错误代码

函数 OSTaskResume()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	任务恢复成功
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_STATE_INVALID	任务处于无效状态
OS_ERR_TASK_NOT_SUSPEND	任务未被挂起
OS_ERR_TASK_RESUME_ISR	在中断中非法恢复任务
OS_ERR_TASK_RESUME_SELF	非法恢复任务本身（当 p_tcb 指向空时）

表 6.3.7 函数 OSTaskResume()错误代码相关描述

5. 注意事项

无。

6. 函数用法举例

```
OS_TCB TaskY;

void TaskX (void *p_arg)
{
    OS_ERR err;

    while (DEF_ON) {
        :
        :
        OSTaskResume ( &TaskY,
                       &err);          /* 恢复被挂起的任务 */
        /* 检查错误代码 */
        :
    }
```

```

:
}
}

```

6.4 μC/OS-III 任务挂起与恢复实验

6.4.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数进行任务的挂起与恢复，本实验设计了四个任务，这四个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建其他任务
task1	用于任务挂起和恢复测试
task2	用于对照测试
task3	用于挂起和恢复任务

表 6.4.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 6-2 UCOS-III 任务挂起与恢复实验》。

6.4.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

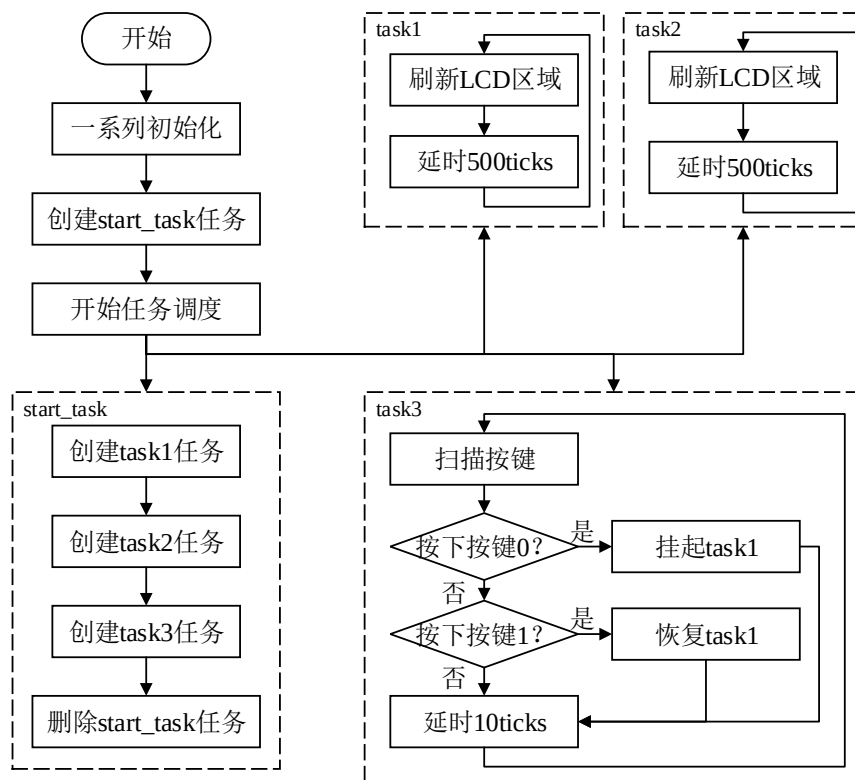


图 6.4.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSTaskSuspend()

此函数用于挂起任务，请参考第 6.3.1 小节《函数 OSTaskSuspend()》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_TASK_SUSPEND_EN 配置为 1。

(2) 函数 OSTaskResume()

此函数用于恢复任务，请参考第 6.3.2 小节《函数 OSTaskResume()》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_TASK_SUSPEND_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));
    OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                    (CPU_CHAR*     ) "task1",
                    (OS_TASK_PTR   ) task1,
                    (void*         ) 0,
                    (OS_PRIO       ) TASK1_PRIO,
                    (CPU_STK*      ) Task1Task_STK,
                    (CPU_STK_SIZE  ) TASK1_STK_SIZE / 10,
```

```

        (CPU_STK_SIZE    )TASK1_STK_SIZE,
        (OS_MSG_QTY      )0,
        (OS_TICK         )0,
        (void*           )0,
        (OS_OPT          )(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*         )&err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      )&Task2Task_TCB,
                 (CPU_CHAR*    )"task2",
                 (OS_TASK_PTR  )task2,
                 (void*        )0,
                 (OS_PRIO      )TASK2_PRIO,
                 (CPU_STK*     )Task2Task_STK,
                 (CPU_STK_SIZE)TASK2_STK_SIZE / 10,
                 (CPU_STK_SIZE)TASK2_STK_SIZE,
                 (OS_MSG_QTY   )0,
                 (OS_TICK      )0,
                 (void*        )0,
                 (OS_OPT       )(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*      )&err);

/* 创建 Task3 */
Task3Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK3_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      )&Task3Task_TCB,
                 (CPU_CHAR*    )"task3",
                 (OS_TASK_PTR  )task3,
                 (void*        )0,
                 (OS_PRIO      )TASK3_PRIO,
                 (CPU_STK*     )Task3Task_STK,
                 (CPU_STK_SIZE)TASK3_STK_SIZE / 10,
                 (CPU_STK_SIZE)TASK3_STK_SIZE,
                 (OS_MSG_QTY   )0,
                 (OS_TICK      )0,
                 (void*        )0,
                 (OS_OPT       )(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*      )&err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);

```

```
}
```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务、task2 任务和 task3 任务，并且 task1 任务、task2 任务和 task3 任务的任务栈都是动态分配的，当然，也可以静态地定任务栈，就像定义一个数组一样。

(2) task1 和 task2 任务

```
/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    uint32_t task1_num = 0;
    OS_ERR err;

    while (1)
    {
        lcd_fill(6, 131, 114, 313, lcd_discolor[++task1_num % 11]);
        lcd_show_xnum(71, 111, task1_num, 3, 16, 0x80, BLUE);

        OSTimeDly(500, OS_OPT_TIME_DLY, &err);
    }
}

/**
 * @brief  task2
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task2(void *p_arg)
{
    uint32_t task2_num = 0;
    OS_ERR err;

    while (1)
    {
        lcd_fill(126, 131, 233, 313, lcd_discolor[11 - (++task2_num % 11)]);
        lcd_show_xnum(191, 111, task2_num, 3, 16, 0x80, BLUE);

        OSTimeDly(500, OS_OPT_TIME_DLY, &err);
    }
}
```

从以上代码中可以看到，task1 和 task2 任务分别每间隔 500ticks 就区域刷新一次屏幕，task1

任务主要是用于测试任务的挂起与恢复，而 task2 任务则用于对照，当 task1 和 task2 任务被创建后，就能够看到屏幕上每间隔 500ticks 进行一次区域刷新；而当 task1 任务被挂起后，屏幕上 task1 任务刷新的区域就不再变化；当 task1 任务被恢复后，屏幕上 task1 任务刷新的区域又再次发生变化。

(4) task3 任务

```
/**
 * @brief   task3
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task3(void *p_arg)
{
    uint8_t key = 0;
    OS_ERR err;

    while (1)
    {
        key = key_scan(0);

        switch (key)
        {
            case KEY0_PRES:                /* 挂起任务 1 */
            {
                OSTaskSuspend(&Task1Task_TCB, &err);
                break;
            }
            case KEY1_PRES:                /* 恢复任务 1 */
            {
                OSTaskResume(&Task1Task_TCB, &err);
                break;
            }
            default:
            {
                break;
            }
        }

        OSTimeDly(10, OS_OPT_TIME_DLY, &err);
    }
}
```

从上面的代码中可以看到，task3 任务负责扫描按键，当检测到 KEY0 按键被按下时候，调用函数 vTaskSuspend()挂起 task1 任务；当检测到 KEY1 按键被按下时，调用函数 vTaskResume()恢复 task1 任务。

6.4.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 6.4.2 LCD 显示内容

其中，每间隔 500ticks，Task1 和 Task2 的屏幕区域就刷新一次，并且 Task1 和 Task2 后方数字也随之加一，此时表示 task1 和 task2 任务均被创建，并且正在运行中。

当按下 KEY0 按键后，task1 任务被挂起，此时，Task1 的屏幕区域不再刷新，并且 Task1 后方的数字也不再改变，表明 task1 任务已被挂起，暂停运行，而 task2 任务不受影响。当按下 KEY1 按键后，task1 任务被恢复，此时，Task1 的屏幕区域再次刷新，并且 Task1 后方的数字也再次改变，表明 task1 任务已经被恢复运行。

第七章 μC/OS-III 启动流程

在前面的章节中，介绍了 μC/OS-III 中任务创建、删除、挂起和恢复等的几个基础任务相关 API 函数的使用方法。本章将讲解 μC/OS-III 系统启动到第一个任务开始运行的一整个流程，也就是 μC/OS-III 的启动流程。

本章分为如下几部分：

- 7.1 μC/OS-III 启动流程
- 7.2 μC/OS-III 屏蔽所有中断
- 7.3 μC/OS-III 内核初始化
- 7.4 μC/OS-III 创建第一个应用任务
- 7.5 μC/OS-III 开始任务调度

7.1 μC/OS-III 启动流程

在以往学习 STM32 裸机的时候，就了解了 STM32 是如何从上电复位开始，一步一步的进入到 main() 函数中的，这就是 STM32 的启动流程，那 μC/OS-III 的启动流程是怎样的呢？

其实在第 2.2 小节《μC/OS-III 移植实验》中可以看到，在 main() 函数中，一开始只是对硬件做了初始化，在最后才调用了自定义函数 uc_os3_demo()。在函数 uc_os3_demo() 中才开始调用了 μC/OS-III 的相关 API 函数，因此函数 uc_os3_demo() 就是用来启动 μC/OS-III 的。

下面来看一下第 2.2 小节《μC/OS-III 移植实验》中的函数 uc_os3_demo()，如下所示：

```
/**
 * @brief   uC/OS-III 例程入口函数
 * @param   无
 * @retval  无
 */
void uc_os3_demo(void)
{
    OS_ERR err;

    lcd_show_string(10, 10, 220, 32, 32, "STM32", RED);
    lcd_show_string(10, 47, 220, 24, 24, "uC/OS-III Porting", RED);
    lcd_show_string(10, 76, 220, 16, 16, "ATOM@ALIENTEK", RED);

    /* 初始化 uC/OS-III */
    OSInit(&err);

    /* 创建 Start Task */
    OSTaskCreate(    (OS_TCB*      ) &StartTask_TCB,
                    (CPU_CHAR*    ) "start_task",
                    (OS_TASK_PTR  ) start_task,
                    (void*        ) 0,
                    (OS_PRIO      ) START_TASK_PRIO,
```

```

        (CPU_STK*      ) StartTask_STK,
        (CPU_STK_SIZE  ) START_STK_SIZE / 10,
        (CPU_STK_SIZE  ) START_STK_SIZE,
        (OS_MSG_QTY    ) 0,
        (OS_TICK       ) 0,
        (void*         ) 0,
        (OS_OPT        ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*       ) &err);

/* 开始任务调度 */
OSStart(&err);

for (;;)
{
    /* 不会进入这里 */
}
}

```

从上面的代码中可以看到，函数 `uc_os3_demo()` 中一共调用了四个 μC/OS-III 相关的 API 函数，在函数 `uc_os3_demo()` 中调用这四个 API 函数的目的如下表所示：

函数	目的
<code>CPU_IntDis()</code>	屏蔽所有中断，确保 μC/OS-III 的启动不被打断
<code>OSInit()</code>	初始化 μC/OS-III 内核
<code>OSTaskCreate()</code>	创建第一个应用任务
<code>OSStart()</code>	启动 μC/OS-III 内核，开始进行任务调度

表 7.1.1 函数 `uc_os3_demo()` 所调用函数的目的

下面就围绕这四个函数，分析 μC/OS-III 的启动流程。

7.2 μC/OS-III 屏蔽所有中断

启动 μC/OS-III 的第一步就是调用函数 `CPU_IntDis()`，该函数通过操作寄存器 `PRIMASK`，实现屏蔽所有的异常和中断，这样是为了保证在 μC/OS-III 的启动过程中，不被异常或中断打断，有关函数 `CPU_IntDis()` 更详细地介绍，请查看第 4.3.2 小节《μC/OS-III 开关中断》。

7.3 μC/OS-III 内核初始化

μC/OS-III 内核的初始化是通过调用函数 `OSInit()` 完成的，函数 `OSInit()` 必须在函数 `OSStart()` 调用之前被调用，函数 `OSInit()` 会初始化 μC/OS-III 内核并检测是否存在错误，如果检测到错误，就会立即返回。

函数 `OSInit()` 的函数原型如下所示：

```
void OSInit(OS_ERR* p_err);
```

可以看到，函数 `OSInit()` 有一个形参 `p_err`，是一个指向接收错误代码变量的指针，如果函数 `OSInit()` 在初始化 μC/OS-III 内核的时候，检测到错误，就会立即返回，并将错误代码通过形参 `p_err` 返回。

函数 `OSInit()` 的代码如下所示：

```
void OSInit(OS_ERR *p_err)
```

```
{
    /* 此宏用于定义异常栈的大小 */
    #if (OS_CFG_ISR_STK_SIZE > 0u)
        /* 这两个局部变量用于辅助配置异常栈
         * 在下面会用到
         */
        CPU_STK          *p_stk;
        CPU_STK_SIZE      size;
    #endif

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
    #ifndef OS_SAFETY_CRITICAL
        if (p_err == (OS_ERR *)0) {
            OS_SAFETY_CRITICAL_EXCEPTION();
            return;
        }
    #endif

    /* μC/OS-III 内核初始化钩子函数
     * 用于完成一些特定于硬件架构的初始化，如下：
     * 1、将异常栈的栈底 (OS_CPU_ExceptStkBase) 按 8 字节对齐
     * 2、检查 FPCCR 寄存器是否为复位值
     * 3、根据配置项 CPU_CFG_NVIC_PRIO_BITS 和配置项 CPU_CFG_KA_IPL_BOUNDARY
     *    初始化全局变量 OS_KA_BASEPRI_Boundary 的值，
     *    该全局变量用于设置 BASEPRI 寄存器，
     *    实现只屏蔽受 μC/OS-III 管理的中断
     */
    OSInitHook();

    /* 中断嵌套计数器 */
    OSIntNestingCtr          = 0u;
    /* μC/OS-III 内核运行状态 */
    OSRunning                 = OS_STATE_OS_STOPPED;
    /* 任务调度器锁定嵌套计数器 */
    OSSchedLockNestingCtr    = 0u;
    /* 指向当前任务的任务控制块的指针 */
    OSTCBCurPtr              = (OS_TCB *)0;
    /* 指向优先级最高的任务的任务控制块的指针 */
    OSTCBHighRdyPtr          = (OS_TCB *)0;
    /* 当前任务的任务优先级 */
    OSPrioCur                = 0u;
    /* 优先级最高的任务的任务优先级 */
    OSPrioHighRdy            = 0u;
}
```

```

    /* 此宏用于使能测量任务调度器的锁定时长 */
#if (OS_CFG_SCHED_LOCK_TIME_MEAS_EN > 0u)
    /* 任务调度器被锁定的开始时间戳 */
    OSSchedLockTimeBegin    = 0u;
    /* 任务调度器被锁定的历史最长时长 */
    OSSchedLockTimeMax      = 0u;
    /* 任务调度器被当前任务锁定的最长时长 */
    OSSchedLockTimeMaxCur   = 0u;
#endif

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    /* 此全局变量用于标志用户是否调用过函数 OSSafetyCriticalStart(),
     * 当函数 OSSafetyCriticalStart() 被调用过后,
     * 表示已经完成了所有的初始化工作, 在此之后将不允许再创建内核对象
     */
    OSSafetyCriticalStartFlag = OS_FALSE;
#endif

    /* 此宏用于使能 μC/OS-III 对时间片调度的支持 */
#if (OS_CFG_SCHED_ROUND_ROBIN_EN > 0u)
    /* 时间片调度使能标志 */
    OSSchedRoundRobinEn = OS_FALSE;
    /* 时间片长度的默认值 */
    OSSchedRoundRobinDfltTimeQuanta = OSCfg_TickRate_Hz / 10u;
#endif

    /* 此宏用于定义异常栈的大小 */
#if (OS_CFG_ISR_STK_SIZE > 0u)
    /* 检查异常栈的配置是否无误 (异常栈的起始地址不为空)
     * 并将异常栈清零
     */
    p_stk = OSCfg_ISRStkBasePtr;
    if (p_stk != (CPU_STK *)0) {
        size = OSCfg_ISRStkSize;
        while (size > 0u) {
            size--;
            *p_stk = 0u;
            p_stk++;
        }
    }
}

```

```

/* 此宏用于使能 RedZone 栈检查功能 */
#if (OS_CFG_TASK_STK_REDZONE_EN > 0u)
/* 初始化异常栈的 RedZone 栈检查功能 */
OS_TaskStkRedzoneInit(OSCfg_ISRStkBasePtr, OSCfg_ISRStkSize);
#endif
#endif

/* 此宏用于使能应用钩子函数 */
#if (OS_CFG_APP_HOOKS_EN > 0u)
/* 此宏用于使能 RedZone 栈检查功能 */
#if (OS_CFG_TASK_STK_REDZONE_EN > 0u)
/* 指向栈溢出钩子函数的指针 */
OS_AppRedzoneHitHookPtr = (OS_APP_HOOK_TCB) 0;
#endif
/* 指向任务创建钩子函数的指针 */
OS_AppTaskCreateHookPtr = (OS_APP_HOOK_TCB) 0;
/* 指向任务删除钩子函数的指针 */
OS_AppTaskDelHookPtr = (OS_APP_HOOK_TCB) 0;
/* 指向任务非法返回钩子函数的指针 */
OS_AppTaskReturnHookPtr = (OS_APP_HOOK_TCB) 0;
/* 指向空闲任务钩子函数的指针 */
OS_AppIdleTaskHookPtr = (OS_APP_HOOK_VOID) 0;
/* 指向统计任务钩子函数的指针 */
OS_AppStatTaskHookPtr = (OS_APP_HOOK_VOID) 0;
/* 指向任务切换钩子函数的指针 */
OS_AppTaskSwHookPtr = (OS_APP_HOOK_VOID) 0;
/* 指向时钟节拍钩子函数的指针 */
OS_AppTimeTickHookPtr = (OS_APP_HOOK_VOID) 0;
#endif

/* 此宏用于定义特定于任务的寄存器数量 */
#if (OS_CFG_TASK_REG_TBL_SIZE > 0u)
/* 任务特定寄存器组读取索引 */
OSTaskRegNextAvailID = 0u;
#endif

/* 初始化就绪态任务任务优先级位图表
* μC/OS-III 将所有就绪态任务的任务优先级以位图的方式记录在 OSPrioTbl[] 中,
* 因为本教程配套实验例程源码中将配置项 OS_CFG_PRIO_MAX, 配置为 32,
* 因此数组 OSPrioTbl 的元素个数为 1,
* 即使用 OSPrioTbl[0] 的 bit0~bit31 记录就绪态任务任务优先级
* 就绪态任务任务优先级位图表的初始化过程, 如下:
* 1、将数组 OSPrioTbl 中的所有元素清零

```

```

* 2、如果未使能配置项 OS_CFG_TASK_IDLE_EN（未使能空闲任务），
* 则强制将空闲任务的优先级（OS_CFG_PRIO_MAX-1）记录到就绪态
* 任务任务优先级位图表中，这样是为了兼容使能了空闲任务的情况
*/
OS_PrioInit();

/* 初始化就绪态任务链表
* 在 μC/OS-III 中定义了 OS_CFG_PRIO_MAX 个就绪态任务链表
* （一个任务优先级对应一个就绪态任务链表），
* 这 OS_CFG_PRIO_MAX 个就绪态任务链表被
* 存放在数组 OSRdyList[OS_CFG_PRIO_MAX] 中，
* 数组 OSRdyList 的一个元素就代表一个就绪态任务链表
* 就绪态任务链表的初始化过程，如下：
* 1、如果使能了配置项 OS_CFG_DBG_EN（使能代码调试功能），
* 则数组 OSRdyList 的每一个元素中都包含一个用于记录
* 该就绪态任务链表中任务数量的变量，初始化时，会将该变量清零
* 2、将每一个就绪态任务链表中指向上一个就绪态任务任务控制块的指针和
* 指向下一个就绪态任务任务控制块的指针清空。
*/
OS_RdyListInit();

/* 此宏用于使能事件标志功能 */
#if (OS_CFG_FLAG_EN > 0u)
/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
/* 用于调式事件标志的变量 */
OSFlagDbgListPtr = (OS_FLAG_GRP *)0;
OSFlagQty = 0u;
#endif
#endif

/* 此宏用于使能内存管理功能 */
#if (OS_CFG_MEM_EN > 0u)
/* 初始化内存管理功能
* 主要用于初始化用于调试内存管理的变量（使能了代码调试功能）
*/
OS_MemInit(p_err);
if (*p_err != OS_ERR_NONE) {
    return;
}
#endif

/* 此宏用于使能消息队列功能或任务内嵌消息队列功能 */

```

```
#if (OS_MSG_EN > 0u)
    /* 初始化消息池
    * 初始化消息池，实际上就是初始化消息池中的消息控制块
    * 消息控制块的初始化过程，如下：
    * 1、消息控制块中指向下一个消息控制块的指针指向下一个消息控制块
    *   （将消息池中的消息控制块链接成一个单向链表）
    * 2、消息控制块中指向消息的指针清空
    * 3、消息控制块中记录消息大小的变量清零
    * 4、消息控制块中记录消息发送时时间戳的变量清零
    */
    S_MsgPoolInit(p_err);
    if (*p_err != OS_ERR_NONE) {
        return;
    }
#endif

    /* 此宏用于使能互斥信号量功能 */
#if (OS_CFG_MUTEX_EN > 0u)
    /* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
        /* 用于调式互斥信号量的变量 */
        OSMutexDbgListPtr = (OS_MUTEX *)0;
        OSMutexQty        = 0u;
#endif
#endif
#endif

    /* 此宏用于使能消息队列功能 */
#if (OS_CFG_Q_EN > 0u)
    /* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
        /* 用于调试消息队列的变量 */
        OSQDbgListPtr = (OS_Q *)0;
        OSQQty        = 0u;
#endif
#endif
#endif

    /* 此宏用于使能信号量功能 */
#if (OS_CFG_SEM_EN > 0u)
    /* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
        /* 用于调试信号量的变量 */
        OSSemDbgListPtr = (OS_SEM *)0;
        OSSemQty        = 0u;
#endif
#endif
#endif
```

```
#endif
#endif

/* 此宏用于定义任务本地存储寄存器的数量 */
#if defined(OS_CFG_TLS_TBL_SIZE) && (OS_CFG_TLS_TBL_SIZE > 0u)
/* 初始化任务本地存储寄存器 */
OS_TLS_Init(p_err);
if (*p_err != OS_ERR_NONE) {
    return;
}
#endif

/* 初始化任务管理
 * 任务管理的初始化过程，如下：
 * 1、将任务数量计数器清零
 * 2、将任务切换次数计数器清零
 */
OS_TaskInit(p_err);
if (*p_err != OS_ERR_NONE) {
    return;
}

/* 此宏用于使能空闲任务 */
#if (OS_CFG_TASK_IDLE_EN > 0u)
/* 初始化空闲任务
 * 空闲任务的初始化过程，如下：
 * 1、将空闲任务运行次数计数器清零
 * 2、根据配置文件中空闲任务的相关配置项创建空闲任务
 */
OS_IdleTaskInit(p_err);
if (*p_err != OS_ERR_NONE) {
    return;
}
#endif

/* 此宏用于使能系统时钟节拍 */
#if (OS_CFG_TICK_EN > 0u)
/* 初始化系统时钟节拍
 * 系统时钟节拍初始化过程，如下：
 * 1、将系统时钟节拍计数器（OSTickCtr）清零
 * 2、将 Tick 任务链表（OSTickList）中指向下一个 Tick 任务控制块的指针清空
 */
OS_TickInit(p_err);
```



```

    if (*p_err != OS_ERR_NONE) {
        return;
    }
#endif

    /* 此宏用于使能任务统计功能 */
#if (OS_CFG_STAT_TASK_EN > 0u)
    /* 初始化任务统计任务
     * 任务统计任务的初始化过程，如下：
     * 1、初始化任务统计功能相关的全局变量
     * 2、根据配置文件中任务统计任务相关的配置项创建任务统计任务
     */
    OS_StatTaskInit(p_err);
    if (*p_err != OS_ERR_NONE) {
        return;
    }
#endif

    /* 此宏用于使能软件定时器功能 */
#if (OS_CFG_TMR_EN > 0u)
    /* 初始化软件定时器功能
     * 软件定时器功能的初始化过程，如下：
     * 1、将软件定时器链表的链表头清空
     * 2、根据配置项 OSCfg_TickRate_Hz 和配置项 OSCfg_TmrTaskRate_Hz
     *    计算软件定时器时间到 Tick 时间二点转换系数 (OSTmrToTicksMult)
     * 3、创建用于保护软件定时器的互斥信号量
     * 4、初始化用于软件定时器的内部变量（仅 μC/OS-III 内核使用）
     * 5、根据配置文件中软件定时器相关的配置项创建软件定时器任务
     */
    OS_TmrInit(p_err);
    if (*p_err != OS_ERR_NONE) {
        return;
    }
#endif

    /* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
    /* 初始化代码调试功能 */
    OS_Dbg_Init();
#endif

    /* μC/OS-III 配置初始化
     * 主要用于保证一些未使用到的变量不会被编译器优化掉

```

```

    */
    OSCfg_Init();

    /* μC/OS-III 内核初始化状态 */
    OSInitialized = OS_TRUE;
}

```

从上面的代码中可以看出，函数 `OSInit()` 就是初始化了各种供 `μC/OS-III` 内核正常运行的全局变量和功能。在做好各种初始化后，会将全局变量 `OSInitialized` 标记为 `OS_TRUE`，表示 `μC/OS-III` 内核已经完成初始化。

7.4 μC/OS-III 创建第一个应用任务

完成 `μC/OS-III` 内核初始化后，在开启 `μC/OS-III` 任务调度之前，还必须至少创建一个应用任务，因为，虽然在函数 `OSInit()` 初始化 `μC/OS-III` 内核的过程中，创建了空闲任务、任务统计任务和软件定时器任务（其中空闲任务是一定会被创建的，而任务统计任务和软件定时器任务会根据配置文件的配置，进行创建或不创建），但是以上这三个任务都属于内核任务，应用程序是无法访问的，并且在 `μC/OS-III` 内核开始任务调度之后，就不会返回，CPU 只会执行已创建的任务，因此在开始任务调度之前，还需要至少创建一个应用任务，这个应用任务来负责初始化应用程序。

下面看一下在第 2.2 小节《`μC/OS-III` 移植实验》中是如何创建第一个应用任务的，具体的代码如下所示：

```

#define START_TASK_PRIO    2                /* 任务优先级 */
#define START_STK_SIZE     512              /* 任务栈大小 */
OS_TCB                    StartTask_TCB;    /* 任务控制块 */
CPU_STK                   StartTask_STK[START_STK_SIZE]; /* 任务栈 */
void start_task(void *p_arg);               /* 任务函数 */

void uc_os3_demo(void)
{
    /* 仅展示创建第一个应用任务，省略其余代码 */

    /* 创建 Start Task */
    OSTaskCreate(
        (OS_TCB*) &StartTask_TCB,
        (CPU_CHAR*) "start_task",
        (OS_TASK_PTR) start_task,
        (void*) 0,
        (OS_PRIO) START_TASK_PRIO,
        (CPU_STK*) StartTask_STK,
        (CPU_STK_SIZE) START_STK_SIZE / 10,
        (CPU_STK_SIZE) START_STK_SIZE,
        (OS_MSG_QTY) 0,
        (OS_TICK) 0,
        (void*) 0,
        (OS_OPT) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),

```

```

(OS_ERR*      ) &err);

/* 仅展示创建第一个应用任务，省略其余代码 */
}

```

通过以上代码，就创建了一个任务名为 `start_task` 的任务，该任务的任务函数为 `start_task()`，那么在开始 μC/OS-III 任务调度之后，任务 `start_task` 就能够参与任务调度，获得 CPU 使用权，这样一来，就能够在 `start_task` 任务中完成应用程序的初始化，以及开始运行应用程序。

值得注意的是，以上例子中，`start_task` 任务的任务优先级被设置为 2，前面说过，应用程序中创建的任务不能够使用数值为 0、1、`OS_CFG_PRIO_MAX-2` 和 `OS_CFG_PRIO_MAX-1` 的任务优先级，因此数值为 2 的任务优先级是应用程序中能够使用的最高任务优先级，将 `start_task` 的任务优先级设置为 2 的目的是，在 `start_task` 任务完成应用程序初始化的过程中，会创建其他应用任务，其他应用任务的任务优先级应按照实际的应用需求进行设置，但只要其他应用任务的任务优先级小于 `start_task` 任务的任务优先级，那么 `start_task` 任务的一整个应用程序初始化流程就不会被其他应用程序打断。

在第 6.1.1 小节《函数 `OSTaskCreate()`》中简单介绍了函数 `OSTaskCreate()` 的使用方法，接下来将分析以下函数 `OSTaskCreate()` 是如何创建一个任务的，这是理解 μC/OS-III 启动流程的关键之一。

函数 `OSTaskCreate()` 的代码如下所示：

```

void OSTaskCreate( OS_TCB*      p_tcb,
                  CPU_CHAR*     p_name,
                  OS_TASK_PTR    p_task,
                  void*          p_arg,
                  OS_PRIO        prio,
                  CPU_STK*       p_stk_base,
                  CPU_STK_SIZE   stk_limit,
                  CPU_STK_SIZE   stk_size,
                  OS_MSG_QTY     q_size,
                  OS_TICK        time_quanta,
                  void*          p_ext,
                  OS_OPT          opt,
                  OS_ERR*        p_err)
{
    CPU_STK_SIZE   i;
#if (OS_CFG_TASK_REG_TBL_SIZE > 0u)
    OS_REG_ID      reg_nbr;
#endif
#if defined(OS_CFG_TLS_TBL_SIZE) && (OS_CFG_TLS_TBL_SIZE > 0u)
    OS_TLS_ID      id;
#endif

    CPU_STK*       p_sp;
    CPU_STK*       p_stk_limit;
    CPU_SR_ALLOC();

```

```
/* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

/* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        *p_err = OS_ERR_ILLEGAL_CREATE_RUN_TIME;
        return;
    }
#endif

/* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
/* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
 * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
 * 因为在中断中不能调用创建任务的函数，
 * 因此返回错误
 */
    if (OSIntNestingCtr > 0u) {
        OS_TRACE_TASK_CREATE_FAILED(p_tcb);
        *p_err = OS_ERR_TASK_CREATE_ISR;
        return;
    }
#endif

/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
/* 指向任务控制块的指针不能为空 */
    if (p_tcb == (OS_TCB *)0) {
        OS_TRACE_TASK_CREATE_FAILED(p_tcb);
        *p_err = OS_ERR_TCB_INVALID;
        return;
    }
/* 指向任务函数的指针不能为空 */
    if (p_task == (OS_TASK_PTR)0u) {
        OS_TRACE_TASK_CREATE_FAILED(p_tcb);
        *p_err = OS_ERR_TASK_INVALID;
    }
#endif
```

```

        return;
    }

    /* 指向任务栈的指针不能为空 */
    if (p_stk_base == (CPU_STK *)0) {
        OS_TRACE_TASK_CREATE_FAILED(p_tcb);
        *p_err = OS_ERR_STK_INVALID;
        return;
    }

    /* 任务栈的大小不能小于配置项 OSCfg_StkSizeMin 的值 */
    if (stk_size < OSCfg_StkSizeMin) {
        OS_TRACE_TASK_CREATE_FAILED(p_tcb);
        *p_err = OS_ERR_STK_SIZE_INVALID;
        return;
    }

    /* “水位”限制值不能超过任务栈的大小 */
    if (stk_limit >= stk_size) {
        OS_TRACE_TASK_CREATE_FAILED(p_tcb);
        *p_err = OS_ERR_STK_LIMIT_INVALID;
        return;
    }

    /* 任务优先级数值不能大于 OS_CFG_PRIO_MAX-1 */
    if ( (prio > (OS_CFG_PRIO_MAX - 2u)) &&
         (prio != (OS_CFG_PRIO_MAX - 1u)) ) {
        OS_TRACE_TASK_CREATE_FAILED(p_tcb);
        *p_err = OS_ERR_PRIO_INVALID;
        return;
    }
#endif

    /* 判断任务优先级是否为最低的任务优先级 (OS_CFG_PRIO_MAX-1)
     * 此任务优先级只能用于创建空闲任务,
     * 如果使用该任务优先级创建的任务不是空闲任务,
     * 则返回错误
     */
    if (prio == (OS_CFG_PRIO_MAX - 1u)) {
#ifdef OS_CFG_TASK_IDLE_EN > 0u
        if (p_tcb != &OSIdleTaskTCB) {
            OS_TRACE_TASK_CREATE_FAILED(p_tcb);
            *p_err = OS_ERR_PRIO_INVALID;
            return;
        }
#else
        OS_TRACE_TASK_CREATE_FAILED(p_tcb);

```

```

        *p_err = OS_ERR_PRIO_INVALID;
        return;
#endif
    }

    /* 初始化任务控制块
     * 初始化任务控制块实际上就是初始化任务控制块中成员变量的值
     * 初始化后任务控制块中成员变量的值，如下：
     * 1、任务名 (NamePtr)："?Task"
     * 2、任务启动时的时间戳 (CyclesStart)：OS_TS_GET()
     * 3、用于代码调式的任务名 (DbgNamePtr)：" "
     * 4、其余的成员变量全部初始化为 0 或空
     */
    OS_TaskInitTCB(p_tcb);

    /* 初始化错误代码为“无错误” */
    *p_err = OS_ERR_NONE;

    /* 如果形参任务选项 (opt) 包含 OS_OPT_TASK_STK_CLR
     * 则将任务栈清 0
     */
    if (( (opt & OS_OPT_TASK_STK_CHK) != 0u) ||
        ((opt & OS_OPT_TASK_STK_CLR) != 0u)) {
        if ((opt & OS_OPT_TASK_STK_CLR) != 0u) {
            p_sp = p_stk_base;
            for (i = 0u; i < stk_size; i++) {
                *p_sp = 0u;
                p_sp++;
            }
        }
    }

    /* 宏 CPU_CFG_STK_GROWTH 用于定义栈的生长方向，
     * 栈的生长方向是与硬件有关的，
     * 当宏 CPU_CFG_STK_GROWTH 被定义为 CPU_STK_GROWTH_LO_TO_HI 时，
     * 表示栈的生长方向为向上生长（数据入栈后，CPU 的栈指针指向下一个更高的栈地址）
     * 当宏 CPU_CFG_STK_GROWTH 被定义为 CPU_STK_GROWTH_HI_TO_LO 时，
     * 表示栈的生长方向为向下生长（数据入栈后，CPU 的栈指针指向下一个更低的栈地址）
     */
    #if (CPU_CFG_STK_GROWTH == CPU_STK_GROWTH_HI_TO_LO)
        /* p_stk_limit 用于指向任务栈“水位”限制的地址
         * 当栈为向下生长时（STM32 就是这种情况），
         * p_stk_limit 就应该为栈的起始地址 p_stk_base（也就是栈满时栈指针指向的地址）
        */
    #endif

```

```

    * 加上（与栈的生长方向相反）“水位”限制的大小
    */
    p_stk_limit = p_stk_base + stk_limit;
#else
    /* 当栈为向上生长时，
    * p_stk_limit 就应该为栈的起始地址 p_stk_base（也就是栈空时栈指针指向的地址）
    * 加上栈的大小（计算出栈满时栈指针指向的地址）
    * 减去（与栈的生长方向相反）“水位”限制的大小
    */
    p_stk_limit = p_stk_base + (stk_size - 1u) - stk_limit;
#endif

/* 初始化任务栈
* 在初始化任务栈的时候，会模拟 CPU 寄存器入栈的操作，
* 将初始数据压入到任务栈中，
* 并且每入栈一个数据，都会更新一次栈指针，
* 一开始的栈指针是指向栈底（任务栈的高地址，因为 STM32 的栈是向下生长的）的，
* 初始化后的栈指针将指向比栈底地址底的某个地址（具体由入栈数据的数量决定），
* 任务栈初始化后，会返回此时的栈顶指针（p_sp），
* 此时任务栈的初始化结果，从栈底开始，从高地址到低地址数据的含义及初始值，如下所示：
* xPSR -> 0x01000000u
* 任务入口函数 -> p_task
* R14 (LR) -> OS_TaskReturn
* R12 -> 0x12121212u
* R3 -> 0x03030303u
* R2 -> 0x02020202u
* R1 -> p_stk_limit
* R0 -> p_arg
* R14 -> 0xFFFFFFFFuL
* R11 -> 0x11111111uL
* R10 -> 0x10101010uL
* R9 -> 0x09090909uL
* R8 -> 0x08080808uL
* R7 -> 0x07070707uL
* R6 -> 0x06060606uL
* R5 -> 0x05050505uL
* R4 -> 0x04040404uL
* 可以看到，初始化后的任务栈中，
* 保存了任务的入口函数和函数的参数，
* 以及任务意外返回后 CPU 执行的函数等信息
*/
p_sp = OSTaskStkInit(    p_task,
                        p_arg,

```

```

        p_stk_base,
        p_stk_limit,
        stk_size,
        opt);

/* 检查任务栈初始化后是否溢出 */
#if (CPU_CFG_STK_GROWTH == CPU_STK_GROWTH_HI_TO_LO)
    if (p_sp < p_stk_base) {
        *p_err = OS_ERR_STK_OVF;
        return;
    }
#else
    if (p_sp > (p_stk_base + stk_size)) {
        *p_err = OS_ERR_STK_OVF;
        return;
    }
#endif

/* 此宏用于使能 RedZone 栈检查功能 */
#if (OS_CFG_TASK_STK_REDZONE_EN > 0u)
/* 初始化任务的 RedZone 栈检查功能
 * 实际上就是在任务栈的末端填充 OS_CFG_TASK_STK_REDZONE_DEPTH 个数据,
 * 所填充的数据值为 OS_STACK_CHECK_VAL,
 * 之后只要检查任务栈末端的 OS_CFG_TASK_STK_REDZONE_DEPTH 个数据是否被修改,
 * 就能够简单判断任务栈是否溢出
 */
    OS_TaskStkRedzoneInit(p_stk_base, stk_size);
#endif

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
/* 指向任务函数的指针 */
    p_tcb->TaskEntryAddr = p_task;
/* 指向任务函数参数的指针 */
    p_tcb->TaskEntryArg = p_arg;
#endif

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
/* 指向任务名的指针 */
    p_tcb->NamePtr = p_name;
#else
    (void)p_name;

```



```
#endif

/* 任务优先级 */
p_tcb->Prio          = prio;

/* 此宏用于使能互斥信号量 */
#if (OS_CFG_MUTEX_EN > 0u)
/* 任务原始优先级 */
p_tcb->BasePrio      = prio;
#endif

/* 更新任务的栈顶指针 */
p_tcb->StkPtr        = p_sp;
/* 指向任务栈“水位”限制的指针 */
p_tcb->StkLimitPtr   = p_stk_limit;

/* 此宏用于使能 μC/OS-III 对时间片调度的支持 */
#if (OS_CFG_SCHED_ROUND_ROBIN_EN > 0u)
/* 设置任务时间片和任务默认时间片
 * 每个任务的任务时间片是可以单独设置的
 */
p_tcb->TimeQuanta     = time_quanta;
if (time_quanta == 0u) {
    p_tcb->TimeQuantaCtr = OSSchedRoundRobinDfltTimeQuanta;
} else {
    p_tcb->TimeQuantaCtr = time_quanta;
}
#else
(void)time_quanta;
#endif

/* 指向用户自定义数据的指针 */
p_tcb->ExtPtr          = p_ext;
/* OS_CFG_DBG_EN: 此宏用于使能代码调试功能
 * OS_CFG_STAT_TASK_STK_CHK_EN: 此宏用于使能在进行任务统计时，检查任务栈的使用情况
 * OS_CFG_TASK_STK_REDZONE_EN: 此宏用于使能 RedZone 栈检查功能
 */
#if ((OS_CFG_DBG_EN > 0u) || \
    (OS_CFG_STAT_TASK_STK_CHK_EN > 0u) || \
    (OS_CFG_TASK_STK_REDZONE_EN > 0u))
/* 指向任务栈起始地址的指针 */
p_tcb->StkBasePtr     = p_stk_base;
/* 任务栈大小 */
```

```

    p_tcb->StkSize          = stk_size;
#endif

    /* 任务操作选项 */
    p_tcb->Opt               = opt;

    /* 此宏用于定义特定于任务的寄存器数量 */
#if (OS_CFG_TASK_REG_TBL_SIZE > 0u)
    /* 将任务中所有特定寄存器清零 */
    for (reg_nbr = 0u; reg_nbr < OS_CFG_TASK_REG_TBL_SIZE; reg_nbr++) {
        p_tcb->RegTbl[reg_nbr] = 0u;
    }
#endif

    /* 此宏用于使能任务内嵌消息队列功能 */
#if (OS_CFG_TASK_Q_EN > 0u)
    /* 初始化任务内嵌消息队列 */
    OS_MsgQInit(&p_tcb->MsgQ,
                q_size);
#else
    (void)q_size;
#endif

    /* 调用任务创建钩子函数 */
    OSTaskCreateHook(p_tcb);

    /* 用于调试，不用理会 */
    OS_TRACE_TASK_CREATE(p_tcb);
    OS_TRACE_TASK_SEM_CREATE(p_tcb, p_name);
#if (OS_CFG_TASK_Q_EN > 0u)
    OS_TRACE_TASK_MSG_Q_CREATE(&p_tcb->MsgQ, p_name);
#endif

    /* 此宏用于定义任务本地存储寄存器的数量 */
#if defined(OS_CFG_TLS_TBL_SIZE) && (OS_CFG_TLS_TBL_SIZE > 0u)
    /* 将所有任务本地存储寄存器清零 */
    for (id = 0u; id < OS_CFG_TLS_TBL_SIZE; id++) {
        p_tcb->TLS_Tbl[id] = 0u;
    }

    /* 调用任务本地存储寄存器创建钩子函数 */
    OS_TLS_TaskCreate(p_tcb);
#endif

    /* 保存中断状态，并进入临界区 */

```

```

CPU_CRITICAL_ENTER();

/* 任务优先级插入就绪态任务优先级位图表 */
OS_PrioInsert(p_tcb->Prio);

/* 任务插入就绪态任务链表 */
OS_RdyListInsertTail(p_tcb);

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
    /* 将任务添加到调试链表中 */
    OS_TaskDbgListAdd(p_tcb);
#endif

/* 更新系统中已创建任务的数量 */
OSTaskQty++;

/* 判断系统是否还未运行
 * 如果系统未运行，
 * 则在退出临界区，并恢复中断状态后直接返回
 * 否则触发任务调度
 */
if (OSRunning != OS_STATE_OS_RUNNING) {
    CPU_CRITICAL_EXIT();
    return;
}

CPU_CRITICAL_EXIT();

OSSched();
}

```

从上面的代码中可以看出，函数 `OSTaskCreate()` 在创建一个任务的时候，会初始化任务控制块中的成员变量，但最重要的还是初始化的任务的任务栈，任务栈中就包含了任务在获得 CPU 使用权时，应该从哪里开始执行、任务函数意外返回，要返回到那里去、任务函数的参数以及任务执行过程中的 CPU 寄存器的值等信息。

7.5 μC/OS-III 开始任务调度

开始 μC/OS-III 任务调度是 μC/OS-III 内核开始运行前的最后一步，在此之前需要先调用函数 `OSInit()` 对 μC/OS-III 内核进行初始化，并至少创建一个应用任务。通过调用函数 `OSStart()` 就能够开始 μC/OS-III 的任务调度。

函数 `OSStart()` 的函数原型如下所示：

```
void OSStart(OS_ERR* p_err)
```

可以看到，函数 `OSStart()` 有一个形参 `p_err`，是一个指向接收错误代码变量的指针，如果函数 `OSStart()` 在开启 μC/OS-III 任务调度的过程中，检测到错误，就会返回，并将错误代码通过形参 `p_err` 返回。如果函数 `OSStart()` 成功开启了 μC/OS-III 的任务调度，那么函数 `OSStart()` 是不

会返回的，会直接开始进行任务调度，并执行任务。

函数 OSStart() 的代码如下所示：

```
void OSStart (OS_ERR *p_err)
{
    OS_OBJ_QTY kernel_task_cnt;

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    /* 在开始任务调度之前，必须先完成 μC/OS-III 内核的初始化（函数 OSInit()）
     * 在函数 OSInit() 初始化完成后会将全局变量 OSInitialized 赋值为 OS_TRUE，
     * 因此判断全局变量 OSInitialized 的值，来判断用户是否在 μC/OS-III 内核完成
     * 初始化之前，非法调用函数 OSStart() 开启 μC/OS-III 任务调度
     */
    if (OSInitialized != OS_TRUE) {
        *p_err = OS_ERR_OS_NOT_INIT;
        return;
    }

    /* 计算已创建的内核任务的数量
     * 内核任务一共有三个，分别为空闲任务、任务统计任务、软件定时器任务
     * 根据配置文件配置的不同，内核任务的数量可能不同
     */
    kernel_task_cnt = 0u;
    #if (OS_CFG_STAT_TASK_EN > 0u)
        kernel_task_cnt++;
    #endif
    #if (OS_CFG_TMR_EN > 0u)
        kernel_task_cnt++;
    #endif
    #if (OS_CFG_TASK_IDLE_EN > 0u)
        kernel_task_cnt++;
    #endif

    /* 在 μC/OS-III 开始任务调度之前，用户必须要至少创建一个应用任务，
     * 因此通过已创建任务的数量是否不大于内核任务的数量，
     * 来判断用户在此之前是否没有创建应用任务
     */
}
```

```

if (OSTaskQty <= kernel_task_cnt) {
    *p_err = OS_ERR_OS_NO_APP_TASK;
    return;
}

/* 判断系统是否已经开始运行，
 * 目的是为了方式重复启动 μC/OS-III 内核
 */
if (OSRunning == OS_STATE_OS_STOPPED) {
    /* 获取就绪态任务中最高的任务优先级
     * 就绪态任务的任务优先级以位图的当时记录在数组 OSPrioTbl 中
     * 通过计算前导零的方式就能够非常方便地获取就绪态任务中最高的任务优先级
     */
    OSPrioHighRdy = OS_PrioGetHighest();
    /* 当前任务的任务优先级设置为最高的就绪态任务优先级
     * 因为接下来就要运行这个任务优先级最高的就绪态任务了
     */
    OSPrioCur = OSPrioHighRdy;
    /* 根据任务优先级从就绪态任务链表中获取任务控制块 */
    OSTCBHighRdyPtr = OSRdyList[OSPrioHighRdy].HeadPtr;
    /* 将指向当前任务控制块的指针指向上面获取到的指针
     * 因为接下来就要运行这个任务优先级最高的就绪态任务了
     */
    OSTCBCurPtr = OSTCBHighRdyPtr;
    /* 标记系统已经启动，防止之后误操作，重复启动系统 */
    OSRunning = OS_STATE_OS_RUNNING;
    /* 启动第一个任务 */
    OSStartHighRdy();
    /* 成功启动第一个任务后，是不会返回的 */
    *p_err = OS_ERR_FATAL_RETURN;
} else {
    *p_err = OS_ERR_OS_RUNNING;
}
}
    
```

从上面的代码中可以看到，函数 `OSStart()` 进行完准备工作后，如果不存在错误，最后会通过调用函数 `OSStartHighRdy()` 进入到第一个任务中去，从此之后就不会再返回了，那么接下来看看函数 `OSStartHighRdy()` 是如何实现的。

函数 `OSStartHighRdy()` 是在文件 `os_cpu_a.asm` 中定义的一个标号，代码如下所示：

```

OSStartHighRdy
; 屏蔽中断
; 防止在切换到任务时发生中断
CPSID    I
    
```

```

; 配置 PendSV 为最低中断优先级
MOV32    R0, NVIC_SYSPRI14
MOV32    R1, NVIC_PENDSV_PRI
STRB     R1, [R0]

; 将 PSP 清零
; 因为开始任务调度本就不是一条不归路
; 因此抛弃之前程序执行过程中使用的栈
; PSP 将被用于任务栈
MOVS     R0, #0
MSR      PSP, R0

; 将 MSP 设置为 OS_CPU_ExceptStkBase
; OS_CPU_ExceptStkBase 在函数 OSInit() 中进行初始化
; 指向异常栈的栈底（按 8 字节对齐）
; MSP 将被用于异常栈
MOV32    R0, OS_CPU_ExceptStkBase
LDR      R1, [R0]
MSR      MSP, R1

; 调用任务切换钩子函数
; 任务切换钩子函数完成的事务，如下：
; 1、调用应用程序中的任务切换钩子函数
; 2、统计任务切换前任务的运行时长
; 3、测量任务的中断禁用时长
; 4、测量任务调度器的锁定时长
; 5、检查任务栈是否溢出
BL       OSTaskSwHook

; 将记录当前任务优先级的变量赋值为就绪态任务中最高的任务优先级
MOV32    R0, OSPrioCur
MOV32    R1, OSPrioHighRdy
LDRB     R2, [R1]
STRB     R2, [R0]

; 将指向当前任务控制块的指针指向就绪态任务中优先级最高的任务控制块
MOV32    R0, OSTCBCurPtr
MOV32    R1, OSTCBHighRdyPtr
LDR      R2, [R1]
STR      R2, [R0]

; 将 PSP 指向任务控制块中的第一个成员变量（StkPtr）
; StkPtr 用于指向任务栈的顶，
    
```

```

; 用于等一下使用 PSP 从任务栈中恢复出 CPU 寄存器的值
; 因为是创建后还未运行的任务，
; 因此任务栈中的值是在任务创建时 (OSTaskCreate())
; 初始化好的默认值
LDR    R0, [R2]
MSR    PSP, R0

; 参考《The Definitive Guide to ARM Cortex-M3 and
; ARM Cortex-M4 Processors》第 4.2.3 小节的 CONTROL register 章节
; CONTROL[2] (SPSEL) = 1    ; Thread 模式使用 PSP
; CONTROL[3] (FPCA) = 0    ; 不使用 FPU (仅对 Cortex-M4 内核有效, Cortex-M3 没有 FPU)
MRS    R0, CONTROL
ORR    R0, R0, #2
BIC    R0, R0, #4
MSR    CONTROL, R0
ISB

; 从任务栈中出栈得到 CPU 寄存器的值
; 这里要结合函数 OSTaskCreate() 创建任务时对任务栈的初始化过程来看
; 这里重点留意 R1 寄存器会从任务栈中得到任务入口函数的地址
LDMFD  SP!, {R4-R11, LR}
LDMFD  SP!, {R0-R3}
LDMFD  SP!, {R12, LR}
LDMFD  SP!, {R1, R2}

; 开中断
CPSIE  I

; 跳转到 R1 处运行，
; 此时 R1 为从任务栈中得到的任务上次执行到的位置，
; 如果任务是第一次运行，
; R1 就指向了任务函数的入口，
; 因此这里就跳转到了系统中优先级最高的任务的任务函数中去运行了
BX     R1
    
```

在执行完函数 `OSStartHighRdy()` 后，就跳转到第一个任务中去执行并开始任务调度了，至此，μC/OS-III 的启动流程也就结束了。

第八章 μC/OS-III 任务切换

RTOS 的核心是任务管理，而任务切换是任务管理的重中之重，系统中任务切换的过程是决定操作系统运行效率和稳定性的因素之一，尤其是对于实时操作系统。因此，对于深入了解和学习 μC/OS-III，μC/OS-III 的任务切换就是必须要掌握的一个知识点。本章就来学习 μC/OS-III 的任务切换。

本章分为如下几部分：

- 8.1 PendSV 异常
- 8.2 μC/OS-III 何时触发任务切换
- 8.3 μC/OS-III 如何触发任务切换
- 8.4 μC/OS-III 如何进行任务切换

8.1 PendSV 异常

PendSV (Pended Service Call, 可挂起服务调用) 异常，是一个对 RTOS 非常重要的异常。正如 PendSV 异常的名称一样，PendSV 异常是一个可以挂起的异常，这主要体现在 PendSV 异常能够被挂起而延迟执行，那么 RTOS 就能够利用 PendSV 异常的这一特点，等待 CPU 处理完其他重要的事务后，再处理 PendSV 异常。

利用 PendSV 异常的可挂起特点，μC/OS-III 将 PendSV 的中断优先级配置为最低的中断优先级，这么一来，PendSV 异常的中断服务函数就会在其他所有中断处理完成后才被执行。μC/OS-III 就是将任务切换的过程放到 PendSV 异常的中断服务函数中处理的。

要挂起 PendSV 异常（触发 PendSV 异常）也非常简单，只需将 ICSR 寄存器（中断控制状态寄存器）中断的 PENDSVSET 为置 1，即可挂起 PendSV 异常，再挂起 PendSV 异常后，PendSV 的中断服务函数并不会立马被执行，但也不会被忽略，而是会等 CPU 处理完所有中断优先级不小于 PendSV 异常中断优先级的中断后，在再处理 PendSV 异常的中断处理函数。

8.2 μC/OS-III 何时触发任务切换

μC/OS-III 要触发任务切换无需在应用程序代码中做任务特殊的处理，因为 μC/OS-III 是一个抢占式的内核，系统会保证当前执行的任务，一定是系统中任务优先级最高的就绪态任务，因此 μC/OS-III 会在一定的条件下自动触发任务切换，μC/OS-III 自动触发任务切换的条件，如下表所示：

条件
任务向另外一个任务发送信号或消息
任务调用延时函数（函数 OSTimeDly()或函数 OSTimeDlyHMSM()）
任务等待一个还未发生的事件发生
任务的挂起状态被终止
任务被创建
任务被删除
内核对象被删除
任务修改了自身或其他任务的任务优先级
任务调用函数 OSTaskSuspend()挂起自身
任务调用函数 OSTaskResume()恢复其他被函数 OSTaskSuspend()挂起的任务

中断嵌套结束
使用函数 OSSchedUnblock()解锁任务调度器
任务调用函数 OSSchedRoundRobinYield()弃用剩余的时间片
在应用程序中调用函数 OSSched()

表 8.2.1 μC/OS-III 自动触发任务切换条件表

当上表中的条件触发时，μC/OS-III 就会自动地触发任务切换。当然，要注意的是，触发任务切换并不一定会切换任务，而仅仅是保证系统运行的任务为系统中任务优先级最高的就绪态任务。

8.3 μC/OS-III 如何触发任务切换

在表 8.2.1 中的条件发生后，μC/OS-III 就会自动地触发任务切换。在 μC/OS-III 中有两个用于触发任务切换的函数，分别为函数 OSSched()和函数 OSIntExit()，这两个函数的不同在于，函数 OSSched()是在任务中使用的，而函数 OSIntExit()是用于中断中的，下面分别来看一下这两个函数。

8.3.1 函数 OSSched()

函数 OSSched()用于在任务中触发任务切换，一般情况下，该函数会在满足表 8.2.1 中的条件（除中断嵌套结束和在应用程序中调用函数 OSSched()）时，被 μC/OS-III 自动调用。当然，应用程序也可以在任务中调用 OSSched()，手动触发任务切换。

函数 OSSched()定义在文件 os_core.c 中，具体的代码如下所示：

```
void OSSched(void)
{
    CPU_SR_ALLOC();

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        /* 如果 μC/OS-III 系统还未运行，
         * 那么是不能进行任务切换的
         */
        if (OSRunning != OS_STATE_OS_RUNNING) {
            return;
        }
    #endif

    /* 通过中断嵌套计数器
     * 判断是否在中断中调用函数 OSSched()，
     * 函数 OSSched() 只能在任务中被调用
     */
    if (OSIntNestingCtr > 0u) {
        return;
    }

    /* 通过任务调度器锁定嵌套计数器
```

```

    * 判断任务调度器是否被锁定，
    * 如果任务调度器被锁定，
    * 那么是不能进行任务切换的
    */
    if (OSSchedLockNestingCtr > 0u) {
        return;
    }

    /* 屏蔽中断， 相当于进入临界区 */
    CPU_INT_DIS();

    /* 获取就绪态任务中最高的任务优先级
    * 就绪态任务的任务优先级以位图的当时记录在数组 OSPrioTbl 中
    * 通过计算前导零的方式就能够非常方便地获取就绪态任务中最高的任务优先级
    */
    OSPrioHighRdy = OS_PrioGetHighest();

    /* 此宏用于使能空闲任务 */
    #if (OS_CFG_TASK_IDLE_EN > 0u)
        /* 根据任务优先级从就绪态任务链表中获取任务控制块 */
        OSTCBHighRdyPtr = OSRdyList[OSPrioHighRdy].HeadPtr;

        /* 如果获取到的任务任务控制块就是当前任务的任务控制块，
        * 那么就无需进行任务切换，
        * 于是直接恢复中断状态，并返回
        */
        if (OSTCBHighRdyPtr == OSTCBCurPtr) {
            CPU_INT_EN();
            return;
        }
    #else
        /* 如果没有使能空闲任务，
        * 但是获取到的任务优先级为 OS_CFG_PRIO_MAX-1，
        * 此任务优先级是分配给空闲任务的，
        * 其他任务是不能使用该任务优先级的，
        * 因为空闲任务没有启用，
        * 因此该任务优先级就不应该有对应的就绪态任务，
        * 因此就无需进行任务切换
        */
        if (OSPrioHighRdy != (OS_CFG_PRIO_MAX - 1u)) {
            OSTCBHighRdyPtr = OSRdyList[OSPrioHighRdy].HeadPtr;
            if (OSTCBHighRdyPtr == OSTCBCurPtr) {
                CPU_INT_EN();
                return;
            }
        }
    #endif
}

```

```

#endif

/* 用于调试，不用理会 */
OS_TRACE_TASK_PREEMPT(OSTCBCurPtr);

/* 此宏用于使能任务分析功能 */
#if (OS_CFG_TASK_PROFILE_EN > 0u)
/* 更新任务的执行次数，
 * 后面马上就要切换到该任务了
 */
OSTCBHighRdyPtr->CtxSwCtr++;
#endif

/* OS_CFG_TASK_PROFILE_EN: 用于使能任务分析功能
 * OS_CFG_DBG_EN: 用于使能代码调试功能
 */
#if ((OS_CFG_TASK_PROFILE_EN > 0u) || (OS_CFG_DBG_EN > 0u))
/* 更新系统执行任务切换的次数
 * 后面马上要进行任务切换了
 */
OSTaskCtxSwCtr++;
#endif

/* 此宏用于定义任务本地存储寄存器的数量 */
#if defined(OS_CFG_TLS_TBL_SIZE) && (OS_CFG_TLS_TBL_SIZE > 0u)
/* 更新系统中使用的 TLS 指针，
 * 切换任务后，
 * TLS 指针应指向任务切换后的任务的任务本地存储寄存器
 */
OS_TLS_TaskSw();
#endif

/* 此宏用于使能空闲任务 */
#if (OS_CFG_TASK_IDLE_EN > 0u)
/* 执行任务切换
 * 实际上仅仅是挂起 PendSV 异常，
 * 当处理完其他中断后，
 * 会自动进入 PendSV 异常的中断服务函数，进行任务切换
 */
OS_TASK_SW();
/* 恢复中断状态，相当于对出临界区 */
CPU_INT_EN();
#else

```

```

/* 如果未使能空闲任务，
 * 那就需要判断任务优先级是否为 OS_CFG_PRIO_MAX-1，
 * 该任务优先级使能由空闲任务使用，
 * 因为未使能空闲任务，
 * 因此该任务优先级就不应该有对应的空闲任务
 * 因此就无需进行任务切换
 */

if ((OSPrioHighRdy != (OS_CFG_PRIO_MAX - 1u))) {
    OS_TASK_SW();
    CPU_INT_EN();
} else {
    /* 无需进行任务切换，
     * 因此重新赋值 OSTCBHighRdyPtr
     */

    OSTCBHighRdyPtr = OSTCBCurPtr;
    CPU_INT_EN();

    /* 下面这个 for 循环实际上就是在未使能空闲任务的情况下，
     * 完成空闲任务应该完成的事务
     */

    for (;;) {
#if (OS_CFG_DBG_EN > 0u) || (OS_CFG_STAT_TASK_EN > 0u)
        CPU_CRITICAL_ENTER();
#endif
#if (OS_CFG_DBG_EN > 0u)
        OSIdleTaskCtr++;
#endif
#if (OS_CFG_STAT_TASK_EN > 0u)
        OSStatTaskCtr++;
#endif

        CPU_CRITICAL_EXIT();
    }

    /* 任务会一直替空闲任务完成事务，
     * 直到系统中有其他就绪态任务使得 CPU 不再“空闲”
     */

    if ((*((volatile OS_PRIO *)&OSPrioHighRdy) != (OS_CFG_PRIO_MAX-1u))) {
        break;
    }
}

}

#endif

```

```

    /* 进行指令同步 */
#ifdef OS_TASK_SW_SYNC
    OS_TASK_SW_SYNC();
#endif
}

```

8.3.2 函数 OSIntExit()

函数 OSIntExit()用于告诉 μC/OS-III 一个中断的中断服务函数执行完毕，并在需要的时候进行任务切换。

函数 OSIntExit()定义在文件 os_core.c 中，具体的代码如下所示：

```

void OSIntExit(void)
{
#ifdef OS_CFG_TASK_STK_REDZONE_EN > 0u
    CPU_BOOLEAN stk_status;
#endif

    CPU_SR_ALLOC();

    /* 如果 μC/OS-III 系统还未运行，
     * 那么是不需要进行中断嵌套计数
     * 和任务切换的
     */
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_ISR_EXIT();
        return;
    }

    /* 屏蔽中断，相当于进入临界区 */
    CPU_INT_DIS();

    /* 在调用函数 OSIntExit()前，应调用过函数 OSIntEnter()
     * 因此如果此时中断嵌套计数器为 0 的话，
     * 则返回
     */
    if (OSIntNestingCtr == 0u) {
        OS_TRACE_ISR_EXIT();
        CPU_INT_EN();
        return;
    }

    /* 更新中断嵌套计数器 */
    OSIntNestingCtr--;

    /* 如果中断嵌套计数器还是大于 0 的话，

```

```

    * 说明当前还处于中断状态，
    * 因此直接返回，无需进行任务切换
    */

if (OSIntNestingCtr > 0u) {
    OS_TRACE_ISR_EXIT();
    CPU_INT_EN();
    return;
}

/* 如果任务调度器锁定嵌套计数器大于 0 的话，
    * 说明当前任务调度器处于锁定状态，
    * 因此直接返回，无需进行任务切换
    */

if (OSSchedLockNestingCtr > 0u) {
    OS_TRACE_ISR_EXIT();
    CPU_INT_EN();
    return;
}

/* 对异常栈进行 RedZone 检查，
    * 主要判断异常栈是否溢出
    */

#if (OS_CFG_ISR_STK_SIZE > 0u)
#if (OS_CFG_TASK_STK_REDZONE_EN > 0u)
    stk_status = OS_TaskStkRedzoneChk(OSCfg_ISRStkBasePtr, OSCfg_ISRStkSize);
    if (stk_status != OS_TRUE) {
        OSRedzoneHitHook((OS_TCB *)0);
    }
#endif
#endif

/* 获取就绪态任务中最高的任务优先级
    * 就绪态任务的任务优先级以位图的当时记录在数组 OSPrioTbl 中
    * 通过计算前导零的方式就能够非常方便地获取就绪态任务中最高的任务优先级
    */

OSPrioHighRdy = OS_PrioGetHighest();
/* 此宏用于使能空闲任务 */

#if (OS_CFG_TASK_IDLE_EN > 0u)
/* 根据任务优先级从就绪态任务链表中获取任务控制块 */
OSTCBHighRdyPtr = OSRdyList[OSPrioHighRdy].HeadPtr;
/* 如果获取到的任务任务控制块就是当前任务的任务控制块，
    * 那么就无需进行任务切换，
    * 于是在检查完任务栈后，恢复中断状态，并返回

```

```

    */
    if (OSTCBHighRdyPtr == OSTCBCurPtr) {
        /* 对当前任务的任务栈进行 RedZone 检查,
        * 主要判断任务栈是否溢出
        */
#ifdef OS_CFG_TASK_STK_REDZONE_EN > 0u
        stk_status = OSTaskStkRedzoneChk((OS_TCB *)0);
        if (stk_status != OS_TRUE) {
            OSRedzoneHitHook(OSTCBCurPtr);
        }
#endif

        OS_TRACE_ISR_EXIT();
        CPU_INT_EN();
        OS_TRACE_TASK_SWITCHED_IN(OSTCBHighRdyPtr);
        return;
    }
#else
    /* 如果没有使能空闲任务,
    * 但是获取到的任务优先级为 OS_CFG_PRIO_MAX-1,
    * 此任务优先级是分配给空闲任务的,
    * 其他任务是不能使用该任务优先级的,
    * 因为空闲任务没有启用,
    * 因此该任务优先级就不应该有对应的就绪态任务,
    * 因此就无需进行任务切换
    */
    if (OSPrioHighRdy != (OS_CFG_PRIO_MAX - 1u)) {
        OSTCBHighRdyPtr = OSRdyList[OSPrioHighRdy].HeadPtr;
        if (OSTCBHighRdyPtr == OSTCBCurPtr) {
            OS_TRACE_ISR_EXIT();
            CPU_INT_EN();
            OS_TRACE_TASK_SWITCHED_IN(OSTCBHighRdyPtr);
            return;
        }
    }
#endif

    /* 此宏用于使能任务分析功能 */
#ifdef OS_CFG_TASK_PROFILE_EN > 0u
    /* 更新任务的执行次数,
    * 后面马上就要切换到该任务了
    */
    OSTCBHighRdyPtr->CtxSwCtr++;
#endif

```

```

/* OS_CFG_TASK_PROFILE_EN: 用于使能任务分析功能
 * OS_CFG_DBG_EN: 用于使能代码调试功能
 */
#if ((OS_CFG_TASK_PROFILE_EN > 0u) || (OS_CFG_DBG_EN > 0u))
/* 更新系统执行任务切换的次数
 * 后面马上要进行任务切换了
 */
OSTaskCtxSwCtr++;
#endif

/* 此宏用于定义任务本地存储寄存器的数量 */
#if defined(OS_CFG_TLS_TBL_SIZE) && (OS_CFG_TLS_TBL_SIZE > 0u)
/* 更新系统中使用的 TLS 指针,
 * 切换任务后,
 * TLS 指针应指向任务切换后的任务的任务本地存储寄存器
 */
OS_TLS_TaskSw();
#endif

/* 用于调试, 不用理会 */
OS_TRACE_ISR_EXIT_TO_SCHEDULER();

/* 执行任务切换
 * 实际上仅仅是挂起 PendSV 异常,
 * 当处理完其他中断后,
 * 会自动进入 PendSV 异常的中断服务函数, 进行任务切换
 */
OSIntCtxSw();

/* 恢复中断状态, 相当于退出临界区 */
CPU_INT_EN();
}

```

8.3.3 挂起 PendSV 异常

通过以上两小节对函数 OSSched()和函数 OSIntExit()的分析, 知道了函数 OSSched()和函数 OSIntExit()分别调用了函数 OS_TASK_SW()和函数 OSIntCtxSw()挂起 PendSV 异常, 以触发任务切换。

首先看一下函数 OS_TASK_SW(), 此函数实际上是一个定义在文件 os_cpu.h 中的宏定义, 具体的定义如下所示:

```
#define OS_TASK_SW()    OSCtxSw()
```

可以看到, 函数 OS_TASK_SW()被定义为 OSCtxSw(), 而函数 OSCtxSw()与函数 OSIntCtxSw()是被定义在文件 os_cpu_a.asm 中用于表示同一段汇编代码的标号, 如下所示:


```

NVIC_INT_CTRL    EQU 0xE00ED04 ; 中断控制状态寄存器的地址
NVIC_PENDSVSET   EQU 0x10000000 ; PENDSVSET 位掩码
    
```

```

OSCtxSw
OSIntCtxSw
; 将 ICSR 寄存器中的 PENDSVSET 位置 1
; 挂起 PendSV 异常
LDR R0, =NVIC_INT_CTRL
LDR R1, =NVIC_PENDSVSET
STR R1, [R0]
BX LR
    
```

从上面的代码中可以看出，函数 `OSCtxSw()` 与函数 `OSIntCtxSw()` 就用于挂起 PendSV 异常的，因为 PendSV 异常的中断优先级被设置为最低的中断优先级，因此在所有中断的中断服务函数处理完成后，就会跳转到 PendSV 异常的中断服务函数中，在 PendSV 的中断服务函数中就进行了任务切换的操作。

8.4 μC/OS-III 如何进行任务切换

通过上一小节分析，了解了在函数 `OSSched()` 和函数 `OSIntExit()` 中会挂起 PendSV 异常，那么 PendSV 异常就能够在所有中断的中断服务函数处理完成后，执行任务切换的操作。

μC/OS-III 提供了 PendSV 异常的中断服务函数，以完成任务切换的操作，该函数为定义在文件 `os_cpu_a.asm` 中的标号 `OS_CPU_PendSVHandler`，具体的代码如下所示：

```

OS_CPU_PendSVHandler
; 屏蔽受 uC/OS-III 管理的中断
; 此处加上开关中断的操作，
; 是为了解决 Cortex-M7，
; 写入 BASEPRI 后不能即时生效的 Bug
CPSID        I
MOV32        R2, OS_KA_BASEPRI_Boundary
LDR          R1, [R2]
MSR          BASEPRI, R1
DSB
ISB
CPSIE        I

; 获取此时的 PSP
; 此时 PSP 为当前任务的任务的栈顶指针
; R0 = PSP
MRS          R0, PSP

; 判断是否使能 FPU
IF {FPU} != "SoftVFP"

; 如果使能了 FPU
    
```

```

; 还需要判断 R14 的 bit4 是否为 1
; 若是，则需要将浮点寄存器中的数据保存到任务栈中
TST          R14, #0x10
IT           EQ
VSTMDBEQ     R0!, {S16-S31}
ENDIF

; 将部分 CPU 寄存器的值模拟入栈到当前任务栈中
; 另外，部分的 CPU 寄存器的值会在进入中断之前，
; 自动入栈到任务栈中，
; 其中就包含了进入中断前，任务执行到的位置（PC 值）
STMFD        R0!, {R4-R11, R14}

; 更新当前任务的任务栈顶指针
; OSTCBCurPtr->StkPtr = R0
MOV32         R5, OSTCBCurPtr
LDR           R1, [R5]
STR           R0, [R1]

; 调用任务切换钩子函数，
; 在调用之前保存 LR 寄存器的值
; R4 = LR
; OSTaskSwHook()
MOV           R4, LR
BL            OSTaskSwHook

; 设置 OSPrioCur 为最高就绪态任务优先级
; OSPrioCur = OSPrioHighRdy
MOV32         R0, OSPrioCur
MOV32         R1, OSPrioHighRdy
LDRB          R2, [R1]
STRB          R2, [R0]

; 设置 OSTCBCurPtr 为最高就绪态任务
; OSTCBCurPtr = OSTCBHighRdyPtr
MOV32         R1, OSTCBHighRdyPtr
LDR           R2, [R1]
STR           R2, [R5]

; 参考《The Definitive Guide to ARM Cortex-M3 and ARM Cortex-M4 Processors》第
8.1.4 小节的 EXC_RETURN 章节
; LR |= 0x2 ; 确保返回任务后使用 PSP
ORR           LR, R4, #0x04

```

```

; 获取任务切换后任务的栈顶
; R0 = OSTCBHighRdyPtr->StkPtr
LDR            R0, [R2]

; 从任务栈中模拟出栈 CPU 寄存器的值到 CPU 寄存器中
; 另外，部分的 CPU 寄存器的值会在退出中断之后，
; 自动从任务栈中出栈到 CPU 寄存器中
; 其中就包含了任务上次被切换时，执行到的位置（PC 值）
LDMFD         R0!, {R4-R11, R14}

; 判断是否使能 FPU
IF {FPU} != "SoftVFP"

; 如果使能了 FPU
; 还需要判断 R14 的 bit4 是否为 1
; 若是，则需要从任务栈中恢复初浮点寄存器中的数据
TST           R14, #0x10
IT            EQ
VLDmiaEQ      R0!, {S16-S31}
ENDIF

; 更新 PSP 指针为此时任务切换后任务的任务栈顶
; 这个 PSP 为任务栈，在退出中断后，
; 会自动从这个栈中出栈 CPU 寄存器的值，
; 其中就包含了任务在上次被切换时，执行到的位置（PC 值）
; 那么在退出中断后，
; 任务就可以从上次被切换时执行到的位置继续执行
; PSP = R0
MSR           PSP, R0

; 取消中断屏蔽
; 此处加上开关中断的操作，
; 是为了解决 Cortex-M7，
; 写入 BASEPRI 后不能即时生效的 Bug
MOV32        R2, #0
CPSID        I
MSR           BASEPRI, R2
DSB
ISB
CPSIE        I

; 退出中断
    
```

; 此时就会根据 PC 指针的值跳转到任务切换后的任务中去执行

BX LR

从上面的代码中可以看出，任务切换的核心就是任务栈中数据的入栈与出栈。入栈操作能够保存当前任务在任务切换前的上下文信息，而出栈操作则是将切换后任务的上下文信息从任务的任务栈总恢复出来，这样任务才能够正常运行。

第九章 μC/OS-III 其他任务相关 API 函数

通过前面几个章节的学习，了解了 μC/OS-III 的一些基础知识，其中任务管理的相关 API 函数也仅是涉及了任务的创建、删除、挂起和恢复等几个任务相关的 API 函数。除此之外，μC/OS-III 还提供了很多与任务管理相关的 API 函数，通过这些函数，用户可以更加灵活地使用 μC/OS-III。本章就来学习 μC/OS-III 中其他与任务管理相关的 API 函数。

本章分为如下几部分：

- 9.1 μC/OS-III 任务相关 API 函数
- 9.2 μC/OS-III 任务优先级修改实验
- 9.3 μC/OS-III 时间片调度实验
- 9.4 μC/OS-III 空闲任务钩子函数实验
- 9.5 μC/OS-III 统计功能实验

9.1 μC/OS-III 任务相关 API 函数

9.1.1 μC/OS-III 任务相关 API 函数预览

在 μC/OS-III 的官方在线文档中，通过导航栏的索引找到“μC/OS-III API Reference”，这里面包含 μC/OS-III 官方对 μC/OS-III 中 API 函数的详细介绍，点开其中的“API – Task Management”，能够看到在线文档列出的 μC/OS-III 中与任务管理相关的 API 函数，如下图所示：

▼ API - Task Management	• OSTaskDel	• OSTaskReturnHook
• OSSchedRoundRobinCfg	• OSTaskDelHook	• OSTaskStkChk
• OSSchedRoundRobinYield	• OSTaskRegGet	• OSTaskStkInit
• OSTaskChangePrio	• OSTaskRegGetID	• OSTaskSuspend
• OSTaskCreate	• OSTaskRegSet	• OSTaskSwHook
• OSTaskCreateHook	• OSTaskResume	• OSTaskTimeQuantaSet

图 9.1.1 μC/OS-III 任务管理相关 API 函数

上图中 μC/OS-III 任务管理相关的 API 函数描述，如下表所示：

函数	描述
OSSchedRoundRobinCfg()	配置时间片调度功能
OSSchedRoundRobinYield()	在时间片到期之前，强制进行任务调度
OSTaskChangePrio()	修改任务优先级
OSTaskCreate()	创建任务
OSTaskCreateHook()	任务创建钩子函数
OSTaskDel()	删除任务
OSTaskDelHook()	任务删除钩子函数

OSTaskRegGet()	获取任务本地存储寄存器的值
OSTaskRegGetID()	获取当前任务本地存储寄存器的索引值
OSTaskRegSet()	修改任务本地存储寄存器的值
OSTaskResume()	恢复任务
OSTaskReturnHook()	任务意外返回钩子函数
OSTaskStkChk()	计算任务栈余量
OSTaskStkInit()	初始化任务栈
OSTaskSuspend()	挂起任务
OSTaskSwHook()	任务切换钩子函数
OSTaskTimeQuantaSet()	修改任务时间片

表 9.1.1 μC/OS-III 任务管理相关 API 函数描述

接着再点开导航栏“μC/OS-III API Reference”下的“API - Miscellaneous”，这里面抱恨了很多杂项 API 函数，这些 API 函数能够帮助用于更快速地完善自己的工程，本文就以杂项函数中比较常用的两个 API 函数为例，为读者介绍，如下表所示：

函数	描述
OSIdleTaskHook()	空闲任务钩子函数
OSSStatTaskCPUUsageInit()	初始化 CPU 使用率统计功能

表 9.1.2 μC/OS-III 部分杂项 API 函数描述

9.1.2 μC/OS-III 相关 API 函数详解

1. 函数 OSSchedRoundRobinCfg()

该函数用于配置时间片调度功能，配置内容为是否开启时间片调度以及时间片的默认长度。

函数 OSSchedRoundRobinCfg() 的函数原型，如下所示：

```
void OSSchedRoundRobinCfg( CPU_BOOLEAN en,
                           OS_TICK      dflt_time_quanta,
                           OS_ERR*      p_err)
```

函数 OSSchedRoundRobinCfg() 的形参描述，如下表所示：

形参	描述
en	是否使能时间片调度
dflt_time_quanta	默认的时间片长度
p_err	指向接收错误代码变量的指针

表 9.1.3 函数 OSSchedRoundRobinCfg() 形参相关描述

函数 OSSchedRoundRobinCfg() 的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.4 函数 OSSchedRoundRobinCfg() 返回值相关描述

函数 OSSchedRoundRobinCfg() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	时间片功能配置成功

表 9.1.5 函数 OSSchedRoundRobinCfg() 错误代码相关描述

2. 函数 OSSchedRoundRobinYield()

该函数用于在任务时间片到期前，强制进行任务调度。

函数 OSSchedRoundRobinYield()的函数原型，如下所示：

```
void OSSchedRoundRobinYield(OS_ERR* p_err)
```

函数 OSSchedRoundRobinYield()的形参描述，如下表所示：

形参	描述
p_err	指向接收错误代码变量的指针

表 9.1.6 函数 OSSchedRoundRobinYield()形参相关描述

函数 OSSchedRoundRobinYield()的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.7 函数 OSSchedRoundRobinYield()返回值相关描述

函数 OSSchedRoundRobinYield()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	函数调用成功
OS_ERR_ROUND_ROBIN_1	不存在同优先级的任务，没有发生任务调度
OS_ERR_ROUND_ROBIN_DISABLED	未使能时间片调度功能
OS_ERR_SCHED_LOCKED	任务调度器已锁定
OS_ERR_YIELD_ISR	在中断中非法调用该函数

表 9.1.8 函数 OSSchedRoundRobinYield()错误代码相关描述

3. 函数 OSTaskChangePrio()

该函数用于修改任务的任务优先级。

函数 OSTaskChangePrio()的函数原型，如下所示：

```
void OSTaskChangePrio(OS_TCB* p_tcb,  
                      OS_PRIO prio_new,  
                      OS_ERR* p_err)
```

函数 OSTaskChangePrio()的形参描述，如下表所示：

形参	描述
p_tcb	指向待修改任务优先级的任务控制块的指针
prio_new	新的任务优先级
p_err	指向接收错误代码变量的指针

表 9.1.9 函数 OSTaskChangePrio()形参相关描述

函数 OSTaskChangePrio()的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.10 函数 OSTaskChangePrio()返回值相关描述

函数 OSTaskChangePrio()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	任务优先级修改成功
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PRIO_INVALID	给定了非法的任务优先级实参
OS_ERR_STATE_INVALID	任务处于无效状态
OS_ERR_TASK_CHANGE_PRIO_ISR	在中断中非法修改任务优先级

表 9.1.11 函数 OSTaskChangePrio()错误代码相关描述

4. 函数 OSTaskCreate()

见第 6.1.1 小节《函数 OSTaskCreate()》。

5. 函数 OSTaskCreateHook()

该函数为任务创建的钩子函数，会在新创建的任务被添加到就绪态任务链表前调用。

函数 OSTaskCreateHook() 的函数原型，如下所示：

```
void OSTaskCreateHook(OS_TCB* p_tcb)
```

函数 OSTaskCreateHook() 的形参描述，如下表所示：

形参	描述
p_tcb	指向新创建任务的任务控制块的指针

表 9.1.12 函数 OSTaskCreateHook() 形参相关描述

函数 OSTaskCreateHook() 的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.13 函数 OSTaskCreateHook() 返回值相关描述

函数 OSTaskCreateHook() 的错误代码描述，如下表所示：

错误代码	描述
无	无

表 9.1.14 函数 OSTaskCreateHook() 错误代码相关描述

6. 函数 OSTaskDel()

见第 6.1.2 小节《函数 OSTaskDel()》。

7. 函数 OSTaskDelHook()

该函数为任务删除的钩子函数，会在被删除任务从所在任务链表中删除后调用。

函数 OSTaskDelHook() 的函数原型，如下所示：

```
void OSTaskDelHook(OS_TCB* p_tcb)
```

函数 OSTaskDelHook() 的形参描述，如下表所示：

形参	描述
p_tcb	指向被删除任务的任务控制块的指针

表 9.1.15 函数 OSTaskDelHook() 形参相关描述

函数 OSTaskDelHook() 的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.16 函数 OSTaskDelHook() 返回值相关描述

函数 OSTaskDelHook() 的错误代码描述，如下表所示：

错误代码	描述
无	无

表 9.1.17 函数 OSTaskDelHook() 错误代码相关描述

8. 函数 OSTaskRegGet()

该函数用于获取指定任务本地寄存器寄存器中的值。

函数 OSTaskRegGet() 的函数原型，如下所示：

```
OS_REG OSTaskRegGet(OS_TCB* p_tcb,
                    OS_REG_ID id,
                    OS_ERR* p_err)
```

函数 OSTaskRegGet() 的形参描述，如下表所示：

形参	描述
p_tcb	指向任务的任务控制块的指针
id	任务本地存储寄存器的索引号
p_err	指向接收错误代码变量的指针

表 9.1.18 函数 OSTaskRegGet()形参相关描述

函数 OSTaskRegGet()的返回值描述，如下表所示：

返回值	描述
OS_REG 类型值	任务本地存储寄存器的值

表 9.1.19 函数 OSTaskRegGet()返回值相关描述

函数 OSTaskRegGet()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	函数调用成功
OS_ERR_REG_ID_INVALID	索引值无效

表 9.1.20 函数 OSTaskRegGet()错误代码相关描述

9. 函数 OSTaskRegGetID()

该函数用于获取任务本地存储寄存器的索引号。

函数 OSTaskRegGetID()的函数原型，如下所示：

```
OS_REG_ID OSTaskRegGetID(OS_ERR* p_err)
```

函数 OSTaskRegGetID()的形参描述，如下表所示：

形参	描述
p_err	指向接收错误代码变量的指针

表 9.1.21 函数 OSTaskRegGetID()形参相关描述

函数 OSTaskRegGetID()的返回值描述，如下表所示：

返回值	描述
OS_REG_ID 类型值	任务本地存储寄存器的索引号

表 9.1.22 函数 OSTaskRegGetID()返回值相关描述

函数 OSTaskRegGetID()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	函数调用成功
OS_ERR_NO_MORE_ID_AVAIL	索引号无效

表 9.1.23 函数 OSTaskRegGetID()错误代码相关描述

10. 函数 OSTaskRegSet()

该函数用于设置任务本地存储寄存器的值。

函数 OSTaskRegSet()的函数原型，如下所示：

```
void OSTaskRegSet(OS_TCB* p_tcb,
                  OS_REG_ID id,
                  OS_REG value,
                  OS_ERR* p_err)
```

函数 OSTaskRegSet()的形参描述，如下表所示：

形参	描述
p_tcb	指向任务的任务控制块的指针
id	任务本地存储寄存器的索引号

value	待设置的值
p_err	指向接收错误代码变量的指针

表 9.1.24 函数 OSTaskRegSet()形参相关描述

函数 OSTaskRegSet()的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.25 函数 OSTaskRegSet()返回值相关描述

函数 OSTaskRegSet()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	函数调用成功
OS_ERR_REG_ID_INVALID	索引值无效

表 9.1.26 函数 OSTaskRegSet()错误代码相关描述

11. 函数 OSTaskResume()

见第 6.3.2 小节《函数 OSTaskResume()》。

12. 函数 OSTaskReturnHook()

该函数为任务意外返回钩子函数，会在任务意外返回后调用。

函数 OSTaskReturnHook()的函数原型，如下所示：

```
void OSTaskReturnHook(OS_TCB* p_tcb)
```

函数 OSTaskReturnHook()的形参描述，如下表所示：

形参	描述
p_tcb	指向任务的任务控制块的指针

表 9.1.27 函数 OSTaskReturnHook()形参相关描述

函数 OSTaskReturnHook()的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.28 函数 OSTaskReturnHook()返回值相关描述

函数 OSTaskReturnHook()的错误代码描述，如下表所示：

错误代码	描述
无	无

表 9.1.29 函数 OSTaskReturnHook()错误代码相关描述

13. 函数 OSTaskStkChk()

该函数用于计算任务的任务栈剩余量。

函数 OSTaskStkChk()的函数原型，如下所示：

```
void OSTaskStkChk(OS_TCB* p_tcb,
                  CPU_STK_SIZE* p_free,
                  CPU_STK_SIZE* p_used,
                  OS_ERR* p_err)
```

函数 OSTaskStkChk()的形参描述，如下表所示：

形参	描述
p_tcb	指向任务的任务控制块的指针
p_free	指向接收任务栈剩余量变量的指针
p_used	指向接收任务栈使用量变量的指针

p_err	指向接收错误代码变量的指针
-------	---------------

表 9.1.30 函数 OSTaskStkChk()形参相关描述

函数 OSTaskStkChk()的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.31 函数 OSTaskStkChk()返回值相关描述

函数 OSTaskStkChk()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	函数调用成功
OS_ERR_PTR_INVALID	指向任务控制块的指针无效
OS_ERR_TASK_NOT_EXIST	任务不存在
OS_ERR_TASK_OPT	任务未指定 OS_OPT_TASK_STK_CHK 选项
OS_ERR_TASK_STK_CHK_ISR	在中断中非法调用该函数

表 9.1.32 函数 OSTaskStkChk()错误代码相关描述

14. 函数 OSTaskStkInit()

该函数用于初始化任务栈，一般在任务创建的时候被调用。

函数 OSTaskStkInit()的函数原型，如下所示：

```
CPU_STK* OSTaskStkInit(OS_TASK_PTR    p_task,
                        void*          p_arg,
                        CPU_STK*       p_stk_base,
                        CPU_STK*       p_stk_limit,
                        CPU_STK_SIZE    stk_size,
                        OS_OPT          opt)
```

函数 OSTaskStkInit()的形参描述，如下表所示：

形参	描述
p_task	指向任务函数的指针
p_arg	指向任务函数参数的指针
p_stk_base	指向任务栈起始地址的指针
p_stk_limit	指向任务栈“水位”限制地址的指针
stk_size	任务栈大小
opt	任务选项

表 9.1.33 函数 OSTaskStkInit()形参相关描述

函数 OSTaskStkInit()的返回值描述，如下表所示：

返回值	描述
CPU_STK 类型指针	指向任务栈初始化后栈顶地址的指针

表 9.1.34 函数 OSTaskStkInit()返回值相关描述

函数 OSTaskStkInit()的错误代码描述，如下表所示：

错误代码	描述
无	无

表 9.1.35 函数 OSTaskStkInit()错误代码相关描述

15. 函数 OSTaskSuspend()

见第 6.3.1 小节《函数 OSTaskSuspend()》。

16. 函数 OSTaskSwHook()

该函数为任务切换的钩子函数，会在任务切换前任务对应的 CPU 寄存器保存到任务栈后调用。

函数 OSTaskSwHook() 的函数原型，如下所示：

```
void OSTaskSwHook(void)
```

函数 OSTaskSwHook() 的形参描述，如下表所示：

形参	描述
无	无

表 9.1.36 函数 OSTaskSwHook() 形参相关描述

函数 OSTaskSwHook() 的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.37 函数 OSTaskSwHook() 返回值相关描述

函数 OSTaskSwHook() 的错误代码描述，如下表所示：

错误代码	描述
无	无

表 9.1.38 函数 OSTaskSwHook() 错误代码相关描述

17. 函数 OSTaskTimeQuantaSet()

该函数用于修改任务的时间片长度。

函数 OSTaskTimeQuantaSet() 的函数原型，如下所示：

```
void OSTaskTimeQuantaSet( OS_TCB* p_tcb,
                           OS_TICK time_quanta,
                           OS_ERR* p_err)
```

函数 OSTaskTimeQuantaSet() 的形参描述，如下表所示：

形参	描述
p_tcb	指向任务函数的指针
time_quanta	时间片长度
p_err	指向接收错误代码变量的指针

表 9.1.39 函数 OSTaskTimeQuantaSet() 形参相关描述

函数 OSTaskTimeQuantaSet() 的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.40 函数 OSTaskTimeQuantaSet() 返回值相关描述

函数 OSTaskTimeQuantaSet() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	函数调用成功
OS_ERR_SET_ISR	在中断中非法调用该函数

表 9.1.41 函数 OSTaskTimeQuantaSet() 错误代码相关描述

18. 函数 OSIdleTaskHook()

该函数为空闲任务的回调函数，在此函数中绝对不能调用可能引发空闲任务挂起的 API 函数。

函数 OSIdleTaskHook() 的函数原型，如下所示：

```
void OSIdleTaskHook(void)
```

函数 OSIdleTaskHook() 的形参描述，如下表所示：

形参	描述
无	无

表 9.1.42 函数 OSIdleTaskHook()形参相关描述

函数 OSIdleTaskHook()的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.43 函数 OSIdleTaskHook()返回值相关描述

函数 OSIdleTaskHook()的错误代码描述，如下表所示：

错误代码	描述
无	无

表 9.1.44 函数 OSIdleTaskHook()错误代码相关描述

19. 函数 OSStatTaskCPUUsageInit()

该函数用于初始化 CPU 使用率统计功能，能够帮助用户优化任务的运行，在实际的开发中开始非常有用的。

函数 OSStatTaskCPUUsageInit()的函数运行，如下所示：

```
void OSStatTaskCPUUsageInit(OS_ERR* p_err)
```

函数 OSStatTaskCPUUsageInit()的形参描述，如下表所示：

形参	描述
p_err	指向接收错误代码变量的指针

表 9.1.45 函数 OSStatTaskCPUUsageInit()形参相关描述

函数 OSStatTaskCPUUsageInit()的返回值描述，如下表所示：

返回值	描述
无	无

表 9.1.46 函数 OSStatTaskCPUUsageInit()返回值相关描述

函数 OSStatTaskCPUUsageInit()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	CPU 使用率统计功能初始化成功
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行

表 9.1.47 函数 OSStatTaskCPUUsageInit()错误代码相关描述

9.2 μC/OS-III 任务优先级修改实验

9.2.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数进行任务优先级的修改，本实验设计了四个任务，这四个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建其他任务
task1	任务优先级修改对象
task2	任务优先级修改对象
task3	用于修改任务优先级

表 9.2.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 9-1 UCOS-III 任务优先级修改实验》。

9.2.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

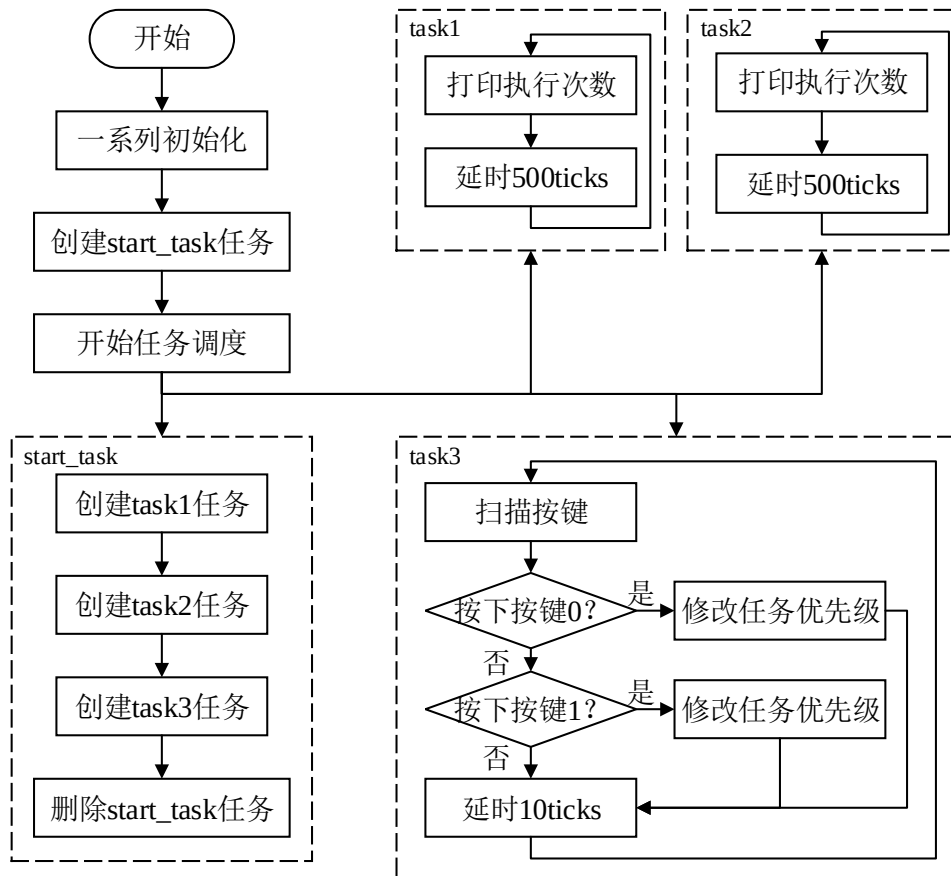


图 9.2.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSTaskChangePrio()

此函数用于修改任务优先级，请参考第 9.1 小节《μC/OS-III 任务相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_TASK_CHANGE_PRIO_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```

/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无

```

```

*/
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                    (CPU_CHAR*    ) "task1",
                    (OS_TASK_PTR  ) task1,
                    (void*        ) 0,
                    (OS_PRIO      ) TASK1_PRIO,
                    (CPU_STK*     ) Task1Task_STK,
                    (CPU_STK_SIZE ) TASK1_STK_SIZE / 10,
                    (CPU_STK_SIZE ) TASK1_STK_SIZE,
                    (OS_MSG_QTY   ) 0,
                    (OS_TICK      ) 0,
                    (void*        ) 0,
                    (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                    (OS_ERR*      ) &err);

    /* 创建 Task2 */
    Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK2_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*      ) &Task2Task_TCB,
                    (CPU_CHAR*    ) "task2",
                    (OS_TASK_PTR  ) task2,
                    (void*        ) 0,
                    (OS_PRIO      ) TASK2_PRIO,
                    (CPU_STK*     ) Task2Task_STK,
                    (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,

```

```

        (CPU_STK_SIZE) TASK2_STK_SIZE,
        (OS_MSG_QTY) 0,
        (OS_TICK) 0,
        (void*) 0,
        (OS_OPT) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*) &err);

/* 创建 Task3 */
Task3Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK3_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*) &Task3Task_TCB,
                (CPU_CHAR*) "task3",
                (OS_TASK_PTR) task3,
                (void*) 0,
                (OS_PRIO) TASK3_PRIO,
                (CPU_STK*) Task3Task_STK,
                (CPU_STK_SIZE) TASK3_STK_SIZE / 10,
                (CPU_STK_SIZE) TASK3_STK_SIZE,
                (OS_MSG_QTY) 0,
                (OS_TICK) 0,
                (void*) 0,
                (OS_OPT) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                (OS_ERR*) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务、task2 任务和 task3 任务，并且 task1 任务、task2 任务和 task3 任务的任务栈都是动态分配的，当然，也可以静态地定任务栈，就像定义一个数组一样。

(2) task1 和 task2 任务

```

/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    uint32_t task1_num = 0;
    CPU_SR_ALLOC();

    while (1)
    {

```



```

        CPU_CRITICAL_ENTER();
        printf("Task1 run %d times\r\n", ++task1_num);
        CPU_CRITICAL_EXIT();
        delay_ms(500);
    }
}

/**
 * @brief   task2
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task2(void *p_arg)
{
    uint32_t task2_num = 0;
    CPU_SR_ALLOC();

    while (1)
    {
        CPU_CRITICAL_ENTER();
        printf("Task2 run %d times\r\n", ++task2_num);
        CPU_CRITICAL_EXIT();
        delay_ms(500);
    }
}

```

从以上代码中可以看到，task1 和 task2 任务分别每间隔 500ms 就通过串口打印一次执行次数，并更新执行次数，而且延时函数使用的是 CPU 忙延时，因此不会触发任务调度。并且，在打印函数被放置于代码的临界区，这是因为，串口打印的内容应该被完整地打印出来，而不应该被其他事情打断。task1 和 task2 任务被设置了不同的任务优先级，当 task1 任务的任务优先级高于 task2 任务的任务优先级时，只有 task1 任务能够运行，反之，则只有 task2 任务能够运行。

(4) task3 任务

```

/**
 * @brief   task3
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task3(void *p_arg)
{
    uint8_t key;
    OS_ERR err;

    while (1)

```

```

{
    key = key_scan(0);

    switch (key)
    {
        case KEY0_PRES:
        {
            OSTaskChangePrio( (OS_TCB*) &Task1Task_TCB,
                               (OS_PRIO) OS_CFG_PRIO_MAX - 5,
                               (OS_ERR*) &err);

            OSTaskChangePrio( (OS_TCB*) &Task2Task_TCB,
                               (OS_PRIO) OS_CFG_PRIO_MAX - 4,
                               (OS_ERR*) &err);

            break;
        }
        case KEY1_PRES:
        {
            OSTaskChangePrio( (OS_TCB*) &Task1Task_TCB,
                               (OS_PRIO) OS_CFG_PRIO_MAX - 4,
                               (OS_ERR*) &err);

            OSTaskChangePrio( (OS_TCB*) &Task2Task_TCB,
                               (OS_PRIO) OS_CFG_PRIO_MAX - 5,
                               (OS_ERR*) &err);

            break;
        }
        default:
        {
            break;
        }
    }

    OSTimeDly(10, OS_OPT_TIME_DLY, &err);
}
}

```

从上面的代码中可以看到，task3 任务负责扫描按键，当检测到 KEY0 按键被按下时候，设置 task1 和 task2 任务的任务优先级，使得 task1 任务的任务优先级高于 task2 任务的任务优先级；当检测到 KEY1 按键被按下时，设置 task1 和 task2 任务的任务优先级，使得 task2 任务的任务优先级高于 task1 任务的任务优先级。

9.2.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：

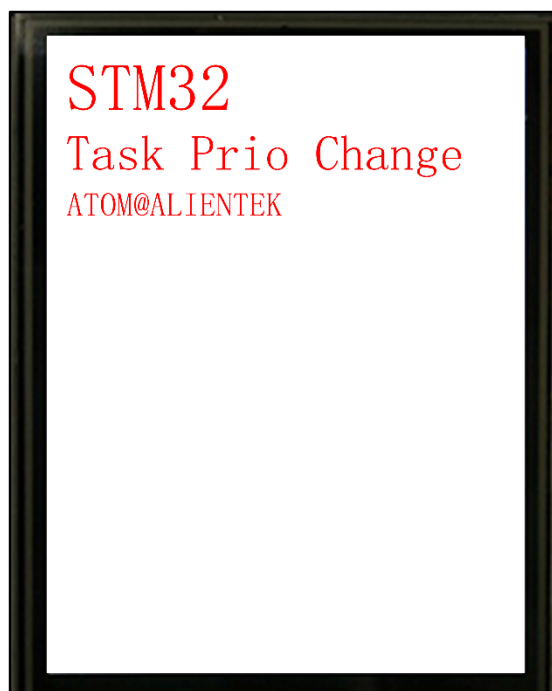


图 9.2.2 LCD 显示内容

同时，通过串口调试助手，就能够看到输出的打印信息，如下图所示：

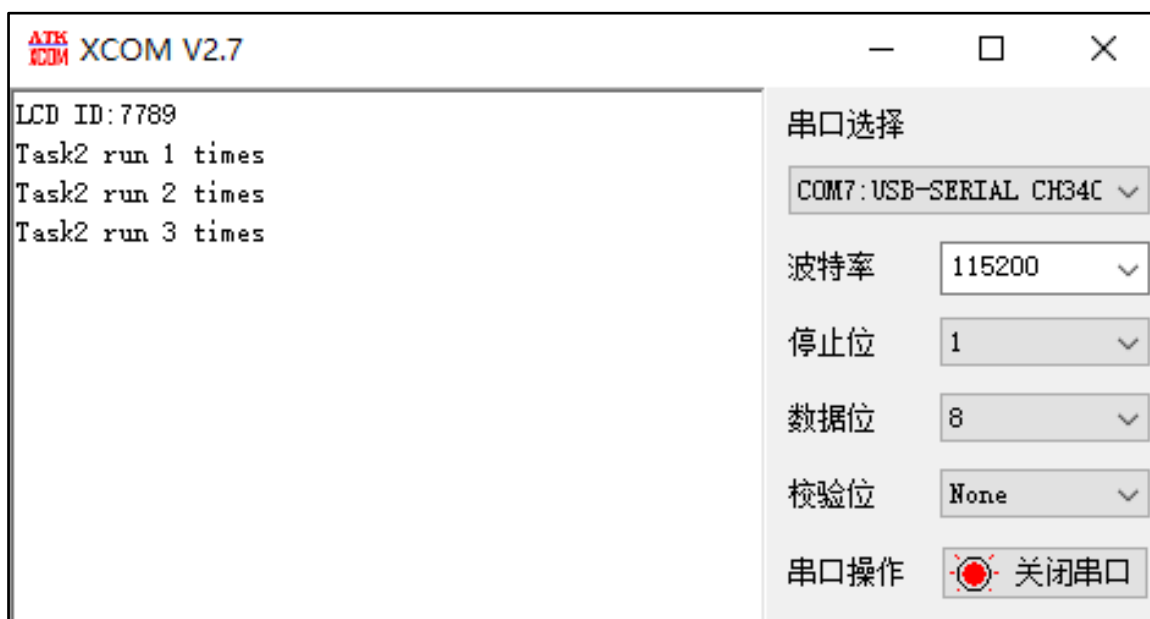


图 9.2.3 串口调试助手显示内容一

从上图中可以看出，一开始只有 task2 任务在运行，这是因为创建任务时，task2 任务的任务优先级是高于 task1 任务的任务优先级的。

接下来按下按键 0，修改 task1 和 task2 任务的任务优先级，使得 task1 任务的任务优先级高于 task2 任务的任务优先级，结果如下图所示：



图 9.2.4 串口调试助手显示内容二

从上图中可以看出，当按键 0 被按下后，成功地修改了 task1 和 task2 任务的任务优，使得 task1 任务的任务优先级高于 task2 任务的任务优先级，因此此时只有 task1 任务在运行。

此时如果再按下按键 1，再次修改 task1 和 task2 任务的任务优，使得 task2 任务的任务优先级高于 task1 任务的任务优先级，那么 task2 任务就能够重新抢占 task1 任务，而运行。

9.3 μC/OS-III 时间片调度实验

9.3.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数开启时间片调度功能，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建其他任务
task1	任务优先级修改对象
task2	任务优先级修改对象

表 9.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 9-2 UCOS-III 时间片调度实验》。

9.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

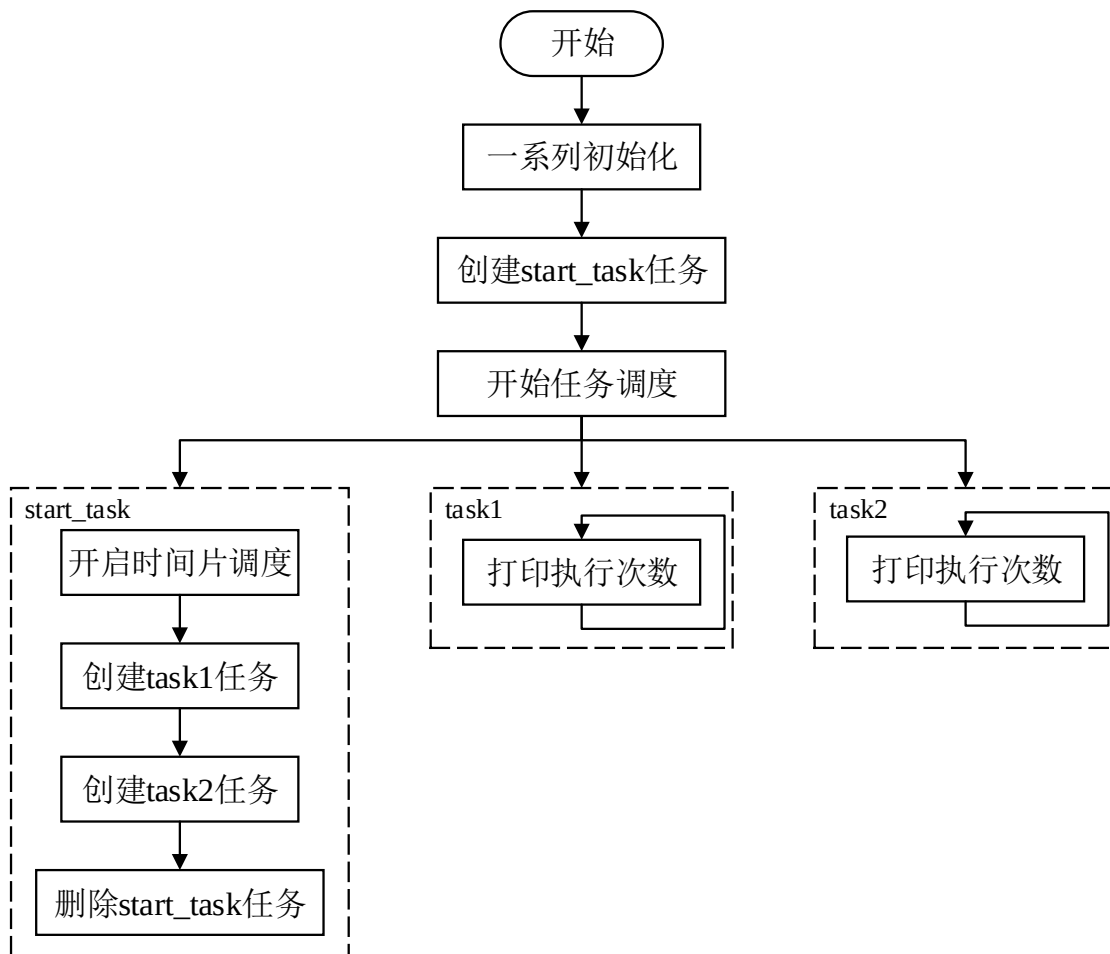


图 9.3.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSSchedRoundRobinCfg()

此函数用于配置时间片调度功能，请参考第 9.1 小节《μC/OS-III 任务相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SCHED_ROUND_ROBIN_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```

/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)
{
    OS_ERR err;

```

```
CPU_INT32U cnts;

/* 初始化 CPU 库 */
CPU_Init();

/* 根据配置的节拍频率配置 SysTick */
cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
OS_CPU_SysTickInit(cnts);

/* 开启时间片调度，时间片设为默认值 */
OSSchedRoundRobinCfg(OS_TRUE, 1, &err);

/* 创建 Task1 */
Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK1_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                  (CPU_CHAR*    ) "task1",
                  (OS_TASK_PTR  ) task1,
                  (void*        ) 0,
                  (OS_PRIO      ) TASK1_PRIO,
                  (CPU_STK*      ) Task1Task_STK,
                  (CPU_STK_SIZE ) TASK1_STK_SIZE / 10,
                  (CPU_STK_SIZE ) TASK1_STK_SIZE,
                  (OS_MSG_QTY    ) 0,
                  (OS_TICK      ) 0,
                  (void*        ) 0,
                  (OS_OPT        ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                  (OS_ERR*      ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      ) &Task2Task_TCB,
                  (CPU_CHAR*    ) "task2",
                  (OS_TASK_PTR  ) task2,
                  (void*        ) 0,
                  (OS_PRIO      ) TASK2_PRIO,
                  (CPU_STK*      ) Task2Task_STK,
                  (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,
                  (CPU_STK_SIZE ) TASK2_STK_SIZE,
                  (OS_MSG_QTY    ) 0,
                  (OS_TICK      ) 0,
                  (void*        ) 0,
```

```

        (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 和 task2 任务，并且在创建任务之前，使用函数 OSSchedRoundRobinCfg() 配置开启了时间片调度，并设置并设置默认的时间片长度为 1，因此接下来 task1 和 task2 任务将轮流运行一个时间片的时长，当然前提是 task1 和 task2 任务的任务优先级设置成一样的任务优先级。

(2) task1 和 task2 任务

```

/**
 * @brief   task1
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task1(void *p_arg)
{
    uint32_t task1_num = 0;
    CPU_SR_ALLOC();

    while (1)
    {
        CPU_CRITICAL_ENTER();
        printf("任务 1 运行次数: %d\r\n", ++task1_num);
        CPU_CRITICAL_EXIT();
    }
}

/**
 * @brief   task2
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task2(void *p_arg)
{
    uint32_t task2_num = 0;
    CPU_SR_ALLOC();

    while (1)
    {
        CPU_CRITICAL_ENTER();
        printf("任务 2 运行次数: %d\r\n", ++task2_num);
    }
}

```

```
CPU_CRITICAL_EXIT();  
  
}  
  
}
```

从上面的代码可以看出，task1 和 task2 任务都是不加延时地一直打印执行次数，因此只要通过串口调试助手，就能够直观地看出，task1 和 task2 任务的运行情况。

9.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 9.3.2 LCD 显示内容

同时，通过串口调试助手，就能够看到输出的打印信息，如下图所示：



图 9.3.3 串口调试助手显示内容

从上图中可以看出，因为配置的时间片长度只有 1，因此 task1 和 task2 任务都只执行一次就进行了任务切换，读者可以自己尝试修改本实验函数 OSSchedRoundRobinCfg() 的参数，再观察一下实验现象。

9.4 μC/OS-III 空闲任务钩子函数实验

9.4.1 功能设计

1. 例程功能

本实验主要用于演示 μC/OS-III 空闲任务钩子函数的使用，本实验直接在第二章中《μC/OS-III 移植实验》的实验工程上做修改，无需额外创建任务。

该实验的实验工程，请参考《UCOS-III 实验例程 9-3 UCOS-III 空闲任务钩子函数》。

9.4.2 软件设计

1. 程序流程图

本实验的程序流程图与第二章中《μC/OS-III 移植实验》的实验流程图一致。

2. μC/OS-III 函数解析

(1) 函数 OSIdleTaskHook()

此函数为空闲任务的回调函数，请参考第 9.1 小节《μC/OS-III 任务相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_APP_HOOKS_EN 配置为 1。

(2) 函数 App_OS_SetAllHooks()

此函数用于初始化所有 μC/OS-III 钩子函数，要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_APP_HOOKS_EN 配置为 1，此函数定义在文件 os_app_hooks.c 中，具体的代码如下所示：

```
void App_OS_SetAllHooks (void)
{
    #if OS_CFG_APP_HOOKS_EN > 0u
        CPU_SR_ALLOC();

        CPU_CRITICAL_ENTER();
        OS_AppIdleTaskHookPtr = App_OS_IdleTaskHook;

        #if (OS_CFG_TASK_STK_REDZONE_EN > 0u)
            OS_AppRedzoneHitHookPtr = App_OS_RedzoneHitHook;
        #endif

        OS_AppStatTaskHookPtr = App_OS_StatTaskHook;

        OS_AppTaskCreateHookPtr = App_OS_TaskCreateHook;
```

```

OS_AppTaskDelHookPtr    = App_OS_TaskDelHook;

OS_AppTaskReturnHookPtr = App_OS_TaskReturnHook;

OS_AppTaskSwHookPtr     = App_OS_TaskSwHook;

OS_AppTimeTickHookPtr   = App_OS_TimeTickHook;
CPU_CRITICAL_EXIT();
#endif
}

```

从上面的代码中可以看出，函数 `App_OS_SetAllHooks()` 就是设置了所有指向钩子函数的指针，其中就包含了空闲任务回调函数 `OSIdleTaskHook()` 会调用的函数指针 `OS_AppIdleTaskHookPtr`，该指针被指向了函数 `App_OS_IdleTaskHook()`，因此当空闲任务调用空闲任务回调函数 `OSIdleTaskHook()` 时，就会调用函数 `App_OS_IdleTaskHook()`。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

`start_task` 任务的入口函数代码如下所示：

```

/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 设置钩子函数 */
    App_OS_SetAllHooks();

    /* 创建 Task1 */

```

```

Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK1_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                  (CPU_CHAR*    ) "task1",
                  (OS_TASK_PTR  ) task1,
                  (void*        ) 0,
                  (OS_PRIO      ) TASK1_PRIO,
                  (CPU_STK*     ) Task1Task_STK,
                  (CPU_STK_SIZE ) TASK1_STK_SIZE / 10,
                  (CPU_STK_SIZE ) TASK1_STK_SIZE,
                  (OS_MSG_QTY   ) 0,
                  (OS_TICK      ) 0,
                  (void*        ) 0,
                  (OS_OPT        ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                  (OS_ERR*      ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      ) &Task2Task_TCB,
                  (CPU_CHAR*    ) "task2",
                  (OS_TASK_PTR  ) task2,
                  (void*        ) 0,
                  (OS_PRIO      ) TASK2_PRIO,
                  (CPU_STK*     ) Task2Task_STK,
                  (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,
                  (CPU_STK_SIZE ) TASK2_STK_SIZE,
                  (OS_MSG_QTY   ) 0,
                  (OS_TICK      ) 0,
                  (void*        ) 0,
                  (OS_OPT        ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                  (OS_ERR*      ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务的函数代码与 第二章中《μC/OS-III 移植实验》的函数代码类似，只是多调用了函数 App_OS_SetAllHooks()以初始化空闲任务回调函数的函数指针 OS_AppIdleTaskHookPtr，令其指向了应用层的函数 App_OS_IdleTaskHook()，因此，只需要在应用层中的函数 App_OS_IdleTaskHook()中添加代码，就会被空闲任务执行。

(2) 函数 App_OS_IdleTaskHook()

```

void App_OS_IdleTaskHook (void)
{

```

```
static unsigned int idle_num = 0;

CPU_SR_ALLOC();

CPU_CRITICAL_ENTER();
if (++idle_num%1000000 == 0)
{
    printf("Idle task runing!\r\n");
}
CPU_CRITICAL_EXIT();
}
```

从上面的代码可以看出，只要空闲任务每执行 1000000 次，就会通过串口打印一次 “Idle task runing!\r\n”。

9.4.3 下载验证

编译并下载代码，整体的实验现象与第二章中《μC/OS-III 移植实验》的实验现象基本类似，不同之处在于，本实验添加了空闲任务的钩子函数，令空闲任务在执行了 1000000 次后通过串口打印 “Idle task runing!\r\n”，如下图所示：

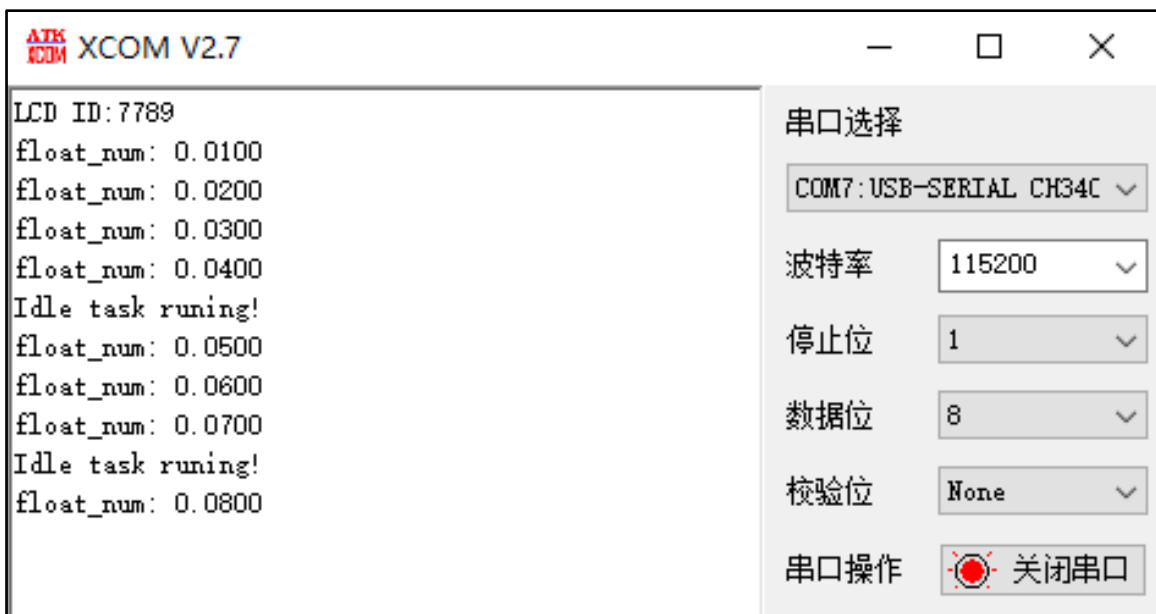


图 9.4.1 串口调试助手显示内容

从上图可以看到，空闲任务钩子函数打印的内容在串口调试助手的窗口中显示出来了。

9.5 μC/OS-III 统计功能实验

9.5.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数统计 CPU 的使用率及任务运行过程中的一些信息，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建其他任务
task1	用于实验测试
task2	用于信息统计信息

表 9.5.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 9-4 UCOS-III 统计功能实验》。

9.5.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

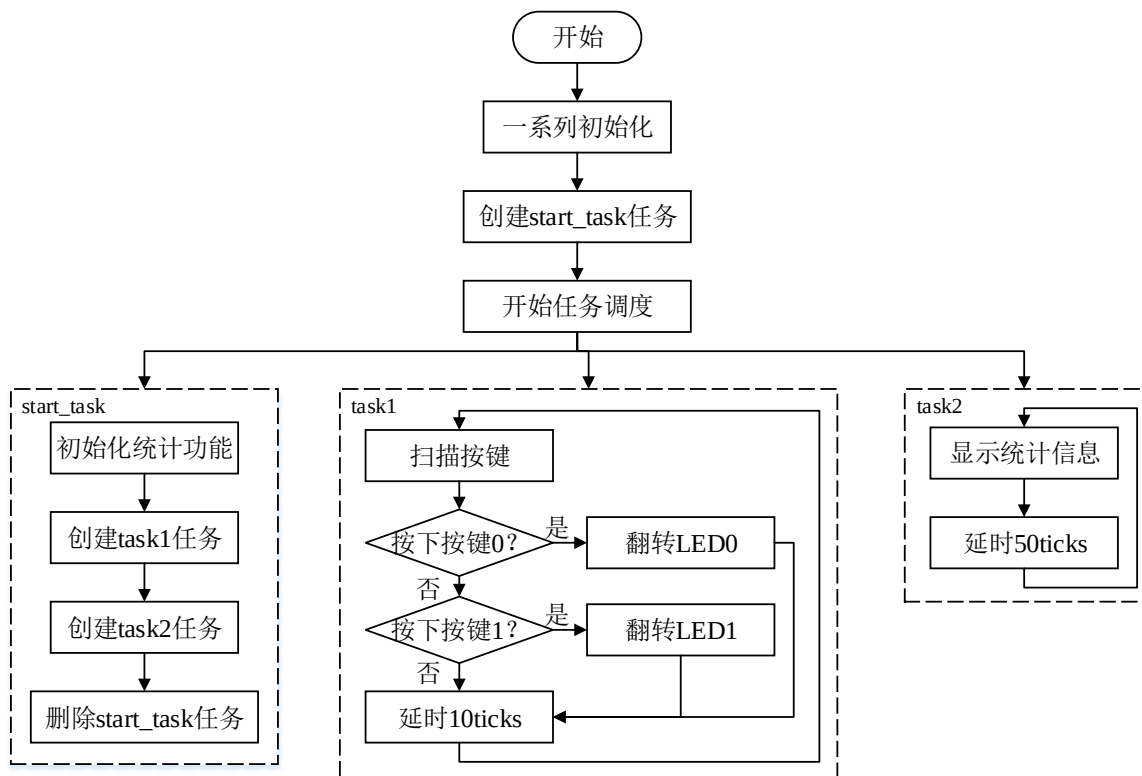


图 9.5.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSStatTaskCPUUsageInit()

此函数用于初始化 CPU 利用率统计功能，请参考第 9.1 小节《μC/OS-III 任务相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_STAT_TASK_EN 配置为 1。同时，由于本函数依赖于时间戳功能，因此请读者参考第 2.1.6 小节配置好 μC/OS-III 的时间戳功能。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```

/**
 * @brief   start_task
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 初始化 CPU 利用率统计 */
    OSStatTaskCPUUsageInit(&err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));
    OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                    (CPU_CHAR*     ) "task1",
                    (OS_TASK_PTR   ) task1,
                    (void*         ) 0,
                    (OS_PRIO       ) TASK1_PRIO,
                    (CPU_STK*      ) Task1Task_STK,
                    (CPU_STK_SIZE  ) TASK1_STK_SIZE / 10,
                    (CPU_STK_SIZE  ) TASK1_STK_SIZE,
                    (OS_MSG_QTY    ) 0,
                    (OS_TICK       ) 0,
                    (void*         ) 0,
                    (OS_OPT        ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                    (OS_ERR*       ) &err);

    /* 创建 Task2 */
    Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK2_STK_SIZE * sizeof(CPU_STK));

```

```
OSTaskCreate(    (OS_TCB*      ) &Task2Task_TCB,
                 (CPU_CHAR*    ) "task2",
                 (OS_TASK_PTR  ) task2,
                 (void*        ) 0,
                 (OS_PRIO     ) TASK2_PRIO,
                 (CPU_STK*     ) Task2Task_STK,
                 (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,
                 (CPU_STK_SIZE ) TASK2_STK_SIZE,
                 (OS_MSG_QTY   ) 0,
                 (OS_TICK      ) 0,
                 (void*        ) 0,
                 (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*      ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}
```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 和 task2 任务，并且在创建任务之前，使用函数 OSStatTaskCPUUsageInit() 初始化了 CPU 使用率统计功能。

(2) task1 任务

```
/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    uint8_t key;
    OS_ERR err;

    while (1)
    {
        key = key_scan(0);

        switch (key)
        {
            case KEY0_PRES:
            {
                LED0_TOGGLED();
                break;
            }
            case KEY1_PRES:
            {
```

```

        LED1_TOGGLE();
        break;
    }
    default:
    {
        break;
    }
}

OSTimeDly(10, OS_OPT_TIME_DLY, &err);
}
}

```

从上面的代码可以看出，task1 任务就是简单地根据按键的按下来翻转 LED。

(3) task2 任务

```

/**
 * @brief task2
 * @paramp_arg : 传入参数(未用到)
 * @retval 无
 */
void task2(void *p_arg)
{
    OS_ERR err;

    while (1)
    {
        lcd_show_num(109, 132, OSStatTaskCPUUsage/100, 3, 16, BLUE);
        lcd_show_xnum(141, 132, OSStatTaskCPUUsage%100, 2, 16, 0x80, BLUE);
        lcd_show_num(173, 132, OSStatTaskCPUUsageMax/100, 3, 16, BLUE);
        lcd_show_xnum(205, 132, OSStatTaskCPUUsageMax%100, 2, 16, 0x80, BLUE);

        lcd_show_num(109, 152, OSIdleTaskTCB.CPUUsage/100, 3, 16, BLUE);
        lcd_show_xnum(141, 152, OSIdleTaskTCB.CPUUsage%100, 2, 16, 0x80, BLUE);
        lcd_show_num(173, 152, OSIdleTaskTCB.CPUUsageMax/100, 3, 16, BLUE);
        lcd_show_xnum(205, 152, OSIdleTaskTCB.CPUUsageMax%100, 2, 16, 0x80, BLUE);

        lcd_show_num(109, 172, OSStatTaskTCB.CPUUsage/100, 3, 16, BLUE);
        lcd_show_xnum(141, 172, OSStatTaskTCB.CPUUsage%100, 2, 16, 0x80, BLUE);
        lcd_show_num(173, 172, OSStatTaskTCB.CPUUsageMax/100, 3, 16, BLUE);
        lcd_show_xnum(205, 172, OSStatTaskTCB.CPUUsageMax%100, 2, 16, 0x80, BLUE);

        lcd_show_num(109, 192, OSTmrTaskTCB.CPUUsage/100, 3, 16, BLUE);
        lcd_show_xnum(141, 192, OSTmrTaskTCB.CPUUsage%100, 2, 16, 0x80, BLUE);
        lcd_show_num(173, 192, OSTmrTaskTCB.CPUUsageMax/100, 3, 16, BLUE);
    }
}

```



```

    lcd_show_xnum(205, 192, OSTmrTaskTCB.CPUUsageMax%100, 2, 16, 0x80, BLUE);

    lcd_show_num(109, 212, Task1Task_TCB.CPUUsage/100, 3, 16, BLUE);
    lcd_show_xnum(141, 212, Task1Task_TCB.CPUUsage%100, 2, 16, 0x80, BLUE);
    lcd_show_num(173, 212, Task1Task_TCB.CPUUsageMax/100, 3, 16, BLUE);
    lcd_show_xnum(205, 212, Task1Task_TCB.CPUUsageMax%100, 2, 16, 0x80, BLUE);

    lcd_show_num(109, 232, Task2Task_TCB.CPUUsage/100, 3, 16, BLUE);
    lcd_show_xnum(141, 232, Task2Task_TCB.CPUUsage%100, 2, 16, 0x80, BLUE);
    lcd_show_num(173, 232, Task2Task_TCB.CPUUsageMax/100, 3, 16, BLUE);
    lcd_show_xnum(205, 232, Task2Task_TCB.CPUUsageMax%100, 2, 16, 0x80, BLUE);

    lcd_show_num(109, 272, Task1Task_TCB.StkUsed, 4, 16, BLUE);
    lcd_show_num(157, 272, TASK1_STK_SIZE, 4, 16, BLUE);

    lcd_show_num(109, 292, Task2Task_TCB.StkUsed, 4, 16, BLUE);
    lcd_show_num(157, 292, TASK2_STK_SIZE, 4, 16, BLUE);

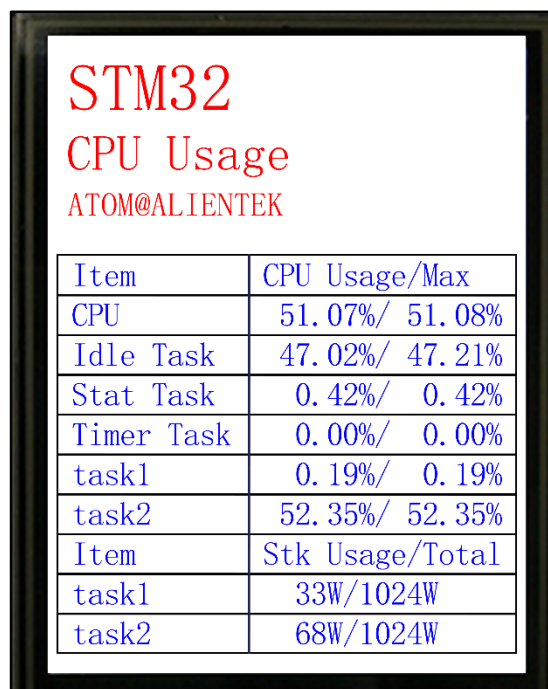
    OSTimeDly(50, OS_OPT_TIME_DLY, &err);
}
}

```

从上面的代码可以看出，task2 任务直接通过全局变量和任务控制块获取 CPU 使用率和任务运行过程中的一些信息，并将这些信息显示到 LCD 上。

9.5.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



The image shows an LCD screen with the following content:

STM32
CPU Usage
ATOM@ALIENTEK

Item	CPU Usage/Max
CPU	51.07%/ 51.08%
Idle Task	47.02%/ 47.21%
Stat Task	0.42%/ 0.42%
Timer Task	0.00%/ 0.00%
task1	0.19%/ 0.19%
task2	52.35%/ 52.35%

Item	Stk Usage/Total
task1	33W/1024W
task2	68W/1024W

图 9.5.2 LCD 显示内容

同时，在 LCD 显示屏上也显示了 CPU 的总利用率、各个任务的 CPU 利用率和任务的栈使用情况。这是可以通过按下按键，使得 task1 任务执行 LED 灯翻转的操作，通过观察 LCD 屏幕，能看到 task1 任务的 CPU 使用率会明显增多。

通过这个任务统计功能，用户就能很直观的看到系统中各个任务的运行情况，以便对各个任务做相应的优化。

第十章 μ C/OS-III 时间管理

在前面的章节实验例程中，频繁地使用了 μ C/OS-III 提供的延时函数，使用延时函数会使任务进入挂起态，直至延时完成或被终止延时，任务才会重新进入就绪态等待任务调度。 μ C/OS-III 是如何对延时任务进行管理的，又是如何判断任务延时是否完成的，这些都是属于 μ C/OS-III 时间管理的相关内容。本章就来学习 μ C/OS-III 时间管理的相关内容。

本章分为如下几部分：

10.1 μ C/OS-III 系统时钟节拍

10.2 μ C/OS-III 任务延时相关函数

10.3 μ C/OS-III 其他系统时钟节拍相关函数

10.1 μ C/OS-III 系统时钟节拍

10.1.1 μ C/OS-III 系统时钟节拍简介

任何的操作系统都需要时钟节拍， μ C/OS-III 也不例外。在 μ C/OS-III 中有一个用于记录系统时钟节拍的计数器，这个计数器是一个全局变量，定义在文件 `os.h` 中，如下所示：

```
OS_EXT OS_TICK OSTickCtr;
```

其中 `OS_EXT` 是一个宏定义，`OS_TICK` 是 μ C/OS-III 重定义的数据类型，上面代码展开后，如下所示：

```
extern unsigned int OSTickCtr;
```

可以看出，这个系统时钟节拍计数器对于 STM32 而言，就是一个全局的 32 比特无符号长整形变量。`OSTickCtr` 就是用来记录自系统启动后，系统时钟节拍产生的个数。每当产生一个系统时钟节拍，`OSTickCtr` 都会被加 1。

系统时钟节拍就像是操作系统的心跳，为操作系统的正常运行提供时基，在 μ C/OS-III 的许多系统服务中，都需要使用到系统时钟节拍，例如软件定时器和一系列需要计算超时的事件。

10.1.2 μ C/OS-III 系统时钟节拍来源

既然系统时钟节拍如此重要，那系统时钟节拍是从哪里来的呢？或者说，系统时钟节拍时如何产生的？提到能够提供准确时基的来源，大家可能会想到硬件定时器，例如 STM32 的 TIM1。当然，使用 TIM1 产生系统时钟节拍为系统提供时基，当然没有问题，但是还有一个更优解，那便是 SysTick（滴答定时器），SysTick 可以说就是为操作系统提供心跳而设计的，因此本教程将以 SysTick 作为产生系统时钟节拍的情况，展开讲解。

既然系统时钟节拍的来源是 SysTick，那么就需要对 SysTick 进行配置，就以第 2.2 小节《 μ C/OS-III 移植实验》这个实验的实验工程为例，在 μ C/OS-III 内核启动前，就已经对 SysTick 进行了多次配置：HAL 库的初始化函数 `HAL_Init()`、STM32 时钟的配置函数 `sys_stm32_clock_init()`、延时初始化函数 `delay_init()`。在以上三个函数中都对 SysTick 做了配置，但是在 μ C/OS-III 内核启动后，还必须使用函数 `OS_CPU_SysTickInit()` 对 SysTick 进行配置。在第 3.2.2 小节《`os_cfg_app.h` 配置文件》中讲解了配置项 `OS_CFG_TICK_RATE_HZ` 的作用，配置项 `OS_CFG_TICK_RATE_HZ` 就是用来配置系统时钟节拍的频率的，因此在 μ C/OS-III 内核系统后，需要根据 SysTick 的时钟频率以及配置项 `OS_CFG_TICK_RATE_HZ` 的值，重新配置 SysTick，如下所示：

```
/* 根据配置的节拍频率配置 SysTick */
cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
OS_CPU_SysTickInit(cnts);
```

从上面的代码中可以看到，在调用函数 `OS_CPU_SysTickInit()` 配置 `SysTick` 之前，先根据 `SysTick` 的时钟频率和配置项 `OS_CFG_TICK_RATE_HZ` 的值计算除了一个 `cnts`，这一步实际上就是为了计算 `SysTick` 的重装载值。

这里要注意的是，虽然 STM32 的 `SysTick` 的时钟频率可以有两种配置，分别为与 CPU 内核同频率和 CPU 内核频率的八分之一，但是，函数 `OS_CPU_SysTickInit()` 在配置 `SysTick` 的时候，是强制将 `SysTick` 配置为与 CPU 内核同频率的，因此在计算 `cnts` 的时候，就直接使用 CPU 内核的频率作为 `SysTick` 的频率。

那么接下来看一下函数 `OS_CPU_SysTickInit()` 是如何配置 `SysTick` 的，函数 `OS_CPU_SysTickInit()` 定义在文件 `os_cpu.c` 中，具体的代码如下所示：

```
void OS_CPU_SysTickInit(CPU_INT32U cnts)
{
    /* 此宏用于使能系统时钟节拍 */
    #if (OS_CFG_TICK_EN > 0u)
        CPU_INT32U prio;
        CPU_INT32U basepri;

        /* 计算受 μC/OS-III 管理的最高中断优先级在中断优先级配置寄存器中表示的值 */
        basepri=(CPU_INT32U) (CPU_CFG_KA_IPL_BOUNDARY<<(8u-CPU_CFG_NVIC_PRIO_BITS));

        /* 设置 SysTick 的重装载寄存器 */
        CPU_REG_SYST_RVR = cnts - 1u;

        /* 获取 SHPR3 寄存器的值 */
        prio = CPU_REG_SCB_SHPRI3;

        /* 将 SHPR3[31:24]清零 */
        prio &= 0x00FFFFFFu;

        /* 将 SHPR3[31:24]设置为受 μC/OS-III 管理的最高优先级 */
        prio |= (basepri << 24u);

        /* 写入 SHPR3 寄存器 */
        CPU_REG_SCB_SHPRI3 = prio;

        /* 使能 SysTick，并设置 SysTick 与 CPU 内核相同的时钟频率 */
        CPU_REG_SYST_CSR |= CPU_REG_SYST_CSR_CLKSOURCE |
                            CPU_REG_SYST_CSR_ENABLE;

        /* 使能 SysTick 的中断 */
        CPU_REG_SYST_CSR |= CPU_REG_SYST_CSR_TICKINT;
    #else
        /* 在未使能系统时钟节拍的情况下
        * 防止编译器警告
        */
        (void)cnts;
    #endif
}
```

可以看到，函数 `OS_CPU_SysTickInit()` 会根据传入的参数 `cnts` 配置 `SysTick` 的重装载值，那么 `SysTick` 就能够按照配置文件 `os_cfg_app.h` 中配置项 `OS_CFG_TICK_RATE_HZ` 的频率发生溢出，并且在上面的代码中，也使能了 `SysTick` 的中断，因此就能够在 `SysTick` 的中断中处理 μC/OS-III 的系统时钟节拍了。

10.1.3 μC/OS-III 系统时钟节拍处理

既然已经知道了 μC/OS-III 系统时钟节拍是来自 `SysTick`，那么 μC/OS-III 系统时钟节拍的处理，也就是在 `SysTick` 的中断服务函数中完成的。在第 2.1.3 小节《修改 SYSTEM 文件》中，就修改了 `SysTick` 的中断服务函数 `SysTick_Handler()`，如下所示：

```
/**
 * @brief   systick 中断服务函数, 使用 OS 时用到
 * @param   ticks: 延时的节拍数
 * @retval  无
 */
void SysTick_Handler(void)
{
    /*OS 开始跑了, 才执行正常的调度处理 */
    if (OSRunning == OS_STATE_OS_RUNNING)
    {
        /* 调用 uC/OS-III 的 SysTick 中断服务函数 */
        OS_CPU_SysTickHandler();
    }
    HAL_IncTick();
}
```

从上面的代码中可以看出，在 `SysTick` 的中断服务函数中，会先通过全局变量 `OS_STATE_OS_RUNNING` 判断 μC/OS-III 内核是否已经开始运行了，只有当 μC/OS-III 内核开始运行之后，才会调用函数 `OS_CPU_SysTickHandler()` 处理系统时钟节拍等事务。

实际上函数 `OS_CPU_SysTickHandler()` 是 μC/OS-III 源代码中提供的 `SysTick` 中断服务函数，μC/OS-III 官方的原意是修改启动文件中的中断向量表，让函数 `OS_CPU_SysTickHandler()` 成为 `SysTick` 的中断服务函数，但是由于本教程是实验例程都是基于 HAL 库开发的，需要调用 HAL 库的函数 `HAL_IncTick()`，并且原本的基础工程就使用了函数 `SysTick_Handler()` 作为 `SysTick` 的中断服务函数，因此只能在函数 `SysTick_Handler()` 中调用函数 `OS_CPU_SysTickHandler()`。

接下来就来看一下函数 `OS_CPU_SysTickHandler()` 是如何处理的，函数 `OS_CPU_SysTickHandler()` 定义在文件 `os_cpu.c` 中，具体的代码如下所示：

```
void OS_CPU_SysTickHandler(void)
{
    CPU_SR_ALLOC();

    CPU_CRITICAL_ENTER();
    /* 进入中断后, 先调用函数 OSIntEnter() */
    OSIntEnter();
    CPU_CRITICAL_EXIT();
}
```

```
/* 处理与 μC/OS-III 系统时钟节拍相关事务 */
OSTimeTick();

/* 中断返回前，调用函数 OSIntExit() */
OSIntExit();
}
```

接下来查看函数 OSTimeTick(), 该函数定义在文件 os_time.c 中, 具体的代码如下所示:

```
void OSTimeTick(void)
{
    /* 如果 μC/OS-III 系统还未运行,
     * 则不进行处理
     */
    if (OSRunning != OS_STATE_OS_RUNNING) {
        return;
    }

    /* 调用系统时钟节拍钩子函数 */
    OSTimeTickHook();

#ifdef OS_CFG_SCHED_ROUND_ROBIN_EN > 0u
    /* 处理时间片 */
    OS_SchedRoundRobin(&OSRdyList[OSPrioCur]);
#endif

#ifdef OS_CFG_TICK_EN > 0u
    /* 更新系统时钟节拍 */
    OS_TickUpdate(1u);
#endif
}
```

可以看到, 在该函数中就会调用系统时钟节拍的钩子函数, 用户可以自行指定这个钩子函数, 就像空闲任务的钩子函数类似。

接着来看一下处理时间片的函数 OS_SchedRoundRobin(), 该函数传入的是当前任务所在的就绪态任务链表, 因为时间片调度就是在最高优先级的任务之间进行调度, 该函数定义在文件 os_core.c 中, 具体的代码如下所示:

```
void OS_SchedRoundRobin(OS_RDY_LIST *p_rdy_list)
{
    OS_TCB *p_tcb;
    CPU_SR_ALLOC();

    /* 检查是否使能了时间片调度 */
    if (OSSchedRoundRobinEn != OS_TRUE) {
        return;
    }
}
```

```

CPU_CRITICAL_ENTER();

/* 获取任务控制块 */
p_tcb = p_rdy_list->HeadPtr;
if (p_tcb == (OS_TCB *)0) {
    CPU_CRITICAL_EXIT();
    return;
}

/* 此宏用于使能空闲任务 */
#if (OS_CFG_TASK_IDLE_EN > 0u)
/* 如果使能了空闲任务，
 * 就需要判断获取到的任务控制块是否为空闲任务，
 * 空闲任务是不需要进行时间片调度的
 */
if (p_tcb == &OSIdleTaskTCB) {
    CPU_CRITICAL_EXIT();
    return;
}
#endif

/* 如果任务剩余时间片大于 0，
 * 就更新任务的剩余时间片（减 1）
 */
if (p_tcb->TimeQuantaCtr > 0u) {
    p_tcb->TimeQuantaCtr--;
}

/* 如果任务剩余时间片更新后依然大于 0，
 * 则说明任务时间片还未超时，
 * 无需进行任务调度，
 * 因此返回
 */
if (p_tcb->TimeQuantaCtr > 0u) {
    CPU_CRITICAL_EXIT();
    return;
}

/* 任务已无剩余时间片，
 * 因此需要切换到同任务优先级的任务去运行，
 * 因此判断任务所在的就绪态任务链表中是否有其他任务，
 * 如果任务所在就绪态任务链表的链表头等于链表尾，

```

```

    * 则说明该任务链表就只有一个任务，
    * 无需进行任务切换，
    * 因此返回
    */
    if (p_rdy_list->HeadPtr == p_rdy_list->TailPtr) {
        CPU_CRITICAL_EXIT();
        return;
    }

    /* 判断任务调度器是否处于锁定状态，
    * 如果任务调度器被锁定，
    * 就无需仅从任务切换，
    * 因此返回
    */
    if (OSSchedLockNestingCtr > 0u) {
        CPU_CRITICAL_EXIT();
        return;
    }

    /* 任务时间片超时，且符合任务切换的条件，
    * 那么接下来就要进行任务切换，
    * 步骤如下：
    * 将当前任务移动到就绪态任务链表的末尾，
    * 获取更新就绪态任务链表的链表头任务
    * 根据任务时间片设置任务剩余时间片
    */
    OS_RdyListMoveHeadToTail(p_rdy_list);
    p_tcb = p_rdy_list->HeadPtr;
    /* 如果任务时间片为 0，
    * 就是用默认时间片
    */
    if (p_tcb->TimeQuanta == 0u) {
        p_tcb->TimeQuantaCtr = OSSchedRoundRobinDfltTimeQuanta;
    } else {
        p_tcb->TimeQuantaCtr = p_tcb->TimeQuanta;
    }

    CPU_CRITICAL_EXIT();
}

```

在处理任务的时间片后，接下来就是调用函数 `OS_TickUpdate()` 更新系统时钟节拍计数器以及处理系统时钟节拍相关的其他事务了。函数 `OS_TickUpdate()` 传入的参数为 1，这个参数代表系统时钟节拍的增量，这里表述更新系统时钟节拍的增量为 1，说明过了一个系统时钟节拍。函数 `OS_TickUpdate()` 定义在文件 `os_tick.c` 中，具体的代码如下所示：


```
void OS_TickUpdate(OS_TICK ticks)
{
    #if (OS_CFG_TS_EN > 0u)
        CPU_TS ts_start;
    #endif

    CPU_SR_ALLOC();

    CPU_CRITICAL_ENTER();

    /* 更新系统时钟节拍计数器 */
    OSTickCtr += ticks;

    /* 用于调式，不用理会 */
    OS_TRACE_TICK_INCREMENT(OSTickCtr);

    /* 此宏用于使能时间戳功能
     * 如果使能了时间戳功能，
     * 则会计算处理 Tick 任务链表的耗时
     */
    #if (OS_CFG_TS_EN > 0u)
        ts_start = OS_TS_GET();
        /* 处理 Tick 任务链表 */
        OS_TickListUpdate(ticks);
        OSTickTime = OS_TS_GET() - ts_start;
        if (OSTickTimeMax < OSTickTime) {
            OSTickTimeMax = OSTickTime;
        }
    #else
        /* 处理 Tick 任务链表 */
        OS_TickListUpdate(ticks);
    #endif

    /* 此宏用于使能动态时钟节拍管理 */
    #if (OS_CFG_DYN_TICK_EN > 0u)
        if (OSTickList.TCB_Ptr != (OS_TCB *)0) {
            OSTickCtrStep = OSTickList.TCB_Ptr->TickRemain;
        } else {
            OSTickCtrStep = 0u;
        }

        OS_DynTickSet(OSTickCtrStep);
    #endif
}
```

```
CPU_CRITICAL_EXIT();
}
```

从上面的代码中可以看出，函数 OS_TickUpdate() 主要完成两件事，分别为更新系统时钟节拍和处理 Tick 任务链表。其中更新系统时钟节拍，就是将系统时钟节拍计数器的值加上该函数的增量入参，而处理 Tick 任务链表，则是调用函数 OS_TickListUpdate() 完成的。函数 OS_TickListUpdate() 定义在文件 os_tick.c 中，具体的代码如下所示：

```
static void OS_TickListUpdate(OS_TICK ticks)
{
    OS_TCB          *p_tcb;
    OS_TICK_LIST     *p_list;
    #if (OS_CFG_DBG_EN > 0u)
        OS_OBJ_QTY    nbr_updated;
    #endif
    #if (OS_CFG_MUTEX_EN > 0u)
        OS_TCB          *p_tcb_owner;
        OS_PRIO          prio_new;
    #endif

    #if (OS_CFG_DBG_EN > 0u)
        nbr_updated      = 0u;
    #endif

    /* 获取 Tick 任务链表 */
    p_list                = &OSTickList;
    /* 获取 Tick 任务链表中的第一个任务
     * 因为 Tick 任务链表中的任务是按照挂起超时先后排序的
     */
    p_tcb                 = p_list->TCB_Ptr;
    /* 如果 Tick 任务链表中没有任务，
     * 则不需要处理
     */
    if (p_tcb != (OS_TCB *)0) {
        /* 先判断时钟节拍的增量会不会导致任务挂起超时 */
        if (p_tcb->TickRemain <= ticks) {
            /* 如果会导致任务挂起超时，
             * 则更新时钟节拍的增量
             * 并清零任务的剩余挂起时间
             */
            ticks = ticks - p_tcb->TickRemain;
            p_tcb->TickRemain = 0u;
        } else {
            /* 如果不会导致任务挂起超时
             * 则更新任务的剩余挂起时间
            */
        }
    }
}
```

```

        */
        p_tcb->TickRemain -= ticks;
    }

    /* 在这个循环中处理完 Tick 任务链表中所有挂起超时的任务 */
    while (p_tcb->TickRemain == 0u) {
#ifdef OS_CFG_DBG_EN > 0u
        nbr_updated++;
#endif

        /* 根据任务的状态，处理挂起超时的任务 */
        switch (p_tcb->TaskState) {
            case OS_TASK_STATE_DLY:
                /* 如果任务只是因延时被挂起，
                 * 那么仅需将延时挂起超时的任务设置为就绪态，
                 * 并将任务添加到就绪态任务链表中即可
                 */
                p_tcb->TaskState = OS_TASK_STATE_RDY;
                OS_RdyListInsert(p_tcb);
                break;

            case OS_TASK_STATE_DLY_SUSPENDED:
                /* 如果任务在延时期间被其他任务使用函数 OSTaskSuspend() 挂起，
                 * 则在任务挂起延时超时后，
                 * 将任务设置为挂起态，
                 * 因为被函数 OSTaskSuspend() 挂起的任务
                 * 只能后函数 OSTaskResume() 恢复
                 */
                p_tcb->TaskState = OS_TASK_STATE_SUSPENDED;
                break;

            default:
                /* 此宏用于使能互斥信号量 */
#ifdef OS_CFG_MUTEX_EN > 0u
                /* 判断任务是否挂起等待互斥信号量
                 * 如果是，则获取持有该互斥信号量的任务
                 */
                p_tcb_owner = (OS_TCB *)0;
                if (p_tcb->PendOn == OS_TASK_PEND_ON_MUTEX) {
                    p_tcb_owner = (OS_TCB *) (
                        (OS_MUTEX *) (
                            (void *)p_tcb->PendObjPtr))
                        ->OwnerTCBPtr;

```

```

    }

#endif

/* 此宏用于使能消息队列功能或任务内嵌消息队列功能 */
#if (OS_MSG_EN > 0u)

/* 因为任务直到挂起超时都未等待到消息
 * 因此设置接收到的消息为空
 */
p_tcb->MsgPtr = (void *)0;
p_tcb->MsgSize = 0u;

#endif

#if (OS_CFG_TS_EN > 0u)
p_tcb->TS = OS_TS_GET();
#endif

/* 将任务从挂起态任务链表中移除 */
OS_PendListRemove(p_tcb);

/* 根据任务的状态，处理任务 */
switch (p_tcb->TaskState) {
    /* 如果任务只是等待事件挂起超时，
     * 那么仅需将等待事件挂起超时的任务设置为就绪态，
     * 并将任务添加到就绪态任务链表即可
     */
    case OS_TASK_STATE_PEND_TIMEOUT:
        OS_RdyListInsert(p_tcb);
        p_tcb->TaskState = OS_TASK_STATE_RDY;
        break;

    /* 如果任务在挂起期间被其他任务调用函数 OSTaskSuspend() 挂起，
     * 则在任务等待事件挂起超时后，
     * 将任务设置为挂起态，
     * 因为被函数 OSTaskSuspend() 挂起的任务
     * 只能后函数 OSTaskResume() 恢复
     */
    case OS_TASK_STATE_PEND_TIMEOUT_SUSPENDED:
        p_tcb->TaskState = OS_TASK_STATE_SUSPENDED;
        break;

    default:
        break;
}

/* 设置任务的挂起状态为挂起超时 */

```

```

        p_tcb->PendStatus = OS_STATUS_PEND_TIMEOUT;
        /* 设置任务挂起等待对象为无 */
        p_tcb->PendOn = OS_TASK_PEND_ON_NOHING;

        /* 此宏用于使能互斥信号量 */
#ifdef (OS_CFG_MUTEX_EN > 0u)
        /* 判断任务是否有等待的互斥信号量 */
        if (p_tcb_owner != (OS_TCB *)0) {
            /* 判断所等待互斥信号量的持有者是否继承了挂起超时任务的任务优先级
             * 如果是，则需要令互斥信号量的持有者继承其他挂起任务的任务优先级
             */
            if ((p_tcb_owner->Prio != p_tcb_owner->BasePrio) &&
                (p_tcb_owner->Prio == p_tcb->Prio)) {
                prio_new = OS_MutexGrpPrioFindHighest(p_tcb_owner);
                prio_new = (prio_new > p_tcb_owner->BasePrio) ?
                    p_tcb_owner->BasePrio :
                    prio_new;
                if (prio_new != p_tcb_owner->Prio) {
                    OS_TaskChangePrio(p_tcb_owner, prio_new);
                    OS_TRACE_MUTEX_TASK_PRIO_DISINHERIT(p_tcb_owner,
                                                            p_tcb_owner->Prio);
                }
            }
        }
    }

#endif

        break;
    }

    /* 获取 Tick 任务链表中的下一个任务 */
    p_list->TCB_Ptr = p_tcb->TickNextPtr;
    p_tcb = p_list->TCB_Ptr;
    /* 如果获取到的任务为空，
     * 则说明 Tick 任务链表遍历完成
     */
    if (p_tcb == (OS_TCB *)0) {
#ifdef (OS_CFG_DBG_EN > 0u)
        p_list->NbrEntries = 0u;
#endif
        break;
    } else {
#ifdef (OS_CFG_DBG_EN > 0u)
        p_list->NbrEntries--;
#endif
    }
}

```

```

        p_tcb->TickPrevPtr = (OS_TCB *)0;
        /* 与本函数前面相同的操作
        * 更新当前遍历到的 Tick 任务链表中任务的剩余挂起时间
        */
        if (p_tcb->TickRemain <= ticks) {
            ticks = ticks - p_tcb->TickRemain;
            p_tcb->TickRemain = 0u;
        } else {
            p_tcb->TickRemain -= ticks;
        }
    }
}

#ifdef OS_CFG_DBG_EN > 0u
    p_list->NbrUpdated = nbr_updated;
#endif
}
    
```

从上面的代码中可以看出，函数 OS_TickListUpdate()主要就是判断 Tick 任务链表中的任务是否超时，如果任务挂起时间超时，则会对齐进行进一步的处理。同时，函数 OS_TickListUpdate()中还设计了一些与互斥信号量优先级继承相关的内容，这个在下文讲解 μC/OS-III 中的互斥信号量时，会再具体分析。

至此 μC/OS-III 系统时钟节拍的处理就分析完了，总结一下，在 μC/OS-III 的时钟节拍处理过程中，最主要就是完成以下两件事：

- 1、更新系统时钟节拍计数器（全局变量 OSTickCtr）；
- 2、处理 Tick 任务链表。

其中系统时钟节拍计数器 OSTickCtr 被用于 μC/OS-III 中许多与时间相关的处理中，还有就是处理 Tick 任务链表，Tick 任务链表主要就是存放一些因延时或等待事件而被挂起的任务，这些任务被挂起时，都设置了一个挂起的超时时间，例如延时的时间就是被延时任务的挂起超时时间，还有最大等待时长就是因等待事件而被挂起的任务的挂起超时时间。

10.2 μC/OS-III 任务延时相关函数

在上一小节中，了解到 μC/OS-III 会在 SysTick 的中断服务函数中更新系统时钟节拍计数器并处理 Tick 任务链表，而 μC/OS-III 中的任务函数函数，本质上就是将任务添加到 Tick 任务链表中，并给定一个挂起的超时时间，在任务被挂起到 Tick 任务链表中时，任务就相当于进行了延时。

μC/OS-III 中于任务延时相关的函数如下表所示：

函数	描述
OSTimeDly()	以系统时钟节拍为单位进行任务延时
OSTimeDlyHMSM()	以时、分、秒、毫秒为单位进行任务延时
OSTimeDlyResume()	恢复任务延时

表 10.2.1 μC/OS-III 任务延时相关函数描述

1. 函数 OSTimeDly()

该函数用于以系统时钟节拍为单位进行任务延时。

函数 OSTimeDly()的函数原型，如下所示：

```
void OSTimeDly( OS_TICK dly,
               OS_OPT  opt,
               OS_ERR  *p_err)
```

函数 OSTimeDly()的形参描述，如下表所示：

形参	描述
dly	任务延时的系统时钟节拍数
opt	延时选项
p_err	指向接收错误代码变量的指针

表 10.2.2 函数 OSTimeDly()形参相关描述

函数 OSTimeDly()的延时选项（opt）描述，如下表所示：

延时选项	描述（OSTickCtr 为系统时钟节拍计数器）
OS_OPT_TIME_DLY	任务延时的结束时刻为 OSTickCtr=OSTickCtr+dly
OS_OPT_TIME_TIMEOUT	任务延时的结束时刻为 OSTickCtr=OSTickCtr+dly
OS_OPT_TIME_MATCH	任务延时的结束时刻为 OSTickCtr=dly
OS_OPT_TIME_PERIODIC	任务延时的结束时刻为 OSTickCtr= OSTCBCurPtr->TickCtrPrev+dly

表 10.2.3 函数 OSTimeDly()延时选项相关描述

函数 OSTimeDly()的返回值描述，如下表所示：

返回值	描述
无	无

表 10.2.4 函数 OSTimeDly()返回值相关描述

函数 OSTimeDly()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	开始任务延时成功
OS_ERR_OPT_INVALID	无效的任务操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_SCHED_LOCKED	任务调度器已锁定
OS_ERR_TIME_DLY_ISR	在中断中非法调用该函数
OS_ERR_TIME_ZERO_DLY	有效的延时时长为 0 个系统时钟节拍

表 10.2.5 函数 OSTimeDly()错误代码相关描述

函数 OSTimeDly()定义在文件 os_time.c 中，具体的代码如下所示：

```
void OSTimeDly( OS_TICK dly,
               OS_OPT  opt,
               OS_ERR  *p_err)
{
    #if (OS_CFG_TICK_EN > 0u)
        CPU_SR_ALLOC();
    #endif

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
    #ifdef OS_SAFETY_CRITICAL
```

```
if (p_err == (OS_ERR *)0) {
    OS_SAFETY_CRITICAL_EXCEPTION();
    return;
}
#endif

/* 此宏用于使能系统时钟节拍
 * 任务延时需要系统时钟节拍的支持
 */
#if (OS_CFG_TICK_EN == 0u)
    *p_err = OS_ERR_TICK_DISABLED;
    return;
#else

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
         * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
         * 因为在中断中不能调用创建任务的函数，
         * 因此返回错误
         */
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_TIME_DLY_ISR;
            return;
        }
    #endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        /* 如果 μC/OS-III 系统还未运行，
         * 那么是不能进行任务延时的
         */
        if (OSRunning != OS_STATE_OS_RUNNING) {
            *p_err = OS_ERR_OS_NOT_RUNNING;
            return;
        }
    #endif

    /* 通过任务调度器锁定嵌套计数器
     * 判断任务调度器是否被锁定，
     * 如果任务调度器被锁定，
     * 那么是不能进行任务延时的
     */

```



```

if (OSSchedLockNestingCtr > 0u) {
    *p_err = OS_ERR_SCHED_LOCKED;
    return;
}

/* 判断任务延时选项参数是否合法 */
switch (opt) {
    case OS_OPT_TIME_DLY:
    case OS_OPT_TIME_TIMEOUT:
    case OS_OPT_TIME_PERIODIC:
    case OS_OPT_TIME_MATCH:
        break;

    default:
        *p_err = OS_ERR_OPT_INVALID;
        return;
}

/* 此宏用于使能系统时钟节拍
 * 任务延时需要系统时钟节拍的支持
 */
#if (OS_CFG_TICK_EN > 0u)
    CPU_CRITICAL_ENTER();

    /* 将任务添加到 Tick 任务链表 */
    OS_TickListInsertDly(OSTCBCurPtr,
                        dly,
                        opt,
                        p_err);

    if (*p_err != OS_ERR_NONE) {
        CPU_CRITICAL_EXIT();
        return;
    }

    /* 将任务从就绪态任务链表中移除 */
    OS_RdyListRemove(OSTCBCurPtr);

    CPU_CRITICAL_EXIT();

    /* 触发任务调度
     * 触发任务调度后，
     * 任务就开始被挂起到 Tick 任务链表中进行延时了
     */

```

```
OSSched();
#endif
}
```

从上面的代码中可以看到，函数 OSTimeDly()进行任务延时就是将任务从就绪态任务链表中移动到 Tick 任务链表中，然后再触发任务调度，这样一来任务的 CPU 使用权就释放掉了，同时任务也被挂起到 Tick 任务链表中进行任务延时了。

2. 函数 OSTimeDlyHMSM()

该函数用于以时、分、秒、毫秒为单位进行任务延时。

函数 OSTimeDlyHMSM()的函数原型，如下所示：

```
void OSTimeDlyHMSM(CPU_INT16U hours,
                   CPU_INT16U minutes,
                   CPU_INT16U seconds,
                   CPU_INT32U milli,
                   OS_OPT opt,
                   OS_ERR *p_err)
```

函数 OSTimeDlyHMSM()的形参描述，如下表所示：

形参	描述
hours	任务延时的小时数
minutes	任务延时的分钟数
seconds	任务延时的秒数
milli	任务延时的毫秒数
opt	延时选项
p_err	指向接收错误代码变量的指针

表 10.2.6 函数 OSTimeDlyHMSM()形参相关描述

函数 OSTimeDlyHMSM()的延时选项（opt）描述，如下表所示：

延时选项	描述（OSTickCtr 为系统时钟节拍计数器）
OS_OPT_TIME_HMSM_STRICT	延时时间参数严格按照实际的时间格式 任务延时的小时数（0~99） 任务延时的分钟数（0~59） 任务延时的秒数（0~59） 任务延时的毫秒数（0~999）
OS_OPT_TIME_HMSM_NON_STRICT	延时时间参数无需严格按照实际的时间格式 任务延时的小时数（0~999） 任务延时的分钟数（0~9999） 任务延时的秒数（0~65535） 任务延时的毫秒数（0~4294967295）
OS_OPT_TIME_DLY	任务延时的结束时刻为 OSTickCtr=OSTickCtr+dly
OS_OPT_TIME_TIMEOUT	任务延时的结束时刻为 OSTickCtr=OSTickCtr+dly
OS_OPT_TIME_PERIODIC	任务延时的结束时刻为 OSTickCtr= OSTCBCurPtr->TickCtrPrev+dly
OS_OPT_TIME_MATCH	任务延时的结束时刻为 OSTickCtr=dly

表 10.2.7 函数 OSTimeDlyHMSM()延时选项相关描述

函数 OSTimeDlyHMSM()的返回值描述，如下表所示：

返回值	描述
无	无

表 10.2.8 函数 OSTimeDlyHMSM()返回值相关描述

函数 OSTimeDlyHMSM()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	开始任务延时成功
OS_ERR_OPT_INVALID	无效的任务延时选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_SCHED_LOCKED	任务调度器已锁定
OS_ERR_TIME_DLY_ISR	在中断中非法调用该函数
OS_ERR_TIME_INVALID_HOURS	无效的任务延时小时数
OS_ERR_TIME_INVALID_MINUTES	无效的任务延时分钟数
OS_ERR_TIME_INVALID_SECONDS	无效的任务延时秒数
OS_ERR_TIME_INVALID_MILLISECONDS	无效的任务延时毫秒数
OS_ERR_TIME_ZERO_DLY	有效的延时时长为 0 个系统时钟节拍

表 10.2.9 函数 OSTimeDlyHMSM()错误代码相关描述

函数 OSTimeDlyHMSM()的函数实现与函数 OSTimeDly()类似，仅仅是多了一步将以“时、分、秒、毫秒”为单位的延时时长转换为以“系统时钟节拍”为单位的延时时长。

3. 函数 OSTimeDlyResume()

该函数用于恢复被添加到 Tick 任务链表中挂起延时的任务。

函数 OSTimeDlyResume()的函数原型，如下所示：

```
void OSTimeDlyResume( OS_TCB *p_tcb,
                      OS_ERR *p_err)
```

函数 OSTimeDlyResume()的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
p_err	指向接收错误代码变量的指针

表 10.2.10 函数 OSTimeDlyResume()形参相关描述

函数 OSTimeDlyResume()的返回值描述，如下表所示：

返回值	描述
无	无

表 10.2.11 函数 OSTimeDlyResume()返回值相关描述

函数 OSTimeDlyResume()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	开始任务延时成功
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_STATE_INVALID	任务处于无效状态
OS_ERR_TASK_NOT_DLY	任务未进行任务延时
OS_ERR_TASK_SUSPEND	任务被函数 OSTaskSuspend()挂起
OS_ERR_TCB_INVALID	无效的任务控制块指针参数
OS_ERR_TIME_DLY_RESUME_ISR	在中断中非法调用该函数

表 10.2.12 函数 OSTimeDlyResume()错误代码相关描述

函数 OSTimeDlyResume()定义在文件 os_time.c 中，具体的代码如下所示：

```
void OSTimeDlyResume( OS_TCB *p_tcb,
                     OS_ERR *p_err)
{
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
         * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
         * 因为在中断中不能调用创建任务的函数，
         * 因此返回错误
         */
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_TIME_DLY_RESUME_ISR;
            return;
        }
    #endif

    /* 此宏用于使能参数检查 */
    #if (OS_CFG_ARG_CHK_EN > 0u)
        /* 指向任务控制块的指针不能为空 */
        if (p_tcb == (OS_TCB *)0) {
            *p_err = OS_ERR_TCB_INVALID;
            return;
        }
    #endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        /* 如果 μC/OS-III 系统还未运行，
         * 那么是不能进行任务切换的
         */
        if (OSRunning != OS_STATE_OS_RUNNING) {
            *p_err = OS_ERR_OS_NOT_RUNNING;
        }
    #endif
}
```

```
        return;
    }
#endif

    CPU_CRITICAL_ENTER();

    /* 根据任务状态进行不同的处理 */
    switch (p_tcb->TaskState) {
        /* 如果任务不处于任务延时状态,
         * 则返回
         */
        case OS_TASK_STATE_RDY:
        case OS_TASK_STATE_PEND:
        case OS_TASK_STATE_PEND_TIMEOUT:
        case OS_TASK_STATE_SUSPENDED:
        case OS_TASK_STATE_PEND_SUSPENDED:
        case OS_TASK_STATE_PEND_TIMEOUT_SUSPENDED:
            CPU_CRITICAL_EXIT();
            *p_err = OS_ERR_TASK_NOT_DLY;
            break;

        case OS_TASK_STATE_DLY:
            /* 如果任务仅处于任务延时状态,
             * 则将任务设置未就绪态,
             * 并将任务从 Tick 任务链表移动到就绪态任务链表中
             */
            p_tcb->TaskState = OS_TASK_STATE_RDY;

#if (OS_CFG_TICK_EN > 0u)
            OS_TickListRemove(p_tcb);
            OS_RdyListInsert(p_tcb);
#endif

            CPU_CRITICAL_EXIT();
            *p_err = OS_ERR_NONE;
            break;

        case OS_TASK_STATE_DLY_SUSPENDED:
            /* 如果任务处于延时状态时,
             * 被其他任务调用函数 OSTaskSuspend() 挂起,
             * 则任务从 Tick 任务链表中移除,
             * 而不能添加到就绪态任务链表,
             * 因为被函数 OSTaskSuspend() 挂起的任务
```

```

        * 只能够函数 OSTaskResume() 恢复
        */

        p_tcb->TaskState = OS_TASK_STATE_SUSPENDED;
#if (OS_CFG_TICK_EN > 0u)
        OS_TickListRemove(p_tcb);
#endif

        CPU_CRITICAL_EXIT();

        *p_err = OS_ERR_TASK_SUSPENDED;
        break;

default:
        /* 无效的任务状态 */
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_STATE_INVALID;
        break;
}

/* 终止任务延时后,
 * 需要触发一次任务调度,
 * 因为该延时任务可能具有较高的任务优先级
 */
OSSched();
}

```

10.3 μC/OS-III 其他系统时钟节拍相关 API 函数

系统时钟节拍功能除了有任务延时相关的几个 API 函数外，μC/OS-III 还提供了其他几个系统时钟节拍相关的 API 函数，如下表所示：

函数	描述
OSTimeGet()	获取当前系统时钟节拍计数器的值
OSTimeSet()	设置系统时钟节拍计数器的值

表 10.3.1 其他系统时钟节拍相关 API 函数描述

1. 函数 OSTimeGet()

该函数用于获取当前系统时钟节拍计数器的值。

函数 OSTimeGet() 的函数原型，如下所示：

```
OS_TICK OSTimeGet(OS_ERR *p_err)
```

函数 OSTimeGet() 的函数形参描述，如下表所示：

形参	描述
p_err	指向接收错误代码变量的指针

表 10.3.2 函数 OSTimeGet() 形参相关描述

函数 OSTimeGet() 的返回值描述，如下表所示：

返回值	描述
无	无

表 10.3.3 函数 OSTimeGet() 返回值相关描述

函数 OSTimeGet() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功获取系统节拍计数器的值

表 10.3.4 函数 OSTimeGet() 错误代码相关描述

2. 函数 OSTimeSet()

该函数用于设置系统时钟节拍计数器的值。

函数 OSTimeSet() 的函数原型，如下所示：

```
void OSTimeSet( OS_TICK ticks,
                OS_ERR *p_err)
```

函数 OSTimeSet() 的函数形参描述，如下表所示：

形参	描述
ticks	系统时钟节拍计数器的新值
p_err	指向接收错误代码变量的指针

表 10.3.5 函数 OSTimeSet() 形参相关描述

函数 OSTimeSet() 的返回值描述，如下表所示：

返回值	描述
无	无

表 10.3.6 函数 OSTimeSet() 返回值相关描述

函数 OSTimeSet() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功获取系统节拍计数器的值

表 10.3.7 函数 OSTimeSet() 错误代码相关描述

第十一章 μC/OS-III 信号量

信号量是操作系统中重要的一部分，信号量是任务间同步的一种机制，信号量可以用在多任务访问统一资源时的资源管理。μC/OS-III 提供了多种信号量，按照信号量的功能可以分为二值信号量、计数型信号量和互斥信号量，不同类新的信号量有其不同的应用场景，合理地使用信号量可以帮助开发者快速开发稳健的系统。本章就来学习 μC/OS-III 中的二值信号量和计数型信号量。

本章分为如下几部分：

- 11.1 μC/OS-III 信号量简介
- 11.2 μC/OS-III 二值信号量
- 11.3 μC/OS-III 共享资源冲突访问实验
- 11.4 μC/OS-III 二值信号量实验
- 11.5 μC/OS-III 计数型信号量
- 11.6 μC/OS-III 计数型信号量实验
- 11.7 μC/OS-III 优先级翻转
- 11.8 μC/OS-III 优先级翻转实验

11.1 μC/OS-III 信号量简介

信号量是一种解决同步问题的机制，可以实现对共享资源的有序访问。其中，“同步”指的是任务间的同步，即信号量可以使得一个任务等待另一个任务完成某件事情后，才继续执行；而“有序访问”指的是对被多任务或中断访问的共享资源（如全局变量）的管理，当一个任务在访问（读取或写入）一个共享资源时，信号量可以防止其他任务或中断在这期间访问（读取或写入）这个共享资源。

举一个例子，假设某个停车场有 100 个停车位（共享资源），这个 100 个停车位对所有人（访问共享资源的任务或中断）开放。如果有一个人要在这个停车场停车，那么就需要先判断这个停车场是否还有空车位（判断信号量是否有资源），如果此时停车场正好有空车位（信号量有资源），那么就可以直接将车开入空车位进行停车（获取信号量成功），如果此时停车场已经没有空车位了（信号量没有资源），那么这个人可以选择不停车（获取信号量失败），也可以选择等待（任务阻塞）其他人将车开出停车场（释放信号量资源），然后再将车停入停车场。

再上面的这个例子中，空车位的数量相当于信号量的资源数，获取信号量相当于占用了空车位，而释放信号量就相当于让出了占用的空车位。信号量用于管理共享资源的场景相当于对共享资源上了个锁，只有任务成功获取到了锁的要是，才能够访问这个共享资源，访问完共享资源后还得归还要是，当然钥匙可以不只一把，即信号量可以有多个资源。

11.2 μC/OS-III 二值信号量

11.2.1 μC/OS-III 二值信号量简介

正如其名，二值信号量实际上就是只有两种情况的信号量，二值信号量只有两种情况，分别为有资源和无资源，这也就意味着，如果把二值信号量比喻成一把锁，那么这把锁只有一把钥匙，当一个任务获取了这个二值信号量之后，在这个任务释放这个二值信号量之前，其他任务都是获取不到这个二值信号量的。二值信号量通常用于互斥访问或任务同步，与之后要讲

解的互斥信号量是比较类似的，但是二值信号量有可能会严重的优先级翻转问题。优先级翻转问题指的是，当一个高优先级任务因获取一个被低优先级任务获取而处于没有资源状态的二值信号量时，这个高优先级的任务将被阻塞，直到低优先级的任务释放这个二值信号量，而在这之前，如果有一个优先级介于高优先级任务和低优先级任务之间的任务就绪，那么这个中等优先级的任务就会抢占低优先级任务的运行，这么一来，这三个任务中优先级最高的任务反而要最后才能运行，这就是二值信号量带来的优先级翻转问题，用户在实际开发中要注意这个问题。

信号量的组成可以简单地理解为一个资源计数器和一个任务等待挂起链表组成的，二值信号量与下文要讲解的计数型信号量的最大的不同之处就在于，组成二值信号量的资源计数器只会用来表示 0 和 1，一次来区分无资源和有资源的两种状态，而计数型信号量的资源计数器就能够表示 0~计数型信号量最大资源数的多种情况，下面看一下组成信号量的结构体（二值信号量和计数型信号量共用一个结构体），该结构体定在文件 os.h 中，具体的代码如下所示：

```
struct os_sem{
    /* 此宏用于使能检查内核对象的类型 */
    #if (OS_OBJ_TYPE_REQ > 0u)
        /* 对象类型，
         * 在信号量初始化时，
         * 信号量的类型会被初始化为 OS_OBJ_TYPE_SEM
         */
        OS_OBJ_TYPE      Type;
    #endif

    /* 此宏用于使能代码调试功能 */
    #if (OS_CFG_DBG_EN > 0u)
        /* 如果使能了代码调试功能，
         * 那么就会为每一个信号量赋一个名称，
         * 方便调试
         */
        CPU_CHAR          *NamePtr;
    #endif

    /* 挂起等待任务链表
     * 当任务因获取了无资源的信号量时，
     * 就可以选择被挂起到该链表，
     * 等待信号量的资源
     */
    OS_PEND_LIST      PendList;

    /* 此宏用于使能代码调试功能 */
    #if (OS_CFG_DBG_EN > 0u)
        /* 一些与代码调试相关的成员变量 */
        OS_SEM          *DbgPrevPtr;
        OS_SEM          *DbgNextPtr;
    #endif
};
```

```

CPU_CHAR      *DbgNamePtr;
#endif

/* 信号量的资源计数器
 * 通过该成员变量判断信号量是否有资源
 */
OS_SEM_CTR    Ctr;

/* 以下的成员变量用于第三方的调试工具 */
#if (OS_CFG_TS_EN > 0u)
    CPU_TS      TS;
#endif
#if (defined(OS_CFG_TRACE_EN) && (OS_CFG_TRACE_EN > 0u))
    CPU_INT16U   SemID;
#endif
};

```

从上面的代码中可以看出，信号量的结构体中，有两个最重要的成员变量，分别为 `PendList`（任务等待挂起链表）和 `Ctr`（信号量资源计数器）。

11.2.2 μC/OS-III 二值信号量相关 API 函数

μC/OS-III 提供了信号量的一些相关操作 API 函数，前面也提到了，二值信号量与计数型信号量是很相似的，因此二值信号量与计数型信号量是共用一套 API 函数的，这些操作函数如下表所示：

函数	描述
<code>OSSemCreate()</code>	创建一个信号量
<code>OSSemDel()</code>	删除一个信号量
<code>OSSemPend()</code>	尝试获取信号量资源
<code>OSSemPendAbort()</code>	终止任务挂起等待信号量资源
<code>OSSemPost()</code>	释放信号量资源
<code>OSSemSet()</code>	强制设置信号量的资源数

表 11.2.1 μC/OS-III 信号量相关 API 函数描述

1. 函数 `OSSemCreate()`

该函数用于创建一个信号量。

函数 `OSSemCreate()` 的函数原型，如下所示：

```

void OSSemCreate( OS_SEM      *p_sem,
                  CPU_CHAR    *p_name,
                  OS_SEM_CTR  cnt,
                  OS_ERR      *p_err)

```

函数 `OSSemCreate()` 的形参描述，如下表所示：

形参	描述
<code>p_sem</code>	指向信号量结构体的指针
<code>p_name</code>	指向作为信号量名的 ASCII 字符串的指针
<code>cnt</code>	信号量资源数的初始值

p_err	指向接收错误代码变量的指针
-------	---------------

表 11.2.2 函数 OSSemCreate()形参相关描述

函数 OSSemCreate()的返回值描述，如下表所示：

返回值	描述
无	无

表 11.2.3 函数 OSSemCreate()返回值相关描述

函数 OSSemCreate()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	信号量创建成功
OS_ERR_CREATE_ISR	在中断中非法调用该函数
OS_ERR_ILLEGAL_CREATE_RUN_TIME	在系统运行过程中非法创建内核对象
OS_ERR_OBJ_PTR_NULL	指向信号量结构体的指针为空

表 11.2.4 函数 OSSemCreate()错误代码相关描述

函数 OSSemCreate()定义在文件 os_sem.c 中，具体的代码如下所示：

```
void OSSemCreate( OS_SEM      *p_sem,
                  CPU_CHAR    *p_name,
                  OS_SEM_CTR  cnt,
                  OS_ERR      *p_err)
{
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        *p_err = OS_ERR_ILLEGAL_CREATE_RUN_TIME;
        return;
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    #endif
}
```

```
*/  
  
if (OSIntNestingCtr > 0u) {  
    *p_err = OS_ERR_CREATE_ISR;  
    return;  
}  
#endif  
  
/* 此宏用于使能参数检查 */  
#if (OS_CFG_ARG_CHK_EN > 0u)  
/* 指向信号量结构体的指针不能为空 */  
if (p_sem == (OS_SEM *)0) {  
    *p_err = OS_ERR_OBJ_PTR_NULL;  
    return;  
}  
#endif  
  
/* 进入临界区 */  
CPU_CRITICAL_ENTER();  
  
/* 此宏用于使能检查内核对象类型 */  
#if (OS_OBJ_TYPE_REQ > 0u)  
#if (OS_CFG_OBJ_CREATED_CHK_EN > 0u)  
/* 通过对象类型是否已经被设置  
 * 判断信号量是否已经被创建  
 */  
if (p_sem->Type == OS_OBJ_TYPE_SEM) {  
    CPU_CRITICAL_EXIT();  
    *p_err = OS_ERR_OBJ_CREATED;  
    return;  
}  
#endif  
#endif  
  
/* 对象类型设置为 OS_OBJ_TYPE_SEM */  
p_sem->Type = OS_OBJ_TYPE_SEM;  
#endif  
  
/* 初始化信号量的资源数 */  
p_sem->Ctr = cnt;  
  
/* 初始化一些用于调试的成员变量 */  
#if (OS_CFG_TS_EN > 0u)  
p_sem->TS = 0u;  
#endif  
#if (OS_CFG_DBG_EN > 0u)
```

```

    p_sem->NamePtr = p_name;
#else
    (void)p_name;
#endif

    /* 初始化该信号量的等待挂起任务链表 */
    OS_PendListInit(&p_sem->PendList);

    /* 此宏用于使能代码调试功能 */
    #if (OS_CFG_DBG_EN > 0u)
        OS_SemDbgListAdd(p_sem);
        OSSemQty++;
    #endif

    /* 用于调试，不用理会 */
    OS_TRACE_SEM_CREATE(p_sem, p_name);

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_NONE;
}

```

2. 函数 OSSemDel()

该函数用于删除一个信号量。

函数 OSSemDel()的函数原型如下所示：

```

OS_OBJ_QTY OSSemDel(    OS_SEM  *p_sem,
                        OS_OPT  opt,
                        OS_ERR  *p_err)

```

函数 OSSemDel()的形参描述，如下表所示：

形参	描述
p_sem	指向信号量结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 11.2.5 函数 OSSemDel()形参相关描述

函数 OSSemDel()的返回值描述，如下表所示：

返回值	描述
OS_OBJ_QTY 类型变量	删除信号量时，被终止挂起任务的数量

表 11.2.6 函数 OSSemDel()返回值相关描述

函数 OSSemDel()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	信号量删除成功
OS_ERR_DEL_ISR	在中断中非法调用该函数
OS_ERR_ILLEGAL_DEL_RUN_TIME	定义了 OS_SAFETY_CRITICAL_IEC61508，且在 OSStart()之后非法地删除内核对象

OS_ERR_OBJ_PTR_NULL	指向信号量结构体的指针为空
OS_ERR_OBJ_TYPE	待被删除内核对象的类型不是信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_TASK_WAITING	有任务挂起等待待被删除的信号量

表 11.2.7 函数 OSSemDel()错误代码相关描述

函数 OSSemDel()定义在文件 os_sem.c 中，具体的代码如下所示：

```
OS_OBJ_QTY OSSemDel( OS_SEM *p_sem,
                     OS_OPT opt,
                     OS_ERR *p_err)
{
    OS_OBJ_QTY    nbr_tasks;
    OS_PEND_LIST  *p_pend_list;
    OS_TCB        *p_tcb;
    CPU_TS        ts;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

    /* 用于调式，不用理会 */
    OS_TRACE_SEM_DEL_ENTER(p_sem, opt);

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        OS_TRACE_SEM_DEL_EXIT(OS_ERR_ILLEGAL_DEL_RUN_TIME);
        *p_err = OS_ERR_ILLEGAL_DEL_RUN_TIME;
        return (0u);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    #endif
}
```

```

    */

    if (OSIntNestingCtr > 0u) {
        OS_TRACE_SEM_DEL_EXIT(OS_ERR_DEL_ISR);
        *p_err = OS_ERR_DEL_ISR;
        return (0u);
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    /* 如果 μC/OS-III 系统还未运行,
     * 那么是不能删除信号量的
     */
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_SEM_DEL_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    /* 指向信号量结构体的指针不能为空 */
    if (p_sem == (OS_SEM *)0) {
        OS_TRACE_SEM_DEL_EXIT(OS_ERR_OBJ_PTR_NULL);
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    /* 此函数只能用于删除信号量 */
    if (p_sem->Type != OS_OBJ_TYPE_SEM) {
        OS_TRACE_SEM_DEL_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

```

```

/* 获取信号量的等待挂起任务链表 */
p_pend_list = &p_sem->PendList;

/* 用于记录终止挂起的任务数量 */
nbr_tasks   = 0u;

/* 根据函数操作选项做不同操作 */
switch (opt) {
    /* 尽在没有任务等待该信号量资源时,
     * 才删除该信号量
     */
    case OS_OPT_DEL_NO_PEND:
        /* 判断信号量的等待挂起任务链表中是否无挂起的任务 */
        if (p_pend_list->HeadPtr == (OS_TCB *)0) {
            #if (OS_CFG_DBG_EN > 0u)
                OS_SemDbgListRemove(p_sem);
                OSSemQty--;
            #endif

            OS_TRACE_SEM_DEL(p_sem);
            /* 去初始化信号量结构体中的成员变量 */
            OS_SemClr(p_sem);
            /* 退出临界区,
             * 并返回信号量删除成功
             */
            CPU_CRITICAL_EXIT();
            *p_err = OS_ERR_NONE;
        } else {
            /* 如果信号量的等待挂起任务链表中有挂起的任务,
             * 则退出临界区,
             * 并返回相应警告
             */
            CPU_CRITICAL_EXIT();
            *p_err = OS_ERR_TASK_WAITING;
        }
        break;

    /* 始终删除信号量 */
    case OS_OPT_DEL_ALWAYS:
        #if (OS_CFG_TS_EN > 0u)
            ts = OS_TS_GET();
        #else
            ts = 0u;
        #endif
}

```



```

    /* 遍历终止等待挂起任务链表中的任务
    * 并记录终止挂起的任务数量
    */
    while (p_pend_list->HeadPtr != (OS_TCB *)0) {
        p_tcb = p_pend_list->HeadPtr;
        OS_PendAbort(    p_tcb,
                        ts,
                        OS_STATUS_PEND_DEL);
        nbr_tasks++;
    }
#if (OS_CFG_DBG_EN > 0u)
    OS_SemDbgListRemove(p_sem);
    OSSemQty--;
#endif

    OS_TRACE_SEM_DEL(p_sem);
    /* 去初始化信号量结构体中的成员变量 */
    OS_SemClr(p_sem);
    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    /* 触发任务调度
    * 因为被终止挂起的任务被重新添加到就绪态任务链表中了
    */
    OSSched();
    *p_err = OS_ERR_NONE;
    break;

default:
    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    /* 返回无效的函数操作选项 */
    *p_err = OS_ERR_OPT_INVALID;
    break;
}

/* 用于调试，不用理会 */
OS_TRACE_SEM_DEL_EXIT(*p_err);

/* 返回被终止挂起任务的数量 */
return (nbr_tasks);
}

```

3. 函数 OSSemPend()

该函数用于获取信号量。

函数 OSSemPend()的函数原型，如下所示：

```
OS_SEM_CTR OS_SemPend(OS_SEM *p_sem,
                      OS_TICK timeout,
                      OS_OPT opt,
                      CPU_TS *p_ts,
                      OS_ERR *p_err)
```

函数 OS_SemPend()的形参描述，如下表所示：

形参	描述
p_sem	指向信号量结构体的指针
timeout	任务挂起等待信号量的最大允许时间
opt	函数操作选项
p_ts	指向接收信号量接收时的时间戳的变量的指针
p_err	指向接收错误代码变量的指针

表 11.2.8 函数 OS_SemPend()形参相关描述

函数 OS_SemPend()的返回值描述，如下表所示：

返回值	描述
OS_SEM_CTR 类型返回值	信号量资源数更新后的值

表 11.2.9 函数 OS_SemPend()返回值相关描述

函数 OS_SemPend()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功获取信号量
OS_ERR_OBJ_DEL	待获取的信号量已经被删除
OS_ERR_OBJ_PTR_NULL	指向信号量结构体的指针为空
OS_ERR_OBJ_TYPE	待获取内核对象的类型不是信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT	任务挂起等待信号量时被终止（还未超时）
OS_ERR_PEND_ISR	在中断中非法调用该函数
OS_ERR_PEND_WOULD_BLOCK	获取的信号量没有资源，并且不挂起任务等待信号量
OS_ERR_SCHED_LOCKED	任务调度器已锁定
OS_ERR_STATUS_INVALID	无效的任务挂起结果
OS_ERR_TIMEOUT	任务挂起等待信号量超时

表 11.2.10 函数 OS_SemPend()错误代码相关描述

函数 OS_SemPend()定义在文件 os_sem.c 中，具体的代码如下所示：

```
OS_SEM_CTR OS_SemPend( OS_SEM *p_sem,
                      OS_TICK timeout,
                      OS_OPT opt,
                      CPU_TS *p_ts,
                      OS_ERR *p_err)
{
    OS_SEM_CTR ctr;
    CPU_SR_ALLOC();

    /* 此宏用于使能时间戳功能 */
```

```
#if (OS_CFG_TS_EN == 0u)
    (void)p_ts;
#endif

/* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

/* 用于调试, 不同理会 */
OS_TRACE_SEM_PEND_ENTER(p_sem, timeout, opt, p_ts);

/* 此宏用于使能系统时钟节拍 */
#if (OS_CFG_TICK_EN == 0u)
    if (timeout != 0u) {
        *p_err = OS_ERR_TICK_DISABLED;
        OS_TRACE_SEM_PEND_FAILED(p_sem);
        OS_TRACE_SEM_PEND_EXIT(OS_ERR_TICK_DISABLED);
        return (0u);
    }
#endif

/* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器,
     * 当 OSIntNestingCtr 大于 0 时, 说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数,
     * 因此返回错误
     */
    if (OSIntNestingCtr > 0u) {
        if ((opt & OS_OPT_PEND_NON_BLOCKING) != OS_OPT_PEND_NON_BLOCKING) {
            OS_TRACE_SEM_PEND_FAILED(p_sem);
            OS_TRACE_SEM_PEND_EXIT(OS_ERR_PEND_ISR);
            *p_err = OS_ERR_PEND_ISR;
            return (0u);
        }
    }
#endif

/* 此宏用于使能检查 μC/OS-III 系统是否运行 */
```

```
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    /* 如果 μC/OS-III 系统还未运行,
     * 那么是不能获取信号量的
     */
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_SEM_PEND_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    /* 指向信号量结构体的指针不能为空 */
    if (p_sem == (OS_SEM *)0) {
        OS_TRACE_SEM_PEND_EXIT(OS_ERR_OBJ_PTR_NULL);
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }

    /* 检查函数的操作选项是否有效 */
    switch (opt) {
        case OS_OPT_PEND_BLOCKING:
        case OS_OPT_PEND_NON_BLOCKING:
            break;

        default:
            OS_TRACE_SEM_PEND_FAILED(p_sem);
            OS_TRACE_SEM_PEND_EXIT(OS_ERR_OPT_INVALID);
            *p_err = OS_ERR_OPT_INVALID;
            return (0u);
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    /* 此函数只能用于获取信号量 */
    if (p_sem->Type != OS_OBJ_TYPE_SEM) {
        OS_TRACE_SEM_PEND_FAILED(p_sem);
        OS_TRACE_SEM_PEND_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
}
```

```
#endif

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 判断信号量是否有资源 */
if (p_sem->Ctr > 0u) {
    /* 更新信号量资源数 */
    p_sem->Ctr--;
#if (OS_CFG_TS_EN > 0u)
    if (p_ts != (CPU_TS *)0) {
        *p_ts = p_sem->TS;
    }
#endif
    ctr = p_sem->Ctr;
    /* 用于调试，不用理会 */
    OS_TRACE_SEM_PEND(p_sem);
    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    /* 用于调试，不用理会 */
    OS_TRACE_SEM_PEND_EXIT(OS_ERR_NONE);
    *p_err = OS_ERR_NONE;
    /* 返回信号量更新后的资源数 */
    return (ctr);
}

/* 判断任务是否不挂起等待信号量 */
if ((opt & OS_OPT_PEND_NON_BLOCKING) != 0u) {
#if (OS_CFG_TS_EN > 0u)
    if (p_ts != (CPU_TS *)0) {
        *p_ts = 0u;
    }
#endif
    /* 任务如果不选择挂起等待信号量
     * 则退出临界区，并返回信号量当前的资源数 (0)
     */
    ctr = p_sem->Ctr;
    CPU_CRITICAL_EXIT();
    OS_TRACE_SEM_PEND_FAILED(p_sem);
    OS_TRACE_SEM_PEND_EXIT(OS_ERR_PEND_WOULD_BLOCK);
    *p_err = OS_ERR_PEND_WOULD_BLOCK;
    return (ctr);
} else {
```

```

/* 如果任务选择挂起等待信号量,
 * 则接下来将要任务添加到信号量的挂起等待任务链表中挂起,
 * 并且需要先判断任务调度器是否被上锁,
 * 如果任务调度器被上锁,
 * 则直接返回相应的错误代码和信号量当前的资源数 (0)
 */
if (OSSchedLockNestingCtr > 0u) {
#ifdef OS_CFG_TS_EN > 0u
    if (p_ts != (CPU_TS *)0) {
        *p_ts = 0u;
    }
#endif

    CPU_CRITICAL_EXIT();
    OS_TRACE_SEM_PEND_FAILED(p_sem);
    OS_TRACE_SEM_PEND_EXIT(OS_ERR_SCHED_LOCKED);
    *p_err = OS_ERR_SCHED_LOCKED;
    return (0u);
}

/* 根据入参的超时时间将任务挂起到信号量的挂起等待任务链表中 */
OS_Pend((OS_PEND_OBJ *) ((void *)p_sem),
        OSTCBCurPtr,
        OS_TASK_PEND_ON_SEM,
        timeout);

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 用于调试, 不用理会 */
OS_TRACE_SEM_PEND_BLOCK(p_sem);

/* 触发任务调度,
 * 因为当前任务被挂起了
 */
OSSched();

/* 发起任务调度后,
 * 当前任务就被挂起了,
 * 因此下面的代码并不会立马被执行,
 * 只有当任务获取到信号量资源、任务挂起超时或任务挂起被中断后等情况下,
 * 才会执行下面的代码
 */

```

```
/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 根据任务不同的挂起结果进行不同的操作 */
switch (OSTCBCurPtr->PendStatus) {
    /* 成功获取信号量 */
    case OS_STATUS_PEND_OK:

#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#endif

        OS_TRACE_SEM_PEND(p_sem);
        *p_err = OS_ERR_NONE;
        break;

    /* 任务挂起等待信号量被中断 */
    case OS_STATUS_PEND_ABORT:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#endif

        OS_TRACE_SEM_PEND_FAILED(p_sem);
        *p_err = OS_ERR_PEND_ABORT;
        break;

    /* 任务挂起等待信号量超时 */
    case OS_STATUS_PEND_TIMEOUT:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = 0u;
        }
#endif

        OS_TRACE_SEM_PEND_FAILED(p_sem);
        *p_err = OS_ERR_TIMEOUT;
        break;

    /* 任务挂起等待的信号量被删除 */
    case OS_STATUS_PEND_DEL:
```

```
#if (OS_CFG_TS_EN > 0u)
    if (p_ts != (CPU_TS *)0) {
        *p_ts = OSTCBCurPtr->TS;
    }
#endif

OS_TRACE_SEM_PEND_FAILED(p_sem);
*p_err = OS_ERR_OBJ_DEL;
break;

/* 无效的任务挂起等待结果 */
default:
    OS_TRACE_SEM_PEND_FAILED(p_sem);
    *p_err = OS_ERR_STATUS_INVALID;
    CPU_CRITICAL_EXIT();
    OS_TRACE_SEM_PEND_EXIT(*p_err);
    return (0u);
}

/* 退出临界区，
 * 并返回信号量当前的资源数
 */
ctr = p_sem->Ctr;
CPU_CRITICAL_EXIT();
OS_TRACE_SEM_PEND_EXIT(*p_err);
return (ctr);
}
```

4. 函数 OSSemPendAbort()

该函数用于终止挂起等待信号量的任务。

函数 OSSemPendAbort()的函数原型，如下所示：

```
OS_OBJ_QTY OSSemPendAbort( OS_SEM  *p_sem,
                           OS_OPT  opt,
                           OS_ERR  *p_err)
```

函数 OSSemPendAbort()的形参描述，如下表所示：

形参	描述
p_sem	指向信号量结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 11.2.11 函数 OSSemPendAbort()形参相关描述

函数 OSSemPendAbort()的返回值描述，如下表所示：

返回值	描述
OS_OBJ_QTY 类型返回值	被终止挂起的任务数量

表 11.2.12 函数 OSSemPendAbort()返回值相关描述

函数 OSSemPendAbort()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	终止任务挂起等待信号量成功
OS_ERR_OBJ_PTR_NULL	指向信号量结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT_ISR	在中断中非法调用该函数
OS_ERR_PEND_ABORT_NONE	没有任务挂起等待该信号量

表 11.2.13 函数 OSSemPendAbort()错误代码相关描述

函数 OSSemPendAbort()定义在文件 os_sem.c 中，具体的代码如下所示：

```
OS_OBJ_QTY OSSemPendAbort( OS_SEM  *p_sem,
                           OS_OPT  opt,
                           OS_ERR  *p_err)
{
    OS_PEND_LIST *p_pend_list;
    OS_TCB      *p_tcb;
    CPU_TS      ts;
    OS_OBJ_QTY  nbr_tasks;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_PEND_ABORT_ISR;
            return (0u);
        }
    #endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        if (OSRunning != OS_STATE_OS_RUNNING) {
            *p_err = OS_ERR_OS_NOT_RUNNING;
            return (0u);
        }
    #endif
}
```

```
/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_sem == (OS_SEM *)0) {
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }

    switch (opt) {
        case OS_OPT_PEND_ABORT_1:
        case OS_OPT_PEND_ABORT_ALL:
        case OS_OPT_PEND_ABORT_1 | OS_OPT_POST_NO_SCHED:
        case OS_OPT_PEND_ABORT_ALL | OS_OPT_POST_NO_SCHED:
            break;

        default:
            *p_err = OS_ERR_OPT_INVALID;
            return (0u);
    }
#endif

/* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_sem->Type != OS_OBJ_TYPE_SEM) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 获取信号量的挂起等待任务链表 */
p_pend_list = &p_sem->PendList;

/* 没有任务挂起等待该信号量，
 * 则直接返回
 */
if (p_pend_list->HeadPtr == (OS_TCB *)0) {
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_PEND_ABORT_NONE;
    return (0u);
}
```

```

/* 记录被终止挂起的任务的数量 */
nbr_tasks = 0u;

#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif

/* 遍历终止所有被挂起的任务 */
while (p_pend_list->HeadPtr != (OS_TCB *)0) {
    /* 获取信号量的挂起等待任务链表 */
    p_tcb = p_pend_list->HeadPtr;
    /* 终止任务的挂起状态
     * 同时会设置任务的挂起结果为“任务挂起被中断”
     */
    OS_PendAbort(    p_tcb,
                    ts,
                    OS_STATUS_PEND_ABORT);

    /* 更新被终止挂起的任务的数量 */
    nbr_tasks++;

    /* 函数的操作选项
     * 可以设置是终止所有挂起的任务
     * 还是终止信号量挂起等待任务链表中的第一个任务
     */
    if (opt != OS_OPT_PEND_ABORT_ALL) {
        break;
    }
}

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 函数的操作选项
 * 可以选择是否进行任务调度
 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

/* 返回被终止挂起的任务的数量 */
*p_err = OS_ERR_NONE;
return (nbr_tasks);

```

```
}
```

5. OSSemPost()

该函数用于释放信号量。

函数 OSSemPost() 的函数原型，如下所示：

```
OS_SEM_CTR OSSemPost( OS_SEM *p_sem,
                      OS_OPT opt,
                      OS_ERR *p_err)
```

函数 OSSemPost() 的形参描述，如下表所示：

形参	描述
p_sem	指向信号量结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 11.2.14 函数 OSSemPost() 形参相关描述

函数 OSSemPost() 的返回值描述，如下表所示：

返回值	描述
OS_SEM_CTR 类型返回值	信号量资源数更新后的值

表 11.2.15 函数 OSSemPost() 返回值相关描述

函数 OSSemPost() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	释放信号量成功
OS_ERR_OBJ_PTR_NULL	指向信号量结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_SEM_OVF	信号量资源数溢出

表 11.2.16 函数 OSSemPost() 错误代码相关描述

函数 OSSemPost() 定义在文件 os_sem.c 中，具体的代码如下所示：

```
OS_SEM_CTR OSSemPost( OS_SEM *p_sem,
                      OS_OPT opt,
                      OS_ERR *p_err)
{
    OS_SEM_CTR    ctr;
    OS_PEND_LIST  *p_pend_list;
    OS_TCB        *p_tcb;
    OS_TCB        *p_tcb_next;
    CPU_TS        ts;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif
}
```

```

    }
#endif

    /* 用于调试，不用理会 */
    OS_TRACE_SEM_POST_ENTER(p_sem, opt);

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        if (OSRunning != OS_STATE_OS_RUNNING) {
            OS_TRACE_SEM_POST_EXIT(OS_ERR_OS_NOT_RUNNING);
            *p_err = OS_ERR_OS_NOT_RUNNING;
            return (0u);
        }
    #endif

    /* 此宏用于使能参数检查 */
    #if (OS_CFG_ARG_CHK_EN > 0u)
        if (p_sem == (OS_SEM *)0) {
            OS_TRACE_SEM_POST_FAILED(p_sem);
            OS_TRACE_SEM_POST_EXIT(OS_ERR_OBJ_PTR_NULL);
            *p_err = OS_ERR_OBJ_PTR_NULL;
            return (0u);
        }
        switch (opt) {
            case OS_OPT_POST_1:
            case OS_OPT_POST_ALL:
            case OS_OPT_POST_1 | OS_OPT_POST_NO_SCHED:
            case OS_OPT_POST_ALL | OS_OPT_POST_NO_SCHED:
                break;

            default:
                OS_TRACE_SEM_POST_FAILED(p_sem);
                OS_TRACE_SEM_POST_EXIT(OS_ERR_OPT_INVALID);
                *p_err = OS_ERR_OPT_INVALID;
                return (0u);
        }
    #endif

    /* 此宏用于使能检查内核对象类型 */
    #if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
        if (p_sem->Type != OS_OBJ_TYPE_SEM) {
            OS_TRACE_SEM_POST_FAILED(p_sem);
            OS_TRACE_SEM_POST_EXIT(OS_ERR_OBJ_TYPE);
        }
    #endif

```

```

        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif

    OS_TRACE_SEM_POST(p_sem);

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();
    /* 获取信号量的挂起等待任务链表
     * 因为可以直接将资源释放给挂起等待的任务
     */
    p_pend_list = &p_sem->PendList;
    /* 判断是否没有任务挂起等待信号量 */
    if (p_pend_list->HeadPtr == (OS_TCB *)0) {
        /* 判断信号量的资源数是否溢出 */
        if (p_sem->Ctr == (OS_SEM_CTR)-1) {
            CPU_CRITICAL_EXIT();
            *p_err = OS_ERR_SEM_OVF;
            OS_TRACE_SEM_POST_EXIT(*p_err);
            return (0u);
        }
        /* 更新信号量的资源数 */
        p_sem->Ctr++;
        ctr = p_sem->Ctr;
    }
    #if (OS_CFG_TS_EN > 0u)
        p_sem->TS = ts;
    #endif

    /* 退出临界区，
     * 并返回信号量的资源数
     */
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_NONE;
    OS_TRACE_SEM_POST_EXIT(*p_err);
    return (ctr);
}

```

```

/* 信号量挂起等待任务链表中有挂起的任务，
 * 此时就要结束任务的挂起状态，
 * 并将资源释放给该任务
 */

/* 获取信号量挂起等待任务链表 */
p_tcb = p_pend_list->HeadPtr;

/* 遍历信号量挂起等待任务链表中的所有任务 */
while (p_tcb != (OS_TCB *)0) {
    p_tcb_next = p_tcb->PendNextPtr;
    /* 结束任务的挂起状态 */
    OS_Post((OS_PEND_OBJ *)((void *)p_sem),
            p_tcb,
            (void *)0,
            0u,
            ts);

    /* 函数的操作选项可以选择
     * 仅解除信号量挂起等待任务链表中一个任务的终止状态，
     * 还是解除信号量挂起等待任务链表中所有任务的终止状态
     */
    if ((opt & OS_OPT_POST_ALL) == 0u) {
        break;
    }

    /* 遍历下一个信号量挂起等待任务链表中的任务 */
    p_tcb = p_tcb_next;
}

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 函数的操作选项
 * 可以选择是否进行任务调度
 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

*p_err = OS_ERR_NONE;
OS_TRACE_SEM_POST_EXIT(*p_err);
return (0u);
}

```

6. 函数 OSSemSet()

该函数用于强制设置信号量的资源数。

函数 OSSemSet() 的函数原型，如下所示：

```
void OSSemSet( OS_SEM      *p_sem,
               OS_SEM_CTR  cnt,
               OS_ERR      *p_err)
```

函数 OSSemSet() 的形参描述，如下表所示：

形参	描述
p_sem	指向信号量结构体的指针
cnt	信号量资源数设置后的值
p_err	指向接收错误代码变量的指针

表 11.2.17 函数 OSSemSet() 形参相关描述

函数 OSSemSet() 的返回值描述，如下表所示：

返回值	描述
无	无

表 11.2.18 函数 OSSemSet() 返回值相关描述

函数 OSSemSet() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	强制设置信号量资源数成功
OS_ERR_OBJ_PTR_NULL	指向信号量结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是信号量
OS_ERR_SET_ISR	在中断中非法调用该函数
OS_ERR_TASK_WAITING	有任务挂起等待信号量，不能修改该信号量的资源数

表 11.2.19 函数 OSSemSet() 错误代码相关描述

函数 OSSemSet() 定义在文件 os_sem.c 中，具体的代码如下所示：

```
void OSSemSet( OS_SEM      *p_sem,
               OS_SEM_CTR  cnt,
               OS_ERR      *p_err)
{
    OS_PEND_LIST *p_pend_list;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        if (OSIntNestingCtr > 0u) {
```



```

        *p_err = OS_ERR_SET_ISR;
        return;
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_sem == (OS_SEM *)0) {
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return;
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_sem->Type != OS_OBJ_TYPE_SEM) {
        *p_err = OS_ERR_OBJ_TYPE;
        return;
    }
#endif

    *p_err = OS_ERR_NONE;
    /* 进入临界区 */
    CPU_CRITICAL_ENTER();
    /* 如果信号量的资源数大于 0
     * 说明没有任务挂起等待该信号量，
     * 那么可以直接设置信号量的资源数
     */
    if (p_sem->Ctr > 0u) {
        p_sem->Ctr = cnt;
    } else {
        /* 如果信号量的资源数不大于 0
         * 那么有可能该信号量存在挂起等待的任务
         * 如果有，那么不能设置信号量的资源数
         */
        p_pend_list = &p_sem->PendList;
        if (p_pend_list->HeadPtr == (OS_TCB *)0) {
            p_sem->Ctr = cnt;
        } else {
            *p_err = OS_ERR_TASK_WAITING;
        }
    }
}

/* 退出临界区 */

```

```
CPU_CRITICAL_EXIT();
}
```

11.3 μC/OS-III 共享资源冲突访问实验

11.3.1 功能设计

1. 例程功能

本实验主要用于演示在不使用信号量的情况下，多任务访问同一个共享资源会出现的问题，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建其他任务
task1	访问共享资源
task2	访问共享资源

表 11.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 11-1 UCOS-III 共享资源冲突访问实验》。

11.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

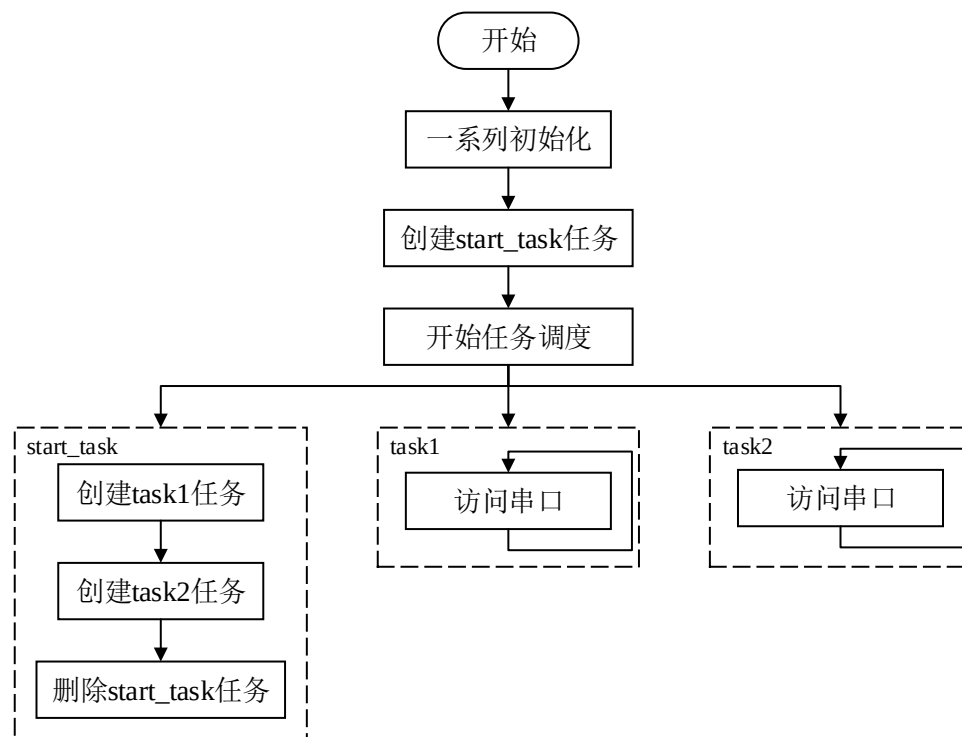


图 11.3.1 程序流程图

2. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief   start_task
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 1, &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                    (CPU_CHAR*    ) "task1",
                    (OS_TASK_PTR  ) task1,
                    (void*        ) 0,
                    (OS_PRIO      ) TASK1_PRIO,
                    (CPU_STK*     ) Task1Task_STK,
                    (CPU_STK_SIZE ) TASK1_STK_SIZE / 10,
                    (CPU_STK_SIZE ) TASK1_STK_SIZE,
                    (OS_MSG_QTY   ) 0,
                    (OS_TICK      ) 0,
                    (void*        ) 0,
                    (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                    (OS_ERR*      ) &err);

    /* 创建 Task2 */
    Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK2_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*      ) &Task2Task_TCB,
```

```

        (CPU_CHAR*      ) "task2",
        (OS_TASK_PTR    ) task2,
        (void*          ) 0,
        (OS_PRIO        ) TASK2_PRIO,
        (CPU_STK*       ) Task2Task_STK,
        (CPU_STK_SIZE   ) TASK2_STK_SIZE / 10,
        (CPU_STK_SIZE   ) TASK2_STK_SIZE,
        (OS_MSG_QTY     ) 0,
        (OS_TICK        ) 0,
        (void*          ) 0,
        (OS_OPT          ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*        ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出, start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务和 task2 任务, 并且 task1 任务和 task2 任务的任务栈都是动态分配的, 当然, 也可以静态地定任务栈, 就像定义一个数组一样。

(2) task1 和 task2 任务

```

/**
 * @brief   task1
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task1(void *p_arg)
{
    while (1)
    {
        printf("This is task1!\r\n");
    }
}

/**
 * @brief   task2
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task2(void *p_arg)
{
    while (1)
    {
        printf("This is task2!\r\n");
    }
}

```

```
}  
}
```

从以上代码中可以看到，task1 和 task2 任务分别访问同一个串口资源，因为是操作系统，多个任务是“同时”运行的，因此这里可以采猜测，task1 任务和 task2 任务会同时访问串口资源，产生冲突，这一点到本实验的下载验证中可以验证。

11.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 11.3.2 LCD 显示内容

同时，通过串口调试助手，就能够看到输出的打印信息，如下图所示：

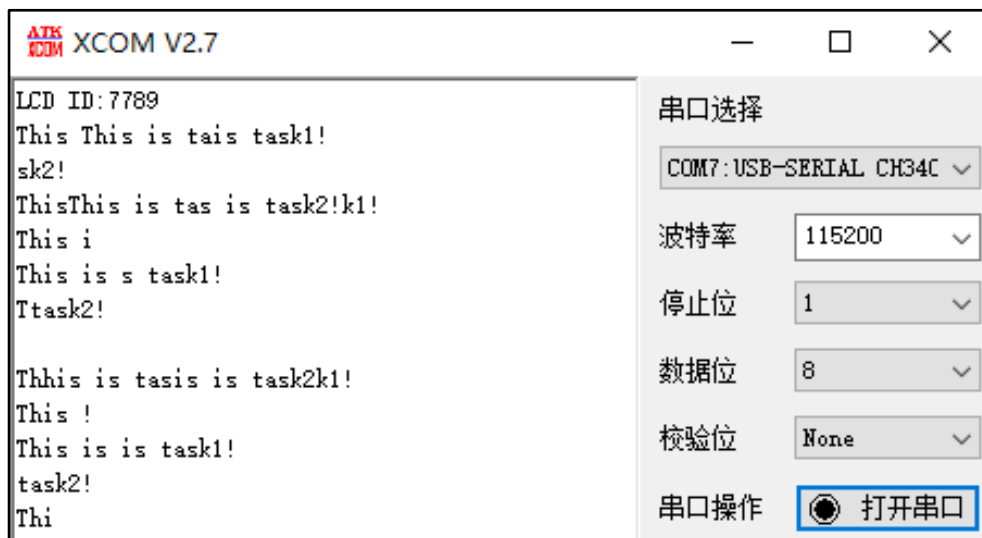


图 13.3.3 串口调试助手显示内容

从上图可以看出，因为 task1 任务与 task2 任务同时访问串口资源，导致串口的输出内容看起来很乱，这是因为在多任务访问同一个资源时，可能会出现一个任务对共享资源的访问还未

结束时，当 CPU 却被其他任务抢占了，而其他任务也访问了这个共享资源，因此就做不到对共享资源的有序访问。那么要如何解决这种问题呢？那就能够使用信号量，在下一小节中，将演示如何使用二值信号量解决共享资源访问冲突的问题。

11.4 μC/OS-III 二值信号量实验

11.4.1 功能设计

1. 例程功能

本实验主要用于演示如何使用二值信号量解决多任务访问共享资源时的冲突问题，做到对共享资源的有序访问，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建二值信号量和其他任务
task1	访问共享资源
task2	访问共享资源

表 11.4.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 11-2 UCOS-III 二值信号量实验》。

11.4.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

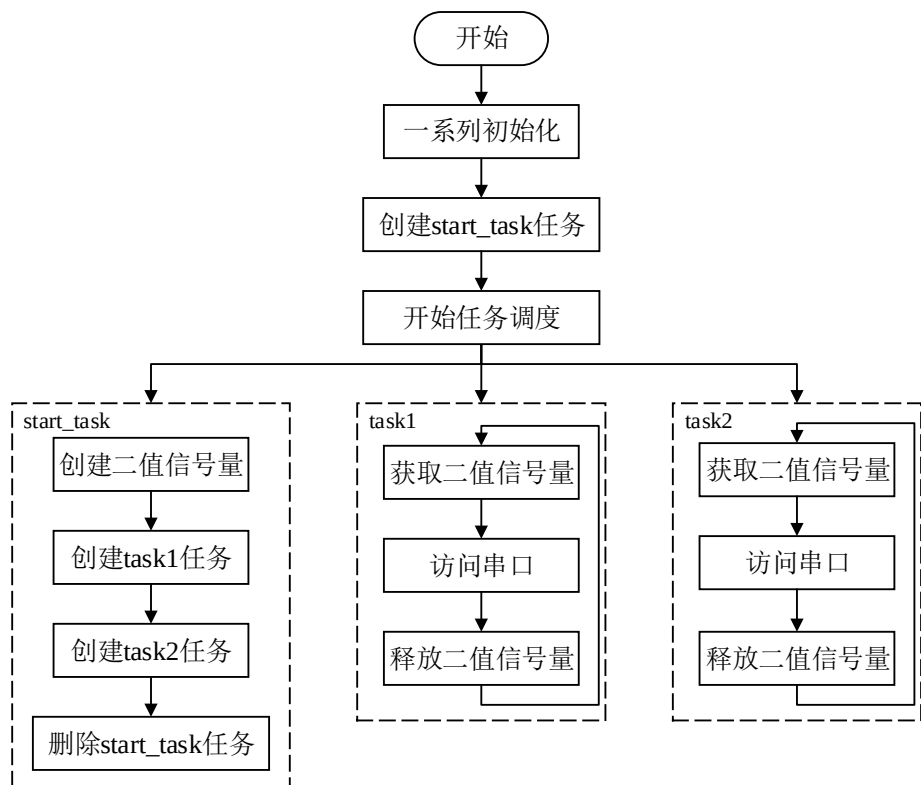


图 11.4.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSSemCreate()

此函数用于创建信号量，请参考第 11.2 小节《μC/OS-III 二值信号量》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SEM_EN 配置为 1。

(2) 函数 OSSemPend()

此函数用于获取信号量，请参考第 11.2 小节《μC/OS-III 二值信号量》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SEM_EN 配置为 1。

(3) 函数 OSSemPost()

此函数用于释放信号量，请参考第 11.2 小节《μC/OS-III 二值信号量》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SEM_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief   start_task
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 1, &err);

    /* 创建二值信号量 */
    OSSemCreate((OS_SEM*) &binary_sem,
                (CPU_CHAR*) "binary sem",
                (OS_SEM_CTR) 1,
                (OS_ERR*) &err);

    /* 创建 Task1 */
}
```

```

Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK1_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                 (CPU_CHAR*    ) "task1",
                 (OS_TASK_PTR  ) task1,
                 (void*        ) 0,
                 (OS_PRIO      ) TASK1_PRIO,
                 (CPU_STK*     ) Task1Task_STK,
                 (CPU_STK_SIZE ) TASK1_STK_SIZE / 10,
                 (CPU_STK_SIZE ) TASK1_STK_SIZE,
                 (OS_MSG_QTY   ) 0,
                 (OS_TICK      ) 0,
                 (void*        ) 0,
                 (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*      ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      ) &Task2Task_TCB,
                 (CPU_CHAR*    ) "task2",
                 (OS_TASK_PTR  ) task2,
                 (void*        ) 0,
                 (OS_PRIO      ) TASK2_PRIO,
                 (CPU_STK*     ) Task2Task_STK,
                 (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,
                 (CPU_STK_SIZE ) TASK2_STK_SIZE,
                 (OS_MSG_QTY   ) 0,
                 (OS_TICK      ) 0,
                 (void*        ) 0,
                 (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*      ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出, start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务和 task2 任务, 并且 task1 任务和 task2 任务的任务栈都是动态分配的, 当然, 也可以静态地定任务栈, 就像定义一个数组一样。同时还创建一个用于保护共享资源的二值信号量。

(2) task1 和 task2 任务

```

/**
 * @brief   task1
 * @param   p_arg : 传入参数(未用到)

```



```
* @retval 无
*/
void task1(void *p_arg)
{
    OS_ERR err;

    while (1)
    {
        /* 获取二值信号量 */
        OSSemPend( (OS_SEM*   ) &binary_sem,
                   (OS_TICK   ) 0,
                   (OS_OPT     ) OS_OPT_PEND_BLOCKING,
                   (CPU_TS*    ) 0,
                   (OS_ERR*    ) &err);

        /* 访问共享资源 */
        printf("This is task1!\r\n");

        /* 释放二值信号量 */
        OSSemPost( (OS_SEM*   ) &binary_sem,
                   (OS_OPT     ) OS_OPT_POST_1,
                   (OS_ERR*    ) &err);
    }
}

/**
 * @brief task2
 * @param p_arg : 传入参数(未用到)
 * @retval 无
 */
void task2(void *p_arg)
{
    OS_ERR err;

    while (1)
    {
        /* 获取二值信号量 */
        OSSemPend( (OS_SEM*   ) &binary_sem,
                   (OS_TICK   ) 0,
                   (OS_OPT     ) OS_OPT_PEND_BLOCKING,
                   (CPU_TS*    ) 0,
                   (OS_ERR*    ) &err);
```

```

/* 访问共享资源 */
printf("This is task2!\r\n");

/* 释放二值信号量 */
OSSemPost( (OS_SEM*) &binary_sem,
            (OS_OPT) OS_OPT_POST_1,
            (OS_ERR*) &err);
}
}

```

从以上代码中可以看到，与上一个实验一样是多任务访问同一个串口资源，但是在访问串口资源前，会先获取二值信号量，如果能成功获取，那么才能够正常地访问串口资源，如果二值信号量已经被其他任务获取了，那么二值信号量就没有资源，此时获取二值信号量就会被挂起。在使用完二值信号量之后，要立马释放二值信号量，让其他任务有使用共享资源的机会。

11.4.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 11.4.2 LCD 显示内容

同时，通过串口调试助手，就能够看到输出的打印信息，如下图所示：



图 13.4.3 串口调试助手显示内容

从上图可以看出，虽然 task1 和 task2 任务依然还是在一直访问串口资源，但是，串口输出的内容却很整洁，这正是二值信号量的功劳，使得多个任务对同一个共享资源的访问变得有序。

11.5 μC/OS-III 计数型信号量

计数型信号量与二值信号量是很相似的，二值信号量的最大资源数只能是 1，因此只有“有资源”和“无资源”两种情况，但计数型信号量的资源数不限于 0 和 1 两种情况，计数型信号量的资源数是可以递归的，只要有任务释放信号量，信号量的资源数就会被加 1，直到溢出，只要有任务获取信号量，那么信号量的资源数就会被减 1，直到 0。

计数型信号量相对于二值信号量，适用于当共享资源可以被多个任务访问的情况。因为计数型信号量于二值信号量十分相似，因此 μC/OS-III 提供的信号量相关的 API 函数，是计数型信号量和二值信号量公用的，并且使用起来也没有什么很大的区别。

11.6 μC/OS-III 计数型信号量实验

11.6.1 功能设计

1. 例程功能

本实验主要用于调用 μC/OS-III 提供的信号量相关的 API 函数使用计数型信号量，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建计数型信号量和其他任务
task1	释放计数型信号量
task2	获取计数型信号量

表 11.6.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 11-3 UCOS-III 计数型信号量实验》。

11.6.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

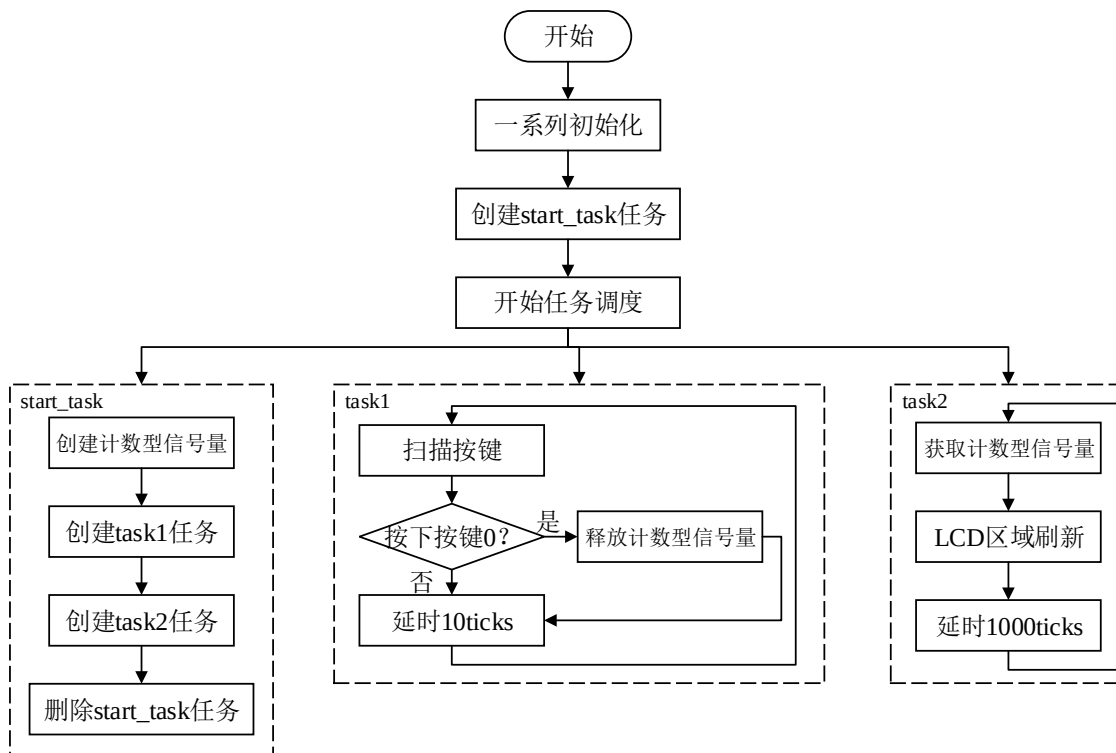


图 11.6.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSSemCreate()

此函数用于创建信号量，请参考第 11.2 小节《μC/OS-III 二值信号量》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SEM_EN 配置为 1。

(2) 函数 OSSemPend()

此函数用于获取信号量，请参考第 11.2 小节《μC/OS-III 二值信号量》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SEM_EN 配置为 1。

(3) 函数 OSSemPost()

此函数用于释放信号量，请参考第 11.2 小节《μC/OS-III 二值信号量》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SEM_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```

/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)
    
```

```
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 1, &err);

    /* 创建计数型信号量 */
    OSSemCreate( (OS_SEM*   )&count_sem,
                 (CPU_CHAR* )"count sem",
                 (OS_SEM_CTR)0,
                 (OS_ERR*   )&err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));
    OSTaskCreate( (OS_TCB*   )&Task1Task_TCB,
                  (CPU_CHAR* )"task1",
                  (OS_TASK_PTR)task1,
                  (void*     )0,
                  (OS_PRIO   )TASK1_PRIO,
                  (CPU_STK*  )Task1Task_STK,
                  (CPU_STK_SIZE)TASK1_STK_SIZE / 10,
                  (CPU_STK_SIZE)TASK1_STK_SIZE,
                  (OS_MSG_QTY)0,
                  (OS_TICK   )0,
                  (void*     )0,
                  (OS_OPT     )(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                  (OS_ERR*   )&err);

    /* 创建 Task2 */
    Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK2_STK_SIZE * sizeof(CPU_STK));
    OSTaskCreate( (OS_TCB*   )&Task2Task_TCB,
                  (CPU_CHAR* )"task2",
                  (OS_TASK_PTR)task2,
```

```

        (void*      )0,
        (OS_PRIO    )TASK2_PRIO,
        (CPU_STK*    )Task2Task_STK,
        (CPU_STK_SIZE)TASK2_STK_SIZE / 10,
        (CPU_STK_SIZE)TASK2_STK_SIZE,
        (OS_MSG_QTY  )0,
        (OS_TICK     )0,
        (void*      )0,
        (OS_OPT       )(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*     )&err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出, start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务和 task2 任务, 并且 task1 任务和 task2 任务的任务栈都是动态分配的, 当然, 也可以静态地定任务栈, 就像定义一个数组一样。同时还创建一个计数型信号量。

(2) task1 任务

```

/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    uint8_t key;
    OS_ERR err;

    while (1)
    {
        key = key_scan(0);

        switch (key)
        {
            case KEY0_PRES:
            {
                /* 释放计数型信号量 */
                OSSemPost((OS_SEM *)&count_sem, (OS_OPT)OS_OPT_POST_ALL, &err);
                lcd_show_xnum(180, 111, count_sem.Ctr, 2, 16, 0, BLUE);
                break;
            }
            default:
            {

```

```

        break;
    }
}

OSTimeDly(10, OS_OPT_TIME_DLY, &err);
}
}

```

从以上代码中可以看到，task1 任务主要负责扫描按键，当检测到按键 0 被按下时，释放计数型信号量，同时还会在 LCD 上显示计数型信号量的资源数，因为是计数型信号量，因此只要重复按下按键，那么计数型信号量的资源数就会一直递增。

(3) task2 任务

```

/**
 * @brief  task2
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task2(void *p_arg)
{
    uint32_t task2_num;
    OS_ERR err;

    while (1)
    {
        /* 获取计数型信号量 */
        OSSemPend( (OS_SEM*   ) &count_sem,
                  (OS_TICK   ) 0,
                  (OS_OPT    ) OS_OPT_PEND_BLOCKING,
                  (CPU_TS*   ) 0,
                  (OS_ERR*   ) &err);

        lcd_show_xnum(180, 111, count_sem.Ctr, 2, 16, 0, BLUE);
        lcd_fill(6, 131, 233, 313, lcd_discolor[++task2_num % 11]);

        OSTimeDly(1000, OS_OPT_TIME_DLY, &err);
    }
}

```

从以上代码中可以看到，task2 任务会每间隔 1000 个 ticks 就获取一次计数型信号量，如果计数型信号量有资源，那么就能成功获取到计数型信号量并刷新 LCD 的显示。

11.6.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 11.6.2 LCD 显示内容一

接着按一下按键 0，观察 LCD 的变化，如下图所示：



图 11.6.3 LCD 显示内容二

从上图中可以看到，当 task1 任务释放一次计数型信号量后，task2 就获取到了计数型信号量，并且立马刷新了 LCD 区域的显示，这也是计数型信号量的资源数依旧为 0 的原因。接下来快速地多次按下按键 0，使得 task1 任务快速地多次释放计数型信号量，观察 LCD 的变化，如下图所示：



图 11.6.4 LCD 显示内容三

此时可以看到，因为计数型信号量的资源数不为 0，因此 task2 任务就能够不断地获取到计数型信号量，并执行刷新 LCD 区域的操作，直到计数型信号量的资源数为 0 为止。

11.7 μC/OS-III 优先级翻转

在使用二值信号量和计数型信号量的时候，会经常地遇到优先级翻转的问题，优先级翻转的问题在抢占式内核中是很常见的，但是在实时操作系统中是不允许出现优先级翻转的现象的，因为优先级翻转会破坏任务执行的预期顺序，可能会导致未知的严重后果，下面就展示了一个优先级翻转的例子，如下图所示：

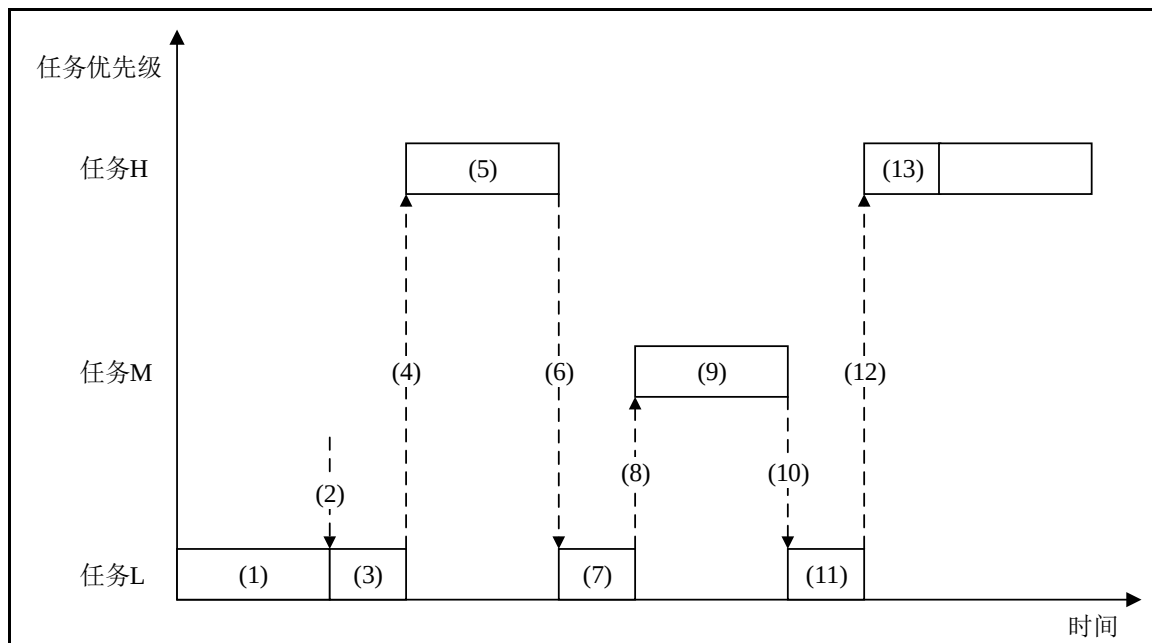


图 11.7.1 优先级翻转示意图

如上图所示，定义：任务 H 为优先级最高的任务，任务 L 为优先级最低的任务，任务 M 为优先级介于任务 H 与任务 L 之间的任务。

- (1) 任务 H 和任务 M 为挂起状态，等待某一事件发生，此时任务 L 正在运行。
- (2) 此时任务 L 要访问共享资源，因此需要获取信号量。
- (3) 任务 L 成功获取信号量，并且此时信号量已无资源，任务 L 开始访问共享资源。
- (4) 此时任务 H 就绪，抢占任务 L 运行。
- (5) 任务 H 开始运行。
- (6) 此时任务 H 要访问共享资源，因此需要获取信号量，但信号量已无资源，因此任务 H 挂起等待信号量资源。
- (7) 任务 L 继续运行。
- (8) 此时任务 M 就绪，抢占任务 L 运行。
- (9) 任务 M 正在运行。
- (10) 任务 M 运行完毕，继续挂起。
- (11) 任务 L 继续运行。
- (12) 此时任务 L 对共享资源的访问操作完成，释放信号量，虽有任务 H 因成功获取信号量，解除挂起状态并抢占任务 L 运行。
- (13) 任务 H 得以运行。

从上面优先级翻转的示例中，可以看出，任务 H 为最高优先级的任务，因此任务 H 执行的操作需要有较高的实时性，但是由于优先级翻转的问题，导致了任务 H 需要等到任务 L 释放信号量才能够运行，并且，任务 L 还会被其他介于任务 H 与任务 L 任务优先级之间的任务 M 抢占，因此任务 H 还需等待任务 M 运行完毕，这显然不符合任务 H 需要的高实时性要求。

11.8 μC/OS-III 优先级翻转实验

11.8.1 功能设计

1. 例程功能

本实验主要用于展示使用 μC/OS-III 的信号量带来的优先级翻转问题，本实验设计了四个任务，这四个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建信号量和其他任务
task1	用于演示 μC/OS-III 的信号量带来的优先级翻转问题
task2	用于演示 μC/OS-III 的信号量带来的优先级翻转问题
task3	用于演示 μC/OS-III 的信号量带来的优先级翻转问题

表 11.8.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 11-4 UCOS-III 优先级翻转实验》。

11.8.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

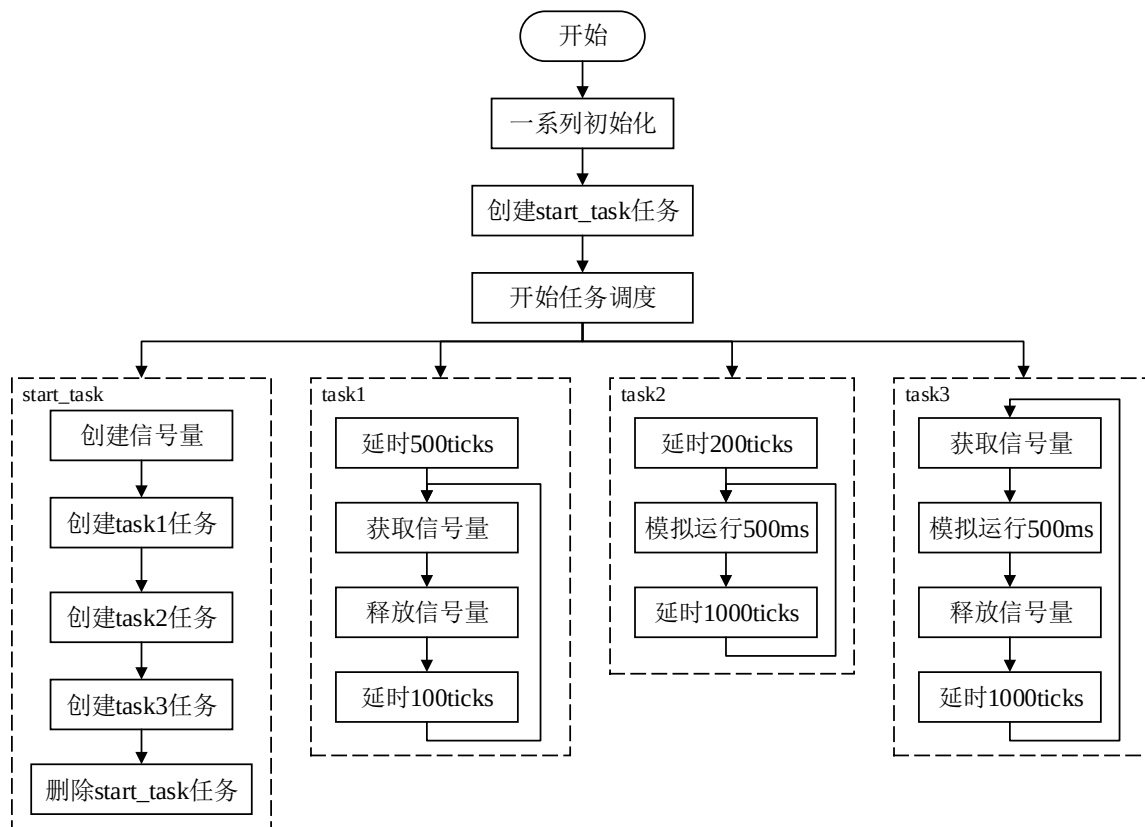


图 11.8.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSSemCreate()

此函数用于创建信号量，请参考第 11.2 小节《μC/OS-III 二值信号量》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SEM_EN 配置为 1。

(2) 函数 OSSemPend()

此函数用于获取信号量，请参考第 11.2 小节《μC/OS-III 二值信号量》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SEM_EN 配置为 1。

(3) 函数 OSSemPost()

此函数用于释放信号量，请参考第 11.2 小节《μC/OS-III 二值信号量》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_SEM_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```

/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)
    
```

```
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建信号量 */
    OSSemCreate( (OS_SEM*   ) &sem,
                 (CPU_CHAR* ) "sem",
                 (OS_SEM_CTR) 1,
                 (OS_ERR*   ) &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK1_STK_SIZE * sizeof(CPU_STK));
    OSTaskCreate( (OS_TCB*   ) &Task1Task_TCB,
                 (CPU_CHAR*   ) "task1",
                 (OS_TASK_PTR ) task1,
                 (void*       ) 0,
                 (OS_PRIO     ) TASK1_PRIO,
                 (CPU_STK*    ) Task1Task_STK,
                 (CPU_STK_SIZE) TASK1_STK_SIZE / 10,
                 (CPU_STK_SIZE) TASK1_STK_SIZE,
                 (OS_MSG_QTY  ) 0,
                 (OS_TICK     ) 0,
                 (void*       ) 0,
                 (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*     ) &err);

    /* 创建 Task2 */
    Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK2_STK_SIZE * sizeof(CPU_STK));
    OSTaskCreate( (OS_TCB*   ) &Task2Task_TCB,
                 (CPU_CHAR*   ) "task2",
                 (OS_TASK_PTR ) task2,
```

```

        (void*          )0,
        (OS_PRIO        )TASK2_PRIO,
        (CPU_STK*       )Task2Task_STK,
        (CPU_STK_SIZE   )TASK2_STK_SIZE / 10,
        (CPU_STK_SIZE   )TASK2_STK_SIZE,
        (OS_MSG_QTY     )0,
        (OS_TICK        )0,
        (void*          )0,
        (OS_OPT          )(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*        )&err);

/* 创建 Task3 */
Task3Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                       TASK3_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*          )&Task3Task_TCB,
                (CPU_CHAR*        )"task3",
                (OS_TASK_PTR      )task3,
                (void*            )0,
                (OS_PRIO          )TASK3_PRIO,
                (CPU_STK*         )Task3Task_STK,
                (CPU_STK_SIZE     )TASK3_STK_SIZE / 10,
                (CPU_STK_SIZE     )TASK3_STK_SIZE,
                (OS_MSG_QTY       )0,
                (OS_TICK          )0,
                (void*            )0,
                (OS_OPT            )(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                (OS_ERR*          )&err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务、task2 任务和 task3 任务，并且 task1 任务、task2 任务和 task3 任务的任务栈都是动态分配的，当然，也可以静态地定任务栈，就像定义一个数组一样。同时还创建一个计数型信号量。

(2) task1 任务、task2 任务和 task3 任务

```

/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    OS_ERR err;

```

```

OSTimeDly(500, OS_OPT_TIME_DLY, &err);
while (1)
{
    printf("task1 ready to pend semaphore\r\n");
    OSSemPend( (OS_SEM*   ) &sem,
               (OS_TICK   ) 0,
               (OS_OPT    ) OS_OPT_PEND_BLOCKING,
               (CPU_TS*   ) 0,
               (OS_ERR*   ) &err);

    printf("task1 has pended semaphore\r\n");
    printf("task1 runing\r\n");
    printf("task1 post semaphore\r\n");
    OSSemPost( (OS_SEM*   ) &sem,
               (OS_OPT    ) OS_OPT_POST_ALL,
               (OS_ERR*   ) &err);
    OSTimeDly(100, OS_OPT_TIME_DLY, &err);
}
}

/**
 * @brief   task2
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task2(void *p_arg)
{
    uint32_t task2_num = 0;
    OS_ERR err;

    OSTimeDly(200, OS_OPT_TIME_DLY, &err);
    while (1)
    {
        for (task2_num=0; task2_num<5; task2_num++)
        {
            printf("task2 runing\r\n");
            delay_ms(100);
        }
        OSTimeDly(1000, OS_OPT_TIME_DLY, &err);
    }
}

/**

```

```

* @brief   task3
* @param   p_arg : 传入参数(未用到)
* @retval  无
*/
void task3(void *p_arg)
{
    uint32_t task3_num = 0;
    OS_ERR err;

    while (1)
    {
        printf("task3 ready to pend semaphore\r\n");
        OSSemPend( (OS_SEM*   ) &sem,
                   (OS_TICK   ) 0,
                   (OS_OPT     ) OS_OPT_PEND_BLOCKING,
                   (CPU_TS*    ) 0,
                   (OS_ERR*    ) &err);

        printf("task3 has pend semaphore\r\n");
        for (task3_num=0; task3_num<5; task3_num++)
        {
            printf("task3 runing\r\n");
            delay_ms(100);
        }
        printf("task3 post semaphore\r\n");
        OSSemPost( (OS_SEM*   ) &sem,
                   (OS_OPT     ) OS_OPT_POST_ALL,
                   (OS_ERR*    ) &err);

        OSTimeDly(1000, OS_OPT_TIME_DLY, &err);
    }
}

```

以上 task1 任务、task2 任务和 task3 任务就是展示了优先级翻转的问题。

11.8.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 11.8.2 LCD 显示内容

同时，通过串口打印了本次实验的相关信息，如下图所示：



图 11.8.3 串口调试助手

1. task3 成功获取信号量，并运行。
2. task2 任务抢占 task3 任务运行。
3. task1 任务抢占 task2 任务运行，task1 任务获取信号量失败，被挂起。
4. task2 任务继续运行。
5. task2 任务运行完毕，task3 任务运行。
6. task3 任务释放信号量，task1 任务获取信号量，解除挂起状态，得以运行。

从串口打印的信息就能看出，优先级最高的 task1 任务，最后才得到 CPU 使用权，这就是优先级翻转带来的问题。那有没有办法解决优先级翻转的问题呢？答案是肯定的，在下一章中，将讲解 μC/OS-III 中的另一种信号量，它就能够很好的优化优先级翻转带来的问题。

第十二章 μC/OS-III 互斥信号量

在上一章中讲解了 μC/OS-III 中的信号量，并且知道了信号量更够解决多任务访问共享资源时带来的冲突问题，但是也发现了信号量会带来另一个无法避免的问题——优先级翻转，并且在实时操作系统中，优先级翻转的问题可能会带来非常严重的后果，为此，本章介绍 μC/OS-III 中的互斥信号量，互斥信号量就能够很好地优化优先级翻转带来的问题。本章就来学习 μC/OS-III 中的互斥信号量。

本章分为如下几部分：

12.1 μC/OS-III 互斥信号量简介

12.2 μC/OS-III 互斥信号量相关 API 函数

12.3 μC/OS-III 互斥信号量实验

12.4 μC/OS-III 互斥信号量优化优先级翻转实验

12.1 μC/OS-III 互斥信号量简介

互斥信号量也叫互斥锁，可以理解为是一种特殊的二值信号量，相对于上一章中讲解的二值信号量，互斥信号量拥有优先级继承的机制使得互斥信号量能够在一定的程度上解决优先级翻转的问题。互斥信号量一般用于那些需要互斥访问的应用中。在互斥访问的应用中，互斥信号量就相当于是一把钥匙，当任务想要访问共享资源的时候，就必须先获取到这把钥匙，当任务访问完共享资源后，就必须归还这把钥匙，这样其他任务就可以拿着这把钥匙再去访问这个共享资源。

互斥信号量的优先级继承机制体现在，当一个互斥信号量正被一个低优先级的任务持有时，如果此时有一个高优先级的任务也尝试获取这个互斥信号量，那么这个高优先级的任务就会因获取不到互斥锁而被挂起，不过接下来，高优先级的任务会将持有互斥信号量的低优先级任务的任务优先级提成到与高优先级任务的任务优先级相同的任务优先级，这个过程就是优先级继承。优先级继承可以尽可能地减少高优先级任务挂起等待互斥锁的时间，并且将优先级翻转问题带来的影响降到最低。

但是优先级继承并不是能完全解决优先级翻转带来的问题，因为优先级继承仅仅是将持有互斥信号量的低优先级任务的任务优先级提高的与高优先级任务相同的任务优先级，而非直接将互斥信号量直接从低优先级任务手上“抢”过来，因此高优先级的任务还是需要等待低优先级的任务释放互斥信号量，高优先级的任务才能够获取到互斥信号量。

下面先看一些互斥信号量的结构体定义，互斥信号量结构体定义在文件 os.h 中，具体的代码如下所示：

```
struct os_mutex{
    /* 此宏用于使能检查内核对象的类型 */
#ifdef (OS_OBJ_TYPE_REQ > 0u)
    /* 对象类型，
     * 在互斥信号量初始化时，
     * 互斥信号量的类型会被初始化为 OS_OBJ_TYPE_MUTEX
     */
    OS_OBJ_TYPE Type;
#endif
}
```

```

    /* 此宏用于代码调试功能 */
#ifdef (OS_CFG_DBG_EN > 0u)
    /* 如果使能了代码调试功能,
     * 那么就会为每一个信号量赋一个名称,
     * 方便调试
     */
    CPU_CHAR *NamePtr;
#endif

    /* 挂起等待任务链表
     * 当任务因尝试获取被占用的互斥信号量时,
     * 就可以选择被挂起到该链表,
     * 等待互斥信号量被释放
     */
    OS_PEND_LIST PendList;

    /* 此宏用于使能代码调试功能 */
#ifdef (OS_CFG_DBG_EN > 0u)
    /* 一些与代码调试相关的成员变量 */
    OS_MUTEX *DbgPrevPtr;
    OS_MUTEX *DbgNextPtr;
    CPU_CHAR *DbgNamePtr;
#endif

    /* 用于将互斥信号量添加到互斥信号量持有者的互斥信号量持有组中 */
    OS_MUTEX *MutexGrpNextPtr;
    /* 指向获取该互斥信号量的任务的任务控制块的指针 */
    OS_TCB *OwnerTCBPtr;
    /* 互斥信号量持有递归计数器
     * 同一个任务在成功获取互斥信号量后
     * 是可以重复多次地获取该互斥信号量的,
     * 但是要释放相同的次数,
     * 才能完全释放互斥信号量,
     * 否则其他任务是无法获取该互斥信号量的
     */
    OS_NESTING_CTR OwnerNestingCtr;

    /* 以下的成员变量用于第三方调试工具 */
#ifdef (OS_CFG_TS_EN > 0u)
    CPU_TS TS;
#endif
#ifdef (defined(OS_CFG_TRACE_EN) && (OS_CFG_TRACE_EN > 0u))

```

```
CPU_INT16U MutexID;
#endif
};
```

12.2 μC/OS-III 互斥信号量相关 API 函数

μC/OS-III 提供了一些列操作互斥信号量的 API 函数，这些操作函数如下表所示：

函数	描述
OSMutexCreate()	创建一个互斥信号量
OSMutexDel()	删除一个互斥信号量
OSMutexPend()	尝试获取互斥信号量
OSMutexPendAbort()	终止任务挂起等待互斥信号量
OSMutexPost()	释放互斥信号量

表 12.2.1 μC/OS-III 信号量相关 API 函数描述

1. 函数 OSMutexCreate()

该函数用于创建一个互斥信号量。

函数 OSMutexCreate()的函数原型，如下所示：

```
void OSMutexCreate( OS_MUTEX*   p_mutex,
                    CPU_CHAR*    p_name,
                    OS_ERR*      p_err)
```

函数 OSMutexCreate()的形参描述，如下表所示：

形参	描述
p_mutex	指向互斥信号量结构体的指针
p_name	指向作为互斥信号量名的 ASCII 字符串的指针
p_err	指向接收错误代码变量的指针

表 12.2.2 函数 OSMutexCreate()形参相关描述

函数 OSMutexCreate()的返回值描述，如下表所示：

返回值	描述
无	无

表 12.2.3 函数 OSMutexCreate()返回值相关描述

函数 OSMutexCreate()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	互斥信号量创建成功
OS_ERR_CREATE_ISR	在中断中非法调用该函数
OS_ERR_ILLEGAL_CREATE_RUN_TIME	在系统运行过程中非法创建内核对象
OS_ERR_OBJ_PTR_NULL	指向互斥信号量结构体的指针为空

表 12.2.4 函数 OSMutexCreate()错误代码相关描述

函数 OSMutexCreate()定义在文件 os_mutex.c 中，具体的代码如下所示：

```
void OSMutexCreate( OS_MUTEX*   p_mutex,
                    CPU_CHAR*    p_name,
                    OS_ERR*      p_err)
{
    CPU_SR_ALLOC();
```

```

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        *p_err = OS_ERR_ILLEGAL_CREATE_RUN_TIME;
        return;
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_CREATE_ISR;
        return;
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    /* 指向互斥信号量结构体的指针不能为空 */
    if (p_mutex == (OS_MUTEX *)0) {
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return;
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 此宏用于使能检查内核对象类型 */
#if (OS_OBJ_TYPE_REQ > 0u)

```

```
#if (OS_CFG_OBJ_CREATED_CHK_EN > 0u)
    /* 通过对象类型是否已经被设置
     * 判断互斥信号量是否已经被创建
     */
    if (p_mutex->Type == OS_OBJ_TYPE_MUTEX) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_OBJ_CREATED;
        return;
    }
#endif

    /* 对象类型设置为 OS_OBJ_TYPE_MUTEX */
    p_mutex->Type = OS_OBJ_TYPE_MUTEX;
#endif

    /* 初始化一些用于调试的成员变量 */
    #if (OS_CFG_DBG_EN > 0u)
        p_mutex->NamePtr = p_name;
    #else
        (void)p_name;
    #endif

    /* 初始化一些成员变量 */
    p_mutex->MutexGrpNextPtr    = (OS_MUTEX *)0;
    p_mutex->OwnerTCBPtr        = (OS_TCB *)0;
    p_mutex->OwnerNestingCtr    = 0u;

    /* 初始化一些用于调试的成员变量 */
    #if (OS_CFG_TS_EN > 0u)
        p_mutex->TS = 0u;
    #endif

    /* 初始化该互斥信号量的等待挂起任务链表 */
    OS_PendListInit(&p_mutex->PendList);

    /* 初始化一些用于调试的成员变量 */
    #if (OS_CFG_DBG_EN > 0u)
        OS_MutexDbgListAdd(p_mutex);
        OSMutexQty++;
    #endif

    /* 用于调试，不用理会 */
    OS_TRACE_MUTEX_CREATE(p_mutex, p_name);
```

```

/* 退出临界区 */
CPU_CRITICAL_EXIT();
*p_err = OS_ERR_NONE;
}

```

2. 函数 OSMutexDel()

该函数用于删除一个互斥信号量。

函数 OSMutexDel()的函数原型，如下所示：

```

OS_OBJ_QTY OSMutexDel(OS_MUTEX* p_mutex,
                      OS_OPT opt,
                      OS_ERR* p_err)

```

函数 OSMutexDel()的形参描述，如下表所示：

形参	描述
p_mutex	指向互斥信号量结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 12.2.5 函数 OSMutexDel()形参相关描述

函数 OSMutexDel()的返回值描述，如下表所示：

返回值	描述
OS_OBJ_QTY 类型返回值	删除互斥信号量时，被终止挂起任务的数量

表 12.2.6 函数 OSMutexDel()返回值相关描述

函数 OSMutexDel()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	互斥信号量删除成功
OS_ERR_DEL_ISR	在中断中非法调用该函数
OS_ERR_ILLEGAL_DEL_RUN_TIME	在系统运行过程中非法创建内核对象
OS_ERR_OBJ_PTR_NULL	指向互斥信号量结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是互斥信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_TASK_WAITING	有任务挂起等待互斥信号量，不能删除互斥信号量

表 12.2.7 函数 OSMutexDel()错误代码相关描述

函数 OSMutexDel()定义在文件 os_mutex.c 中，具体的代码如下所示：

```

OS_OBJ_QTY OSMutexDel(OS_MUTEX* p_mutex,
                      OS_OPT opt,
                      OS_ERR* p_err)
{
    OS_OBJ_QTY    nbr_tasks;
    OS_PEND_LIST* p_pend_list;
    OS_TCB*       p_tcb;
    OS_TCB*       p_tcb_owner;
    CPU_TS        ts;
    #if (OS_CFG_MUTEX_EN > 0u)
    OS_PRIO        prio_new;

```

```
#endif

CPU_SR_ALLOC();

/* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

OS_TRACE_MUTEX_DEL_ENTER(p_mutex, opt);

/* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        OS_TRACE_MUTEX_DEL_EXIT(OS_ERR_ILLEGAL_DEL_RUN_TIME);
        *p_err = OS_ERR_ILLEGAL_DEL_RUN_TIME;
        return (0u);
    }
#endif

/* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    if (OSIntNestingCtr > 0u) {
        OS_TRACE_MUTEX_DEL_EXIT(OS_ERR_DEL_ISR);
        *p_err = OS_ERR_DEL_ISR;
        return (0u);
    }
#endif

/* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_MUTEX_DEL_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }
}
```

```
#endif

/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
/* 指向互斥信号量结构体的指针不能为空 */
if (p_mutex == (OS_MUTEX *)0) {
    OS_TRACE_MUTEX_DEL_EXIT(OS_ERR_OBJ_PTR_NULL);
    *p_err = OS_ERR_OBJ_PTR_NULL;
    return (0u);
}
#endif

/* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
if (p_mutex->Type != OS_OBJ_TYPE_MUTEX) {
    OS_TRACE_MUTEX_DEL_EXIT(OS_ERR_OBJ_TYPE);
    *p_err = OS_ERR_OBJ_TYPE;
    return (0u);
}
#endif

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 获取互斥信号量的挂起等待任务链表 */
p_pend_list = &p_mutex->PendList;
/* 用于记录被终止挂起任务的数量 */
nbr_tasks = 0u;

/* 根据不同的函数操作选项，
 * 作不同的处理
 */
switch (opt) {
/* 仅在没有挂起等待任务时，
 * 才删除互斥信号量
 */
case OS_OPT_DEL_NO_PEND:
/* 判断互斥信号量的挂起等待任务链表是否为空 */
if (p_pend_list->HeadPtr == (OS_TCB *)0) {
/* 此宏用于代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
    OS_MutexDbgListRemove(p_mutex);
    OSMutexQty--;
#endif

```



```
#endif

    /* 用于调试，不用理会 */
    OS_TRACE_MUTEX_DEL(p_mutex);
    /* 判断互斥信号量是否正被其他任务持有 */
    if (p_mutex->OwnerTCBPtr != (OS_TCB *)0) {
        /* 将互斥信号量从任务的持有互斥信号量组中移除 */
        OS_MutexGrpRemove(p_mutex->OwnerTCBPtr, p_mutex);
    }
    /* 去初始化互斥信号量 */
    OS_MutexClr(p_mutex);
    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_NONE;
} else {
    /* 有任务正挂起等待该互斥信号量
     * 因此退出临界区，
     * 不删除互斥信号量
     */
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_TASK_WAITING;
}
break;

/* 始终删除互斥信号量 */
case OS_OPT_DEL_ALWAYS:
#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif

/* 终止挂起所有挂起等待该互斥信号量的任务 */
while (p_pend_list->HeadPtr != (OS_TCB *)0) {
    p_tcb = p_pend_list->HeadPtr;
    OS_PendAbort(p_tcb,
                ts,
                OS_STATUS_PEND_DEL);

    nbr_tasks++;
}

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
    OS_MutexDbgListRemove(p_mutex);
    OSMutexQty--;
#endif
#endif
```

```

/* 用于调试，不用理会 */
OS_TRACE_MUTEX_DEL(p_mutex);
/* 判断该互斥信号量是否正被其他任务持有
 * 如果是，
 * 则将该互斥信号量从任务的持有互斥信号量组中移除
 */
p_tcb_owner = p_mutex->OwnerTCBPtr;
if (p_tcb_owner != (OS_TCB *)0) {
    OS_MutexGrpRemove(p_tcb_owner, p_mutex);
}

/* 因为互斥信号量具有优先级继承的机制
 * 因此当任务持有的互斥信号量被删除时，
 * 需要判断该任务是否持有其他互斥信号量，
 * 如果有，
 * 则还需要判断任务是否需要继承新的任务优先级
 */
if (p_tcb_owner != (OS_TCB *)0) {
    if (p_tcb_owner->Prio != p_tcb_owner->BasePrio) {
        prio_new = OS_MutexGrpPrioFindHighest(p_tcb_owner);
        prio_new = (prio_new > p_tcb_owner->BasePrio) ?
            p_tcb_owner->BasePrio :
            prio_new;
        OS_TaskChangePrio(p_tcb_owner, prio_new);
        OS_TRACE_MUTEX_TASK_PRIO_DISINHERIT( p_tcb_owner,
            p_tcb_owner->Prio);
    }
}

/* 去初始化互斥信号量 */
OS_MutexClr(p_mutex);
/* 退出临界区 */
CPU_CRITICAL_EXIT();
/* 触发任务调度 */
OSSched();
*p_err = OS_ERR_NONE;
break;

/* 无效的任务操作选项 */
default:
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_OPT_INVALID;
    break;

```

```

}

OS_TRACE_MUTEX_DEL_EXIT(*p_err);

/* 返回被终止挂起任务的数量 */
return (nbr_tasks);
}

```

3. 函数 OSMutexPend()

该函数用于获取互斥信号量。

函数 OSMutexPend()的函数原型，如下所示：

```

void OSMutexPend( OS_MUTEX*   p_mutex,
                  OS_TICK      timeout,
                  OS_OPT        opt,
                  CPU_TS*       p_ts,
                  OS_ERR*       p_err)

```

函数 OSMutexPend()的形参描述，如下表所示：

形参	描述
p_mutex	指向互斥信号量结构体的指针
timeout	任务挂起等待互斥信号量的最大允许时间
opt	函数操作选项
p_ts	指向接收互斥信号量接收时的时间戳的变量的指针
p_err	指向接收错误代码变量的指针

表 12.2.8 函数 OSMutexPend()形参相关描述

函数 OSMutexPend()的返回值描述，如下表所示：

返回值	描述
无	无

表 12.2.9 函数 OSMutexPend()返回值相关描述

函数 OSMutexPend()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功获取互斥信号量
OS_ERR_MUTEX_OWNER	任务重复获取互斥信号量
OS_ERR_MUTEX_OVF	互斥信号量持有递归计数器计溢出
OS_ERR_OBJ_DEL	待获取的互斥信号量已经被删除
OS_ERR_OBJ_PTR_NULL	指向互斥信号量结构体的指针为空
OS_ERR_OBJ_TYPE	待获取内核对象的类型不是互斥信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT	任务挂起等待互斥信号量时被终止（还未超时）
OS_ERR_PEND_ISR	在中断中非法调用该函数
OS_ERR_PEND_WOULD_BLOCK	获取互斥信号量失败，并且不挂起任务等待互斥信号量
OS_ERR_SCHED_LOCKED	任务调度器已锁定
OS_ERR_STATUS_INVALID	无效的任务挂起结果

OS_ERR_TIMEOUT	任务挂起等待互斥信号量超时
----------------	---------------

表 12.2.10 函数 OSMutexPend()错误代码相关描述

函数 OSMutexPend()定义在文件 os_mutex.c 中，具体的代码如下所示：

```
void OSMutexPend( OS_MUTEX*   p_mutex,
                  OS_TICK     timeout,
                  OS_OPT      opt,
                  CPU_TS*     p_ts,
                  OS_ERR*     p_err)
{
    OS_TCB *p_tcb;
    CPU_SR_ALLOC();

    #if (OS_CFG_TS_EN == 0u)
        (void)p_ts;
    #endif

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
    #ifdef OS_SAFETY_CRITICAL
        if (p_err == (OS_ERR *)0) {
            OS_SAFETY_CRITICAL_EXCEPTION();
            return;
        }
    #endif

    /* 用于调试，不用理会 */
    OS_TRACE_MUTEX_PEND_ENTER(p_mutex, timeout, opt, p_ts);

    /* 此宏用于使能系统时钟节拍 */
    #if (OS_CFG_TICK_EN == 0u)
        if (timeout != 0u) {
            *p_err = OS_ERR_TICK_DISABLED;
            OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
            OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_TICK_DISABLED);
            return;
        }
    #endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
         * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
         * 因为在中断中不能调用创建任务的函数，
         * 因此返回错误
        */
    #endif
}
```

```
*/  
  
if (OSIntNestingCtr > 0u) {  
    OS_TRACE_MUTEX_PEND_FAILED(p_mutex);  
    OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_PEND_ISR);  
    *p_err = OS_ERR_PEND_ISR;  
    return;  
}  
#endif  
  
/* 此宏用于使能检查 μC/OS-III 系统是否运行 */  
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)  
    if (OSRunning != OS_STATE_OS_RUNNING) {  
        OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_OS_NOT_RUNNING);  
        *p_err = OS_ERR_OS_NOT_RUNNING;  
        return;  
    }  
#endif  
  
/* 此宏用于使能参数检查 */  
#if (OS_CFG_ARG_CHK_EN > 0u)  
    if (p_mutex == (OS_MUTEX *)0) {  
        OS_TRACE_MUTEX_PEND_FAILED(p_mutex);  
        OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_OBJ_PTR_NULL);  
        *p_err = OS_ERR_OBJ_PTR_NULL;  
        return;  
    }  
    switch (opt) {  
        case OS_OPT_PEND_BLOCKING:  
        case OS_OPT_PEND_NON_BLOCKING:  
            break;  
  
        default:  
            OS_TRACE_MUTEX_PEND_FAILED(p_mutex);  
            OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_OPT_INVALID);  
            *p_err = OS_ERR_OPT_INVALID;  
            return;  
    }  
#endif  
  
/* 此宏用于使能检查内核对象类型 */  
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)  
    if (p_mutex->Type != OS_OBJ_TYPE_MUTEX) {  
        OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
```

```

    OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_OBJ_TYPE);
    *p_err = OS_ERR_OBJ_TYPE;
    return;
}
#endif

/* 进入临界区 */
CPU_CRITICAL_ENTER();
/* 通过互斥信号量的持有递归计数器
 * 判断互斥信号量是否已经被其他任务持有
 */
if (p_mutex->OwnerNestingCtr == 0u) {
    /* 设置互斥信号量的持有者为当前任务 */
    p_mutex->OwnerTCBPtr = OSTCBCurPtr;
    /* 设置互斥信号量的持有递归计数器为 1 */
    p_mutex->OwnerNestingCtr = 1u;
}
#if (OS_CFG_TS_EN > 0u)
    if (p_ts != (CPU_TS *)0) {
        *p_ts = p_mutex->TS;
    }
#endif

/* 将互斥信号量添加到当前任务的持有互斥信号量组中 */
OS_MutexGrpAdd(OSTCBCurPtr, p_mutex);
/* 退出临界区 */
CPU_CRITICAL_EXIT();
OS_TRACE_MUTEX_PEND(p_mutex);
OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_NONE);
*p_err = OS_ERR_NONE;
return;
}

/* 通过判断互斥信号量的持有者是否为当前任务
 * 从而进行互斥信号量持有递归
 */
if (OSTCBCurPtr == p_mutex->OwnerTCBPtr) {
    /* 判断互斥信号量的持有递归计数器是否溢出 */
    if (p_mutex->OwnerNestingCtr == (OS_NESTING_CTR)-1) {
        CPU_CRITICAL_EXIT();
        OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
        OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_MUTEX_OVF);
        *p_err = OS_ERR_MUTEX_OVF;
        return;
    }
}

```

```

        /* 更新互斥信号量的持有递归计数器 */
        p_mutex->OwnerNestingCtr++;
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = p_mutex->TS;
        }
#endif

        /* 退出临界区 */
        CPU_CRITICAL_EXIT();
        OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
        OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_MUTEX_OWNER);
        *p_err = OS_ERR_MUTEX_OWNER;
        return;
    }

    /* 下面的情况为互斥信号量已经被其他任务持有了 */

    /* 该函数操作选项表示
     * 不挂起等待互斥信号量被释放
     */
    if ((opt & OS_OPT_PEND_NON_BLOCKING) != 0u) {
        /* 直接退出临界区，
         * 并返回
         */
        CPU_CRITICAL_EXIT();
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = 0u;
        }
#endif
        OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
        OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_PEND_WOULD_BLOCK);
        *p_err = OS_ERR_PEND_WOULD_BLOCK;
        return;
    } else {
        /* 如果选择挂起等待互斥信号量被释放，
         * 那就需要判断任务调度器是否被锁定
         */
        if (OSSchedLockNestingCtr > 0u) {
            /* 如果任务调度器被锁定，
             * 那么直接退出临界区，
             * 并返回
             */

```

```

        CPU_CRITICAL_EXIT();
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = 0u;
        }
#endif

        OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
        OS_TRACE_MUTEX_PEND_EXIT(OS_ERR_SCHED_LOCKED);
        *p_err = OS_ERR_SCHED_LOCKED;
        return;
    }
}

/* 下面就要将任务挂起等待互斥信号量被释放了 */

/* 获取互斥信号量的持有者
 * 并进行根据需要进行优先级继承
 * 保证互斥信号量持有者的任务优先级不低于当前要被挂起的任务
 */
p_tcb = p_mutex->OwnerTCBPtr;
if (p_tcb->Prio > OSTCBCurPtr->Prio) {
    OS_TaskChangePrio(p_tcb, OSTCBCurPtr->Prio);
    OS_TRACE_MUTEX_TASK_PRIO_INHERIT(p_tcb, p_tcb->Prio);
}

/* 将当前任务添加到互斥信号量的挂起等待任务链表中 */
OS_Pend( (OS_PEND_OBJ *) ((void *)p_mutex),
        OSTCBCurPtr,
        OS_TASK_PEND_ON_MUTEX,
        timeout);

/* 退出临界区 */
CPU_CRITICAL_EXIT();
OS_TRACE_MUTEX_PEND_BLOCK(p_mutex);
/* 进行任务调度 */
OSSched();

/* 进行任务调度后，当前任务就被挂起了，
 * 下面的代码是在任务解除挂起状态后才能够被执行
 */

/* 进入临界区 */
CPU_CRITICAL_ENTER();
/* 根据任务的挂起结果进行处理 */

```



```
switch (OSTCBCurPtr->PendStatus) {
    /* 成功地等待到了互斥信号量 */
    case OS_STATUS_PEND_OK:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#endif
        OS_TRACE_MUTEX_PEND(p_mutex);
        *p_err = OS_ERR_NONE;
        break;

    /* 任务挂起被终止 */
    case OS_STATUS_PEND_ABORT:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#endif
        OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
        *p_err = OS_ERR_PEND_ABORT;
        break;

    /* 任务挂起超时 */
    case OS_STATUS_PEND_TIMEOUT:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = 0u;
        }
#endif
        OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
        *p_err = OS_ERR_TIMEOUT;
        break;

    /* 挂起等待的互斥信号量被删除 */
    case OS_STATUS_PEND_DEL:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#endif
        OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
        *p_err = OS_ERR_OBJ_DEL;
```

```

        break;

        /* 无效的挂起结果 */
        default:
            OS_TRACE_MUTEX_PEND_FAILED(p_mutex);
            *p_err = OS_ERR_STATUS_INVALID;
            break;
    }

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    OS_TRACE_MUTEX_PEND_EXIT(*p_err);
}

```

4. 函数 OSMutexPendAbort()

该函数用于终止任务挂起等待互斥信号量。

函数 OSMutexPendAbort()的函数原型，如下所示：

```

OS_OBJ_QTY OSMutexPendAbort( OS_MUTEX* p_mutex,
                             OS_OPT      opt,
                             OS_ERR*     p_err)

```

函数 OSMutexPendAbort()的形参描述，如下表所示：

形参	描述
p_mutex	指向互斥信号量结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 12.2.11 函数 OSMutexPendAbort()形参相关描述

函数 OSMutexPendAbort()的返回值描述，如下表所示：

返回值	描述
OS_OBJ_QTY 类型变量	被终止挂起任务的数量

表 12.2.12 函数 OSMutexPendAbort()返回值相关描述

函数 OSMutexPendAbort()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	终止任务挂起等待互斥信号量成功
OS_ERR_OBJ_PTR_NULL	指向互斥信号量结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是互斥信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT_ISR	在中断中非法调用该函数
OS_ERR_PEND_ABORT_NONE	没有任务挂起等待该互斥信号量

表 12.2.13 函数 OSMutexPendAbort()错误代码相关描述

函数 OSMutexPendAbort()定义在文件 os_mutex.c 中，具体的代码如下所示：

```

OS_OBJ_QTY OSMutexPendAbort( OS_MUTEX* p_mutex,
                             OS_OPT      opt,
                             OS_ERR*     p_err)
{

```

```

OS_PEND_LIST*   p_pend_list;
OS_TCB*         p_tcb;
OS_TCB*         p_tcb_owner;
CPU_TS          ts;
OS_OBJ_QTY      nbr_tasks;
OS_PRIO         prio_new;
CPU_SR_ALLOC();

/* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return ((OS_OBJ_QTY)0u);
    }
#endif

/* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_PEND_ABORT_ISR;
        return (0u);
    }
#endif

/* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }
#endif

/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_mutex == (OS_MUTEX *)0) {
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }

```

```

switch (opt) {
    case OS_OPT_PEND_ABORT_1:
    case OS_OPT_PEND_ABORT_ALL:
    case OS_OPT_PEND_ABORT_1 | OS_OPT_POST_NO_SCHED:
    case OS_OPT_PEND_ABORT_ALL | OS_OPT_POST_NO_SCHED:
        break;

    default:
        *p_err = OS_ERR_OPT_INVALID;
        return (0u);
}
#endif

#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    /* 此宏用于使能检查内核对象类型 */
    if (p_mutex->Type != OS_OBJ_TYPE_MUTEX) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();
    /* 获取互斥信号量的挂起等待任务链表 */
    p_pend_list = &p_mutex->PendList;
    /* 如果互斥信号量的挂起等待任务链表为空
     * 则不需要终止挂起任务，
     * 因此直接返回
     */
    if (p_pend_list->HeadPtr == (OS_TCB *)0) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_PEND_ABORT_NONE;
        return (0u);
    }
    /* 用于记录被终止挂起任务的数量 */
    nbr_tasks = 0u;

#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif

```

```

/* 遍历互斥信号量的挂起等待任务链表 */
while (p_pend_list->HeadPtr != (OS_TCB *)0) {
    /* 获取被挂起的任务 */
    p_tcb = p_pend_list->HeadPtr;
    /* 将任务从互斥信号量的挂起等待任务链表中移除 */
    OS_PendAbort(    p_tcb,
                    ts,
                    OS_STATUS_PEND_ABORT);
    /* 获取互斥信号量的持有者 */
    p_tcb_owner = p_mutex->OwnerTCBPtr;
    prio_new = p_tcb_owner->Prio;
    /* 判断互斥信号量的持有者是否继承了当前遍历到的被挂起的任务的任务优先级 */
    if (    (p_tcb_owner->Prio != p_tcb_owner->BasePrio) &&
        (p_tcb_owner->Prio == p_tcb->Prio)) {
        /* 如果是的话,
        * 则需要互斥信号量的持有者继承其他被挂起任务的任务优先级,
        * 因此当前被遍历到的被挂起的任务要被终止挂起了
        */
        prio_new = OS_MutexGrpPrioFindHighest(p_tcb_owner);
        prio_new = (prio_new > p_tcb_owner->BasePrio) ?
                    p_tcb_owner->BasePrio :
                    prio_new;
    }

    /* 修改互斥信号量持有者的优先级 */
    if(prio_new != p_tcb_owner->Prio) {
        OS_TaskChangePrio(p_tcb_owner, prio_new);
        OS_TRACE_MUTEX_TASK_PRIO_DISINHERIT(p_tcb_owner, p_tcb_owner->Prio);
    }
    /* 记录被终止挂起任务的数量 */
    nbr_tasks++;
    if (opt != OS_OPT_PEND_ABORT_ALL) {
        break;
    }
}

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 根据函数操作选项进行任务调度 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

```

```
*p_err = OS_ERR_NONE;
return (nbr_tasks);
}
```

5. 函数 OSMutexPost()

该函数用于释放互斥信号量。

函数 OSMutexPost()的函数原型，如下所示：

```
void OSMutexPost( OS_MUTEX* p_mutex,
                  OS_OPT    opt,
                  OS_ERR*   p_err);
```

函数 OSMutexPost()的形参描述，如下表所示：

形参	描述
p_mutex	指向互斥信号量结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 12.2.14 函数 OSMutexPost()形参相关描述

函数 OSMutexPost()的返回值描述，如下表所示：

返回值	描述
无	无

表 12.2.15 函数 OSMutexPost()返回值相关描述

函数 OSMutexPost()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	释放互斥信号量成功
OS_ERR_MUTEX_NESTING	还未完全释放互斥信号量（持有递归计数器不为 0）
OS_ERR_MUTEX_NOT_OWNER	释放互斥信号量的任务不是互斥信号量的持有者
OS_ERR_OBJ_PTR_NULL	指向互斥信号量结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是互斥信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_POST_ISR	在中断中非法调用该函数

表 12.2.16 函数 OSMutexPost()错误代码相关描述

函数 OSMutexPost()定义在文件 os_mutex.c 中，具体的代码如下所示：

```
void OSMutexPost( OS_MUTEX* p_mutex,
                  OS_OPT    opt,
                  OS_ERR*   p_err)
{
    OS_PEND_LIST* p_pend_list;
    OS_TCB*       p_tcb;
    CPU_TS        ts;
    OS_PRIO       prio_new;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
```

```

    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    /* 用于调试，不用理会 */
    OS_TRACE_MUTEX_POST_ENTER(p_mutex, opt);

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    if (OSIntNestingCtr > 0u) {
        OS_TRACE_MUTEX_POST_FAILED(p_mutex);
        OS_TRACE_MUTEX_POST_EXIT(OS_ERR_POST_ISR);
        *p_err = OS_ERR_POST_ISR;
        return;
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_MUTEX_POST_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return;
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_mutex == (OS_MUTEX *)0) {
        OS_TRACE_MUTEX_POST_FAILED(p_mutex);
        OS_TRACE_MUTEX_POST_EXIT(OS_ERR_OBJ_PTR_NULL);
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return;
    }

    switch (opt) {
        case OS_OPT_POST_NONE:

```

```

        case OS_OPT_POST_NO_SCHED:
            break;

        default:
            OS_TRACE_MUTEX_POST_FAILED(p_mutex);
            OS_TRACE_MUTEX_POST_EXIT(OS_ERR_OPT_INVALID);
            *p_err = OS_ERR_OPT_INVALID;
            return;
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_mutex->Type != OS_OBJ_TYPE_MUTEX) {
        OS_TRACE_MUTEX_POST_FAILED(p_mutex);
        OS_TRACE_MUTEX_POST_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return;
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();
    /* 只有互斥信号量的持有者才能够释放互斥信号量 */
    if (OSTCBCurPtr != p_mutex->OwnerTCBPtr) {
        CPU_CRITICAL_EXIT();
        OS_TRACE_MUTEX_POST_FAILED(p_mutex);
        OS_TRACE_MUTEX_POST_EXIT(OS_ERR_MUTEX_NOT_OWNER);
        *p_err = OS_ERR_MUTEX_NOT_OWNER;
        return;
    }

    /* 用于调试，不用理会 */
    OS_TRACE_MUTEX_POST(p_mutex);

#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
    p_mutex->TS = ts;
#else
    ts = 0u;
#endif

    /* 更新互斥信号的持有递归计数器的值 */

```



```

p_mutex->OwnerNestingCtr--;

/* 如果互斥信号量的持有递归计数器的值为 0
 * 说明任务已经完全释放互斥信号量
 * 否则就保持获取
 */

if (p_mutex->OwnerNestingCtr > 0u) {
    CPU_CRITICAL_EXIT();
    OS_TRACE_MUTEX_POST_EXIT(OS_ERR_MUTEX_NESTING);
    *p_err = OS_ERR_MUTEX_NESTING;
    return;
}

/* 如果互斥信号量的持有递归计数器为 0
 * 那么说明互斥信号量的所有者已经完全释放了互斥信号量
 */

/* 从任务的互斥信号量拥有组中移除互斥互斥信号量 */
OS_MutexGrpRemove(OSTCBCurPtr, p_mutex);
/* 获取互斥信号量的挂起等待任务链表 */
p_pend_list = &p_mutex->PendList;
/* 判断是否没有任务正挂起等待互斥信号量
 * 如果没有则直接返回
 */

if (p_pend_list->HeadPtr == (OS_TCB *)0) {
    p_mutex->OwnerTCBPtr = (OS_TCB *)0;
    p_mutex->OwnerNestingCtr = 0u;
    CPU_CRITICAL_EXIT();
    OS_TRACE_MUTEX_POST_EXIT(OS_ERR_NONE);
    *p_err = OS_ERR_NONE;
    return;
}

/* 如果互斥信号量的挂起等待任务链表中有挂起的任务，
 * 那么要将挂起的任务设置为互斥信号量的所有者
 */

/* 判断当前任务是否进行了优先级继承 */
if (OSTCBCurPtr->Prio != OSTCBCurPtr->BasePrio) {
    prio_new = OS_MutexGrpPrioFindHighest(OSTCBCurPtr);
    prio_new = (prio_new > OSTCBCurPtr->BasePrio) ?
                OSTCBCurPtr->BasePrio :
                prio_new;
    if (prio_new > OSTCBCurPtr->Prio) {

```

```

        OS_RdyListRemove(OSTCBCurPtr);
        OSTCBCurPtr->Prio = prio_new;
        OS_TRACE_MUTEX_TASK_PRIO_DISINHERIT(OSTCBCurPtr, prio_new);
        OS_PrioInsert(prio_new);
        OS_RdyListInsertTail(OSTCBCurPtr);
        OSPrioCur = prio_new;
    }
}

/* 获取互斥信号量的挂起等待任务链表 */
p_tcb = p_pend_list->HeadPtr;
/* 设置挂起等待互斥信号量的任务为互斥信号量的持有者 */
p_mutex->OwnerTCBPtr = p_tcb;
p_mutex->OwnerNestingCtr = 1u;
OS_MutexGrpAdd(p_tcb, p_mutex);
/* 将互斥信号量释放给挂起等待互斥信号量的任务
 * 这将终止挂起该任务
 */
OS_Post((OS_PEND_OBJ*)((void*)p_mutex),
        p_tcb,
        (void*)0,
        0u,
        ts);
/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 根据函数操作选项进行任务调度 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

OS_TRACE_MUTEX_POST_EXIT(OS_ERR_NONE);
*p_err = OS_ERR_NONE;
}

```

12.3 μC/OS-III 互斥信号量实验

12.3.1 功能设计

1. 例程功能

本实验主要用于展示使用 μC/OS-III 提供的 API 函数来使用互斥信号量，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
-----	--------

start_task	用于创建互斥信号量和其他任务
task1	访问共享资源
task2	访问共享资源

表 12.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 12-1 UCOS-III 互斥信号量实验》。

12.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

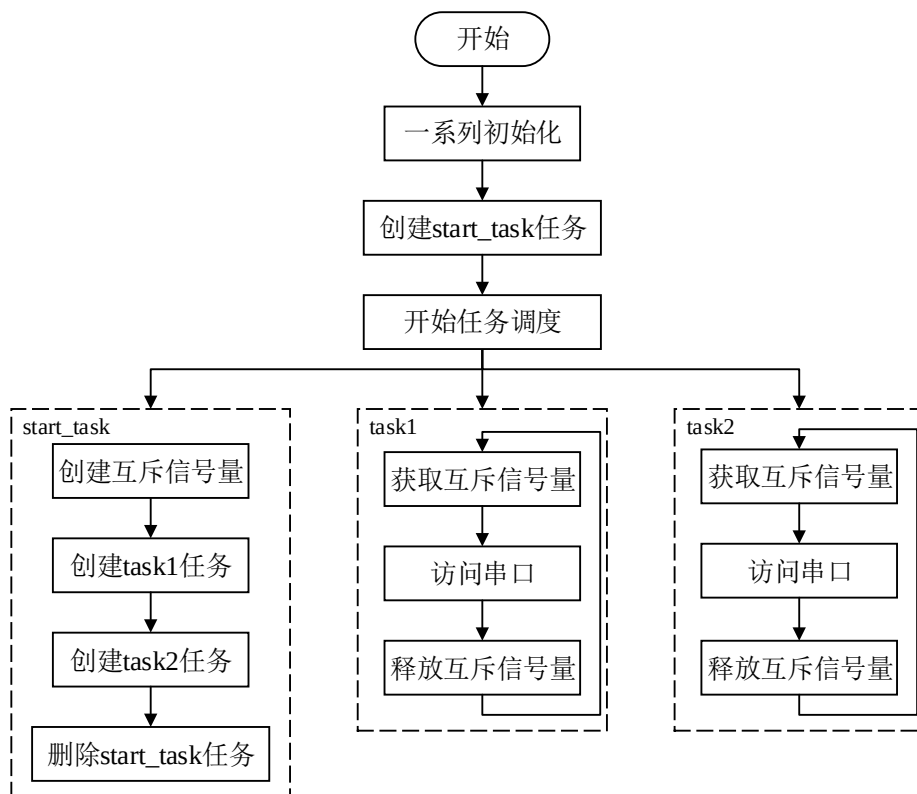


图 12.3.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSMutexCreate()

此函数用于创建互斥信号量，请参考第 12.2 小节《μC/OS-III 互斥信号量相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_MUTEX_EN 配置为 1。

(2) 函数 OSMutexPend()

此函数用于获取互斥信号量，请参考第 12.2 小节《μC/OS-III 互斥信号量相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_MUTEX_EN 配置为 1。

(3) 函数 OSMutexPost()

此函数用于释放互斥信号量，请参考第 12.2 小节《μC/OS-III 互斥信号量相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_MUTEX_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 1, &err);

    /* 创建互斥信号量 */
    OSMutexCreate( (OS_MUTEX* )&mutex,
                  (CPU_CHAR* )"mutex",
                  (OS_ERR* )&err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc( SRAMIN,
                                         TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate( (OS_TCB* )&Task1Task_TCB,
                  (CPU_CHAR* )"task1",
                  (OS_TASK_PTR )task1,
                  (void* )0,
                  (OS_PRIO )TASK1_PRIO,
                  (CPU_STK* )Task1Task_STK,
                  (CPU_STK_SIZE )TASK1_STK_SIZE / 10,
                  (CPU_STK_SIZE )TASK1_STK_SIZE,
                  (OS_MSG_QTY )0,
                  (OS_TICK )0,
                  (void* )0,
```

```

        (OS_OPT          ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*         ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                       TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(   (OS_TCB*      ) &Task2Task_TCB,
                (CPU_CHAR*    ) "task2",
                (OS_TASK_PTR  ) task2,
                (void*        ) 0,
                (OS_PRIO      ) TASK2_PRIO,
                (CPU_STK*     ) Task2Task_STK,
                (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,
                (CPU_STK_SIZE ) TASK2_STK_SIZE,
                (OS_MSG_QTY   ) 0,
                (OS_TICK      ) 0,
                (void*        ) 0,
                (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                (OS_ERR*      ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务和 task2 任务，并且 task1 任务和 task2 任务的任务栈都是动态分配的，当然，也可以静态地定任务栈，就像定义一个数组一样。同时还创建一个用于保护共享资源的互斥信号量。

(2) task1 和 task2 任务

```

/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    OS_ERR err;

    while (1)
    {
        /* 获取互斥信号量 */
        OSMutexPend(   (OS_MUTEX* ) &mutex,
                      (OS_TICK    ) 0,
                      (OS_OPT     ) OS_OPT_PEND_BLOCKING,
                      (CPU_TS*    ) 0,

```

```

(OS_ERR* )&err);

/* 访问共享资源 */
printf("This is task1!\r\n");

/* 释放互斥信号量 */
OSMutexPost( (OS_MUTEX* )&mutex,
              (OS_OPT )OS_OPT_POST_NONE,
              (OS_ERR* )&err);
}
}

/**
 * @brief task2
 * @param p_arg : 传入参数(未用到)
 * @retval 无
 */
void task2(void *p_arg)
{
    OS_ERR err;

    while (1)
    {
        /* 获取互斥信号量 */
        OSMutexPend( (OS_MUTEX* )&mutex,
                     (OS_TICK )0,
                     (OS_OPT )OS_OPT_PEND_BLOCKING,
                     (CPU_TS* )0,
                     (OS_ERR* )&err);

        /* 访问共享资源 */
        printf("This is task2!\r\n");

        /* 释放互斥信号量 */
        OSMutexPost( (OS_MUTEX* )&mutex,
                     (OS_OPT )OS_OPT_POST_NONE,
                     (OS_ERR* )&err);
    }
}

```

从以上代码中可以看到，本实验是让多任务访问同一个串口资源，但是在访问串口资源前，会先获取互斥信号量，如果能成功获取，那么才能够正常地访问串口资源，如果互斥信号量已经被其他任务占有，此时获取互斥信号量就会被挂起。在使用完互斥信号量之后，要立马释放互斥信号量，让其他任务有使用共享资源的机会。

12.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 12.3.2 LCD 显示内容

同时，通过串口调试助手，就能够看到输出的打印信息，如下图所示：



图 12.3.3 串口调试助手显示内容

从上图可以看出，虽然 task1 和 task2 任务是在一直访问串口资源的，但是，串口输出的内容却很整洁，这正是互斥信号量的功劳，使得多个任务对同一个共享资源的访问变得有序。

12.4 μC/OS-III 互斥信号量优化优先级翻转实验

12.4.1 功能设计

1. 例程功能

通过上一小节的实验，可以发现，互斥信号量与二值信号量是很相似的，但是互斥信号量

较二值信号量多了优先级继承的机制，这使得互斥信号量在优化优先级翻转的问题上有着很好的表现，本实验将展示使用互斥信号量的优先级继承机制，优化优先级翻转的问题。本实验设计了四个任务，这四个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建互斥信号量和其他任务
task1	用于演示互斥信号量优化优先级翻转问题
task2	用于演示互斥信号量优化优先级翻转问题
task3	用于演示互斥信号量优化优先级翻转问题

表 12.4.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 12-2 UCOS-III 互斥信号量优化优先级翻转实验》。

12.4.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

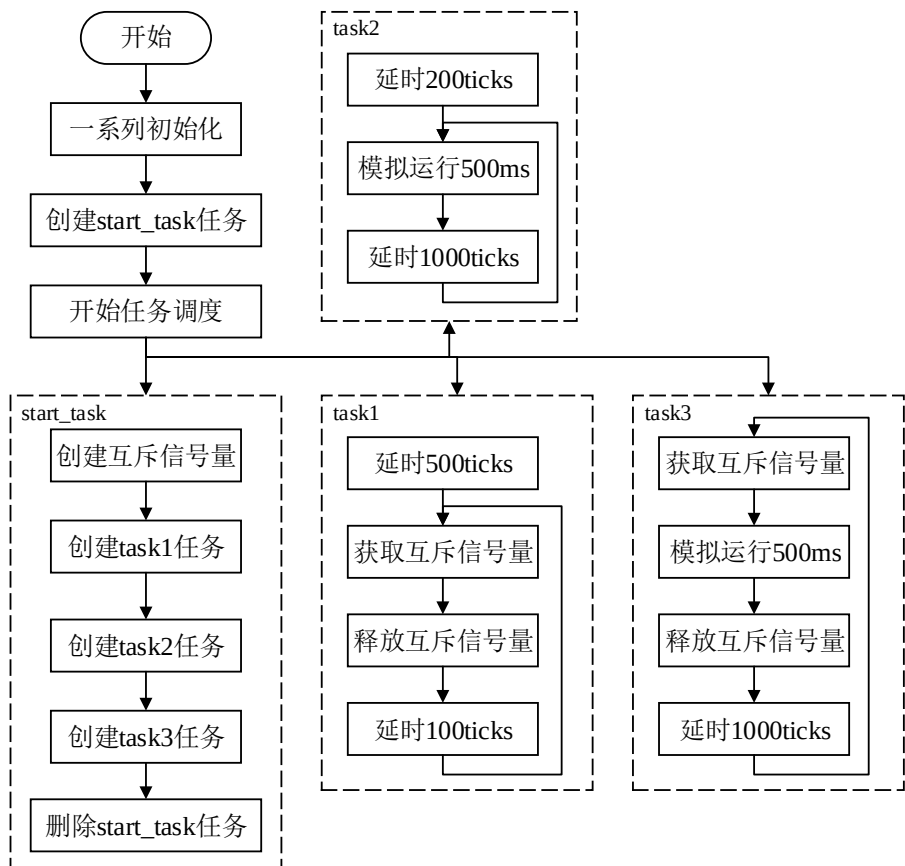


图 12.4.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSMutexCreate()

此函数用于创建互斥信号量，请参考第 12.2 小节《μC/OS-III 互斥信号量相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_MUTEX_EN 配置为 1。

(2) 函数 OSMutexPend()

此函数用于获取互斥信号量，请参考第 12.2 小节《μC/OS-III 互斥信号量相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_MUTEX_EN 配置为 1。

(3) 函数 OSMutexPost()

此函数用于释放互斥信号量，请参考第 12.2 小节《μC/OS-III 互斥信号量相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_MUTEX_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief   start_task
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建互斥信号量 */
    OSMutexCreate( (OS_MUTEX*) &mutex,
                  (CPU_CHAR*) "mutex",
                  (OS_ERR*) &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc( SRAMIN,
                                          TASK1_STK_SIZE * sizeof(CPU_STK));
    OSTaskCreate( (OS_TCB*) &Task1Task_TCB,
                  (CPU_CHAR*) "task1",
                  (OS_TASK_PTR) task1,
                  (void*) 0,
```

```

        (OS_PRIO          ) TASK1_PRIO,
        (CPU_STK          *) Task1Task_STK,
        (CPU_STK_SIZE     ) TASK1_STK_SIZE / 10,
        (CPU_STK_SIZE     ) TASK1_STK_SIZE,
        (OS_MSG_QTY       ) 0,
        (OS_TICK          ) 0,
        (void*             ) 0,
        (OS_OPT           ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*          ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                       TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*          ) &Task2Task_TCB,
                 (CPU_CHAR*        ) "task2",
                 (OS_TASK_PTR      ) task2,
                 (void*            ) 0,
                 (OS_PRIO          ) TASK2_PRIO,
                 (CPU_STK*         ) Task2Task_STK,
                 (CPU_STK_SIZE     ) TASK2_STK_SIZE / 10,
                 (CPU_STK_SIZE     ) TASK2_STK_SIZE,
                 (OS_MSG_QTY       ) 0,
                 (OS_TICK          ) 0,
                 (void*            ) 0,
                 (OS_OPT           ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*          ) &err);

/* 创建 Task3 */
Task3Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                       TASK3_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*          ) &Task3Task_TCB,
                 (CPU_CHAR*        ) "task3",
                 (OS_TASK_PTR      ) task3,
                 (void*            ) 0,
                 (OS_PRIO          ) TASK3_PRIO,
                 (CPU_STK*         ) Task3Task_STK,
                 (CPU_STK_SIZE     ) TASK3_STK_SIZE / 10,
                 (CPU_STK_SIZE     ) TASK3_STK_SIZE,
                 (OS_MSG_QTY       ) 0,
                 (OS_TICK          ) 0,
                 (void*            ) 0,
                 (OS_OPT           ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*          ) &err);
    
```

```
/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}
```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务、task2 任务和 task3 任务，并且 task1 任务、task2 任务和 task3 任务的任务栈都是动态分配的，当然，也可以静态地定任务栈，就像定义一个数组一样。同时还创建一个互斥信号量。

(2) task1 任务、task2 任务和 task3 任务

```
/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    OS_ERR err;

    OSTimeDly(500, OS_OPT_TIME_DLY, &err);
    while (1)
    {
        printf("task1 ready to pend mutex\r\n");
        OSMutexPend(    (OS_MUTEX*) &mutex,
                        (OS_TICK) 0,
                        (OS_OPT) OS_OPT_PEND_BLOCKING,
                        (CPU_TS*) 0,
                        (OS_ERR*) &err);

        printf("task1 has pended mutex\r\n");
        printf("task1 runing\r\n");
        printf("task1 post mutex\r\n");
        OSMutexPost(    (OS_MUTEX*) &mutex,
                        (OS_OPT) OS_OPT_POST_NONE,
                        (OS_ERR*) &err);

        OSTimeDly(100, OS_OPT_TIME_DLY, &err);
    }
}

/**
 * @brief  task2
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task2(void *p_arg)
{

```

```
uint32_t task2_num = 0;
OS_ERR err;

OSTimeDly(200, OS_OPT_TIME_DLY, &err);
while (1)
{
    for (task2_num=0; task2_num<5; task2_num++)
    {
        printf("task2 runing\r\n");
        delay_ms(100);
    }
    OSTimeDly(1000, OS_OPT_TIME_DLY, &err);
}

/**
 * @brief   task3
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task3(void *p_arg)
{
    uint32_t task3_num = 0;
    OS_ERR err;

    while (1)
    {
        printf("task3 ready to pend mutex\r\n");
        OSMutexPend(    (OS_MUTEX* )&mutex,
                        (OS_TICK   )0,
                        (OS_OPT    )OS_OPT_PEND_BLOCKING,
                        (CPU_TS*   )0,
                        (OS_ERR*   )&err);
        printf("task3 has pend mutex\r\n");
        for (task3_num=0; task3_num<5; task3_num++)
        {
            printf("task3 runing\r\n");
            delay_ms(100);
        }
        printf("task3 post mutex\r\n");
        OSMutexPost(    (OS_MUTEX* )&mutex,
                        (OS_OPT    )OS_OPT_POST_NONE,
                        (OS_ERR*   )&err);
    }
}
```

```
OSTimeDly(1000, OS_OPT_TIME_DLY, &err);
}
}
```

以上 task1、task2 和 task3 任务就是展示了使用互斥信号量优化优先级翻转的问题。

12.4.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 12.4.2 LCD 显示内容

同时，通过串口打印了本次实验的相关信息，如下图所示：

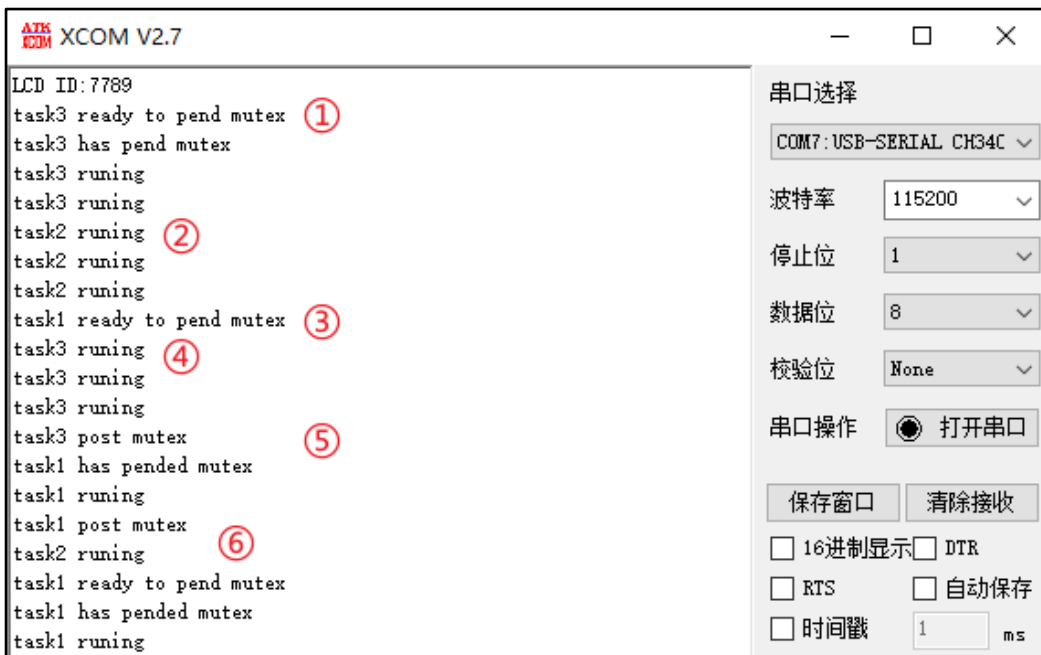


图 12.4.3 串口调试助手

1. task3 成功获取互斥信号量，并运行。
2. task2 任务抢占 task3 任务运行。
3. task1 任务抢占 task2 任务运行，task1 任务获取互斥信号量失败，被挂起，同时将互斥信号量的持有者进行优先级继承，此时 task3 任务的任务优先级已经提高到与 task1 任务相同的任务优先级。
4. 因为 task3 任务的任务优先级被提高了，因此 task3 抢占了 task2 任务运行。
5. task3 任务释放互斥信号量，此时 task3 任务的任务优先级会被恢复到优先级继承之前的原始优先级。
6. 因为互斥信号量已经被释放了，因此终止了 task1 任务的挂起状态，并让 task1 任务成为互斥信号量的持有者

从串口打印的信息就能看出，相较于使用二值信号量和计数型信号量，由于互斥信号量具有优先级继承的机制，因此在本实验中，任务优先级最高的 task1 任务并没有因为获取不到互斥信号量，而一直被其他比自身任务优先级低的任务抢占，而仅仅只需要等待互斥信号量的持有者释放互斥信号量，即可运行，这么一来系统的实时性得到了保障。

第十三章 μC/OS-III 消息队列

在前面的章节中，讲解了 μC/OS-III 中的几种信号量，信号量一般用于任务之间的同步，但是在实际的项目开发当中，经常会遇到需要在任务之间进行通信，任务间的通信，指的是消息的传递，μC/OS-III 提供了消息队列的机制，通过这一机制，就能够很好地在任务间传递消息。本章就来学习 μC/OS-III 中的消息队列。

本章分为如下几部分：

13.1 μC/OS-III 消息队列简介

13.2 μC/OS-III 消息队列相关 API 函数

13.3 μC/OS-III 消息队列实验

13.1 μC/OS-III 消息队列简介

任务与其他任务或任务与中断间的通讯一般可以通过全局变量或消息队列来完成，如果是使用全局变量的话，那么这个全局变量将被作为任务与任务之间或任务与中断之间的共享资源，因此开发者在设计时还需要考虑该共享资源的互斥访问问题，并且当全局变量被一个任务或中断访问更新后，通讯中全局变量的接收任务无法知道该全局变量是否已经被更新，从而无法实时地获取该全局变量的最新数据，正是由于使用全局变量进行任务与任务或任务与中断之间的通讯存在诸多的弊端，因此诞生了消息队列这一机制。

消息队列中包含了消息，这些消息可以通过消息队列进行传输，也可以以直接送到指定的任务中，因为每个任务都可以有独自的内嵌消息队列（这在后面的章节中会进行讲解）。通过消息队列传输的消息，可以发送给多个任务。

下面来看一下消息的数据类型结构体，消息的数据类型结构体定义在文件 os.h 中，具体的代码如下所示：

```
struct os_msg{
    OS_MSG*      NextPtr;      /* 指向下一条消息的指针 */
    void*        MsgPtr;       /* 指向消息内容的指针 */
    OS_MSG_SIZE MsgSize;       /* 消息内容的大小，单位：字节 */

#ifdef (OS_CFG_TS_EN > 0u)
    CPU_TS      MsgTS;         /* 消息发送时的时间戳 */
#endif
};
```

可以看到，消息的结构中包含了一个指向消息内容的指针 `MsgPtr`，其数据类型为 `void*`，这可以理解为是一个“万能指针”，`MsgPtr` 可以指向任何的数据，甚至可以指向一个函数，因此消息的发送方和接收方必须按照实现约定好的方式去发送和接收消息，只有这样，消息的接收方才能够正确地解析接收到的消息。

前面说到，消息是通过消息队列进行传输的，任务和中断都能够操作消息队列，但是中断只能往消息队列中发送消息，而不能从消息队列中接收消息，如下图所示：

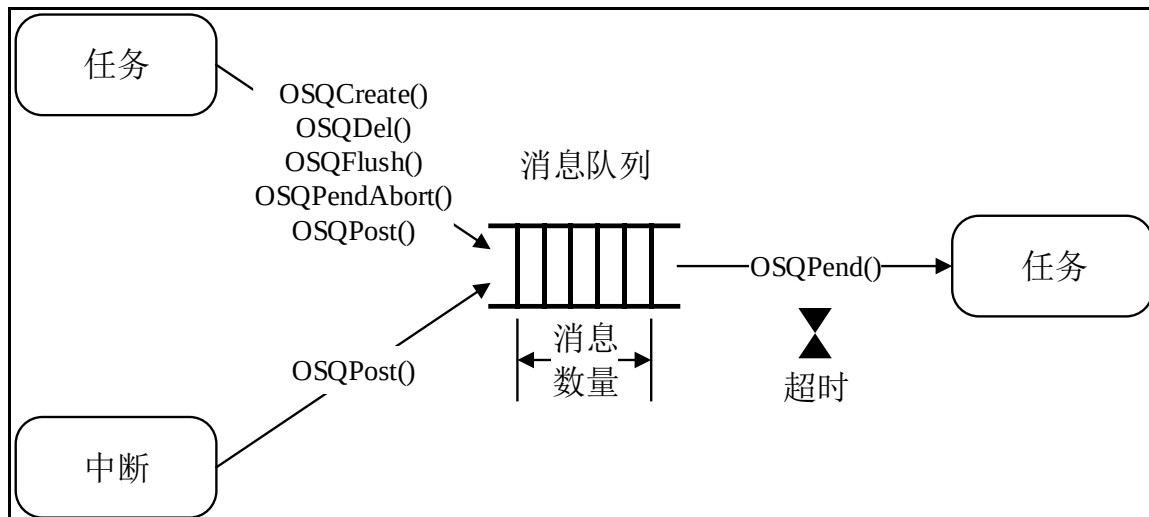


图 13.1.1 消息队列操作示意图

消息队列中的消息是以 LIFO 的方式传输消息的，即以后进先出的方式传输消息。LIFO 的传输机制在任务或中断需要向任务发送紧急的消息的情况下显得非常有用，在这种情况下，紧急的消息将比消息队列中已有的其他非紧急消息更早地被任务接收。

在上图中可以看到，在靠近接收任务的地方有一个代表超时“沙漏”，这表示任务从消息队列中接收消息是可以指定超时时间的，这个超时时间说明接收消息的任务愿意在消息队列中无消息的时候，等待一定的时间，如果超过了指定的这段时间，消息队列中还是没有消息，那么将接收任务将收到超时相应的错误代码，当然，也可以指定等待的时间为无限长，那么接收消息的任务将一直被挂起，直到消息队列中有消息。

消息队列中还包含了挂起等待任务链表，这意味着，可以有多个任务同时等待同一个消息队列中的消息，如下图所示：

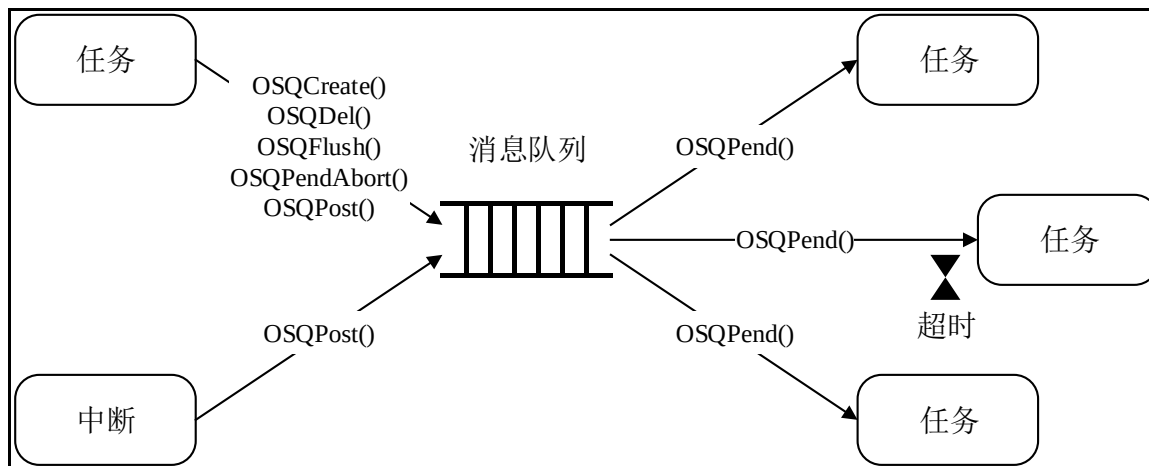


图 13.1.2 多任务等待消息示意图

在这种情况下，发送消息的任务或中断，可以指定发送消息到挂起等待任务链表中任务优先级最高的任务或者是发送到挂起等待任务链表中的所有任务。如果因接收到消息而被解除挂起状态的任务的任务优先级高于发送消息的任务的任务优先级，那么 μC/OS-III 将会立马转去执行任务优先级高的接收任务。

13.2 μC/OS-III 消息队列相关 API 函数

μC/OS-III 提供了一些列操作消息队列的 API 函数，这些操作函数如下表所示：

函数	描述
OSQCreate()	创建一个消息队列
OSQDel()	删除一个消息队列
OSQFlush()	清空消息队列中的所有消息
OSQPend()	获取消息队列中的消息
OSQPendAbort()	终止任务挂起等待消息队列
OSQPost()	发送消息到消息队列

表 13.2.1 μC/OS-III 消息队列相关 API 函数描述

1. 函数 OSQCreate()

该函数用于创建一个消息队列。

函数 OSQCreate() 的函数原型，如下所示：

```
void OSQCreate(OS_Q*      p_q,
               CPU_CHAR*  p_name,
               OS_MSG_QTY max_qty,
               OS_ERR*    p_err)
```

函数 OSQCreate() 的形参描述，如下表所示：

形参	描述
p_q	指向消息队列结构体的指针
p_name	指向作为消息队列名的 ASCII 字符串的指针
max_qty	消息队列的大小
p_err	指向接收错误代码变量的指针

表 13.2.2 函数 OSQCreate() 形参相关描述

函数 OSQCreate() 的返回值描述，如下表所示：

返回值	描述
无	无

表 13.2.3 函数 OSQCreate() 返回值相关描述

函数 OSQCreate() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	消息队列创建成功
OS_ERR_CREATE_ISR	在中断中非法调用该函数
OS_ERR_ILLEGAL_CREATE_RUN_TIME	在系统运行过程中非法创建内核对象
OS_ERR_OBJ_PTR_NULL	指向消息队列结构体的指针为空
OS_ERR_Q_SIZE	消息队列的大小为非法值 0

表 13.2.4 函数 OSQCreate() 错误代码相关描述

函数 OSQCreate() 定义在文件 os_q.c 中，具体的代码如下所示：

```
void OSQCreate(OS_Q*      p_q,
               CPU_CHAR*  p_name,
               OS_MSG_QTY max_qty,
               OS_ERR*    p_err)
```

```
{  
    CPU_SR_ALLOC();  
  
    /* 此宏通常用于使能检查 p_err 指针的合法性 */  
#ifdef OS_SAFETY_CRITICAL  
    if (p_err == (OS_ERR *)0) {  
        OS_SAFETY_CRITICAL_EXCEPTION();  
        return;  
    }  
#endif  
  
    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */  
#ifdef OS_SAFETY_CRITICAL_IEC61508  
    if (OSSafetyCriticalStartFlag == OS_TRUE) {  
        *p_err = OS_ERR_ILLEGAL_CREATE_RUN_TIME;  
        return;  
    }  
#endif  
  
    /* 此宏用于使能检查是否在中断中非法调用相关函数 */  
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)  
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，  
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中  
     * 因为在中断中不能调用创建任务的函数，  
     * 因此返回错误  
     */  
    if (OSIntNestingCtr > 0u) {  
        *p_err = OS_ERR_CREATE_ISR;  
        return;  
    }  
#endif  
  
    /* 此宏用于使能参数检查 */  
#if (OS_CFG_ARG_CHK_EN > 0u)  
    if (p_q == (OS_Q *)0) {  
        *p_err = OS_ERR_OBJ_PTR_NULL;  
        return;  
    }  
    if (max_qty == 0u) {  
        *p_err = OS_ERR_Q_SIZE;  
        return;  
    }  
#endif
```

```
/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 此宏用于使能检查内核对象类型 */
#if (OS_OBJ_TYPE_REQ > 0u)
#if (OS_CFG_OBJ_CREATED_CHK_EN > 0u)
    if (p_q->Type == OS_OBJ_TYPE_Q) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_OBJ_CREATED;
        return;
    }
#endif
#endif

/* 对象类型设置为 OS_OBJ_TYPE_Q */
p_q->Type = OS_OBJ_TYPE_Q;
#endif

/* 初始化一些用于调试的成员变量 */
#if (OS_CFG_DBG_EN > 0u)
    p_q->NamePtr = p_name;
#else
    (void)p_name;
#endif

/* 初始化消息队列结构体中的成员变量 */
OS_MsgQInit(&p_q->MsgQ, max_qty);

/* 初始化消息队列结构体中的挂起等待任务链表 */
OS_PendListInit(&p_q->PendList);

/* 初始化一些用于调试的成员变量 */
#if (OS_CFG_DBG_EN > 0u)
    OS_QDbgListAdd(p_q);
    OSQQty++;
#endif

/* 用于调试，不用理会 */
OS_TRACE_Q_CREATE(p_q, p_name);

/* 退出临界区 */
CPU_CRITICAL_EXIT();
*p_err = OS_ERR_NONE;
}
```

2. 函数 OSQDel()

该函数用于删除一个消息队列。

函数 OSQDel() 的函数原型，如下所示：

```
OS_OBJ_QTY OSQDel( OS_Q* p_q,
                   OS_OPT opt,
                   OS_ERR* p_err)
```

函数 OSQDel() 的形参描述，如下表所示：

形参	描述
p_q	指向消息队列结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 13.2.5 函数 OSQDel() 形参相关描述

函数 OSQDel() 的返回值描述，如下表所示：

返回值	描述
OS_OBJ_QTY 类型返回值	删除消息队列时，被终止挂起任务的数量

表 13.2.6 函数 OSQDel() 返回值相关描述

函数 OSQDel() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	消息队列删除成功
OS_ERR_DEL_ISR	在中断中非法调用该函数
OS_ERR_ILLEGAL_DEL_RUN_TIME	在系统运行过程中非法创建内核对象
OS_ERR_OBJ_PTR_NULL	指向消息队列结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是消息队列
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_TASK_WAITING	有任务挂起等待消息队列，不能删除消息队列

表 13.2.7 函数 OSQDel() 错误代码相关描述

函数 OSQDel() 定义在文件 os_q.c 中，具体的代码如下所示：

```
OS_OBJ_QTY OSQDel( OS_Q* p_q,
                   OS_OPT opt,
                   OS_ERR* p_err)
{
    OS_OBJ_QTY    nbr_tasks;
    OS_PEND_LIST* p_pend_list;
    OS_TCB*       p_tcb;
    CPU_TS        ts;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif
}
```

```

    }
#endif

    OS_TRACE_Q_DEL_ENTER(p_q, opt);

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        OS_TRACE_Q_DEL_EXIT(OS_ERR_ILLEGAL_DEL_RUN_TIME);
        *p_err = OS_ERR_ILLEGAL_DEL_RUN_TIME;
        return (0u);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    if (OSIntNestingCtr > 0u) {
        OS_TRACE_Q_DEL_EXIT(OS_ERR_DEL_ISR);
        *p_err = OS_ERR_DEL_ISR;
        return (0u);
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_Q_DEL_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_q == (OS_Q *)0) {
        OS_TRACE_Q_DEL_EXIT(OS_ERR_OBJ_PTR_NULL);
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }

```

```

    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_q->Type != OS_OBJ_TYPE_Q) {
        OS_TRACE_Q_DEL_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 获取消息队列的挂起等待任务链表 */
    p_pend_list = &p_q->PendList;
    /* 用于记录被终止挂起任务的数量 */
    nbr_tasks = 0u;

    /* 根据不同的函数操作选项，
     * 作不同的处理
     */
    switch (opt) {
        /* 仅在没有挂起等待任务时，
         * 才删除消息队列
         */
        case OS_OPT_DEL_NO_PEND:
            /* 判断消息队列的挂起等待任务链表是否为空 */
            if (p_pend_list->HeadPtr == (OS_TCB *)0) {
                /* 此宏用于代码调试功能 */
                #if (OS_CFG_DBG_EN > 0u)
                    OS_QDbgListRemove(p_q);
                    OSQQty--;
                #endif

                /* 用于调试，不用理会 */
                OS_TRACE_Q_DEL(p_q);
                /* 去初始化消息队列 */
                OS_QClr(p_q);
                /* 退出临界区 */
                CPU_CRITICAL_EXIT();
                *p_err = OS_ERR_NONE;
            } else {

```

```

        /* 有任务正挂起等待该消息队列
        * 因此退出临界区,
        * 不删除消息队列
        */
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_TASK_WAITING;
    }
    break;

    /* 始终删除消息队列 */
    case OS_OPT_DEL_ALWAYS:
#if (OS_CFG_TS_EN > 0u)
        ts = OS_TS_GET();
#else
        ts = 0u;
#endif

    /* 终止挂起所有挂起等待该消息队列的任务 */
    while (p_pend_list->HeadPtr != (OS_TCB *)0) {
        p_tcb = p_pend_list->HeadPtr;
        OS_PendAbort(    p_tcb,
                        ts,
                        OS_STATUS_PEND_DEL);

        nbr_tasks++;
    }

    /* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
        OS_QDbgListRemove(p_q);
        OSQQty--;
#endif

    /* 用于调试, 不用理会 */
    OS_TRACE_Q_DEL(p_q);

    /* 去初始化消息队列 */
    OS_QClr(p_q);
    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    /* 触发任务调度 */
    OSSched();
    *p_err = OS_ERR_NONE;
    break;

    /* 无效的任务操作选项 */
    default:

```

```

        CPU_CRITICAL_EXIT();

        *p_err = OS_ERR_OPT_INVALID;
        break;
    }

    OS_TRACE_Q_DEL_EXIT(*p_err);

    /* 返回被终止挂起任务的数量 */
    return (nbr_tasks);
}

```

3. 函数 OSQFlush()

该函数用于清空消息队列中的所有消息。

函数 OSQFlush()的函数原型，如下所示：

```

OS_MSG_QTY OSQFlush(    OS_Q*    p_q,
                        OS_ERR* p_err)

```

函数 OSQFlush()的形参描述，如下表所示：

形参	描述
p_q	指向消息队列结构体的指针
p_err	指向接收错误代码变量的指针

表 13.2.8 函数 OSQFlush()形参相关描述

函数 OSQFlush()的返回值描述，如下表所示：

返回值	描述
OS_MSG_QTY 类型返回值	被释放的消息数量

表 13.2.9 函数 OSQFlush()返回值相关描述

函数 OSQFlush()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	消息队列清空成功
OS_ERR_FLUSH_ISR	在中断中非法调用该函数
OS_ERR_OBJ_PTR_NULL	指向消息队列结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是消息队列
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行

表 13.2.10 函数 OSQFlush()错误代码相关描述

函数 OSQFlush()定义在文件 os_q.c 中，具体的代码如下所示：

```

OS_MSG_QTY OSQFlush(    OS_Q*    p_q,
                        OS_ERR* p_err)
{
    OS_MSG_QTY entries;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
    }
#endif
}

```



```
        return (0u);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_FLUSH_ISR;
        return (0u);
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_q == (OS_Q *)0) {
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_q->Type != OS_OBJ_TYPE_Q) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();
```

```

/* 清空消息队列中的所有消息 */
entries = OS_MsgQFreeAll (&p_q->MsgQ);

/* 退出临界区 */
CPU_CRITICAL_EXIT();

*p_err = OS_ERR_NONE;

/* 返回被释放的消息数量 */
return (entries);
}

```

4. 函数 OSQPend()

该函数用于获取消息队列中的消息。

函数 OSQPend()的函数原型，如下所示：

```

void *OSQPend( OS_Q*      p_q,
               OS_TICK    timeout,
               OS_OPT      opt,
               OS_MSG_SIZE* p_msg_size,
               CPU_TS*     p_ts,
               OS_ERR*     p_err)

```

函数 OSQPend()的形参描述，如下表所示：

形参	描述
p_q	指向消息队列结构体的指针
timeout	任务挂起等待消息队列的最大允许时间
opt	函数操作选项
p_msg_size	指向用于接收消息大小变量的指针
p_ts	指向接收消息队列接收时的时间戳的变量的指针
p_err	指向接收错误代码变量的指针

表 13.2.11 函数 OSQPend()形参相关描述

函数 OSQPend()的返回值描述，如下表所示：

返回值	描述
void *类型返回值	指向消息的指针

表 13.2.12 函数 OSQPend()返回值相关描述

函数 OSQPend()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	消息接收成功
OS_ERR_OBJ_DEL	指定的消息队列已经被删除
OS_ERR_OBJ_PTR_NULL	指向消息队列结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是消息队列
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT	任务挂起等待消息队列时被终止（还未超时）
OS_ERR_PEND_ISR	在中断中非法调用该函数
OS_ERR_PEND_WOULD_BLOCK	获取消息队列失败，并且不挂起任务等待互斥信号量
OS_ERR_PTR_INVALID	参数 p_msg_size 指针为空
OS_ERR_SCHED_LOCKED	任务调度器已锁定

OS_ERR_TIMEOUT

任务挂起等待消息队列超时

表 13.2.13 函数 OSQPend()错误代码相关描述

函数 OSQPend()定义在文件 os_q.c 中，具体的代码如下所示：

```
void *OSQPend( OS_Q*      p_q,
               OS_TICK    timeout,
               OS_OPT      opt,
               OS_MSG_SIZE* p_msg_size,
               CPU_TS*     p_ts,
               OS_ERR*     p_err)
{
    void *p_void;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return ((void *)0);
    }
#endif

    OS_TRACE_Q_PEND_ENTER(p_q, timeout, opt, p_msg_size, p_ts);

    /* 此宏用于使能系统时钟节拍 */
    #if (OS_CFG_TICK_EN == 0u)
        if (timeout != 0u) {
            *p_err = OS_ERR_TICK_DISABLED;
            OS_TRACE_Q_PEND_FAILED(p_q);
            OS_TRACE_Q_PEND_EXIT(OS_ERR_TICK_DISABLED);
            return ((void *)0);
        }
    #endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
         * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
         * 因为在中断中不能调用创建任务的函数，
         * 因此返回错误
         */
        if (OSIntNestingCtr > 0u) {
            if ((opt & OS_OPT_PEND_NON_BLOCKING) != OS_OPT_PEND_NON_BLOCKING) {
                OS_TRACE_Q_PEND_FAILED(p_q);
            }
        }
    #endif
}
```

```

        OS_TRACE_Q_PEND_EXIT(OS_ERR_PEND_ISR);
        *p_err = OS_ERR_PEND_ISR;
        return ((void *)0);
    }
}
#endif

/* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_Q_PEND_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return ((void *)0);
    }
#endif

/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_q == (OS_Q *)0) {
        OS_TRACE_Q_PEND_FAILED(p_q);
        OS_TRACE_Q_PEND_EXIT(OS_ERR_OBJ_PTR_NULL);
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return ((void *)0);
    }
    if (p_msg_size == (OS_MSG_SIZE *)0) {
        OS_TRACE_Q_PEND_FAILED(p_q);
        OS_TRACE_Q_PEND_EXIT(OS_ERR_PTR_INVALID);
        *p_err = OS_ERR_PTR_INVALID;
        return ((void *)0);
    }
    switch (opt) {
        case OS_OPT_PEND_BLOCKING:
        case OS_OPT_PEND_NON_BLOCKING:
            break;

        default:
            OS_TRACE_Q_PEND_FAILED(p_q);
            OS_TRACE_Q_PEND_EXIT(OS_ERR_OPT_INVALID);
            *p_err = OS_ERR_OPT_INVALID;
            return ((void *)0);
    }
#endif

```

```

/* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_q->Type != OS_OBJ_TYPE_Q) {
        OS_TRACE_Q_PEND_FAILED(p_q);
        OS_TRACE_Q_PEND_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return ((void *)0);
    }
#endif

if (p_ts != (CPU_TS *)0) {
    *p_ts = 0u;
}

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/*从消息队列中获取消息 */
p_void = OS_MsgQGet(    &p_q->MsgQ,
                        p_msg_size,
                        p_ts,
                        p_err);

/* 判断是否获取成功 */
if (*p_err == OS_ERR_NONE) {
    /* 获取成功则直接返回 */
    OS_TRACE_Q_PEND(p_q);
    CPU_CRITICAL_EXIT();
    OS_TRACE_Q_PEND_EXIT(OS_ERR_NONE);
    return (p_void);
}

/* 获取消息失败，
 * 接下来根据函数操作选项判断是否需要挂起等待消息
 */
if ((opt & OS_OPT_PEND_NON_BLOCKING) != 0u) {
    /* 不挂起等待消息，
     * 则直接返回
     */
    CPU_CRITICAL_EXIT();
    OS_TRACE_Q_PEND_FAILED(p_q);
    OS_TRACE_Q_PEND_EXIT(OS_ERR_PEND_WOULD_BLOCK);
    *p_err = OS_ERR_PEND_WOULD_BLOCK;
}

```

```

        return ((void *)0);
    } else {
        /* 选择挂起等待消息还要判断任务调度器是否被上锁，
        * 若任务调度器被上锁，
        * 则直接返回
        */
        if (OSSchedLockNestingCtr > 0u) {
            CPU_CRITICAL_EXIT();
            OS_TRACE_Q_PEND_FAILED(p_q);
            OS_TRACE_Q_PEND_EXIT(OS_ERR_SCHED_LOCKED);
            *p_err = OS_ERR_SCHED_LOCKED;
            return ((void *)0);
        }
    }

    /* 将任务添加到消息队列的挂起等待任务链表中 */
    OS_Pend( (OS_PEND_OBJ *) ((void *)p_q),
            OSTCBCurPtr,
            OS_TASK_PEND_ON_Q,
            timeout);

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    OS_TRACE_Q_PEND_BLOCK(p_q);

    /* 触发任务调度，
    * 触发任务调度后，
    * 任务将处于挂起状态
    */
    OSSched();

    /* 只有当任务结束挂起状态后，
    * 才会执行以下代码
    */

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 根据不同的挂起结果进行处理 */
    switch (OSTCBCurPtr->PendStatus) {
        /* 成功地等待到了消息 */
        case OS_STATUS_PEND_OK:
            p_void = OSTCBCurPtr->MsgPtr;

```

```

        *p_msg_size = OSTCBCurPtr->MsgSize;
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#endif

    OS_TRACE_Q_PEND(p_q);
    *p_err = OS_ERR_NONE;
    break;

    /* 任务挂起被终止 */
case OS_STATUS_PEND_ABORT:
    p_void = (void *)0;
    *p_msg_size = 0u;
#if (OS_CFG_TS_EN > 0u)
    if (p_ts != (CPU_TS *)0) {
        *p_ts = OSTCBCurPtr->TS;
    }
#endif

    OS_TRACE_Q_PEND_FAILED(p_q);
    *p_err = OS_ERR_PEND_ABORT;
    break;

    /* 任务挂起等待消息队列超时 */
case OS_STATUS_PEND_TIMEOUT:
    p_void = (void *)0;
    *p_msg_size = 0u;
    OS_TRACE_Q_PEND_FAILED(p_q);
    *p_err = OS_ERR_TIMEOUT;
    break;

    /* 任务等待的消息队列被删除 */
case OS_STATUS_PEND_DEL:
    p_void = (void *)0;
    *p_msg_size = 0u;
#if (OS_CFG_TS_EN > 0u)
    if (p_ts != (CPU_TS *)0) {
        *p_ts = OSTCBCurPtr->TS;
    }
#endif

    OS_TRACE_Q_PEND_FAILED(p_q);
    *p_err = OS_ERR_OBJ_DEL;
    break;

```

```

/* 无效的挂起结果 */
default:
    p_void = (void *)0;
    *p_msg_size = 0u;
    OS_TRACE_Q_PEND_FAILED(p_q);
    *p_err = OS_ERR_STATUS_INVALID;
    break;
}

/* 退出临界区,
 * 并返回
 */
CPU_CRITICAL_EXIT();
OS_TRACE_Q_PEND_EXIT(*p_err);
return (p_void);
}

```

5. 函数 OSQPendAbort()

该函数用于终止任务挂起等待消息队列。

函数 OSQPendAbort()的函数原型，如下所示：

```

OS_OBJ_QTY OSQPendAbort( OS_Q*   p_q,
                        OS_OPT  opt,
                        OS_ERR* p_err)

```

函数 OSQPendAbort()的形参描述，如下表所示：

形参	描述
p_q	指向消息队列结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 13.2.14 函数 OSQPendAbort()形参相关描述

函数 OSQPendAbort()的返回值描述，如下表所示：

返回值	描述
OS_OBJ_QTY 类型返回值	被终止挂起任务的数量

表 13.2.15 函数 OSQPendAbort()返回值相关描述

函数 OSQPendAbort()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	终止任务挂起等待消息队列成功
OS_ERR_OBJ_PTR_NULL	指向消息队列结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是消息队列
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT_ISR	在中断中非法调用该函数
OS_ERR_PEND_ABORT_NONE	没有任务挂起等待消息队列

表 13.2.16 函数 OSQPendAbort()错误代码相关描述

函数 OSQPendAbort() 定义在文件 os_q.c 中，具体的代码如下所示：

```
OS_OBJ_QTY OSQPendAbort( OS_Q* p_q,
                          OS_OPT opt,
                          OS_ERR* p_err)
{
    OS_PEND_LIST* p_pend_list;
    OS_TCB* p_tcb;
    CPU_TS ts;
    OS_OBJ_QTY nbr_tasks;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
         * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
         * 因为在中断中不能调用创建任务的函数，
         * 因此返回错误
         */
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_PEND_ABORT_ISR;
            return (0u);
        }
    #endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        if (OSRunning != OS_STATE_OS_RUNNING) {
            *p_err = OS_ERR_OS_NOT_RUNNING;
            return (0u);
        }
    #endif

    /* 此宏用于使能参数检查 */
    #if (OS_CFG_ARG_CHK_EN > 0u)
        if (p_q == (OS_Q *)0) {
```

```

        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }

    switch (opt) {
        case OS_OPT_PEND_ABORT_1:
        case OS_OPT_PEND_ABORT_ALL:
        case OS_OPT_PEND_ABORT_1 | OS_OPT_POST_NO_SCHED:
        case OS_OPT_PEND_ABORT_ALL | OS_OPT_POST_NO_SCHED:
            break;

        default:
            *p_err = OS_ERR_OPT_INVALID;
            return (0u);
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_q->Type != OS_OBJ_TYPE_Q) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 获取消息队列的挂起等待任务链表 */
    p_pend_list = &p_q->PendList;
    /* 如果消息队列的挂起等待任务链表为空
     * 则不需要终止挂起任务，
     * 因此直接返回
     */
    if (p_pend_list->HeadPtr == (OS_TCB *)0) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_PEND_ABORT_NONE;
        return (0u);
    }

    /* 用于记录被终止挂起任务的数量 */
    nbr_tasks = 0u;
#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();

```

```
#else
    ts = 0u;
#endif

/* 遍历消息队列的挂起等待任务链表 */
while (p_pend_list->HeadPtr != (OS_TCB *)0) {
    /* 获取被挂起的任务 */
    p_tcb = p_pend_list->HeadPtr;

    /* 将任务从消息队列的挂起等待任务链表中移除 */
    OS_PendAbort(    p_tcb,
                    ts,
                    OS_STATUS_PEND_ABORT);

    /* 记录被终止挂起任务的数量 */
    nbr_tasks++;
    if (opt != OS_OPT_PEND_ABORT_ALL) {
        break;
    }
}

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 根据函数操作选项进行任务调度 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

*p_err = OS_ERR_NONE;
return (nbr_tasks);
}
```

6. 函数 OSQPost()

该函数用于发送消息到消息队列。

函数 OSQPost()的函数原型，如下所示：

```
void OSQPost(    OS_Q*        p_q,
                void*        p_void,
                OS_MSG_SIZE msg_size,
                OS_OPT        opt,
                OS_ERR*       p_err)
```

函数 OSQPost()的形参描述，如下表所示：

形参	描述
p_q	指向消息队列结构体的指针

p_void	指向消息的指针
msg_size	消息的大小，单位：字节
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 13.2.17 函数 OSQPost()形参相关描述

函数 OSQPost()的返回值描述，如下表所示：

返回值	描述
无	无

表 13.2.18 函数 OSQPost()返回值相关描述

函数 OSQPost()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功发送消息到消息队列
OS_ERR_MSG_POLL_EMPTY	消息队列满
OS_ERR_OBJ_PTR_NULL	指向消息队列结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是消息队列
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_Q_MAX	消息队列满

表 13.2.19 函数 OSQPost()错误代码相关描述

函数 OSQPost()定义在文件 os_q.c 中，具体的代码如下所示：

```
void OSQPost( OS_Q*      p_q,
              void*      p_void,
              OS_MSG_SIZE msg_size,
              OS_OPT      opt,
              OS_ERR*     p_err)
{
    OS_OPT      post_type;
    OS_PEND_LIST* p_pend_list;
    OS_TCB*     p_tcb;
    OS_TCB*     p_tcb_next;
    CPU_TS      ts;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    OS_TRACE_Q_POST_ENTER(p_q, p_void, msg_size, opt);
```

```

/* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_Q_POST_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return;
    }
#endif

/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_q == (OS_Q *)0) {
        OS_TRACE_Q_POST_FAILED(p_q);
        OS_TRACE_Q_POST_EXIT(OS_ERR_OBJ_PTR_NULL);
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return;
    }

    switch (opt) {
        case OS_OPT_POST_FIFO:
        case OS_OPT_POST_LIFO:
        case OS_OPT_POST_FIFO | OS_OPT_POST_ALL:
        case OS_OPT_POST_LIFO | OS_OPT_POST_ALL:
        case OS_OPT_POST_FIFO | OS_OPT_POST_NO_SCHED:
        case OS_OPT_POST_LIFO | OS_OPT_POST_NO_SCHED:
        case OS_OPT_POST_FIFO | (OS_OPT) (OS_OPT_POST_ALL|OS_OPT_POST_NO_SCHED):
        case OS_OPT_POST_LIFO | (OS_OPT) (OS_OPT_POST_ALL|OS_OPT_POST_NO_SCHED):
            break;

        default:
            OS_TRACE_Q_POST_FAILED(p_q);
            OS_TRACE_Q_POST_EXIT(OS_ERR_OPT_INVALID);
            *p_err = OS_ERR_OPT_INVALID;
            return;
    }
#endif

/* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_q->Type != OS_OBJ_TYPE_Q) {
        OS_TRACE_Q_POST_FAILED(p_q);
        OS_TRACE_Q_POST_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return;
    }

```

```

    }
#endif

    /* 此宏用于使能时间戳功能 */
#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif

    OS_TRACE_Q_POST(p_q);

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 获取消息队列的挂起等待任务链表 */
    p_pend_list = &p_q->PendList;
    /* 判断是否有任务挂起等待消息队列，
     * 如果没有，则直接将消息存储消息队列，
     * 如果有，则直接将消息传递给挂起的任务
     */
    if (p_pend_list->HeadPtr == (OS_TCB *)0) {

        /* 根据函数操作选项将消息存入消息队列 */
        if ((opt & OS_OPT_POST_LIFO) == 0u) {
            post_type = OS_OPT_POST_FIFO;
        } else {
            post_type = OS_OPT_POST_LIFO;
        }
        OS_MsgQPut(&p_q->MsgQ,
                   p_void,
                   msg_size,
                   post_type,
                   ts,
                   p_err);

        /* 退出临界区并返回 */
        CPU_CRITICAL_EXIT();
        OS_TRACE_Q_POST_EXIT(*p_err);
        return;
    }

    /* 遍历消息队列的挂起等待任务链表 */

```

```

p_tcb = p_pend_list->HeadPtr;
while (p_tcb != (OS_TCB *)0) {
    p_tcb_next = p_tcb->PendNextPtr;

    /* 终止挂起挂起的任务，
     * 并传入消息
     */
    OS_Post(    (OS_PEND_OBJ *)((void *)p_q),
                p_tcb,
                p_void,
                msg_size,
                ts);

    /* 如果不将消息释放给所有挂起等待消息队列的任务，
     * 则退出遍历
     */
    if ((opt & OS_OPT_POST_ALL) == 0u) {
        break;
    }
    p_tcb = p_tcb_next;
}

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 根据函数选项进行任务调度 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

*p_err = OS_ERR_NONE;
OS_TRACE_Q_POST_EXIT(*p_err);
}

```

13.3 μC/OS-III 消息队列实验

13.3.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数使用消息队列发送和接收消息，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建消息队列和其他任务

task1	用于扫描按键，并将键值作为消息发送到消息队列
task2	从消息队列中获取键值消息，并作相应的解释

表 13.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 13 UCOS-III 消息队列实验》。

13.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

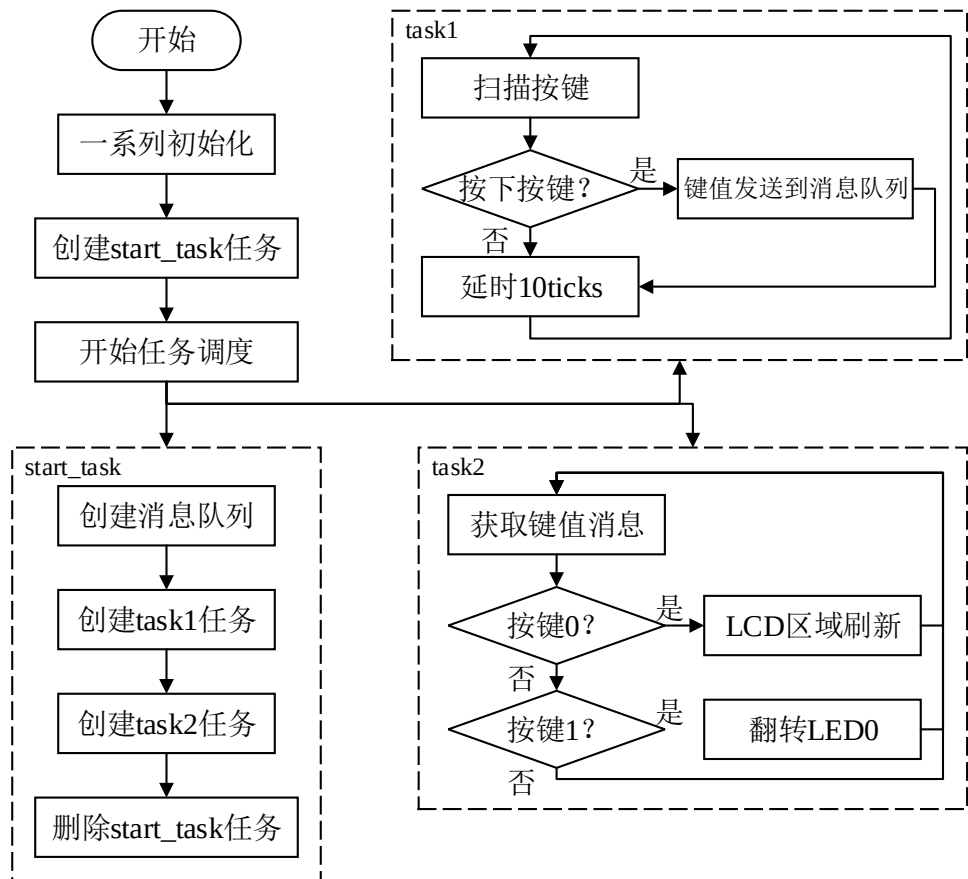


图 13.3.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSQCreate()

此函数用于创建消息队列，请参考第 13.2 小节《μC/OS-III 消息队列相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_Q_EN 配置为 1。

(2) 函数 OSQPend()

此函数用于从消息队列中获取消息，请参考第 13.2 小节《μC/OS-III 消息队列相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_Q_EN 配置为 1。

(3) 函数 OSQPost()

此函数用于发送消息到消息队列，请参考第 13.2 小节《μC/OS-III 消息队列相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_Q_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)
{
    OS_ERR      err;
    CPU_INT32U  cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建消息队列 */
    OSQCreate( (OS_Q*      )&q,
               (CPU_CHAR*  )"q",
               (OS_MSG_QTY)1,
               (OS_ERR*     )&err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc( SRAMIN,
                                         TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate( (OS_TCB*      )&Task1Task_TCB,
                  (CPU_CHAR*    )"task1",
                  (OS_TASK_PTR  )task1,
                  (void*         )0,
                  (OS_PRIO      )TASK1_PRIO,
                  (CPU_STK       *)Task1Task_STK,
                  (CPU_STK_SIZE )TASK1_STK_SIZE / 10,
                  (CPU_STK_SIZE )TASK1_STK_SIZE,
                  (OS_MSG_QTY    )0,
                  (OS_TICK       )0,
```

```

        (void*          )0,
        (OS_OPT         ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*        ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                       TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*          ) &Task2Task_TCB,
                 (CPU_CHAR*        ) "task2",
                 (OS_TASK_PTR      ) task2,
                 (void*            ) 0,
                 (OS_PRIO          ) TASK2_PRIO,
                 (CPU_STK*         ) Task2Task_STK,
                 (CPU_STK_SIZE     ) TASK2_STK_SIZE / 10,
                 (CPU_STK_SIZE     ) TASK2_STK_SIZE,
                 (OS_MSG_QTY       ) 0,
                 (OS_TICK          ) 0,
                 (void*            ) 0,
                 (OS_OPT           ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*          ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出, start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务和 task2 任务, 并且 task1 任务和 task2 任务的任务栈都是动态分配的, 当然, 也可以静态地定任务栈, 就像定义一个数组一样。同时还创建一个消息队列。

(2) task1 任务

```

/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    uint8_t key;
    OS_ERR  err;

    while (1)
    {
        key = key_scan(0);

        if (key != 0)

```

```

    {
        OSQPost(    (OS_Q*)      ) &q,
                    (void*)      ) &key,
                    (OS_MSG_SIZE ) sizeof(key),
                    (OS_OPT      ) OS_OPT_POST_FIFO,
                    (OS_ERR*)     ) &err);
    }

    OSTimeDly(10, OS_OPT_TIME_DLY, &err);
}
}

```

从上面的代码中可以看出，task1 任务还是比较简单的，就是扫描按键，然后将扫描到的按键键值作为消息发送到消息队列中。

(3) task2 任务

```

/**
 * @brief   task2
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task2(void *p_arg)
{
    uint32_t    task2_num = 0;
    uint8_t*    key;
    OS_MSG_SIZE size;
    OS_ERR      err;

    while (1)
    {
        key = (uint8_t *)OSQPend( (OS_Q *) &q,
                                   (OS_TICK) 0,
                                   (OS_OPT) OS_OPT_PEND_BLOCKING,
                                   (OS_MSG_SIZE *) &size,
                                   (CPU_TS *) 0,
                                   (OS_ERR *) &err);

        switch (*key)
        {
            case KEY0_PRES:
            {
                lcd_fill(6, 131, 233, 313, lcd_discolor[++task2_num % 11]);
                break;
            }
        }
    }
}

```

```

        case KEY1_PRES:
        {
            LED0_TOGGLE();
            break;
        }

        default:
        {
            break;
        }
    }
}

```

从上面的代码中可以看出，task2 任务就是不断地挂起等待获取消息队列中的消息，获取到消息后，将消息作为按键键值进行解析，然后对解析出的不同按键键值结果进行不同的解释，当接收到按键键值 0 的消息时，对 LCD 区域进行刷新，当接收到按键键值 1 的消息时，则改变 LED0 的状态。

13.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：

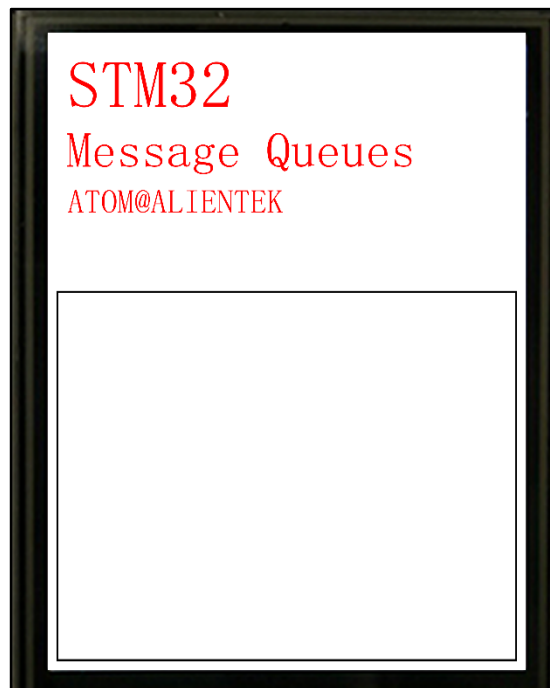


图 13.3.2 LCD 显示内容一

接着按下按键 0，使得 task1 任务往消息队列中发送按键键值为 0 的消息，然后 task2 任务就能够从消息队列中获取到并解析出内容为按键键值 0 的消息，接着执行刷新 LCD 的区域显示的操作，如下图所示：



图 13.3.3 LCD 显示内容二

如果多按几下按键 0，那么 task2 任务就会多次接收到键值 0 的消息，从而多次地去刷新 LCD。如果按键按键 1，那么 task1 任务就会发送键值为 1 的消息，task2 任务就会收到该消息，闭关翻转 LED0。

第十四章 μC/OS-III 任务内嵌信号量

信号量是用于任务同步的一种常用方法，因此 μC/OS-III 为每一个任务都分配了一个到任务控制块结构体中的任务内嵌信号量，这样不仅简化了代码，并且还提高了信号量的工作效率，本章就来学习 μC/OS-III 中的任务内嵌信号量。

本章分为如下几部分：

14.1 μC/OS-III 任务内嵌信号量简介

14.2 μC/OS-III 任务内嵌信号量相关 API 函数

14.3 μC/OS-III 任务内嵌信号量实验

14.1 μC/OS-III 任务内嵌信号量简介

任务内嵌信号量本质上就是一个信号量，与第十一章《μC/OS-III 信号量》中的信号量是类似的，但是任务内嵌信号量并不需要信号量这么一个中间的内核对象，任务内嵌信号量是分配于每一个任务的任务控制块结构体中的，因此每一个任务都有独自的任务内嵌信号量，任务内嵌信号量只能被该任务获取，但是可以由其他任务或者中断释放，如下图所示：

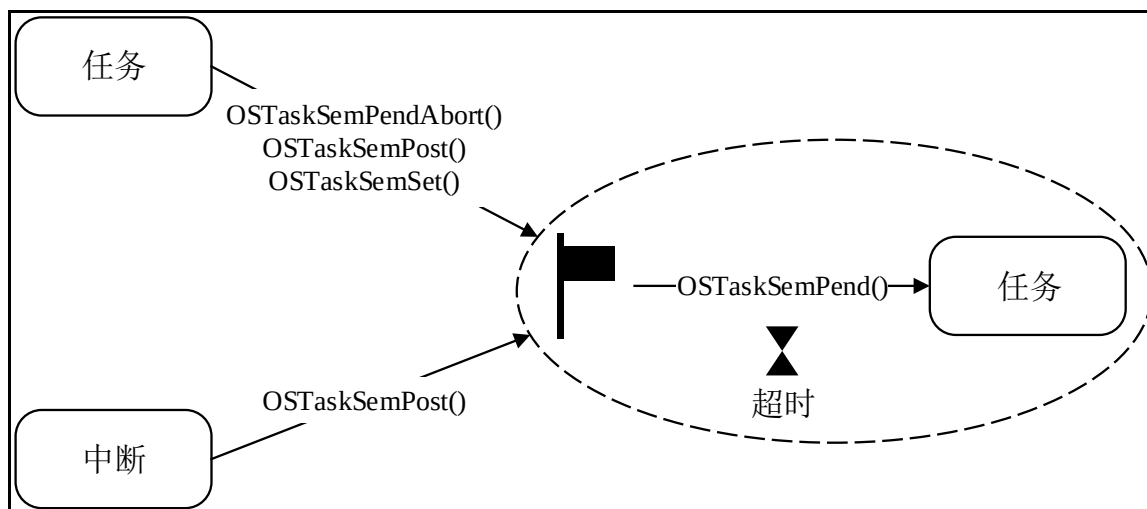


图 14.1.1 任务内嵌信号量操作示意图

如上如所示，当任务或中断需要往指定任务的内嵌信号量发出信号时，是需要调用相应的 API 函数即可，每个任务的内嵌信号量在创建的时候都已经被创建好了，并且发出的信号能够直接到达指定的任务中，因此使用内嵌信号量的效率比使用内核对象的信号量高得多，在实际的开发当中，可以优先考虑使用任务内嵌信号量。

14.2 μC/OS-III 任务内嵌信号量相关 API 函数

μC/OS-III 提供了一些列操作任务内嵌信号量的 API 函数，这些操作函数如下表所示：

函数	描述
OSTaskSemPend()	获取任务内嵌信号量
OSTaskSemPendAbort()	终止任务挂起等待任务内嵌信号量
OSTaskSemPost()	释放指定任务的任务内嵌信号量

OSTaskSemSet()	强制设置指定的任务内嵌信号量为指定值
----------------	--------------------

表 14.2.1 μC/OS-III 任务内嵌信号量相关 API 函数描述

1. 函数 OSTaskSemPend()

该函数用于获取任务内嵌信号量。

函数 OSTaskSemPend() 的函数原型，如下所示：

```
OS_SEM_CTR OSTaskSemPend( OS_TICK timeout,
                           OS_OPT opt,
                           CPU_TS* p_ts,
                           OS_ERR* p_err)
```

函数 OSTaskSemPend() 的形参描述，如下表所示：

形参	描述
timeout	任务挂起等待任务内嵌信号量的最大允许时间
opt	函数操作选项
p_ts	指向接收任务内嵌信号量接收时的时间戳的变量的指针
p_err	指向接收错误代码变量的指针

表 14.2.2 函数 OSTaskSemPend() 形参相关描述

函数 OSTaskSemPend() 的返回值描述，如下表所示：

返回值	描述
OS_SEM_CTR 类型返回值	任务内嵌信号量更新后的资源数

表 14.2.3 函数 OSTaskSemPend() 返回值相关描述

函数 OSTaskSemPend() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功获取任务内嵌信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT	任务挂起等待信号量时被终止（还未超时）
OS_ERR_PEND_ISR	在中断中非法调用该函数
OS_ERR_PEND_WOULD_BLOCK	获取的信号量没有资源，并且不挂起任务等待信号量
OS_ERR_SCHED_LOCKED	任务调度器被锁定
OS_ERR_STATUS_INVALID	无效的任务挂起结果
OS_ERR_TIMEOUT	任务挂起等待任务内嵌信号量超时

表 14.2.4 函数 OSTaskSemPend() 错误代码相关描述

函数 OSTaskSemPend() 定义在文件 os_task.c 中，具体的代码如下所示：

```
OS_SEM_CTR OSTaskSemPend( OS_TICK timeout,
                           OS_OPT opt,
                           CPU_TS* p_ts,
                           OS_ERR* p_err)
{
    OS_SEM_CTR ctr;
    CPU_SR_ALLOC();

    /* 此宏用于使能时间戳功能 */
    #if (OS_CFG_TS_EN == 0u)
```

```

        (void)p_ts;
    #endif

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
    #ifndef OS_SAFETY_CRITICAL
        if (p_err == (OS_ERR *)0) {
            OS_SAFETY_CRITICAL_EXCEPTION();
            return (0u);
        }
    #endif

    OS_TRACE_TASK_SEM_PEND_ENTER(OSTCBCurPtr, timeout, opt, p_ts);

    /* 此宏用于使能系统时钟节拍 */
    #if (OS_CFG_TICK_EN == 0u)
        if (timeout != 0u) {
            OS_TRACE_TASK_SEM_PEND_FAILED(OSTCBCurPtr);
            OS_TRACE_TASK_SEM_PEND_EXIT(OS_ERR_TICK_DISABLED);
            *p_err = OS_ERR_TICK_DISABLED;
            return (0u);
        }
    #endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
         * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
         * 因为在中断中不能调用创建任务的函数，
         * 因此返回错误
         */
        if (OSIntNestingCtr > 0u) {
            OS_TRACE_TASK_SEM_PEND_FAILED(OSTCBCurPtr);
            OS_TRACE_TASK_SEM_PEND_EXIT(OS_ERR_PEND_ISR);
            *p_err = OS_ERR_PEND_ISR;
            return (0u);
        }
    #endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        if (OSRunning != OS_STATE_OS_RUNNING) {
            OS_TRACE_TASK_SEM_PEND_EXIT(OS_ERR_OS_NOT_RUNNING);
            *p_err = OS_ERR_OS_NOT_RUNNING;
        }
    #endif

```



```

        return (0u);
    }
#endif

    /* 此宏用于使能参数检查 */
    #if (OS_CFG_ARG_CHK_EN > 0u)
        switch (opt) {
            case OS_OPT_PEND_BLOCKING:
            case OS_OPT_PEND_NON_BLOCKING:
                break;

            default:
                OS_TRACE_TASK_SEM_PEND_FAILED(OSTCBCurPtr);
                OS_TRACE_TASK_SEM_PEND_EXIT(OS_ERR_OPT_INVALID);
                *p_err = OS_ERR_OPT_INVALID;
                return (0u);
        }
    #endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();
    /* 判断信号量是否有资源 */
    if (OSTCBCurPtr->SemCtr > 0u) {
        /* 更新信号量资源数 */
        OSTCBCurPtr->SemCtr--;
        ctr = OSTCBCurPtr->SemCtr;
    #if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
    #if (OS_CFG_TASK_PROFILE_EN > 0u)
    #if (OS_CFG_TS_EN > 0u)
        OSTCBCurPtr->SemPendTime = OS_TS_GET() - OSTCBCurPtr->TS;
        if (OSTCBCurPtr->SemPendTimeMax < OSTCBCurPtr->SemPendTime) {
            OSTCBCurPtr->SemPendTimeMax = OSTCBCurPtr->SemPendTime;
        }
    #endif
    #endif
    #endif

    /* 用于调试，不用理会 */
    OS_TRACE_TASK_SEM_PEND(OSTCBCurPtr);
    /* 退出临界区 */
    CPU_CRITICAL_EXIT();

```

```
/* 用于调试，不用理会 */
OS_TRACE_TASK_SEM_PEND_EXIT(OS_ERR_NONE);
*p_err = OS_ERR_NONE;
/* 返回信号量更新后的资源数 */
return (ctr);
}

/* 判断任务是否不挂起等待信号量 */
if ((opt & OS_OPT_PEND_NON_BLOCKING) != 0u) {
    /* 任务如果不选择挂起等待信号量
     * 则退出临界区，并返回信号量当前的资源数 (0)
     */
    CPU_CRITICAL_EXIT();
#if (OS_CFG_TS_EN > 0u)
    if (p_ts != (CPU_TS *)0) {
        *p_ts = 0u;
    }
#endif
    OS_TRACE_TASK_SEM_PEND_FAILED(OSTCBCurPtr);
    OS_TRACE_TASK_SEM_PEND_EXIT(OS_ERR_PEND_WOULD_BLOCK);
    *p_err = OS_ERR_PEND_WOULD_BLOCK;
    return (0u);
} else {
    /* 如果任务选择挂起等待信号量，
     * 则接下来将要吧任务添加到信号量的挂起等待任务链表中挂起，
     * 并且需要先判断任务调度器是否被上锁，
     * 如果任务调度器被上锁，
     * 则直接返回相应的错误代码和信号量当前的资源数 (0)
     */

    if (OSSchedLockNestingCtr > 0u) {
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = 0u;
        }
#endif
        CPU_CRITICAL_EXIT();
        OS_TRACE_TASK_SEM_PEND_FAILED(OSTCBCurPtr);
        OS_TRACE_TASK_SEM_PEND_EXIT(OS_ERR_SCHED_LOCKED);
        *p_err = OS_ERR_SCHED_LOCKED;
        return (0u);
    }
}
```

```

/* 根据入参的超时时间将任务挂起到信号量的挂起等待任务链表中 */
OS_Pend( (OS_PEND_OBJ *)0,
        OSTCBCurPtr,
        OS_TASK_PEND_ON_TASK_SEM,
        timeout);

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 用于调试，不用理会 */
OS_TRACE_TASK_SEM_PEND_BLOCK(OSTCBCurPtr);

/* 触发任务调度，
 * 因为当前任务被挂起了
 */
OSSched();

/* 发起任务调度后，
 * 当前任务就被挂起了，
 * 因此下面的代码并不会立马被执行，
 * 只有当任务获取到信号量资源、任务挂起超时或任务挂起被中断后等情况下，
 * 才会执行下面的代码
 */

/* 进入临界区 */
CPU_CRITICAL_ENTER();
/* 根据任务不同的挂起结果进行不同的操作 */
switch (OSTCBCurPtr->PendStatus) {
    /* 成功获取信号量 */
    case OS_STATUS_PEND_OK:
#ifdef OS_CFG_TS_EN > 0u
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#ifdef OS_CFG_TASK_PROFILE_EN > 0u
        if (OS_CFG_TS_EN > 0u)
            OSTCBCurPtr->SemPendTime = OS_TS_GET() - OSTCBCurPtr->TS;
            if (OSTCBCurPtr->SemPendTimeMax < OSTCBCurPtr->SemPendTime) {
                OSTCBCurPtr->SemPendTimeMax = OSTCBCurPtr->SemPendTime;
            }
#endif
#endif
#endif
}

```

```

#endif

    OS_TRACE_TASK_SEM_PEND(OSTCBCurPtr);
    *p_err = OS_ERR_NONE;
    break;

    /* 任务挂起等待信号量被中断 */
    case OS_STATUS_PEND_ABORT:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#endif

    OS_TRACE_TASK_SEM_PEND_FAILED(OSTCBCurPtr);
    *p_err = OS_ERR_PEND_ABORT;
    break;

    /* 任务挂起等待信号量超时 */
    case OS_STATUS_PEND_TIMEOUT:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = 0u;
        }
#endif

    OS_TRACE_TASK_SEM_PEND_FAILED(OSTCBCurPtr);
    *p_err = OS_ERR_TIMEOUT;
    break;

    /* 无效的任务挂起等待结果 */
    default:
        OS_TRACE_TASK_SEM_PEND_FAILED(OSTCBCurPtr);
        *p_err = OS_ERR_STATUS_INVALID;
        break;
}

/* 退出临界区，
 * 并返回信号量当前的资源数
 */
ctr = OSTCBCurPtr->SemCtr;
CPU_CRITICAL_EXIT();
OS_TRACE_TASK_SEM_PEND_EXIT(*p_err);
return (ctr);
}

```

2. 函数 OSTaskSemPendAbort()

该函数用于终止任务挂起等待任务内嵌信号量。

函数 OSTaskSemPendAbort() 的函数原型，如下所示：

```
CPU_BOOLEAN OSTaskSemPendAbort(OS_TCB* p_tcb,
                                OS_OPT opt,
                                OS_ERR* p_err)
```

函数 OSTaskSemPendAbort() 的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 14.2.5 函数 OSTaskSemPendAbort() 形参相关描述

函数 OSTaskSemPendAbort() 的返回值描述，如下表所示：

返回值	描述
CPU_BOOLEAN	终止任务挂起是否成功

表 14.2.6 函数 OSTaskSemPendAbort() 返回值相关描述

函数 OSTaskSemPendAbort() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功终止任务挂起等待任务内嵌信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT_ISR	在中断中非法调用该函数
OS_ERR_PEND_ABORT_NONE	任务没有挂起等待任务内嵌信号量
OS_ERR_PEND_ABORT_SELF	任务终止自己挂起等待任务内嵌信号量

表 14.2.7 函数 OSTaskSemPendAbort() 错误代码相关描述

函数 OSTaskSemPendAbort() 定义在文件 os_task.c 中，具体的代码如下所示：

```
CPU_BOOLEAN OSTaskSemPendAbort(OS_TCB* p_tcb,
                                OS_OPT opt,
                                OS_ERR* p_err)
{
    CPU_TS ts;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    if (OSIntNestingCtr > 0u) {
```

```
        *p_err = OS_ERR_PEND_ABORT_ISR;
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    switch (opt) {
        case OS_OPT_POST_NONE:
        case OS_OPT_POST_NO_SCHED:
            break;

        default:
            *p_err = OS_ERR_OPT_INVALID;
            return (OS_FALSE);
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();
    /* 无法终止任务自身挂起,
     * 因此返回错误
     */
    if ((p_tcb == (OS_TCB *)0) || (p_tcb == OSTCBCurPtr)) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_PEND_ABORT_SELF;
        return (OS_FALSE);
    }

    /* 任务需要是挂起等待任务内嵌信号量
     * 才能使用该 API 函数终止挂起
     */
    if (p_tcb->PendOn != OS_TASK_PEND_ON_TASK_SEM) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_PEND_ABORT_NONE;
        return (OS_FALSE);
    }
}
```

```

}

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 进入临界区 */
CPU_CRITICAL_ENTER();

#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif

/* 终止任务的挂起状态
 * 同时会设置任务的挂起结果为“任务挂起被中断”
 */
OS_PendAbort(    p_tcb,
                ts,
                OS_STATUS_PEND_ABORT);

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 根据函数操作选项进行任务调度 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

*p_err = OS_ERR_NONE;
return (OS_TRUE);
}

```

3. 函数 OSTaskSemPost()

该函数用于释放指定任务的内嵌信号量。

函数 OSTaskSemPost()的函数原型，如下所示：

```

OS_SEM_CTR OSTaskSemPost( OS_TCB* p_tcb,
                          OS_OPT opt,
                          OS_ERR* p_err)

```

函数 OSTaskSemPost()的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 14.2.8 函数 OSTaskSemPost()形参相关描述

函数 OSTaskSemPost() 的返回值描述，如下表所示：

返回值	描述
OS_SEM_CTR 类型返回值	任务内嵌信号量更新后的资源数

表 14.2.9 函数 OSTaskSemPost() 返回值相关描述

函数 OSTaskSemPost() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功释放指定任务的任务内嵌信号量
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_SEM_OVF	信号量资源数溢出
OS_ERR_STATE_INVALID	任务处于无效状态

表 14.2.10 函数 OSTaskSemPost() 错误代码相关描述

函数 OSTaskSemPost() 定义在文件 os_task.c 中，具体的代码如下所示：

```
OS_SEM_CTR OSTaskSemPost( OS_TCB* p_tcb,
                           OS_OPT  opt,
                           OS_ERR* p_err)
{
    OS_SEM_CTR  ctr;
    CPU_TS      ts;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

    /* 用于调试，不用理会 */
    OS_TRACE_TASK_SEM_POST_ENTER(p_tcb, opt);

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        if (OSRunning != OS_STATE_OS_RUNNING) {
            OS_TRACE_TASK_SEM_POST_EXIT(OS_ERR_OS_NOT_RUNNING);
            *p_err = OS_ERR_OS_NOT_RUNNING;
            return (0u);
        }
    #endif

    /* 此宏用于使能参数检查 */
    #if (OS_CFG_ARG_CHK_EN > 0u)
```



```

switch (opt) {
    case OS_OPT_POST_NONE:
    case OS_OPT_POST_NO_SCHED:
        break;

    default:
        OS_TRACE_TASK_SEM_POST_FAILED(p_tcb);
        OS_TRACE_TASK_SEM_POST_EXIT(OS_ERR_OPT_INVALID);
        *p_err = OS_ERR_OPT_INVALID;
        return (0u);
}
#endif

#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif

OS_TRACE_TASK_SEM_POST(p_tcb);

/* 进入临界区 */
CPU_CRITICAL_ENTER();
if (p_tcb == (OS_TCB *)0) {
    p_tcb = OSTCBCurPtr;
}
#if (OS_CFG_TS_EN > 0u)
    p_tcb->TS = ts;
#endif

*p_err = OS_ERR_NONE;

/* 根据任务的不同状态进行处理 */
switch (p_tcb->TaskState) {
    case OS_TASK_STATE_RDY:
    case OS_TASK_STATE_DLY:
    case OS_TASK_STATE_SUSPENDED:
    case OS_TASK_STATE_DLY_SUSPENDED:
        /* 判断信号量资源计数器是否溢出 */
        if (p_tcb->SemCtr == (OS_SEM_CTR)-1) {
            CPU_CRITICAL_EXIT();
            *p_err = OS_ERR_SEM_OVF;
            OS_TRACE_SEM_POST_EXIT(*p_err);
            return (0u);

```

```

    }

    /* 更新信号量资源计数器 */
    p_tcb->SemCtr++;
    ctr = p_tcb->SemCtr;
    CPU_CRITICAL_EXIT();
    break;

case OS_TASK_STATE_PEND:
case OS_TASK_STATE_PEND_TIMEOUT:
case OS_TASK_STATE_PEND_SUSPENDED:
case OS_TASK_STATE_PEND_TIMEOUT_SUSPENDED:
    /* 判断任务是否已经挂起等待任务内嵌信号量 */
    if (p_tcb->PendOn == OS_TASK_PEND_ON_TASK_SEM) {
        /* 释放任务的任务内嵌信号量 */
        OS_Post(    (OS_PEND_OBJ *)0,
                    p_tcb,
                    (void *)0,
                    0u,
                    ts);

        ctr = p_tcb->SemCtr;
        CPU_CRITICAL_EXIT();

        /* 根据函数操作选项进行任务调度 */
        if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
            OSSched();
        }
    } else {
        /* 判断信号量的资源计数器是否溢出 */
        if (p_tcb->SemCtr == (OS_SEM_CTR)-1) {
            CPU_CRITICAL_EXIT();
            *p_err = OS_ERR_SEM_OVF;
            OS_TRACE_SEM_POST_EXIT(*p_err);
            return (0u);
        }

        /* 更新信号量的资源计数器 */
        p_tcb->SemCtr++;
        ctr = p_tcb->SemCtr;
        CPU_CRITICAL_EXIT();
    }

    break;

/* 无效的任务状态 */
default:
    CPU_CRITICAL_EXIT();

```

```

        *p_err = OS_ERR_STATE_INVALID;
        ctr = 0u;
        break;
    }

    /* 用于调试，不用理会 */
    OS_TRACE_TASK_SEM_POST_EXIT(*p_err);

    /* 返回更新后的信号量资源数 */
    return (ctr);
}

```

4. 函数 OSTaskSemSet()

该函数用于强制设置任务内嵌信号量的资源数。

函数 OSTaskSemSet() 的函数原型，如下所示：

```

OS_SEM_CTR OSTaskSemSet( OS_TCB*    p_tcb,
                          OS_SEM_CTR cnt;
                          OS_ERR*    p_err)

```

函数 OSTaskSemSet() 的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
cnt	指定的信号量资源数
p_err	指向接收错误代码变量的指针

表 14.2.11 函数 OSTaskSemSet() 形参相关描述

函数 OSTaskSemSet() 的返回值描述，如下表所示：

返回值	描述
OS_SEM_CTR 类型返回值	任务内嵌信号量设置前的资源数

表 14.2.12 函数 OSTaskSemSet() 返回值相关描述

函数 OSTaskSemSet() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功设置任务内嵌信号量资源数
OS_ERR_SET_ISR	在中断中非法调用该函数
OS_ERR_TASK_WAITING	任务正在等待任务内嵌信号量，资源数设置失败

表 14.2.13 函数 OSTaskSemSet() 错误代码相关描述

函数 OSTaskSemSet() 定义在文件 os_task.c 中，具体的代码如下所示：

```

OS_SEM_CTR OSTaskSemSet( OS_TCB*    p_tcb,
                          OS_SEM_CTR cnt,
                          OS_ERR*    p_err)
{
    OS_SEM_CTR ctr;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL

```

```

if (p_err == (OS_ERR *)0) {
    OS_SAFETY_CRITICAL_EXCEPTION();
    return (0u);
}
#endif

/* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
/* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
 * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
 * 因为在中断中不能调用创建任务的函数，
 * 因此返回错误
 */
if (OSIntNestingCtr > 0u) {
    *p_err = OS_ERR_SET_ISR;
    return (0u);
}
#endif

/* 进入临界区 */
CPU_CRITICAL_ENTER();

if (p_tcb == (OS_TCB *)0) {
    p_tcb = OSTCBCurPtr;
}

/* 判断任务是否正等待任务内嵌信号量 */
if (( (p_tcb->TaskState & OS_TASK_STATE_PEND) != 0u) &&
    (p_tcb->PendOn == OS_TASK_PEND_ON_TASK_SEM)) {
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_TASK_WAITING;
    return (0u);
}

/* 获取任务内嵌信号量设置前的资源数 */
ctr = p_tcb->SemCtr;
/* 设置任务内嵌信号量的资源数 */
p_tcb->SemCtr = (OS_SEM_CTR)cnt;

/* 退出临界区 */
CPU_CRITICAL_EXIT();

*p_err = OS_ERR_NONE;
return (ctr);

```

}

14.3 μC/OS-III 任务内嵌信号量实验

14.3.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数使用任务内嵌信号量，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建其他任务
task1	用于扫描按键，并释放 task2 任务的任务内嵌信号量
task2	获取任务内嵌信号量

表 14.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 14 UCOS-III 任务内嵌信号量实验》。

14.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

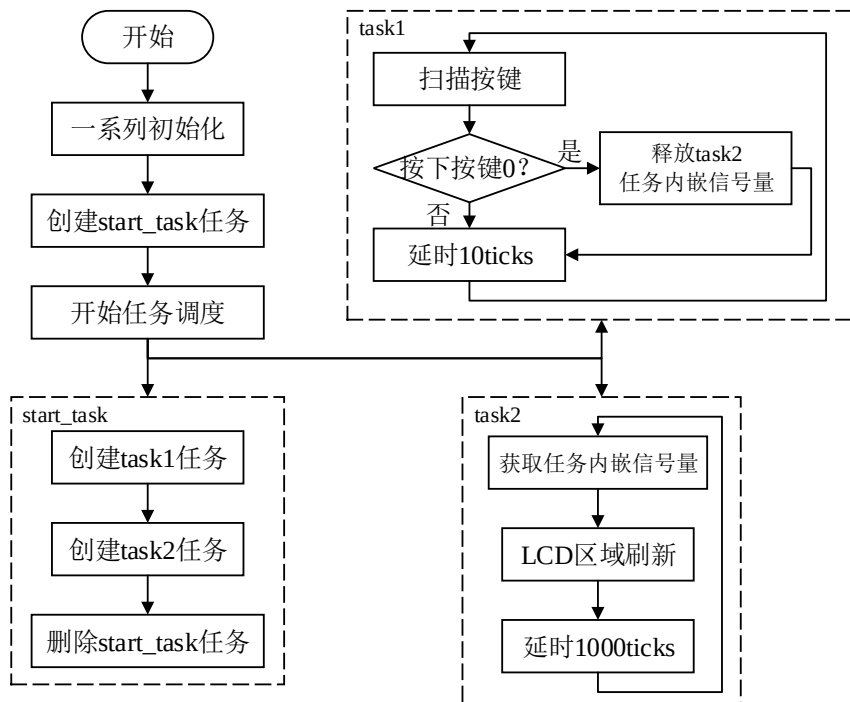


图 14.3.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSTaskSemPost()

此函数用于释放任务内嵌信号量，请参考第 14.2 小节《μC/OS-III 任务内嵌信号量相关 API 函数》。

(2) 函数 OSTaskSemPend()

此函数用于获取任务内嵌信号量，请参考第 14.2 小节《μC/OS-III 任务内嵌信号量相关 API 函数》。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief   start_task
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*)    &Task1Task_TCB,
                    (CPU_CHAR*)  "task1",
                    (OS_TASK_PTR) task1,
                    (void*)       0,
                    (OS_PRIO)     TASK1_PRIO,
                    (CPU_STK)     *)Task1Task_STK,
                    (CPU_STK_SIZE)TASK1_STK_SIZE / 10,
                    (CPU_STK_SIZE)TASK1_STK_SIZE,
                    (OS_MSG_QTY)  0,
                    (OS_TICK)     0,
                    (void*)       0,
                    (OS_OPT)      (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
```

```

(OS_ERR*      ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                         TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(   (OS_TCB*      ) &Task2Task_TCB,
                (CPU_CHAR*    ) "task2",
                (OS_TASK_PTR  ) task2,
                (void*        ) 0,
                (OS_PRIO      ) TASK2_PRIO,
                (CPU_STK*     ) Task2Task_STK,
                (CPU_STK_SIZE ) TASK2_STK_SIZE / 10,
                (CPU_STK_SIZE ) TASK2_STK_SIZE,
                (OS_MSG_QTY   ) 0,
                (OS_TICK      ) 0,
                (void*        ) 0,
                (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                (OS_ERR*      ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出, start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务和 task2 任务, 并且 task1 任务和 task2 任务的任务栈都是动态分配的, 当然, 也可以静态地定任务栈, 就像定义一个数组一样。

(2) task1 任务

```

/**
 * @brief   task1
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task1(void *p_arg)
{
    uint8_t key;
    OS_ERR err;

    while (1)
    {
        key = key_scan(0);

        switch (key)
        {
            case KEY0_PRES:

```

```

        {
            OSTaskSemPost( (OS_TCB*   ) &Task2Task_TCB,
                           (OS_OPT    ) OS_OPT_POST_NONE,
                           (OS_ERR*   ) &err);

            break;
        }
        default:
        {
            break;
        }
    }

    OSTimeDly(10, OS_OPT_TIME_DLY, &err);
}
}

```

从上面的代码中可以看出，task1 任务还是比较简单的，就是扫描按键，然后当检测到按键 0 被按下时，释放 task2 任务的任务内嵌信号量。

(3) task2 任务

```

/**
 * @brief  task2
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task2(void *p_arg)
{
    OS_ERR err;
    uint32_t task2_num = 0;

    while (1)
    {
        OSTaskSemPend( (OS_TICK    ) 0,
                      (OS_OPT     ) OS_OPT_PEND_BLOCKING,
                      (CPU_TS*    ) 0,
                      (OS_ERR*    ) &err);

        lcd_show_xnum(124, 111, ++task2_num, 3, 16, 0, BLUE);
        lcd_fill(6, 131, 233, 313, lcd_discolor[task2_num % 11]);
    }
}

```

从上面的代码中可以看出，task2 任务就是不断地获取任务内嵌信号量，当成功获取到任务内嵌信号量后，就刷新 LCD 的显示，同时会在 LCD 上显示 task2 任务的执行次数。

14.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 14.3.2 LCD 显示内容一

接着按下按键 0，使得 task1 任务释放 task2 任务的任务内嵌信号量，然后 task2 任务就能够获取到任务内嵌信号量，然后刷新 LCD 的区域显示，如下图所示：



图 14.3.3 LCD 显示内容二

如果多按几下按键 0，那么 task1 任务就会释放多次 task2 任务的任务内嵌信号量，那么也就能看到 LCD 刷新了多次，并且在 LCD 上也会显示 task2 任务的执行次数。

第十五章 μC/OS-III 任务内嵌消息队列

与任务内嵌信号量一样，消息队列也有任务内嵌的“版本”，叫作任务内嵌消息队列，μC/OS-III 为每一个任务都分配了一个任务内嵌消息队列，这意味着，任务内嵌消息队列是每一个任务独自拥有的，这样不仅简化了代码，并且还提高了消息队列的工作效率，本章就来学习 μC/OS-III 中的任务内嵌消息队列。

本章分为如下几部分：

15.1 μC/OS-III 任务内嵌消息队列简介

15.2 μC/OS-III 任务内嵌消息队列相关 API 函数

15.3 μC/OS-III 任务内嵌消息队列实验

15.1 μC/OS-III 任务内嵌消息队列简介

任务内嵌消息队列本质上就是一个消息队列，与第十二章《μC/OS-III 消息队列》中介绍的消息队列是类似的，但是任务内嵌消息队列并不需要消息队列这么一个中间的内核对象，任务内嵌消息队列是分配于每一个任务的任务控制块结构体中的，因此每一个任务都有独自的任务内嵌消息队列，任务内嵌消息队列只能被该任务接收，但是可以由其他任务或中断发送，如下图所示：

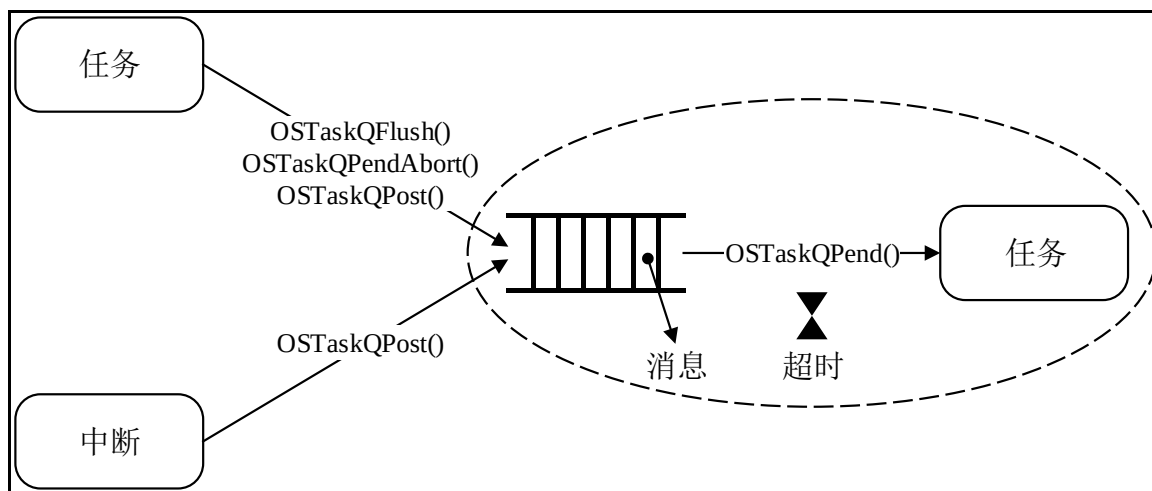


图 15.1.1 任务内嵌消息队列操作示意图

如上图所示，当任务或中断需要往指定任务的内嵌消息队列发送消息时，只需要调用相应的 API 函数即可，每个任务的内嵌消息队列是在任务创建的时候就已经被创建好的了，并且发出的消息能够直接到达指定任务的任务内嵌消息队列中因此使用任务内嵌消息队列的效率会比使用内核对象的消息队列高得多，在实际开发当中，可以优先考虑使用任务内嵌消息队列。

15.2 μC/OS-III 任务内嵌消息队列相关 API 函数

μC/OS-III 提供了一些列操作任务内嵌消息队列的 API 函数，这些操作函数如下表所示：

函数	描述
OSTaskQFlush()	清空任务内嵌消息队列中的所有消息
OSTaskQPend()	获取任务内嵌消息队列中的消息

OSTaskQPendAbort()	终止任务挂起等待任务内嵌消息队列
OSTaskQPost()	发送消息到任务内嵌消息队列

表 15.2.1 μC/OS-III 任务内嵌消息队列相关 API 函数描述

1. 函数 OSTaskQFlush()

该函数用于清空任务内嵌消息队列中的所有消息。

函数 OSTaskQFlush() 的函数原型，如下所示：

```
OS_MSG_QTY OSTaskQFlush( OS_TCB* p_tcb,
                          OS_ERR* p_err)
```

函数 OSTaskQFlush() 的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
p_err	指向接收错误代码变量的指针

表 15.2.2 函数 OSTaskQFlush() 形参相关描述

函数 OSTaskQFlush() 的返回值描述，如下表所示：

返回值	描述
OS_MSG_QTY 类型返回值	被释放的消息数量

表 15.2.3 函数 OSTaskQFlush() 返回值相关描述

函数 OSTaskQFlush() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	任务内嵌消息队列清空成功
OS_ERR_FLUSH_ISR	在中断中非法调用该函数
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行

表 15.2.4 函数 OSTaskQFlush() 错误代码相关描述

函数 OSTaskQFlush() 定义在文件 os_task.c 中，具体的代码如下所示：

```
OS_MSG_QTY OSTaskQFlush( OS_TCB* p_tcb,
                          OS_ERR* p_err)
{
    OS_MSG_QTY entries;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
         * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
         * 因为在中断中不能调用创建任务的函数，
         */
    #endif
}
```

```

    * 因此返回错误
    */
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_FLUSH_ISR;
        return (0u);
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }
#endif

    if (p_tcb == (OS_TCB *)0) {
        CPU_CRITICAL_ENTER();
        p_tcb = OSTCBCurPtr;
        CPU_CRITICAL_EXIT();
    }

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 清空任务内嵌消息队列中的所有消息 */
    entries = OS_MsgQFreeAll(&p_tcb->MsgQ);

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_NONE;

    /* 返回被释放的消息数量 */
    return (entries);
}

```

2. 函数 OSTaskQPend()

该函数用于获取任务内嵌消息队列中的消息。

函数 OSTaskQPend() 的函数原型，如下所示：

```

void *OSTaskQPend( OS_TICK      timeout,
                  OS_OPT        opt,
                  OS_MSG_SIZE*  p_msg_size,
                  CPU_TS*       p_ts,
                  OS_ERR*       p_err)

```

函数 OSTaskQPend() 的形参描述，如下表所示：

形参	描述
timeout	任务挂起等待任务内嵌消息队列的最大允许时间
opt	函数操作选项
p_msg_size	指向用于接收消息大小变量的指针
p_ts	指向接收消息队列接收时的时间戳的变量的指针
p_err	指向接收错误代码变量的指针

表 15.2.5 函数 OSTaskQPend() 形参相关描述

函数 OSTaskQPend() 的返回值描述，如下表所示：

返回值	描述
void * 类型返回值	指向消息的指针

表 15.2.6 函数 OSTaskQPend() 返回值相关描述

函数 OSTaskQPend() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	消息接收成功
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT	任务挂起等待任务内嵌消息队列时被终止（还未超时）
OS_ERR_PEND_ISR	在中断中非法调用该函数
OS_ERR_PEND_WOULD_BLOCK	获取消息队列失败，并且不挂起任务等待互斥信号量
OS_ERR_PTR_INVALID	参数 p_msg_size 指针为空
OS_ERR_SCHED_LOCKED	任务调度器已锁定
OS_ERR_TIMEOUT	任务挂起等待消息队列超时

表 15.2.7 函数 OSTaskQPend() 错误代码相关描述

函数 OSTaskQPend() 定义在文件 os_task.c 中，具体的代码如下所示：

```
void *OSTaskQPend( OS_TICK      timeout,
                  OS_OPT      opt,
                  OS_MSG_SIZE* p_msg_size,
                  CPU_TS*      p_ts,
                  OS_ERR*      p_err)
{
    OS_MSG_Q*    p_msg_q;
    void*         p_void;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return ((void *)0);
    }
#endif
}
```

```

OS_TRACE_TASK_MSG_Q_PEND_ENTER( &OSTCBCurPtr->MsgQ,
                                timeout,
                                opt,
                                p_msg_size,
                                p_ts);

/* 此宏用于使能系统时钟节拍 */
#if (OS_CFG_TICK_EN == 0u)
    if (timeout != 0u) {
        *p_err = OS_ERR_TICK_DISABLED;
        OS_TRACE_TASK_MSG_Q_PEND_EXIT(OS_ERR_TICK_DISABLED);
        return ((void *)0);
    }
#endif

/* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    if (OSIntNestingCtr > 0u) {
        OS_TRACE_TASK_MSG_Q_PEND_EXIT(OS_ERR_PEND_ISR);
        *p_err = OS_ERR_PEND_ISR;
        return ((void *)0);
    }
#endif

/* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_TASK_MSG_Q_PEND_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return ((void *)0);
    }
#endif

/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_msg_size == (OS_MSG_SIZE * 0)) {
        OS_TRACE_TASK_MSG_Q_PEND_EXIT(OS_ERR_PTR_INVALID);
        *p_err = OS_ERR_PTR_INVALID;
    }

```

```

        return ((void *)0);
    }
    switch (opt) {
        case OS_OPT_PEND_BLOCKING:
        case OS_OPT_PEND_NON_BLOCKING:
            break;

        default:
            OS_TRACE_TASK_MSG_Q_PEND_EXIT(OS_ERR_OPT_INVALID);
            *p_err = OS_ERR_OPT_INVALID;
            return ((void *)0);
    }
#endif

    if (p_ts != (CPU_TS *)0) {
        *p_ts = 0u;
    }

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /*从任务内嵌消息队列中获取消息 */
    p_msg_q = &OSTCBCurPtr->MsgQ;
    p_void = OS_MsgQGet( p_msg_q,
                        p_msg_size,
                        p_ts,
                        p_err);

    /* 判断是否获取成功 */
    if (*p_err == OS_ERR_NONE) {
#ifdef OS_CFG_TASK_PROFILE_EN > 0u
    #if OS_CFG_TS_EN > 0u
        if (p_ts != (CPU_TS *)0) {
            OSTCBCurPtr->MsgQPendTime = OS_TS_GET() - *p_ts;
            if (OSTCBCurPtr->MsgQPendTimeMax < OSTCBCurPtr->MsgQPendTime) {
                OSTCBCurPtr->MsgQPendTimeMax = OSTCBCurPtr->MsgQPendTime;
            }
        }
    #endif
#endif
}
#endif
#endif

    /* 获取成功则直接返回 */
    OS_TRACE_TASK_MSG_Q_PEND(p_msg_q);
    CPU_CRITICAL_EXIT();

```

```

    OS_TRACE_TASK_MSG_Q_PEND_EXIT(OS_ERR_NONE);
    return (p_void);
}

/* 获取消息失败,
 * 接下来根据函数操作选项判断是否需要挂起等待消息
 */
if ((opt & OS_OPT_PEND_NON_BLOCKING) != 0u) {
    /* 不挂起等待消息,
     * 则直接返回
     */
    *p_err = OS_ERR_PEND_WOULD_BLOCK;
    CPU_CRITICAL_EXIT();
    OS_TRACE_TASK_MSG_Q_PEND_FAILED(p_msg_q);
    OS_TRACE_TASK_MSG_Q_PEND_EXIT(OS_ERR_PEND_WOULD_BLOCK);
    return ((void *)0);
} else {
    /* 选择挂起等待消息还要判断任务调度器是否被上锁,
     * 若任务调度器被上锁,
     * 则直接返回
     */
    if (OSSchedLockNestingCtr > 0u) {
        CPU_CRITICAL_EXIT();
        OS_TRACE_TASK_MSG_Q_PEND_FAILED(p_msg_q);
        OS_TRACE_TASK_MSG_Q_PEND_EXIT(OS_ERR_SCHED_LOCKED);
        *p_err = OS_ERR_SCHED_LOCKED;
        return ((void *)0);
    }
}

/* 将任务添加到任务内嵌消息队列的挂起等待任务链表中 */
OS_Pend( (OS_PEND_OBJ *)0,
        OSTCBCurPtr,
        OS_TASK_PEND_ON_TASK_Q,
        timeout);

/* 退出临界区 */
CPU_CRITICAL_EXIT();
OS_TRACE_TASK_MSG_Q_PEND_BLOCK(p_msg_q);

/* 触发任务调度,
 * 触发任务调度后,
 * 任务将处于挂起状态
 */

```



```

    */

OSSched();

/* 只有当任务结束挂起状态后，
 * 才会执行以下代码
 */

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 根据不同的挂起结果进行处理 */
switch (OSTCBCurPtr->PendStatus) {
    /* 成功地等待到了消息 */
    case OS_STATUS_PEND_OK:
        p_void = OSTCBCurPtr->MsgPtr;
        *p_msg_size = OSTCBCurPtr->MsgSize;
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
#if (OS_CFG_TASK_PROFILE_EN > 0u)
            OSTCBCurPtr->MsgQPendTime = OS_TS_GET() - OSTCBCurPtr->TS;
            if (OSTCBCurPtr->MsgQPendTimeMax < OSTCBCurPtr->MsgQPendTime) {
                OSTCBCurPtr->MsgQPendTimeMax = OSTCBCurPtr->MsgQPendTime;
            }
#endif
#endif
        }
    #endif
}

OS_TRACE_TASK_MSG_Q_PEND(p_msg_q);
*p_err = OS_ERR_NONE;
break;

/* 任务挂起被终止 */
case OS_STATUS_PEND_ABORT:
    p_void = (void *)0;
    *p_msg_size = 0u;
    if (p_ts != (CPU_TS *)0) {
        *p_ts = 0u;
    }
    OS_TRACE_TASK_MSG_Q_PEND_FAILED(p_msg_q);
    *p_err = OS_ERR_PEND_ABORT;
    break;

/* 任务挂起等待消息队列超时

```

```

        * 或是无效的挂起结果
        */

case OS_STATUS_PEND_TIMEOUT:
default:
    p_void = (void *)0;
    *p_msg_size = 0u;
#if (OS_CFG_TS_EN > 0u)
    if (p_ts != (CPU_TS *)0) {
        *p_ts = OSTCBCurPtr->TS;
    }
#endif

    OS_TRACE_TASK_MSG_Q_PEND_FAILED(p_msg_q);
    *p_err = OS_ERR_TIMEOUT;
    break;

}

/* 退出临界区,
 * 并返回
 */
CPU_CRITICAL_EXIT();
OS_TRACE_TASK_MSG_Q_PEND_EXIT(*p_err);
return (p_void);
}

```

3. 函数 OSTaskQPendAbort()

该函数用于终止任务挂起等待任务内嵌消息队列。

函数 OSTaskQPendAbort()的函数原型，如下所示：

```

CPU_BOOLEAN OSTaskQPendAbort( OS_TCB* p_tcb,
                               OS_OPT opt,
                               OS_ERR* p_err)

```

函数 OSTaskQPendAbort()的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 15.2.8 函数 OSTaskQPendAbort()形参相关描述

函数 OSTaskQPendAbort()的返回值描述，如下表所示：

返回值	描述
CPU_BOOLEAN	终止任务挂起是否成功

表 15.2.9 函数 OSTaskQPendAbort()返回值相关描述

函数 OSTaskQPendAbort()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	终止任务挂起等待任务内嵌消息队列成功
OS_ERR_OPT_INVALID	无效的函数操作选项

OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT_ISR	在中断中非法调用该函数
OS_ERR_PEND_ABORT_NONE	没有任务挂起等待任务内嵌消息队列
OS_ERR_PEND_ABORT_SELF	任务终止自己挂起等待任务内嵌消息队列

表 15.2.10 函数 OSTaskQPendAbort() 错误代码相关描述

函数 OSTaskQPendAbort() 定义在文件 os_task.c 中，具体的代码如下所示：

```
CPU_BOOLEAN OSTaskQPendAbort ( OS_TCB* p_tcb,
                                OS_OPT opt,
                                OS_ERR* p_err)
{
    CPU_TS ts;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    /* 全局变量 OSIntNestingCtr 为中断嵌套计数器，
     * 当 OSIntNestingCtr 大于 0 时，说明当前处于中断中
     * 因为在中断中不能调用创建任务的函数，
     * 因此返回错误
     */
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_PEND_ABORT_ISR;
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能参数检查 */
```

```

#if (OS_CFG_ARG_CHK_EN > 0u)
    switch (opt) {
        case OS_OPT_POST_NONE:
        case OS_OPT_POST_NO_SCHED:
            break;

        default:
            *p_err = OS_ERR_OPT_INVALID;
            return (OS_FALSE);
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

#if (OS_CFG_ARG_CHK_EN > 0u)
    if ((p_tcb == (OS_TCB *)0) || (p_tcb == OSTCBCurPtr)) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_PEND_ABORT_SELF;
        return (OS_FALSE);
    }
#endif

    /* 判断任务是否正挂起等待任务内嵌消息队列 */
    if (p_tcb->PendOn != OS_TASK_PEND_ON_TASK_Q) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_PEND_ABORT_NONE;
        return (OS_FALSE);
    }

#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif

    /* 将任务从任务内嵌消息队列的挂起等待任务链表中移除 */
    OS_PendAbort(    p_tcb,
                    ts,
                    OS_STATUS_PEND_ABORT);

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();

```

```
/* 根据函数操作选项进行任务调度 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

*p_err = OS_ERR_NONE;
return (OS_TRUE);
}
```

4. 函数 OSTaskQPost()

该函数用于发送消息到任务内嵌消息队列。

函数 OSTaskQPost()的函数原型，如下所示：

```
void OSTaskQPost( OS_TCB*    p_tcb,
                  void*      p_void,
                  OS_MSG_SIZE msg_size,
                  OS_OPT      opt,
                  OS_ERR*     p_err)
```

函数 OSTaskQPost()的形参描述，如下表所示：

形参	描述
p_tcb	指向任务控制块的指针
p_void	指向消息的指针
msg_size	消息的大小，单位：字节
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 15.2.11 函数 OSTaskQPost()形参相关描述

函数 OSTaskQPost()的返回值描述，如下表所示：

返回值	描述
无	无

表 15.2.12 函数 OSTaskQPost()返回值相关描述

函数 OSTaskQPost()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功发送消息到任务内嵌消息队列
OS_ERR_MSG_POLL_EMPTY	任务内嵌消息队列满
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_Q_MAX	任务内嵌消息队列满
OS_ERR_STATE_INVALID	任务处于无效状态

表 15.2.13 函数 OSTaskQPost()错误代码相关描述

函数 OSTaskQPost()定义在文件 os_task.c 中，具体的代码如下所示：

```
void OSTaskQPost( OS_TCB*    p_tcb,
                  void*      p_void,
                  OS_MSG_SIZE msg_size,
                  OS_OPT      opt,
                  OS_ERR*     p_err)
```

```
{
    CPU_TS ts;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    OS_TRACE_TASK_MSG_Q_POST_ENTER(&p_tcb->MsgQ, p_void, msg_size, opt);

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        if (OSRunning != OS_STATE_OS_RUNNING) {
            OS_TRACE_TASK_MSG_Q_POST_EXIT(OS_ERR_OS_NOT_RUNNING);
            *p_err = OS_ERR_OS_NOT_RUNNING;
            return;
        }
    #endif

    /* 此宏用于使能参数检查 */
    if (OS_CFG_ARG_CHK_EN > 0u)
        switch (opt) {
            case OS_OPT_POST_FIFO:
            case OS_OPT_POST_LIFO:
            case OS_OPT_POST_FIFO | OS_OPT_POST_NO_SCHED:
            case OS_OPT_POST_LIFO | OS_OPT_POST_NO_SCHED:
                break;

            default:
                OS_TRACE_TASK_MSG_Q_POST_FAILED(&p_tcb->MsgQ);
                OS_TRACE_TASK_MSG_Q_POST_EXIT(OS_ERR_OPT_INVALID);
                *p_err = OS_ERR_OPT_INVALID;
                return;
        }
    #endif

    /* 此宏用于使能时间戳功能 */
    if (OS_CFG_TS_EN > 0u)
        ts = OS_TS_GET();
}
```

```
#else
    ts = 0u;
#endif

OS_TRACE_TASK_MSG_Q_POST(&p_tcb->MsgQ);

*p_err = OS_ERR_NONE;

/* 进入临界区 */
CPU_CRITICAL_ENTER();

if (p_tcb == (OS_TCB *)0) {
    p_tcb = OSTCBCurPtr;
}

/* 根据任务的不同状态进行处理 */
switch (p_tcb->TaskState) {
    case OS_TASK_STATE_RDY:
    case OS_TASK_STATE_DLY:
    case OS_TASK_STATE_SUSPENDED:
    case OS_TASK_STATE_DLY_SUSPENDED:
        /* 将消息存入任务内嵌消息队列 */
        OS_MsgQPut(&p_tcb->MsgQ,
                    p_void,
                    msg_size,
                    opt,
                    ts,
                    p_err);

        /* 退出临界区 */
        CPU_CRITICAL_EXIT();
        break;

    case OS_TASK_STATE_PEND:
    case OS_TASK_STATE_PEND_TIMEOUT:
    case OS_TASK_STATE_PEND_SUSPENDED:
    case OS_TASK_STATE_PEND_TIMEOUT_SUSPENDED:
        /* 判断任务是否已经挂起等待任务内嵌消息队列 */
        if (p_tcb->PendOn == OS_TASK_PEND_ON_TASK_Q) {
            /* 释放任务的任务内嵌消息队列 */
            OS_Post((OS_PEND_OBJ *)0,
                    p_tcb,
                    p_void,
                    msg_size,
```

```

        ts);

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();

    /* 根据函数操作选项进行任务调度 */
    if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
        OSSched();
    }
} else {
    /* 将消息存入任务内嵌消息队列 */
    OS_MsgQPut(&p_tcb->MsgQ,
               p_void,
               msg_size,
               opt,
               ts,
               p_err);

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
}
break;

/* 无效的任务状态 */
default:
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_STATE_INVALID;
    break;
}

OS_TRACE_TASK_MSG_Q_POST_EXIT(*p_err);
}

```

15.3 μC/OS-III 任务内嵌消息队列实验

15.3.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数使用任务内嵌消息队列，本实验设计了三个任务，这三个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建其他任务
task1	用于扫描按键，并将键值发送到 task2 任务的任务内嵌消息队列
task2	用于获取任务内嵌消息队列中的消息

表 15.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 15 UCOS-III 任务内嵌消息队列实验》。

15.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

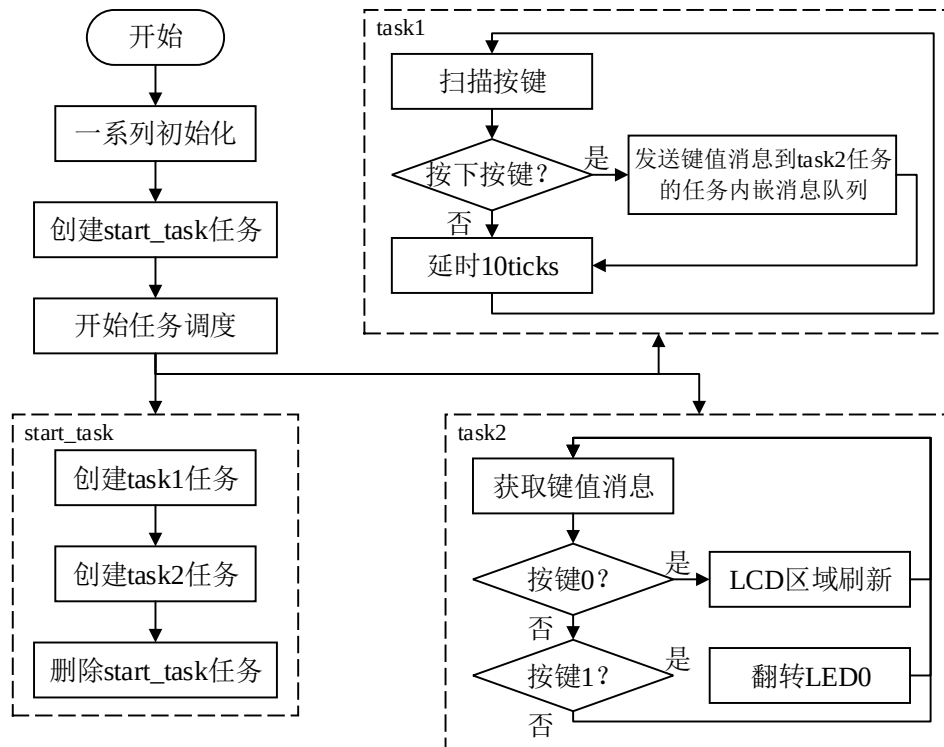


图 15.3.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSTaskQPost()

此函数用于发送消息到任务内嵌消息队列，请参考第 15.2 小节《μC/OS-III 任务内嵌消息队列相关 API 函数》。

(2) 函数 OSTaskQPend()

此函数用于接收任务内嵌消息队列中的消息，请参考第 15.2 小节《μC/OS-III 任务内嵌消息队列相关 API 函数》。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```

/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)

```

```
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                    (CPU_CHAR*    ) "task1",
                    (OS_TASK_PTR  ) task1,
                    (void*        ) 0,
                    (OS_PRIO      ) TASK1_PRIO,
                    (CPU_STK      *) Task1Task_STK,
                    (CPU_STK_SIZE) TASK1_STK_SIZE / 10,
                    (CPU_STK_SIZE) TASK1_STK_SIZE,
                    (OS_MSG_QTY   ) 0,
                    (OS_TICK      ) 0,
                    (void*        ) 0,
                    (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                    (OS_ERR*      ) &err);

    /* 创建 Task2 */
    Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK2_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*      ) &Task2Task_TCB,
                    (CPU_CHAR*    ) "task2",
                    (OS_TASK_PTR  ) task2,
                    (void*        ) 0,
                    (OS_PRIO      ) TASK2_PRIO,
                    (CPU_STK*     ) Task2Task_STK,
                    (CPU_STK_SIZE) TASK2_STK_SIZE / 10,
                    (CPU_STK_SIZE) TASK2_STK_SIZE,
                    (OS_MSG_QTY   ) 0,
```

```

        (OS_TICK          ) 0,
        (void*            ) 0,
        (OS_OPT           ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*          ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出, start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务和 task2 任务, 并且 task1 任务和 task2 任务的任务栈都是动态分配的, 当然, 也可以静态地定任务栈, 就像定义一个数组一样。

(2) task1 任务

```

/**
 * @brief   task1
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task1(void *p_arg)
{
    uint8_t key;
    OS_ERR err;

    while (1)
    {
        key = key_scan(0);

        if (key != 0)
        {
            OSTaskQPost(    (OS_TCB*          ) &Task2Task_TCB,
                            (void*            ) &key,
                            (OS_MSG_SIZE      ) sizeof(key),
                            (OS_OPT           ) OS_OPT_POST_FIFO,
                            (OS_ERR*          ) &err);
        }

        OSTimeDly(10, OS_OPT_TIME_DLY, &err);
    }
}

```

从上面的代码中可以看出, task1 任务还是比较简单的, 就是扫描按键, 然后将扫描到的键值作为消息发送到 task2 任务的任务内嵌消息队列中。

(3) task2 任务

```

/**
 * @brief   task2

```

```

* @param   p_arg : 传入参数(未用到)
* @retval  无
*/
void task2(void *p_arg)
{
    uint8_t*   key;
    OS_MSG_SIZE size;
    OS_ERR      err;
    uint32_t    task2_num = 0;

    while (1)
    {
        key = (uint8_t *)OSTaskQPend( (OS_TICK      )0,
                                       (OS_OPT       )OS_OPT_PEND_BLOCKING,
                                       (OS_MSG_SIZE*)  )&size,
                                       (CPU_TS*)      )0,
                                       (OS_ERR*)      )&err);

        switch (*key)
        {
            case KEY0_PRES:
            {
                lcd_fill(6, 131, 233, 313, lcd_discolor[++task2_num % 11]);
                break;
            }
            case KEY1_PRES:
            {
                LED0_TOGGLE();
                break;
            }
            default:
            {
                break;
            }
        }
    }
}

```

从上面的代码中可以看出，task2 任务就是不断地挂起获取任务内嵌消息队列中的消息，获取到消息后，将消息解析为键值，然后对不同的按键键值作不同的解释。

15.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 15.3.2 LCD 显示内容一

接着按下按键 0, 使得 task1 任务往 task2 任务的任务内嵌消息队列中发送键值为 0 的消息, 然后 task2 任务就能够获取到该消息, 然后刷新 LCD 的区域显示, 如下图所示:



图 15.3.3 LCD 显示内容二

如果多按几下按键 0, 那么 task2 任务就会多次接收到键值 0 的消息, 从而多次地去刷新 LCD。如果按键按键 1, 那么 task1 任务就会发送键值为 1 的消息, task2 任务就会收到该消息, 闭关翻转 LED0。

第十六章 μC/OS-III 事件标志

事件标志与信号量一样属于任务间同步的一种机制，但是信号量一般用于任务间的单事件同步，而对于任务间的多事件同步，仅仅使用信号量就显得力不从心了。为此 μC/OS-III 提供了事件标志这一机制，事件标志就能够很好地处理任务间的多任务同步。本章就来学习 μC/OS-III 中事件标志的相关内容。

本章分为如下几部分：

- 16.1 μC/OS-III 事件标志简介
- 16.2 μC/OS-III 事件标志相关 API 函数
- 16.3 μC/OS-III 事件标志实验

16.1 μC/OS-III 事件标志简介

16.1.1 事件标志

事件标志是一个用于指示事件是否发生的比特位，因为一个事件是否发生只有两种情况，分别为事件发生和事件未发生，因此只需一个比特位就能够表示事件是否发生，μC/OS-III 用 1 表示事件发生，用 0 表示事件未发生。

16.1.2 事件标志组

事件表示组是多个事件的集合，下面来看一下事件标志组的结构体，事件标志组的结构体定义在文件 os.h 中，具体的代码如下所示：

```
struct os_flag_grp{
    /* 此宏用于使能检查内核对象的类型 */
    #if (OS_OBJ_TYPE_REQ > 0u)
        /* 对象类型，
         * 在信号量初始化时，
         * 信号量的类型会被初始化为 OS_OBJ_TYPE_FLAG
         */
        OS_OBJ_TYPE Type;
    #endif

    /* 此宏用于使能代码调试功能 */
    #if (OS_CFG_DBG_EN > 0u)
        /* 如果使能了代码调试功能，
         * 那么就会为每一个信号量赋一个名称，
         * 方便调试
         */
        CPU_CHAR* NamePtr;
    #endif

    /* 挂起等待任务链表
```

```

    * 当任务获取的事件还未发生时，
    * 就可以选择被挂起到该链表，
    * 等待事件发生
    */
    OS_PEND_LIST PendList;

    /* 此宏用于使能代码调试功能 */
#ifdef (OS_CFG_DBG_EN > 0u)
    /* 一些与代码调试相关的成员变量 */
    OS_FLAG_GRP* DbgPrevPtr;
    OS_FLAG_GRP* DbgNextPtr;
    CPU_CHAR*    DbgNamePtr;
#endif

    /* 事件表示的集合
    * 对于 32 位 MCU，
    * 这一成员变量是一个 32 位的无符号数，
    * 每一比特位表示一个事件标志，
    * 因此一个事件标志组最多容纳 32 个事件标志
    */
    OS_FLAGS      Flags;

    /* 以下的成员变量用于第三方的调试工具 */
#ifdef (OS_CFG_TS_EN > 0u)
    CPU_TS        TS;
#endif
#ifdef (defined(OS_CFG_TRACE_EN) && (OS_CFG_TRACE_EN > 0u))
    CPU_INT16U    FlagID;
#endif
};

```

从上面的代码中可以看到，事件标志组的结构体中包含了一个数据类型为 OS_FLAGS 的成员变量 Flags，对于 32 位的 MCU 而言，数据类型 OS_FLAGS 实际上就是 32 位的无符号整型，因此成员变量 Flags 就包含了 32 比特，而 Flags 的每一个比特就能够用来存储一个事件标志。当成员变量 Flags 的某一比特位被置一时，就说明这一比特位对应的事件发生了。

16.1.3 事件标志操作简介

事件标志组由多个事件标志组成，因此任务就能够很方便地通过多个事件的发生进行任务间的同步。通过事件标志，任务还能够通过多个事件之间的逻辑关系（与、或关系）进行任务间的同步，如下图所示：

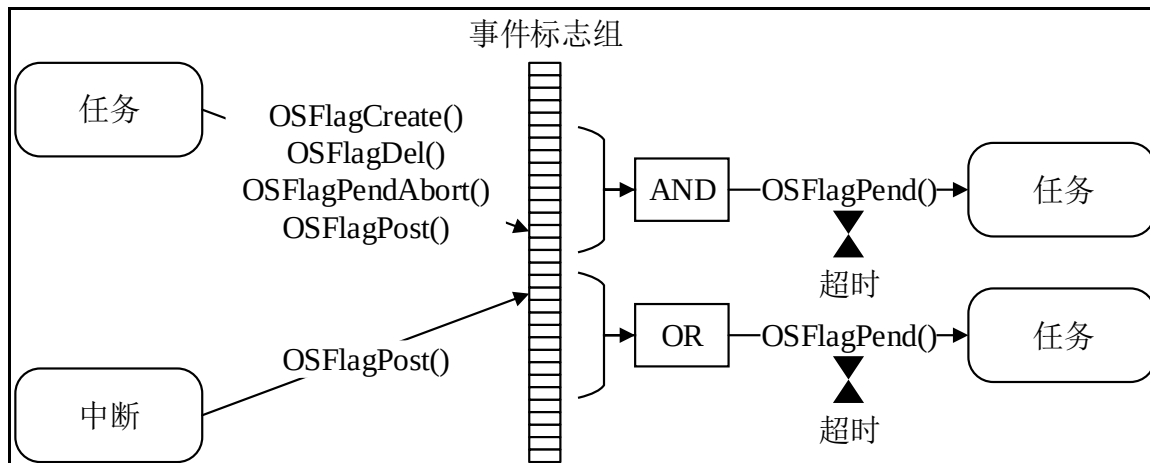


图 16.1.1 事件标志组操作示意图

16.2 μC/OS-III 事件标志相关 API 函数

μC/OS-III 提供了一些列操作事件标志的 API 函数，这些操作函数如下表所示：

函数	描述
OSFlagCreate()	创建一个事件标志组
OSFlagDel()	删除一个事件标志组
OSFlagPend()	等待事件标志组中的事件
OSFlagPendAbort()	终止挂起等待事件标志组中的事件
OSFlagPendGetFlagRdy()	获取任务等待到的事件
OSFlagPost()	设置事件标志组中的事件

表 16.2.1 μC/OS-III 事件标志相关 API 函数描述

1. 函数 OSFlagCreate()

该函数用于创建一个事件标志组。

函数 OSFlagCreate() 的函数原型，如下所示：

```
void OSFlagCreate( OS_FLAG_GRP*   p_grp,
                  CPU_CHAR*       p_name,
                  OS_FLAGS         flags,
                  OS_ERR*         p_err)
```

函数 OSFlagCreate() 的形参描述，如下表所示：

形参	描述
p_grp	指向事件标志组结构体的指针
p_name	指向作为事件标志组名的 ASCII 字符串的指针
flags	事件标志组的初始值
p_err	指向接收错误代码变量的指针

表 16.2.2 函数 OSFlagCreate() 形参相关描述

函数 OSFlagCreate() 的返回值描述，如下表所示：

返回值	描述
无	无

表 16.2.3 函数 OSFlagCreate() 返回值相关描述

函数 OSFlagCreate() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	事件标志组创建成功
OS_ERR_CREATE_ISR	在中断中非法调用该函数
OS_ERR_ILLEGAL_CREATE_RUN_TIME	在系统运行过程中非法创建内核对象
OS_ERR_OBJ_PTR_NULL	指向事件标志组结构体的指针为空

表 16.2.4 函数 OSFlagCreate()错误代码相关描述

函数 OSFlagCreate()定义在文件 os_flag.c 中，具体的代码如下所示：

```
void OSFlagCreate( OS_FLAG_GRP*   p_grp,
                  CPU_CHAR*       p_name,
                  OS_FLAGS        flags,
                  OS_ERR*         p_err)
{
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        *p_err = OS_ERR_ILLEGAL_CREATE_RUN_TIME;
        return;
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    {
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_CREATE_ISR;
            return;
        }
    }

    /* 此宏用于使能参数检查 */
    if (OS_CFG_ARG_CHK_EN > 0u)
    {
        if (p_grp == (OS_FLAG_GRP *)0) {
            *p_err = OS_ERR_OBJ_PTR_NULL;
            return;
        }
    }
}
```

```

    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 此宏用于使能检查内核对象的类型 */
    #if (OS_OBJ_TYPE_REQ > 0u)
    #if (OS_CFG_OBJ_CREATED_CHK_EN > 0u)
    /* 通过对象类型是否已经被设置
     * 判断事件标志组是否已经被创建
     */
    if (p_grp->Type == OS_OBJ_TYPE_FLAG) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_OBJ_CREATED;
        return;
    }
    #endif
    /* 对象类型设置位 OS_OBJ_TYPE_FLAG */
    p_grp->Type = OS_OBJ_TYPE_FLAG;
#endif

    /* 此宏用于使能代码调试功能 */
    #if (OS_CFG_DBG_EN > 0u)
    /* 如果使能了代码调试功能,
     * 则设置事件标志组的名称
     */
    p_grp->NamePtr = p_name;
    #else
    (void)p_name;
    #endif

    /* 事件标志赋初值 */
    p_grp->Flags = flags;

    /* 此宏用于使能时间戳功能 */
    #if (OS_CFG_TS_EN > 0u)
    p_grp->TS = 0u;
    #endif

    /* 初始化挂起等待任务链表 */
    OS_PendListInit(&p_grp->PendList);

```

```

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)

/* 初始化一些用于代码调试的成员变量 */
OS_FlagDbgListAdd(p_grp);
OSFlagQty++;

#endif

/* 用于调试，不用理会 */
OS_TRACE_FLAG_CREATE(p_grp, p_name);

/* 退出临界区 */
CPU_CRITICAL_EXIT();
*p_err = OS_ERR_NONE;
}

```

2. 函数 OSFlagDel()

该函数用于删除一个事件标志组。

函数 OSFlagDel()的函数原型，如下所示：

```

OS_OBJ_QTY OSFlagDel( OS_FLAG_GRP* p_grp,
                      OS_OPT      opt,
                      OS_ERR*      p_err);

```

函数 OSFlagDel()的形参描述，如下表所示：

形参	描述
p_grp	指向事件标志组结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 16.2.5 函数 OSFlagDel()形参相关描述

函数 OSFlagDel()的返回值描述，如下表所示：

返回值	描述
OS_OBJ_QTY 类型返回值	被终止挂起等待事件标志任务的数量

表 16.2.6 函数 OSFlagDel()返回值相关描述

函数 OSFlagDel()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	事件标志组删除成功
OS_ERR_DEL_ISR	在中断中非法调用该函数
OS_ERR_ILLEGAL_DEL_RUN_TIME	在系统运行过程中非法删除内核对象
OS_ERR_OBJ_PTR_NULL	指向事件标志组结构体的指针为空
OS_ERR_OBJ_TYPE	待被删除内核对象的类型不是事件标志组
OS_ERR_OPT_INVALID	无效的任务操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_TASK_WAITING	有任务挂起等待待被删除的事件标志组

表 16.2.7 函数 OSFlagDel()错误代码相关描述

函数 OSFlagDel()定义在文件 os_flag.c 中，具体的代码如下所示：

```

OS_OBJ_QTY OSFlagDel( OS_FLAG_GRP* p_grp,

```

```

        OS_OPT        opt,
        OS_ERR*       p_err)
{
    OS_OBJ_QTY        nbr_tasks;
    OS_PEND_LIST*     p_pend_list;
    OS_TCB*           p_tcb;
    CPU_TS             ts;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

    OS_TRACE_FLAG_DEL_ENTER(p_grp, opt);

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        OS_TRACE_FLAG_DEL_EXIT(OS_ERR_ILLEGAL_DEL_RUN_TIME);
        *p_err = OS_ERR_ILLEGAL_DEL_RUN_TIME;
        return (0u);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_DEL_ISR;
        OS_TRACE_FLAG_DEL_EXIT(OS_ERR_DEL_ISR);
        return (0u);
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_FLAG_DEL_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }

```

```

    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_grp == (OS_FLAG_GRP *)0) {
        OS_TRACE_FLAG_DEL_EXIT(OS_ERR_OBJ_PTR_NULL);
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_grp->Type != OS_OBJ_TYPE_FLAG) {
        OS_TRACE_FLAG_DEL_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 获取事件标志组的挂起等待任务链表 */
    p_pend_list = &p_grp->PendList;

    /* 用于记录被终止挂起任务的数量 */
    nbr_tasks = 0u;

    /* 根据不同的函数操作选项，
     * 作不同的处理
     */
    switch (opt) {
        /* 仅在没有挂起等待任务时，
         * 才删除事件标志组
         */
        case OS_OPT_DEL_NO_PEND:
            /* 判断事件标志组的挂起等待任务链表是否为空 */
            if (p_pend_list->HeadPtr == (OS_TCB *)0) {
                /* 此宏用于代码调试功能 */
                #if (OS_CFG_DBG_EN > 0u)
                    OS_FlagDbgListRemove(p_grp);
                #endif
            }
        }
    }

```

```

        OSFlagQty--;
#endif

        /* 用于调试，不用理会 */
        OS_TRACE_FLAG_DEL(p_grp);
        /* 去初始化事件标志组 */
        OS_FlagClr(p_grp);
        /* 退出临界区 */
        CPU_CRITICAL_EXIT();

        *p_err = OS_ERR_NONE;
    } else {
        /* 有任务正挂起等待该事件标志组
        * 因此退出临界区，
        * 不删除事件标志组
        */
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_TASK_WAITING;
    }
    break;

    /* 始终删除事件标志组 */
    case OS_OPT_DEL_ALWAYS:
#if (OS_CFG_TS_EN > 0u)
        ts = OS_TS_GET();
#else
        ts = 0u;
#endif

        /* 终止挂起所有挂起等待该事件标志组的任务 */
        while (p_pend_list->HeadPtr != (OS_TCB *)0) {
            p_tcb = p_pend_list->HeadPtr;
            OS_PendAbort(    p_tcb,
                            ts,
                            OS_STATUS_PEND_DEL);

            nbr_tasks++;
        }

        /* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
        OS_FlagDbgListRemove(p_grp);
        OSFlagQty--;
#endif

        /* 用于调试，不用理会 */
        OS_TRACE_FLAG_DEL(p_grp);
        /* 去初始化消息队列 */

```

```

        OS_FlagClr(p_grp);

        /* 退出临界区 */
        CPU_CRITICAL_EXIT();

        /* 触发任务调度 */
        OSSched();

        *p_err = OS_ERR_NONE;
        break;

        /* 无效的任务操作选项 */
        default:
            CPU_CRITICAL_EXIT();
            *p_err = OS_ERR_OPT_INVALID;
            break;
    }

    OS_TRACE_FLAG_DEL_EXIT(*p_err);

    /* 返回被终止挂起任务的数量 */
    return (nbr_tasks);
}

```

3. 函数 OSFlagPend()

该函数用于等待事件标志组中的事件。

函数 OSFlagPend()的函数原型，如下所示：

```

OS_FLAGS OSFlagPend(    OS_FLAG_GRP*   p_grp,
                        OS_FLAGS        flags,
                        OS_TICK         timeout,
                        OS_OPT          opt,
                        CPU_TS*         p_ts,
                        OS_ERR*         p_err)

```

函数 OSFlagPend()的形参描述，如下表所示：

形参	描述
p_grp	指向事件标志组结构体的指针
flags	等待的事件标志
timeout	任务挂起等待事件标志的最大允许时间
opt	函数操作选项
p_ts	指向接收等待到事件时的时间戳的变量的指针
p_err	指向接收错误代码变量的指针

表 16.2.8 函数 OSFlagPend()形参相关描述

函数 OSFlagPend()的返回值描述，如下表所示：

返回值	描述
OS_FLAGS 类型返回值	任务实际等待到的事件标志

表 16.2.9 函数 OSFlagPend()返回值相关描述

函数 OSFlagPend()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	没有错误
OS_ERR_OBJ_DEL	指定的事件标志组已经被删除
OS_ERR_OBJ_PTR_NULL	指向事件标志组结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是事件标志组
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT	任务挂起等待事件标志组时被终止（还未超时）
OS_ERR_PEND_ISR	在中断中非法调用该函数
OS_ERR_PEND_WOULD_BLOCK	等待事件失败，并且不挂起任务等待事件
OS_ERR_SCHED_LOCKED	任务调度器被锁定
OS_ERR_STATUS_INVALID	无效的任务挂起结果
OS_ERR_TIMEOUT	任务挂起等待事件标志组超时

表 16.2.10 函数 OSFlagPend()错误代码相关描述

函数 OSFlagPend()定义在文件 os_flag.c 中，具体的代码如下所示：

```
OS_FLAGS OSFlagPend( OS_FLAG_GRP* p_grp,
                    OS_FLAGS      flags,
                    OS_TICK       timeout,
                    OS_OPT        opt,
                    CPU_TS*       p_ts,
                    OS_ERR*       p_err)
{
    CPU_BOOLEAN consume;
    OS_FLAGS      flags_rdy;
    OS_OPT        mode;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

    OS_TRACE_FLAG_PEND_ENTER(p_grp, flags, timeout, opt, p_ts);

    /* 此宏用于使能系统时钟节拍 */
    if (OS_CFG_TICK_EN == 0u)
        if (timeout != 0u) {
            *p_err = OS_ERR_TICK_DISABLED;
            OS_TRACE_FLAG_PEND_FAILED(p_grp);
            OS_TRACE_FLAG_PEND_EXIT(OS_ERR_TICK_DISABLED);
        }
}
```



```

        return ((OS_FLAGS)0);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    if (OSIntNestingCtr > 0u) {
        if ((opt & OS_OPT_PEND_NON_BLOCKING) != OS_OPT_PEND_NON_BLOCKING) {
            *p_err = OS_ERR_PEND_ISR;
            OS_TRACE_FLAG_PEND_FAILED(p_grp);
            OS_TRACE_FLAG_PEND_EXIT(OS_ERR_PEND_ISR);
            return ((OS_FLAGS)0);
        }
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_FLAG_PEND_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_grp == (OS_FLAG_GRP *)0) {
        OS_TRACE_FLAG_PEND_FAILED(p_grp);
        OS_TRACE_FLAG_PEND_EXIT(OS_ERR_OBJ_PTR_NULL);
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }

    switch (opt) {
        case OS_OPT_PEND_FLAG_CLR_ALL:
        case OS_OPT_PEND_FLAG_CLR_ANY:
        case OS_OPT_PEND_FLAG_SET_ALL:
        case OS_OPT_PEND_FLAG_SET_ANY:
        case OS_OPT_PEND_FLAG_CLR_ALL | OS_OPT_PEND_FLAG_CONSUME:
        case OS_OPT_PEND_FLAG_CLR_ANY | OS_OPT_PEND_FLAG_CONSUME:
        case OS_OPT_PEND_FLAG_SET_ALL | OS_OPT_PEND_FLAG_CONSUME:
        case OS_OPT_PEND_FLAG_SET_ANY | OS_OPT_PEND_FLAG_CONSUME:
        case OS_OPT_PEND_FLAG_CLR_ALL | OS_OPT_PEND_NON_BLOCKING:

```

```

        case OS_OPT_PEND_FLAG_CLR_ANY | OS_OPT_PEND_NON_BLOCKING:
        case OS_OPT_PEND_FLAG_SET_ALL | OS_OPT_PEND_NON_BLOCKING:
        case OS_OPT_PEND_FLAG_SET_ANY | OS_OPT_PEND_NON_BLOCKING:
        case OS_OPT_PEND_FLAG_CLR_ALL | \
            (OS_OPT) (OS_OPT_PEND_FLAG_CONSUME | \
                OS_OPT_PEND_NON_BLOCKING):
        case OS_OPT_PEND_FLAG_CLR_ANY | \
            (OS_OPT) (OS_OPT_PEND_FLAG_CONSUME | \
                OS_OPT_PEND_NON_BLOCKING):
        case OS_OPT_PEND_FLAG_SET_ALL | \
            (OS_OPT) (OS_OPT_PEND_FLAG_CONSUME | \
                OS_OPT_PEND_NON_BLOCKING):
        case OS_OPT_PEND_FLAG_SET_ANY | \
            (OS_OPT) (OS_OPT_PEND_FLAG_CONSUME | \
                OS_OPT_PEND_NON_BLOCKING):
            break;

    default:
        OS_TRACE_FLAG_PEND_FAILED(p_grp);
        OS_TRACE_FLAG_PEND_EXIT(OS_ERR_OPT_INVALID);
        *p_err = OS_ERR_OPT_INVALID;
        return (0u);
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_grp->Type != OS_OBJ_TYPE_FLAG) {
        OS_TRACE_FLAG_PEND_FAILED(p_grp);
        OS_TRACE_FLAG_PEND_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

    /* 根据函数操作选项判断是否需要取反被获取的事件标志 */
    if ((opt & OS_OPT_PEND_FLAG_CONSUME) != 0u) {
        consume = OS_TRUE;
    } else {
        consume = OS_FALSE;
    }

    if (p_ts != (CPU_TS *)0) {

```

```

        *p_ts = 0u;
    }

    /* 获取函数操作选项中等待事件标志的操作
    * OS_OPT_PEND_FLAG_CLR_ALL : 0x01 // 等待入参 flags 中所有指定的事件被清 0
    * OS_OPT_PEND_FLAG_CLR_AND : 0x01 // 等待入参 flags 中所有指定的事件被清 0
    * OS_OPT_PEND_FLAG_CLR_ANY : 0x02 // 等待入参 flags 中任意指定的事件被清 0
    * OS_OPT_PEND_FLAG_CLR_OR : 0x02 // 等待入参 flags 中任意指定的事件被清 0
    * OS_OPT_PEND_FLAG_SET_ALL : 0x04 // 等待入参 flags 中所有指定的事件被置 1
    * OS_OPT_PEND_FLAG_SET_AND : 0x04 // 等待入参 flags 中所有指定的事件被置 1
    * OS_OPT_PEND_FLAG_SET_ANY : 0x08 // 等待入参 flags 中任意指定的事件被置 1
    * OS_OPT_PEND_FLAG_SET_OR : 0x08 // 等待入参 flags 中任意指定的事件被置 1
    */
    mode = opt & OS_OPT_PEND_FLAG_MASK;

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 根据函数操作选项进行处理 */
    switch (mode) {
        /* 等待入参 flags 中所有指定的事件被置 1 */
        case OS_OPT_PEND_FLAG_SET_ALL:
            /* 获取事件标志组中入参 flags 指定的事件 */
            flags_rdy = (p_grp->Flags & flags);
            /* 判断等待的事件是否都被置 1 */
            if (flags_rdy == flags) {
                /* 判断是否需要取反获取到的事件 */
                if (consume == OS_TRUE) {
                    /* 清 0 获取到的事件 */
                    p_grp->Flags &= ~flags_rdy;
                }
                /* 将获取到的事件存入任务控制块中的特定成员变量中 */
                OSTCBCurPtr->FlagsRdy = flags_rdy;
            }
            #if (OS_CFG_TS_EN > 0u)
                if (p_ts != (CPU_TS *)0) {
                    *p_ts = p_grp->TS;
                }
            #endif

            /* 退出临界区，
            * 并返回获取到的事件标志
            */
            CPU_CRITICAL_EXIT();
            OS_TRACE_FLAG_PEND(p_grp);

```

```

        OS_TRACE_FLAG_PEND_EXIT(OS_ERR_NONE);
        *p_err = OS_ERR_NONE;
        return (flags_rdy);
    } else {
        /* 这个 else 的情况是等待的事件并没有全部被置 1 */

        /* 判断是否选择挂起等待事件标志 */
        if ((opt & OS_OPT_PEND_NON_BLOCKING) != 0u) {
            /* 不选择挂起等待事件标志,
             * 则退出临界区并返回相应的错误
             */
            CPU_CRITICAL_EXIT();
            OS_TRACE_FLAG_PEND_FAILED(p_grp);
            OS_TRACE_FLAG_PEND_EXIT(OS_ERR_PEND_WOULD_BLOCK);
            *p_err = OS_ERR_PEND_WOULD_BLOCK;
            return ((OS_FLAGS)0);
        } else {
            /* 如果任务选择挂起等待事件标志,
             * 则接下来将要把任务添加到事件标志组的挂起等待任务链表中挂起,
             * 并且需要先判断任务调度器是否被上锁,
             * 如果任务调度器被上锁,
             * 则直接返回相应的错误代码
             */
            if (OSSchedLockNestingCtr > 0u) {
                CPU_CRITICAL_EXIT();
                OS_TRACE_FLAG_PEND_FAILED(p_grp);
                OS_TRACE_FLAG_PEND_EXIT(OS_ERR_SCHED_LOCKED);
                *p_err = OS_ERR_SCHED_LOCKED;
                return (0u);
            }
        }

        /* 将任务添加到事件标志组的挂起等待任务链表中 */
        OS_FlagBlock( p_grp,
                     flags,
                     opt,
                     timeout);

        /* 退出临界区 */
        CPU_CRITICAL_EXIT();
    }
    break;

```

```

/* 等待入参 flags 中任意指定的事件被置 1 */
case OS_OPT_PEND_FLAG_SET_ANY:
    /* 获取事件标志组中入参 flags 指定的事件 */
    flags_rdy = (p_grp->Flags & flags);
    /* 通过判断获取到的事件标志是否不为 0,
    * 判断是否由任意的事件标志被置 1
    */
    if (flags_rdy != 0u) {
        /* 判断是否需要取反获取到的事件 */
        if (consume == OS_TRUE) {
            /* 清 0 获取到的事件 */
            p_grp->Flags &= ~flags_rdy;
        }
        /* 将获取到的事件存入任务控制块中的特定成员变量中 */
        OSTCBCurPtr->FlagsRdy = flags_rdy;
#ifdef OS_CFG_TS_EN > 0u
        if (p_ts != (CPU_TS *)0) {
            *p_ts = p_grp->TS;
        }
#endif

        /* 退出临界区,
        * 并返回获取到的事件标志
        */
        CPU_CRITICAL_EXIT();
        OS_TRACE_FLAG_PEND(p_grp);
        OS_TRACE_FLAG_PEND_EXIT(OS_ERR_NONE);
        *p_err = OS_ERR_NONE;
        return (flags_rdy);
    } else {
        /* 这个 else 的情况是等待的事件全部没有被置 1 */

        /* 判断是否选择挂起等待事件标志 */
        if ((opt & OS_OPT_PEND_NON_BLOCKING) != 0u) {
            /* 不选择挂起等待事件标志,
            * 则退出临界区并返回相应的错误
            */
            CPU_CRITICAL_EXIT();
            OS_TRACE_FLAG_PEND_EXIT(OS_ERR_PEND_WOULD_BLOCK);
            *p_err = OS_ERR_PEND_WOULD_BLOCK;
            return ((OS_FLAGS)0);
        } else {
            /* 如果任务选择挂起等待事件标志,
            * 则接下来将要任务添加到事件标志组的挂起等待任务链表中挂起,

```

```

        * 并且需要先判断任务调度器是否被上锁,
        * 如果任务调度器被上锁,
        * 则直接返回相应的错误代码
        */
        if (OSSchedLockNestingCtr > 0u) {
            CPU_CRITICAL_EXIT();
            OS_TRACE_FLAG_PEND_EXIT(OS_ERR_SCHED_LOCKED);
            *p_err = OS_ERR_SCHED_LOCKED;
            return ((OS_FLAGS)0);
        }
    }

    /* 将任务添加到事件标志组的挂起等待任务链表中 */
    OS_FlagBlock( p_grp,
                 flags,
                 opt,
                 timeout);

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
}

break;

/* 此宏用于使能等待事件标志清 0 功能 */
#if (OS_CFG_FLAG_MODE_CLR_EN > 0u)
    /* 等待入参 flags 中所有指定的事件被清 0 */
    case OS_OPT_PEND_FLAG_CLR_ALL:
        /* 获取事件标志组中入参 flags 指定的事件 */
        flags_rdy = (OS_FLAGS)(~p_grp->Flags & flags);
        /* 判断等待的事件标志是否都被清 0 */
        if (flags_rdy == flags) {
            /* 判断是否需要取反获取到的事件 */
            if (consume == OS_TRUE) {
                /* 清 0 获取到的事件 */
                p_grp->Flags |= flags_rdy;
            }

            /* 将获取到的事件存入任务控制块中的特定成员变量中 */
            OSTCBCurPtr->FlagsRdy = flags_rdy;
        }
#endif
    if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = p_grp->TS;
        }
#endif
#endif

```

```

        /* 退出临界区,
        * 并返回获取到的事件标志
        */

        CPU_CRITICAL_EXIT();
        OS_TRACE_FLAG_PEND(p_grp);
        OS_TRACE_FLAG_PEND_EXIT(OS_ERR_NONE);
        *p_err = OS_ERR_NONE;
        return (flags_rdy);
    } else {

        /* 这个 else 的情况是等待的事件没有全部被清 0 */

        /* 判断是否选择挂起等待事件标志 */
        if ((opt & OS_OPT_PEND_NON_BLOCKING) != 0u) {
            /* 不选择挂起等待事件标志,
            * 则退出临界区并返回相应的错误
            */

            CPU_CRITICAL_EXIT();
            OS_TRACE_FLAG_PEND_EXIT(OS_ERR_PEND_WOULD_BLOCK);
            *p_err = OS_ERR_PEND_WOULD_BLOCK;
            return ((OS_FLAGS)0);
        } else {
            /* 如果任务选择挂起等待事件标志,
            * 则接下来将要把任务添加到事件标志组的挂起等待任务链表中挂起,
            * 并且需要先判断任务调度器是否被上锁,
            * 如果任务调度器被上锁,
            * 则直接返回相应的错误代码
            */

            if (OSSchedLockNestingCtr > 0u) {
                CPU_CRITICAL_EXIT();
                OS_TRACE_FLAG_PEND_EXIT(OS_ERR_SCHED_LOCKED);
                *p_err = OS_ERR_SCHED_LOCKED;
                return (0);
            }

            /* 将任务添加到事件标志组的挂起等待任务链表中 */
            OS_FlagBlock( p_grp,
                        flags,
                        opt,
                        timeout);

            /* 退出临界区 */
            CPU_CRITICAL_EXIT();
        }
    }
}

```

```

        break;

/* 等待入参 flags 中任意指定的事件被清 0 */
case OS_OPT_PEND_FLAG_CLR_ANY:
    /* 获取事件标志组中入参 flags 指定的事件 */
    flags_rdy = (~p_grp->Flags & flags);
    /* 判断是否有任意等待的事件标志被清 0 */
    if (flags_rdy != 0u) {
        /* 判断是否需要取反获取到的事件 */
        if (consume == OS_TRUE) {
            /* 清 0 获取到的事件 */
            p_grp->Flags |= flags_rdy;
        }
        /* 将获取到的事件存入任务控制块中的特定成员变量中 */
        OSTCBCurPtr->FlagsRdy = flags_rdy;
#ifdef OS_CFG_TS_EN > 0u
        if (p_ts != (CPU_TS *)0) {
            *p_ts = p_grp->TS;
        }
#endif

        /* 退出临界区，
         * 并返回获取到的事件标志
         */
        CPU_CRITICAL_EXIT();
        OS_TRACE_FLAG_PEND(p_grp);
        OS_TRACE_FLAG_PEND_EXIT(OS_ERR_NONE);
        *p_err = OS_ERR_NONE;
        return (flags_rdy);
    } else {
        /* 这个 else 的情况是等待的事件全部没有被清 0 */

        /* 判断是否选择挂起等待事件标志 */
        if ((opt & OS_OPT_PEND_NON_BLOCKING) != 0u) {
            /* 不选择挂起等待事件标志，
             * 则退出临界区并返回相应的错误
             */
            CPU_CRITICAL_EXIT();
            OS_TRACE_FLAG_PEND_EXIT(OS_ERR_PEND_WOULD_BLOCK);
            *p_err = OS_ERR_PEND_WOULD_BLOCK;
            return ((OS_FLAGS)0);
        } else {
            /* 如果任务选择挂起等待事件标志，
             * 则接下来将要任务添加到事件标志组的挂起等待任务链表中挂起，

```



```

        * 并且需要先判断任务调度器是否被上锁,
        * 如果任务调度器被上锁,
        * 则直接返回相应的错误代码
        */

        if (OSSchedLockNestingCtr > 0u) {
            CPU_CRITICAL_EXIT();
            OS_TRACE_FLAG_PEND_EXIT(OS_ERR_SCHED_LOCKED);
            *p_err = OS_ERR_SCHED_LOCKED;
            return (0u);
        }
    }

    /* 将任务添加到事件标志组的挂起等待任务链表中 */
    OS_FlagBlock( p_grp,
                  flags,
                  opt,
                  timeout);

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
}
break;
#endif

/* 无效的函数操作选项 */
default:
    /* 退出临界区,
    * 并返回相应错误
    */
    CPU_CRITICAL_EXIT();
    OS_TRACE_FLAG_PEND_FAILED(p_grp);
    OS_TRACE_FLAG_PEND_EXIT(OS_ERR_OPT_INVALID);
    *p_err = OS_ERR_OPT_INVALID;
    return (0u);
}

/* 用于调试, 不用理会 */
OS_TRACE_FLAG_PEND_BLOCK(p_grp);

/* 触发任务调度 */
OSSched();

/* 如果任务被挂起等待事件标志的话,

```

```
* 那么在进行任务调度后，
* 任务就会被挂起，
* 只有当任务挂起结束后才会执行下面的代码
*/

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 根据任务不同的挂起结果进行不同的操作 */
switch (OSTCBCurPtr->PendStatus) {
    /* 成功等待到了相应的事件标志 */
    case OS_STATUS_PEND_OK:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#endif
        OS_TRACE_FLAG_PEND(p_grp);
        *p_err = OS_ERR_NONE;
        break;

    /* 任务挂起等待事件标志被中断 */
    case OS_STATUS_PEND_ABORT:
#if (OS_CFG_TS_EN > 0u)
        if (p_ts != (CPU_TS *)0) {
            *p_ts = OSTCBCurPtr->TS;
        }
#endif
        CPU_CRITICAL_EXIT();
        OS_TRACE_FLAG_PEND_FAILED(p_grp);
        *p_err = OS_ERR_PEND_ABORT;
        break;

    /* 任务挂起等待事件标志超时 */
    case OS_STATUS_PEND_TIMEOUT:
        if (p_ts != (CPU_TS *)0) {
            *p_ts = 0u;
        }
        CPU_CRITICAL_EXIT();
        OS_TRACE_FLAG_PEND_FAILED(p_grp);
        *p_err = OS_ERR_TIMEOUT;
        break;
```

```

        /* 任务挂起等待的事件标志组被删除 */
        case OS_STATUS_PEND_DEL:
#if (OS_CFG_TS_EN > 0u)
            if (p_ts != (CPU_TS *)0) {
                *p_ts = OSTCBCurPtr->TS;
            }
#endif

            CPU_CRITICAL_EXIT();
            OS_TRACE_FLAG_PEND_FAILED(p_grp);
            *p_err = OS_ERR_OBJ_DEL;
            break;

        /* 无效的任务挂起等待结果 */
        default:
            CPU_CRITICAL_EXIT();
            OS_TRACE_FLAG_PEND_FAILED(p_grp);
            *p_err = OS_ERR_STATUS_INVALID;
            break;
    }

    /* 如果有错误，这里就返回 */
    if (*p_err != OS_ERR_NONE) {
        OS_TRACE_FLAG_PEND_EXIT(*p_err);
        return (0u);
    }

    /* 获取任务等待到的事件标志 */
    flags_rdy = OSTCBCurPtr->FlagsRdy;

    /* 判断是否需要取反获取到的事件 */
    if (consume == OS_TRUE) {
        /* 根据函数操作选项进行处理 */
        switch (mode) {
            /* 等待事件标志置 1 */
            case OS_OPT_PEND_FLAG_SET_ALL:
            case OS_OPT_PEND_FLAG_SET_ANY:
                /* 将获取到的事件标志清 0 */
                p_grp->Flags &= ~flags_rdy;
                break;

            /* 此宏用于使能等待事件标志清 0 功能 */
#if (OS_CFG_FLAG_MODE_CLR_EN > 0u)
            /* 等待事件标志清 0 */

```

```

        case OS_OPT_PEND_FLAG_CLR_ALL:
        case OS_OPT_PEND_FLAG_CLR_ANY:
            /* 将获取到的事件标志置 1 */
            p_grp->Flags |= flags_rdy;
            break;
#endif

/* 无效的函数操作选项 */
default:
    CPU_CRITICAL_EXIT();
    OS_TRACE_FLAG_PEND_EXIT(OS_ERR_OPT_INVALID);
    *p_err = OS_ERR_OPT_INVALID;
    return (0u);
}

/* 退出临界区，
 * 并返回获取到的事件标志
 */
CPU_CRITICAL_EXIT();
OS_TRACE_FLAG_PEND_EXIT(OS_ERR_NONE);
*p_err = OS_ERR_NONE;
return (flags_rdy);
}

```

4. 函数 OSFlagPendAbort()

该函数用于终止任务挂起等待事件标志组。

函数 OSFlagPendAbort()的函数原型，如下所示：

```

OS_OBJ_QTY OSFlagPendAbort(OS_FLAG_GRP*   p_grp,
                           OS_OPT          opt,
                           OS_ERR*         p_err)

```

函数 OSFlagPendAbort()的形参描述，如下表所示：

形参	描述
p_grp	指向事件标志组结构体的指针
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 16.2.11 函数 OSFlagPendAbort()形参相关描述

函数 OSFlagPendAbort()的返回值描述，如下表所示：

返回值	描述
OS_OBJ_QTY 类型返回值	被终止挂起任务的数量

表 16.2.12 函数 OSFlagPendAbort()返回值相关描述

函数 OSFlagPendAbort()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	终止任务挂起等待事件标志组成功

OS_ERR_OBJ_PTR_NULL	指向事件标志组结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是事件标志组
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ABORT_ISR	在中断中非法调用该函数
OS_ERR_PEND_ABORT_NONE	没有任务挂起等待事件标志组

表 16.2.13 函数 OSFlagPendAbort()错误代码相关描述

函数 OSFlagPendAbort()定义在文件 os_flag.c 中，具体的代码如下所示：

```
OS_OBJ_QTY OSFlagPendAbort(OS_FLAG_GRP*   p_grp,
                           OS_OPT         opt,
                           OS_ERR*        p_err)
{
    OS_PEND_LIST*   p_pend_list;
    OS_TCB*         p_tcb;
    CPU_TS          ts;
    OS_OBJ_QTY      nbr_tasks;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return ((OS_OBJ_QTY)0u);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_PEND_ABORT_ISR;
            return (0u);
        }
    #endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        if (OSRunning != OS_STATE_OS_RUNNING) {
            *p_err = OS_ERR_OS_NOT_RUNNING;
            return (0u);
        }
    #endif

    /* 此宏用于使能参数检查 */
```

```

#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_grp == (OS_FLAG_GRP *)0) {
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }
    switch (opt) {
        case OS_OPT_PEND_ABORT_1:
        case OS_OPT_PEND_ABORT_ALL:
        case OS_OPT_PEND_ABORT_1 | OS_OPT_POST_NO_SCHED:
        case OS_OPT_PEND_ABORT_ALL | OS_OPT_POST_NO_SCHED:
            break;

        default:
            *p_err = OS_ERR_OPT_INVALID;
            return (0u);
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_grp->Type != OS_OBJ_TYPE_FLAG) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();
    p_pend_list = &p_grp->PendList;
    if (p_pend_list->HeadPtr == (OS_TCB *)0) {
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_PEND_ABORT_NONE;
        return (0u);
    }

    /* 记录被终止挂起任务的数量 */
    nbr_tasks = 0u;

#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif
#endif

```

```

/* 遍历终止所有被挂起的任务 */
while (p_pend_list->HeadPtr != (OS_TCB *)0) {
    /* 获取事件标志组的挂起等待任务链表 */
    p_tcb = p_pend_list->HeadPtr;
    /* 终止任务的挂起状态
     * 同时会设置任务的挂起结果为“任务挂起被中断”
     */
    OS_PendAbort( p_tcb,
                  ts,
                  OS_STATUS_PEND_ABORT);

    /* 更新被终止挂起的任务的数量 */
    nbr_tasks++;
    /* 函数的操作选项
     * 可以设置是终止所有挂起的任务
     * 还是终止事件标志组挂起等待任务链表中的第一个任务
     */
    if (opt != OS_OPT_PEND_ABORT_ALL) {
        break;
    }
}

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 函数的操作选项
 * 可以选择是否进行任务调度
 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

/* 返回被终止挂起的任务的数量 */
*p_err = OS_ERR_NONE;
return (nbr_tasks);
}

```

5. 函数 OSFlagPendGetFlagsRdy()

该函数用于获取任务等待到的事件。

函数 OSFlagPendGetFlagsRdy()的函数原型，如下所示：

```
OS_FLAGS OSFlagPendGetFlagsRdy(OS_ERR *p_err)
```

函数 OSFlagPendGetFlagsRdy()的形参描述，如下表所示：

形参	描述
p_err	指向接收错误代码变量的指针

表 16.2.14 函数 OSFlagPendGetFlagsRdy()形参相关描述

函数 OSFlagPendGetFlagsRdy() 的返回值描述，如下表所示：

返回值	描述
OS_FLAGS 类型返回值	任务获取到的事件标志

表 16.2.15 函数 OSFlagPendGetFlagsRdy() 返回值相关描述

函数 OSFlagPendGetFlagsRdy() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	获取任务获取到的事件标志成功
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_PEND_ISR	在中断中非法调用该函数

表 16.2.16 函数 OSFlagPendGetFlagsRdy() 错误代码相关描述

函数 OSFlagPendGetFlagsRdy() 定义在文件 os_flag.c 中，具体的代码如下所示：

```
OS_FLAGS OSFlagPendGetFlagsRdy(OS_ERR *p_err)
{
    OS_FLAGS flags;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return ((OS_FLAGS)0);
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    #if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        if (OSRunning != OS_STATE_OS_RUNNING) {
            *p_err = OS_ERR_OS_NOT_RUNNING;
            return (0u);
        }
    #endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    #if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_PEND_ISR;
            return (0u);
        }
    #endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 获取任务获取到的事件标志 */
```



```

flags = OSTCBCurPtr->FlagsRdy;

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 返回获取到的事件标志 */
*p_err = OS_ERR_NONE;

return (flags);
}

```

6. 函数 OSFlagPost()

该函数用于设置事件标志组中的事件。

函数 OSFlagPost()的函数原型，如下所示：

```

OS_FLAGS OSFlagPost( OS_FLAG_GRP* p_grp,
                     OS_FLAGS      flags,
                     OS_OPT        opt,
                     OS_ERR*       p_err)

```

函数 OSFlagPost()的形参描述，如下表所示：

形参	描述
p_grp	指向事件标志组结构体的指针
flags	等待的事件标志
opt	函数操作选项
p_err	指向接收错误代码变量的指针

表 16.2.17 函数 OSFlagPost()形参相关描述

函数 OSFlagPost()的返回值描述，如下表所示：

返回值	描述
OS_FLAGS 类型返回值	事件标志组更新后的事件标志值

表 16.2.18 函数 OSFlagPost()返回值相关描述

函数 OSFlagPost()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	设置事件标志成功
OS_ERR_OBJ_PTR_NULL	指向事件标志组结构体的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是事件标志组
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行

表 16.2.19 函数 OSFlagPost()错误代码相关描述

函数 OSFlagPost()定义在文件 os_flag.c 中，具体的代码如下所示：

```

OS_FLAGS OSFlagPost( OS_FLAG_GRP * p_grp,
                     OS_FLAGS      flags,
                     OS_OPT        opt,
                     OS_ERR*       p_err)
{
    OS_FLAGS      flags_cur;
    OS_FLAGS      flags_rdy;
    OS_OPT        mode;
    OS_PEND_LIST* p_pend_list;

```

```

OS_TCB*      p_tcb;
OS_TCB*      p_tcb_next;
CPU_TS       ts;
CPU_SR_ALLOC();

/* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

OS_TRACE_FLAG_POST_ENTER(p_grp, flags, opt);

/* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        OS_TRACE_FLAG_POST_EXIT(OS_ERR_OS_NOT_RUNNING);
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (0u);
    }
#endif

/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_grp == (OS_FLAG_GRP *)0) {
        OS_TRACE_FLAG_POST_FAILED(p_grp);
        OS_TRACE_FLAG_POST_EXIT(OS_ERR_OBJ_PTR_NULL);
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return (0u);
    }
#endif

/* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_grp->Type != OS_OBJ_TYPE_FLAG) {
        OS_TRACE_FLAG_POST_FAILED(p_grp);
        OS_TRACE_FLAG_POST_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

```

```

#if (OS_CFG_TS_EN > 0u)
    ts = OS_TS_GET();
#else
    ts = 0u;
#endif

OS_TRACE_FLAG_POST(p_grp);

/* 根据不同的函数操作选项,
 * 作不同的处理
 */
switch (opt) {
    /* 设置事件标志 */
    case OS_OPT_POST_FLAG_SET:
    case OS_OPT_POST_FLAG_SET | OS_OPT_POST_NO_SCHED:
        CPU_CRITICAL_ENTER();
        p_grp->Flags |= flags;
        break;

    /* 清零事件标志 */
    case OS_OPT_POST_FLAG_CLR:
    case OS_OPT_POST_FLAG_CLR | OS_OPT_POST_NO_SCHED:
        CPU_CRITICAL_ENTER();
        p_grp->Flags &= ~flags;
        break;

    /* 无效的函数操作选项 */
    default:
        *p_err = OS_ERR_OPT_INVALID;
        OS_TRACE_FLAG_POST_EXIT(*p_err);
        return (0u);
}

/* 此宏用于使能时间戳功能 */
#if (OS_CFG_TS_EN > 0u)
    p_grp->TS = ts;
#endif

/* 获取事件标志组的挂起等待任务链表 */
p_pend_list = &p_grp->PendList;

/* 判断事件标志组的挂起等待任务链表是否为空 */
if (p_pend_list->HeadPtr == (OS_TCB *)0) {

```

```

    /* 退出临界区,
    * 并返回更新后的事件标志
    */
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_NONE;
    OS_TRACE_FLAG_POST_EXIT(*p_err);
    return (p_grp->Flags);
}

/* 遍历挂起等待任务链表 */
p_tcb = p_pend_list->HeadPtr;
while (p_tcb != (OS_TCB *)0) {
    p_tcb_next = p_tcb->PendNextPtr;
    /* 获取挂起任务的挂起操作选项中等待事件标志的操作
    * OS_OPT_PEND_FLAG_CLR_ALL : 0x01 // 等待入参 flags 中所有指定的事件被清 0
    * OS_OPT_PEND_FLAG_CLR_AND : 0x01 // 等待入参 flags 中所有指定的事件被清 0
    * OS_OPT_PEND_FLAG_CLR_ANY : 0x02 // 等待入参 flags 中任意指定的事件被清 0
    * OS_OPT_PEND_FLAG_CLR_OR : 0x02 // 等待入参 flags 中任意指定的事件被清 0
    * OS_OPT_PEND_FLAG_SET_ALL : 0x04 // 等待入参 flags 中所有指定的事件被置 1
    * OS_OPT_PEND_FLAG_SET_AND : 0x04 // 等待入参 flags 中所有指定的事件被置 1
    * OS_OPT_PEND_FLAG_SET_ANY : 0x08 // 等待入参 flags 中任意指定的事件被置 1
    * OS_OPT_PEND_FLAG_SET_OR : 0x08 // 等待入参 flags 中任意指定的事件被置 1
    */
    mode = p_tcb->FlagsOpt & OS_OPT_PEND_FLAG_MASK;
    /* 根据挂起任务的挂起操作选项进行处理 */
    switch (mode) {
        /* 等待入参 flags 中所有指定的事件被置 1 */
        case OS_OPT_PEND_FLAG_SET_ALL:
            /* 获取挂起任务等待的事件标志 */
            flags_rdy = (p_grp->Flags & p_tcb->FlagsPend);
            /* 判断获取到的事件标志是否全部被置 1 */
            if (flags_rdy == p_tcb->FlagsPend) {
                /* 将挂起任务添加到就绪态任务链表 */
                OS_FlagTaskRdy(p_tcb,
                               flags_rdy,
                               ts);
            }
            break;

        /* 等待入参 flags 中任意指定的事件被置 1 */
        case OS_OPT_PEND_FLAG_SET_ANY:
            /* 获取挂起任务等待的事件标志 */
            flags_rdy = (p_grp->Flags & p_tcb->FlagsPend);

```

```

        /* 判断是否有任务获取到的事件标志被置 1 */
        if (flags_rdy != 0u) {
            /* 将任务添加到就绪态任务链表 */
            OS_FlagTaskRdy( p_tcb,
                           flags_rdy,
                           ts);
        }
        break;

    /* 此宏用于使能等待事件标志清 0 功能 */
#ifdef (OS_CFG_FLAG_MODE_CLR_EN > 0u)
    /* 等待入参 flags 中所有指定的事件被清 0 */
    case OS_OPT_PEND_FLAG_CLR_ALL:
        /* 获取挂起任务等待的事件标志 */
        flags_rdy = (OS_FLAGS)(~p_grp->Flags & p_tcb->FlagsPend);
        /* 判断获取到的事件标志是否都被清 0 */
        if (flags_rdy == p_tcb->FlagsPend) {
            /* 将任务添加到就绪态任务链表 */
            OS_FlagTaskRdy( p_tcb,
                           flags_rdy,
                           ts);
        }
        break;

    /* 等待入参 flags 中任意指定的事件被清 0 */
    case OS_OPT_PEND_FLAG_CLR_ANY:
        /* 获取挂起任务等待的事件标志 */
        flags_rdy = (OS_FLAGS)(~p_grp->Flags & p_tcb->FlagsPend);
        /* 判断是否有任意获取到的事件标志被清 0 */
        if (flags_rdy != 0u) {
            /* 将任务添加到就绪态任务链表 */
            OS_FlagTaskRdy( p_tcb,
                           flags_rdy,
                           ts);
        }
        break;
#endif

    /* 无效的挂起任务的挂起操作选项 */
    default:
        CPU_CRITICAL_EXIT();
        *p_err = OS_ERR_FLAG_PEND_OPT;
        OS_TRACE_FLAG_POST_EXIT(*p_err);

```

```

        return (0u);
    }

    p_tcb = p_tcb_next;
}

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 函数的操作选项
 * 可以选择是否进行任务调度
 */
if ((opt & OS_OPT_POST_NO_SCHED) == 0u) {
    OSSched();
}

/* 进入临界区 */
CPU_CRITICAL_ENTER();
/* 获取更新后的事件标志 */
flags_cur = p_grp->Flags;
/* 退出临界区 */
CPU_CRITICAL_EXIT();
*p_err = OS_ERR_NONE;

OS_TRACE_FLAG_POST_EXIT(*p_err);

/* 返回更新后的事件标志 */
return (flags_cur);
}

```

16.3 μC/OS-III 事件标志实验

16.3.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数使用事件标志，本实验设计了四个任务，这四个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建事件标志组和其他任务
task1	用于扫描按键，并设置事件标志
task2	用于等待事件标志
task3	用于实时显示事件标志组中事件标志的值

表 16.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 16 UCOS-III 事件标志实验》。

16.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

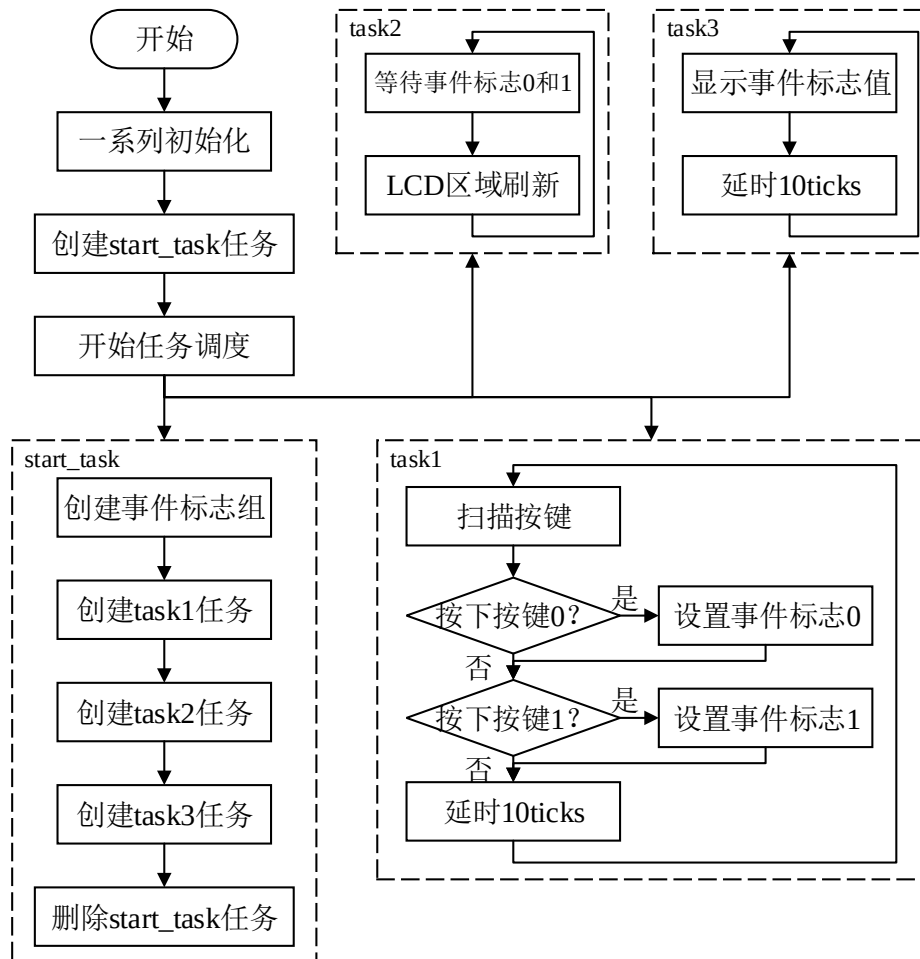


图 16.3.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSFlagCreate()

此函数用于创建事件标志组，请参考第 16.2 小节《μC/OS-III 事件标志相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_FLAG_EN 配置为 1。

(2) 函数 OSFlagPend()

此函数用于等待事件标志，请参考第 16.2 小节《μC/OS-III 事件标志相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_FLAG_EN 配置为 1。

(3) 函数 OSFlagPost()

此函数用于设置事件标志，请参考第 16.2 小节《μC/OS-III 事件标志相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_FLAG_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建事件标志 */
    OSFlagCreate(    (OS_FLAG_GRP*) &flag,
                    (CPU_CHAR*)    "flag",
                    (OS_FLAGS)      0,
                    (OS_ERR*)       &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*)      &Task1Task_TCB,
                    (CPU_CHAR*)     "task1",
                    (OS_TASK_PTR)    task1,
                    (void*)          0,
                    (OS_PRIO)        TASK1_PRIO,
                    (CPU_STK)        *)Task1Task_STK,
                    (CPU_STK_SIZE)   TASK1_STK_SIZE / 10,
                    (CPU_STK_SIZE)   TASK1_STK_SIZE,
                    (OS_MSG_QTY)     0,
                    (OS_TICK)        0,
```



```

        (void*          )0,
        (OS_OPT         ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
        (OS_ERR*        ) &err);

/* 创建 Task2 */
Task2Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                       TASK2_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*          ) &Task2Task_TCB,
                 (CPU_CHAR*        ) "task2",
                 (OS_TASK_PTR      ) task2,
                 (void*            ) 0,
                 (OS_PRIO          ) TASK2_PRIO,
                 (CPU_STK*         ) Task2Task_STK,
                 (CPU_STK_SIZE     ) TASK2_STK_SIZE / 10,
                 (CPU_STK_SIZE     ) TASK2_STK_SIZE,
                 (OS_MSG_QTY       ) 0,
                 (OS_TICK          ) 0,
                 (void*            ) 0,
                 (OS_OPT           ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*         ) &err);

/* 创建 Task3 */
Task3Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                       TASK3_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*          ) &Task3Task_TCB,
                 (CPU_CHAR*        ) "task3",
                 (OS_TASK_PTR      ) task3,
                 (void*            ) 0,
                 (OS_PRIO          ) TASK3_PRIO,
                 (CPU_STK*         ) Task3Task_STK,
                 (CPU_STK_SIZE     ) TASK3_STK_SIZE / 10,
                 (CPU_STK_SIZE     ) TASK3_STK_SIZE,
                 (OS_MSG_QTY       ) 0,
                 (OS_TICK          ) 0,
                 (void*            ) 0,
                 (OS_OPT           ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*         ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务、task2 任务和 task3 任务，并且 task1 任务、task2 任务和 task3 任务的任务栈都是动态分配的，当然，

也可以静态地定任务栈，就像定义一个数组一样，同时还调用函数 OSFlagCreate() 创建了一个用于本次实验的事件标志组。

(2) task1 任务

```
/**
 * @brief task1
 * @param p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    uint8_t key;
    OS_ERR err;

    while (1)
    {
        key = key_scan(0);

        switch (key)
        {
            case KEY0_PRES:
            {
                OSFlagPost( (OS_FLAG_GRP*) &flag,
                            (OS_FLAGS) FLAGBIT_0,
                            (OS_OPT) OS_OPT_POST_FLAG_SET,
                            (OS_ERR*) &err);

                break;
            }
            case KEY1_PRES:
            {
                OSFlagPost( (OS_FLAG_GRP*) &flag,
                            (OS_FLAGS) FLAGBIT_1,
                            (OS_OPT) OS_OPT_POST_FLAG_SET,
                            (OS_ERR*) &err);

                break;
            }
            default:
            {
                break;
            }
        }

        OSTimeDly(10, OS_OPT_TIME_DLY, &err);
    }
}
```

```
}
```

可以看到 task1 任务就是用于扫描按键，当检测到按键 0 被按下时，就设置事件标志组中的事件标志 0，当检测到按键 1 被按下时，就设置事件标志组中的事件标志 1，每间隔 10ticks 就扫描一次按键。

(3) task2 任务

```
/**
 * @brief   task2
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task2(void *p_arg)
{
    uint32_t task2_num = 0;
    OS_ERR err;
    CPU_SR_ALLOC();

    while (1)
    {
        OSFlagPend( (OS_FLAG_GRP*) &flag,
                    (OS_FLAGS) FLAGBIT_ALL,
                    (OS_TICK) 0,
                    (OS_OPT) OS_OPT_PEND_FLAG_SET_ALL |
                             OS_OPT_PEND_FLAG_CONSUME |
                             OS_OPT_PEND_BLOCKING,
                    (CPU_TS *) 0,
                    (OS_ERR *) &err);

        CPU_CRITICAL_ENTER();
        lcd_fill(6, 131, 233, 313, lcd_discolor[++task2_num % 11]);
        CPU_CRITICAL_EXIT();
    }
}
```

从上面的代码中可以看出，task2 任务会等待事件标志组中的事件标志 0 和事件标志 1，只有当事件标志 0 和事件标志 1 同时被置 1 时，task2 任务才能够执行，并且在 task2 任务成功等待到事件标志 0 和事件标志 1 后，会清零事件标志 0 和事件标志 1。

(4) task3 任务

```
/**
 * @brief   task3
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task3(void *p_arg)
{
```

```
OS_FLAGS flags;
OS_ERR err;
CPU_SR_ALLOC();

while (1)
{
    flags = flag.Flags;

    CPU_CRITICAL_ENTER();
    lcd_show_xnum(163, 110, flags, 1, 16, 0, BLUE);
    CPU_CRITICAL_EXIT();

    OSTimeDly(10, OS_OPT_TIME_DLY, &err);
}
}
```

task3 任务比较简单，就是不断地获取事件标志组中事件标志的值，然后实时地将事件标志组中事件标志的值显示在 LCD 上，但值得一提的是，从上面的代码中可以看出，操作 LCD 的代码是处于临界区的，这是因为在本次实验中，task2 任务和 task3 任务都会操作 LCD，在临界区中操作 LCD 就是为了避免任务在访问 LCD 这一共享资源的时候，产生访问冲突，因此在 task2 任务的任务代码中，也能够看到操作 LCD 的代码被放在了临界区中。

16.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 16.3.2 LCD 显示内容一

接着按下按键 0，使得 task1 任务将事件标志组中的事件标志 0 置 1，随后立马就能看到 task3 任务在 LCD 上显示了事件标志组中事件标志的值为 1，如下图所示：



图 16.3.3 LCD 显示内容二

但此时 task2 任务还没有刷新 LCD，因为 task2 任务等待的时事件标志 0 和事件标志 1。接下来按下按键 1，使得 task1 任务设置事件标志组中的事件标志 1，如下图所示：



图 16.3.4 LCD 显示内容三

可以看到，task2 任务刷新了 LCD，同时因为 task2 任务等待到了事件标志 0 和事件标志 1，清 0 了事件标志组中的事件标志 0 和事件标志 1，因此 task3 任务在 LCD 显示的事件标志组中事件标志的值变回了 0。

第十七章 μC/OS-III 软件定时器

定时器可以说是每个 MCU 都有的外设，有的 MCU 自带的定时器有着十分强大的功能，但是最常用的还是定时器的基础功能——定时，通过定时器的定时功能，能够在定时器定时结束时去完成一些周期性的事务。MCU 自带的定时器为硬件定时器，本章讲解的定时器为 μC/OS-III 提供的软件定时器，软件定时器相对于硬件定时器有着使用简单、成本低等优先。本章就来学习 μC/OS-III 中软件定时器相关的内容。

本章分为如下几部分：

17.1 μC/OS-III 软件定时器简介

17.2 μC/OS-III 软件定时器相关 API 函数

17.3 μC/OS-III 软件定时器实验

17.1 μC/OS-III 软件定时器简介

17.1.1 软件定时器

软件定时器指的是由软件实现的定时器，并不是由具体的硬件组成，μC/OS-III 提供的软件定时器是一种向下计数的定时器，当设置好一个定时器超时时间后，软件定时器便从这个定时器时间对应的数值开始往下递减，每经过一个系统时钟节拍，软件定时器的计数值就往下减一，当软件定时器的计数值减到 0 的时候，软件定时器就会自动地去调用其对应的超时回调函数。

下面来看一下软件定时器的控制块结构体，软件定时器的控制块结构体定义在文件 os.h 中，具体的代码如下所示：

```
struct os_tmr{
    /* 此宏用于使能检查内核对象的类型 */
    #if (OS_OBJ_TYPE_REQ > 0u)
        /* 对象类型，
         * 在软件定时器初始化时，
         * 软件定时器的类型会被初始化为 OS_OBJ_TYPE_TMR
         */
        OS_OBJ_TYPE Type;
    #endif

    /* 此宏用于使能代码调试功能 */
    #if (OS_CFG_DBG_EN > 0u)
        /* 如果使能了代码调试功能，
         * 那么就会为每一个软件定时器赋一个名称，
         * 方便调试
         */
        CPU_CHAR *NamePtr;
    #endif

    OS_TMR_CALLBACK_PTR CallbackPtr; /* 超时回调函数 */
}
```

```
void*          CallbackPtrArg; /* 超时回调函数参数 */
OS_TMR*       NextPtr;        /* 指向软件定时器链表中下一个软件定时器的指针 */
OS_TMR*       PrevPtr;        /* 指向软件定时器链表中上一个软件定时器的指针 */
OS_TICK       Remain;         /* 剩余超时时间 */
OS_TICK       Dly;             /* 开启延迟时间 */
OS_TICK       Period;         /* 周期时间 */
OS_OPT        Opt;             /* 操作选项，单次定时器或周期定时器 */
OS_STATE      State;           /* 状态 */

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
    OS_TMR *DbgPrevPtr;
    OS_TMR *DbgNextPtr;
#endif
};
```

下面从软件定时器的控制块结构中“拎”出几个比较重要的成员变量进行讲解，如下所示：

1. 成员变量 CallbackPtr

该成员变量是一个指向软件定时器超时回调函数的指针，当软件定时器定时超时的时候，就会调用该成员变量指向的函数。

2. 成员变量 CallbackPtrArg

该成员变量是一个指向软件定时器超时回调函数参数的指针，当软件定时器超时的時候，就会将该成员变量指向的参数传入软件定时器的超时回调函数，通过这一成员变量，可令多个软件定时器使用同一个超时回调函数，只要让软件定时器的超时回调函数传入指向该软件定时器控制块结构体的指针，接着在软件定时器的超时回调函数中判断入参是指向哪一个软件定时器控制块结构体的指针，即可进行不同的处理。

3. 成员变量 Opt

该成员变量在软件定时器被创建的时候被初始化，并且只有两种可能的值，分别为 OS_OPT_TMR_ONE_SHOT 和 OS_OPT_TMR_PERIODIC，其中 OS_OPT_TMR_ONE_SHOT 代表该软件定时器是单次定时器，单次定时器在开启定时后，只会超时一次，便停止定时了，若要单次定时器再次定时，需要再次调用软件定时器的开启 API 函数开启单次定时器；OS_OPT_TMR_PERIODIC 代表该软件定时器是周期定时器，周期定时器在超时的时候，会自动地根据设置好的定时周期进行重新定时，若不调用相关 API 函数停止周期定时器，那么周期定时器会自动地开启定时和超时。

4. 成员变量 Remain

该成员变量就是软件定时器的定时器计数器，当软件定时器开启定时器时，会为该成员变量赋一个初值，开启定时器后，软件定时器任务会在每经过一个系统时钟节拍就将该成员变量递减 1，当该成员变量第一次从初值减到 0 时，说明该软件定时器超时了。

5. 成员变量 Dly

该成员变量定义了软件定时器的延时开启时间，当调用相关 API 函数开启软件定时器定时后，软件定时器会延时该成员变量定义的系统时钟节拍数后，才会真正地开始定时。

6. 成员变量 Period

该成员变量定义了周期定时器的定时周期，该成员变量仅对周期定时器有效，当周期定时器超时后，会将成员变量 Remain 重新赋值为该成员变量定义的值，然后重新开启该软件定时

器的定时器。

17.1.2 软件定时器超时回调函数

软件定时器的超时回调函数是完全由用户自定义的，并且会在软件定时器定时器超时的時候被自动地调用，且可以设置函数的传入参数。通过软件定时器的回调函数，可以实现点亮或关闭一个 LED 灯、启动电机等一系列的操作。但是要特别注意的是，绝对不能在软件定时器的超时回调函数中使用可能会导致任务被阻塞、被挂起甚至被删除的函数，例如调用函数 `OSTimeDly()`、函数 `OSTimeDlyHMSM()`、函数 `OSxxxPend()` 等任何可能导致软件定时器任务被阻塞、被挂起甚至被删除的函数。

17.1.3 单次定时器和周期定时器

μC/OS-III 提供了两种软件定时器，分别为单次定时器和周期定时器：

1. 单次定时器

单次定时器一旦定时超时，只会执行一次其软件定时器超时回调函数，超时后可以被手动重新开启，但单次定时器不会自动重新开启定时。

2. 周期定时器

周期定时器一旦被开启，会在每次超时时，自动地重新启动定时器，从而周期地执行其软件定时器回调函数。

17.1.4 软件定时器的状态

软件定时器一共可以有四种状态，分别为未使用态、停止态、运行态和完成态，这四种软件定时器状态的描述如下所示：

1. 未使用态

当软件定时器被定义但还未被创建或软件定时器被删除时，软件定时器就处于未使用态。

2. 停止态

当软件定时器被创建但还未开启定时器或处于运行态的软件定时器被停止时，软件定时器就处于停止态。

3. 运行态

当处于停止态的软件定时器被启动或单次定时器定时超时后重新启动或周期定时器定时超时后自动重新启动时，软件定时器处于运行态。

4. 完成态

当单次定时器定时超时后，软件定时器处于完成态。

17.1.5 软件定时器任务

软件定时器任务负责更新软件定时器的定时器计数器，并且在软件定时器超时的時候，调用软件定时器的软件定时器超时回调函数，因此不能在软件定时器的超时回调函数中使用可能到时任务被阻塞、被挂起甚至被删除的函数。

在第 7.3 小节《μC/OS-III 内核初始化》中的函数 `OSInit()` 中调用函数 `OS_TmrInit()` 进行了 μC/OS-III 软件定时器功能的初始化，其中就创建了软件定时器任务，下面简单地看一下函数 `OS_TmrInit()`，该函数定义在文件 `os_tmr.c` 中，具体的代码如下所示：

```
void OS_TmrInit(OS_ERR *p_err)
{
```



```

/* 省略其他代码,
 * 仅保留创建软件定时器任务的相关代码
 */

/* ..... */

OSTmrToTicksMult = OSCfg_TickRate_Hz / OSCfg_TmrTaskRate_Hz;

/* ..... */

OSTaskCreate(    &OSTmrTaskTCB,
#if (OS_CFG_DBG_EN == 0u)
    (CPU_CHAR *)0,
#else
    (CPU_CHAR *) "uC/OS-III Timer Task",
#endif

    OS_TmrTask,          /* 软件定时器任务函数 */
    (void *)0,
    OSCfg_TmrTaskPrio,
    OSCfg_TmrTaskStkBasePtr,
    OSCfg_TmrTaskStkLimit,
    OSCfg_TmrTaskStkSize,
    0u,
    0u,
    (void *)0,
    (OS_OPT_TASK_STK_CHK | \
     (OS_OPT) ( OS_OPT_TASK_STK_CLR | \
                OS_OPT_TASK_NO_TLS)),
    p_err);
}

```

可以看到，软件定时器的任务入口函数为函数 OS_TmrTask()，该函数定义在文件 os_tmr.c 中，具体的代码如下所示：

```

void OS_TmrTask(void *p_arg)
{
    OS_TMR_CALLBACK_PTR p_fnct;
    OS_TMR*              p_tmr;
    OS_TICK              timeout;
    OS_TICK              elapsed;
    OS_TICK              time;
#if (OS_CFG_TS_EN > 0u)
    CPU_TS              ts_start;
#endif
    CPU_SR_ALLOC();
}

```

```

(void)p_arg;

/* 通过互斥信号量保护对全局变量软件定时器链表的访问 */
OS_TmrLock();

for (;;) {
    /* 获取 timeout 时间，
     * 软件定时器任务将挂起 timeout 的时长等待软件定时器超时，
     * 然后才去处理软件定时器
     */
    if (OSTmrListPtr == (OS_TMR *)0) {
        /* 当软件定时器链表中没有软件定时器时，
         * 就将 timeout 设置为 0，
         * 表示将软件定时器一直挂起，
         * 如果有任务创建了开启了软件定时器，
         * 那么才会解除软件定时器任务的挂起状态
         */
        timeout = 0u;
    } else {
        /* 如果软件定时器链表中有任务，
         * 就将 timeout 设置为软件定时器的剩余超时时间，
         * 表示将软件定时器任务挂起等待软件定时器超时，
         * 然后再去出去软件定时器
         */
        timeout = OSTmrListPtr->Remain;
    }

    /* 释放互斥信号量（方便其他任务获取），
     * 将软件定时器任务挂起 timeout 的时间，
     * 因为在 timeout 时间到达之前，
     * 没有软件定时器会超时，
     * 软件定时器任务也就不需要处理任务软件定时器，
     * 因此将软件定时器任务挂起
     */
    OS_TmrCondWait(timeout);

    /* 如果此时软件定时器链表中还是没有软件定时器，
     * 那么就不需要进行下面的处理，
     * 直接重新开始 for 循环
     */
    if (OSTmrListPtr == (OS_TMR *)0) {
        continue;
    }
}

```

```

    }

    /* 此宏用于使能时间戳功能 */
#if (OS_CFG_TS_EN > 0u)
    /* 记录开始处理软件定时器时的时间戳 */
    ts_start = OS_TS_GET();
#endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 此宏用于使能动态时钟节拍管理 */
#if (OS_CFG_DYN_TICK_EN > 0u)
    /* 获取当前的系统时钟节拍，
     * 如果使能了动态时钟节拍管理，
     * 则还需要做相应的处理
     */
    time = OSTickCtr + OS_DynTickGet();
#else
    /* 获取当前的系统时钟节拍，
     * 主要用于计算距离上次处理软件定时器过去了多少了系统时钟节拍
     */
    time = OSTickCtr;
#endif

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();

    /* 计算距离上次处理软件定时器过去了多少了系统时钟节拍 */
    elapsed = time - OSTmrTaskTickBase;
    /* 记录下本次处理软件定时器前的系统时钟节拍
     * 方便下次处理软件定时器前使用
     */
    OSTmrTaskTickBase = time;

    /* 下面开始更新软件定时器链表中软件定时器的超时剩余时间 */

    /* 获取软件定时器链表中的软件定时器
     * 用于下面遍历软件定时器链表
     */
    p_tmr = OSTmrListPtr;

    /* 更新软件定时器任务链表中的软件定时器的剩余超时时间（成员变量 Remain），

```

* 直到 elapsed 为 0，
 * 因为软件定时器链表中的软件定时器是按照超时等剩余时间长短排序的，
 * 并且软件定时器链表中软件定时器的超时时间为当前软件定时器的 Remain
 * 与此链表前面所有软件定时器的 Remain 的总和，如下所示：

	[-----]		[-----]		[-----]
	Tmr1	---\	Tmr2	---\	Tmr3
	Remain=3	---/	Remain=1	---/	Remain=5
	[-----]		[-----]		[-----]
* 实际剩余	3		3+1=4		3+1+5=9
* 超时时间					
*/					

```
while ((elapsed != 0u) && (p_tmr != (OS_TMR *)0)) {
    if (elapsed > p_tmr->Remain) {
        elapsed -= p_tmr->Remain;
        p_tmr->Remain = 0u;
    } else {
        p_tmr->Remain -= elapsed;
        elapsed = 0u;
    }

    p_tmr = p_tmr->NextPtr;
}

/* 下面开始处理超时的软件定时器 */

/* 获取软件定时器链表中的软件定时器
 * 用于下面遍历软件定时器链表
 */
p_tmr = OSTmrListPtr;

/* 通过软件定时器的 Remain 判断软件定时器是否超时 */
while ((p_tmr != (OS_TMR *)0) && (p_tmr->Remain == 0u)) {
    /* 设置软件定时器的状态为超时状态 */
    p_tmr->State = OS_TMR_STATE_TIMEOUT;

    /* 调用软件定时器的超时回调函数，
     * 并传入软件定时器的超时回调函数参数
     */
    p_fnct = p_tmr->CallbackPtr;
    if (p_fnct != (OS_TMR_CALLBACK_PTR)0u) {
        (*p_fnct)(p_tmr, p_tmr->CallbackPtrArg);
    }
}
```

```

/* 处理处于超时状态的软件定时器 */
if (p_tmr->State == OS_TMR_STATE_TIMEOUT) {
    /* 将软件定时器从软件定时器链表中移除 */
    OSTmrUnlink(p_tmr, OSTmrTaskTickBase);

    /* 根据软件定时器是周期定时器还是单次定时器，
     * 进行不同的处理
     */
    if (p_tmr->Opt == OS_OPT_TMR_PERIODIC) {
        /* 如果是周期定时器，
         * 那么超时后重新设置周期定时器的状态为运行态，
         * 重新赋值 Remain 为 Period，
         * 然后将周期定时器再次添加到软件定时器链表中进行定时
         */
        p_tmr->State = OS_TMR_STATE_RUNNING;
        p_tmr->Remain = p_tmr->Period;
        OSTmrLink(p_tmr, OSTmrTaskTickBase);
    } else {
        /* 如果是单次定时器，
         * 那么超时后就无需再次定时器了，
         * 因此清空单次定时器的软件定时器指针，
         * 清零单次定时器的 Remain，
         * 并且将单次定时器的状态设置为完成态
         */
        p_tmr->PrevPtr = (OS_TMR *)0;
        p_tmr->NextPtr = (OS_TMR *)0;
        p_tmr->Remain = 0u;
        p_tmr->State = OS_TMR_STATE_COMPLETED;
    }
}

p_tmr = OSTmrListPtr;
}

/* 此宏用于使能时间戳功能 */
#if (OS_CFG_TS_EN > 0u)
    /* 计算处理软件定时器的时间戳耗时 */
    OSTmrTaskTime = OS_TS_GET() - ts_start;
    if (OSTmrTaskTimeMax < OSTmrTaskTime) {
        OSTmrTaskTimeMax = OSTmrTaskTime;
    }
#endif
}

```

```
}
```

其中，软件定时器任务在处理软件定时器之前，或将自身挂起等待一段时间，等待软件定时器链表中有软件定时器要超时的时候，在会取消挂起状态，来处理软件定时器，并且，在挂起软件定时器任务之前还会释放用于保护软件定时器链表的互斥信号量，这一系列的操作都是在函数 OS_TmrCoudWait()中完成的，接下来就来看一下这个函数，函数 OS_TmrCoudWait()定义在文件 os_tmr.c 中，具体的代码如下所示：

```
static void OS_TmrCondWait(OS_TICK timeout)
{
    OS_TCB*      p_tcb;
    OS_PEND_LIST* p_pend_list;
    CPU_TS       ts;
    CPU_SR_ALLOC();

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();
#ifdef OS_CFG_TS_EN > 0u
    ts = OS_TS_GET();
    OSTmrMutex.TS = ts;
#else
    ts = 0u;
#endif

    /* 准备将互斥信号量释放给其他任务 */
    OS_MutexGrpRemove(&OSTmrTaskTCB, &OSTmrMutex);
    p_pend_list = &OSTmrMutex.PendList;

    /* 恢复任务优先级（互斥信号量可能导致优先级继承） */
    if (OSTmrTaskTCB.Prio != OSTmrTaskTCB.BasePrio) {
        OS_TRACE_MUTEX_TASK_PRIO_DISINHERIT(&OSTmrTaskTCB, OSTmrTaskTCB.Prio);
        OSTmrTaskTCB.Prio = OSTmrTaskTCB.BasePrio;
        OSPrioCur = OSTmrTaskTCB.BasePrio;
    }

    /* 判断是否没有其他任务正挂起等待互斥信号量 */
    if (p_pend_list->HeadPtr == (OS_TCB *)0) {
        OSTmrMutex.OwnerTCBPtr = (OS_TCB *)0;
        OSTmrMutex.OwnerNestingCtr = 0u;
    } else {
        /* 如果有任务正挂起等待互斥信号量
         * 那么就将互斥信号量释放给该任务
         */
        p_tcb = p_pend_list->HeadPtr;
        OSTmrMutex.OwnerTCBPtr = p_tcb;
    }
}
```

```

OSTmrMutex.OwnerNestingCtr = 1u;
OS_MutexGrpAdd(p_tcb, &OSTmrMutex);

OS_Post(    (OS_PEND_OBJ *) ((void *) &OSTmrMutex),
            p_tcb,
            (void *) 0,
            0u,
            ts);
}

/* 将软件定时器任务添加到挂起态任务链表
 * 等待下一个软件定时器超时时间到达
 * 设置的挂起超时时间就是下一个软件定时器超时的剩余时间
 */
OS_Pend( (OS_PEND_OBJ *) ((void *) &OSTmrCond),
        &OSTmrTaskTCB,
        OS_TASK_PEND_ON_COND,
        timeout);

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 触发任务调度 */
OSSched();

/* 触发任务调度后,
 * 软件定时器任务就被挂起了,
 * 只有当软件定时器挂起超时或被其他任务终止挂起状态后,
 * 才会继续执行一下代码
 */

/* 进入临界区 */
CPU_CRITICAL_ENTER();
/* 判断使能够获取互斥信号量 */
if (OSTmrMutex.OwnerTCBPtr == (OS_TCB *) 0) {
    /* 能获得互斥信号量就获取互斥信号量 */
    OS_MutexGrpAdd(&OSTmrTaskTCB, &OSTmrMutex);
    OSTmrMutex.OwnerTCBPtr = &OSTmrTaskTCB;
    OSTmrMutex.OwnerNestingCtr = 1u;
    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
} else {
    /* 不能获取互斥信号量 */

```

```

p_tcb = OSTmrMutex.OwnerTCBPtr;

/* 根据需要对互斥信号量当前的持有者进行任务优先级继承 */
if (p_tcb->Prio > OSTmrTaskTCB.Prio) {
    OS_TaskChangePrio(p_tcb, OSTmrTaskTCB.Prio);
}

/* 将软件定时器添加到互斥信号量的挂起等待任务链表中 */
OS_Pend(    (OS_PEND_OBJ *) ((void *)&OSTmrMutex),
            &OSTmrTaskTCB,
            OS_TASK_PEND_ON_MUTEX,
            0u);

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 进行任务调度 */
OSSched();

/* 只有在软件定时器任务成功获取到互斥信号量之后，
 * 软件定时器任务才会解除挂起状态，
 * 该函数才会返回
 */
}
}

```

17.1.6 软件定时器定时频率

实际上软件定时器的定时器频率并不等于系统时钟节拍的频率，在第 17.1.5 小节《软件定时器任务》的函数 `OS_TmrInit()` 代码中可以看到一下代码：

```
OSTmrToTicksMult = OSCfg_TickRate_Hz / OSCfg_TmrTaskRate_Hz;
```

可以看到，在初始化软件定时器功能的时候，计算了一个系数——`OSTmrToTicksMult`，这个系数就是用于将软件定时器的时间转换为系统时钟节拍的节拍数，在计算这一系数的时候使用到了两个变量，分别为 `OSCfg_TickRate_Hz` 和 `OSCfg_TmrTaskRate_Hz`，这两个变量又分别由文件 `os_cfg_app.h` 中的两个宏定义定义，分别为宏定义 `OS_CFG_TICK_RATE_HZ` 和宏定义 `OS_CFG_TMR_TASK_RATE_HZ`（详见第 3.2.2 小节《`os_cfg_app.h` 配置文件》）。

正是因为软件定时器的定时频率与系统时钟节拍的节拍频率不同，才导致如果设置一个软件定时器定时器的时间为 10，但在开启软件定时器后，软件定时器并不一定会在 10 个系统时钟节拍后超时，而是会在 $10 * \text{OSTmrToTicksMult}$ 个系统时钟节拍后超时。

例如本教程的实验例程中将宏 `OS_CFG_TMR_TASK_RATE_HZ` 配置为 10，这代表着软件定时器的定时器频率为 10Hz，那么当开启了一个定时器时长为 10 的软件定时器后，软件定时器将会在大约 1000 毫秒后超时（因为频率为 10Hz，所以周期为 100ms，那么总的定时器周期就为 $10 * 100\text{ms} = 1000\text{ms}$ ）。

17.1.7 软件定时器时序图

1. 单次定时器（`dly>0`，`period=0`）

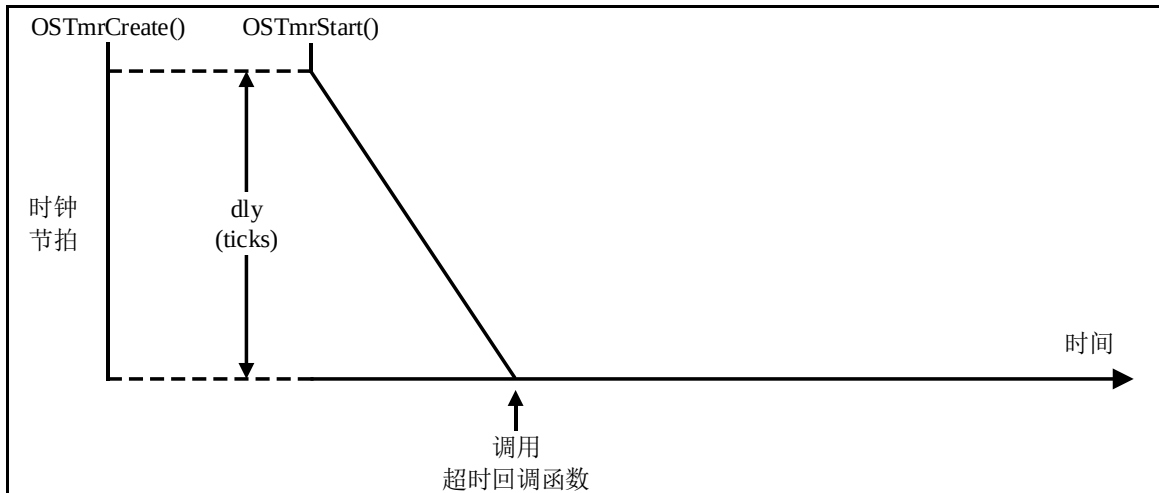


图 17.1.1 单次定时器 ($dly > 0$, $period = 0$)

2. 重新触发单次定时器

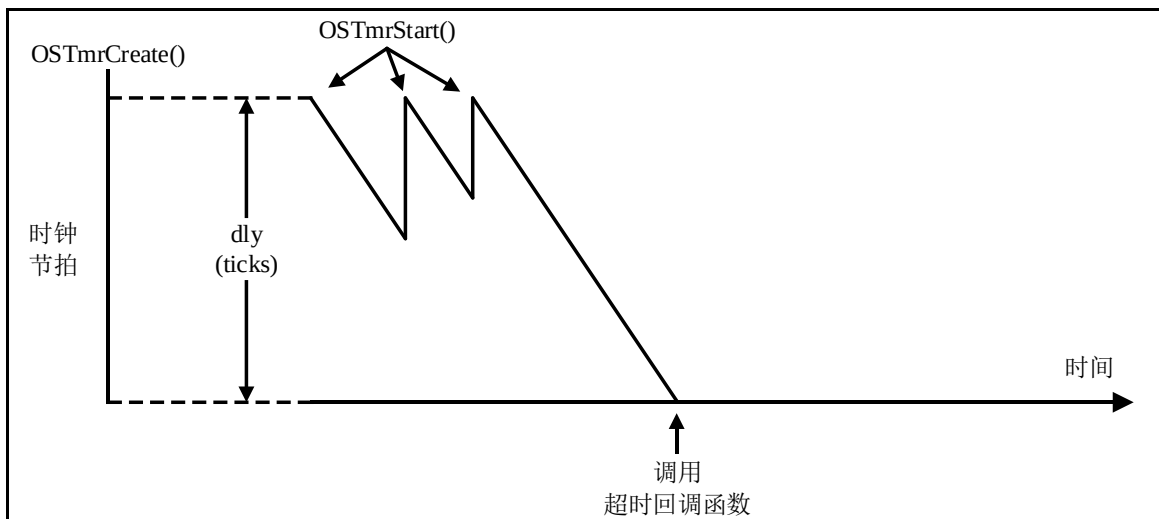


图 17.1.2 重新触发单次定时器

3. 周期定时器 ($dly = 0$, $period > 0$)

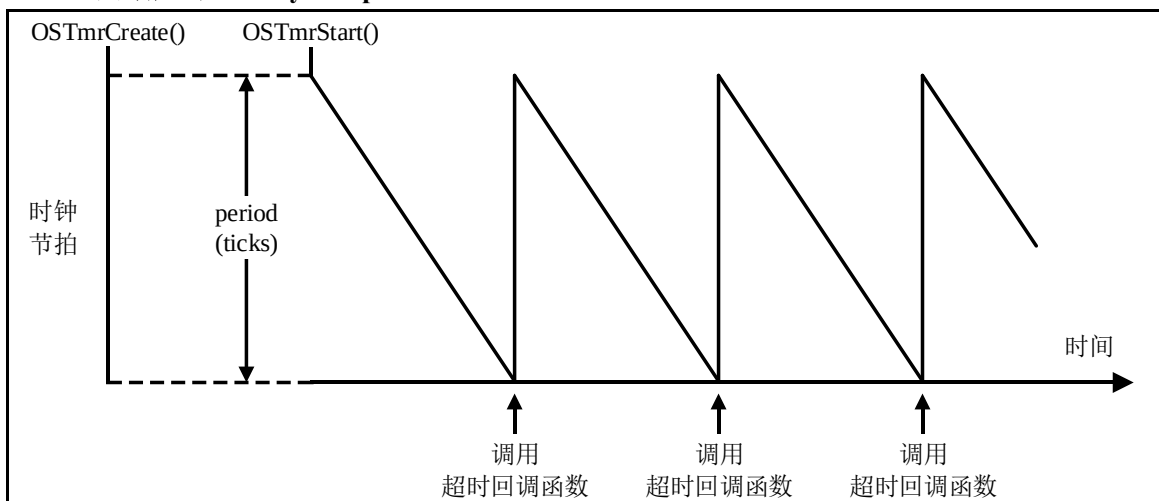


图 17.1.3 周期定时器 ($dly = 0$, $period > 0$)

4. 周期定时器 ($dly > 0$, $period > 0$)

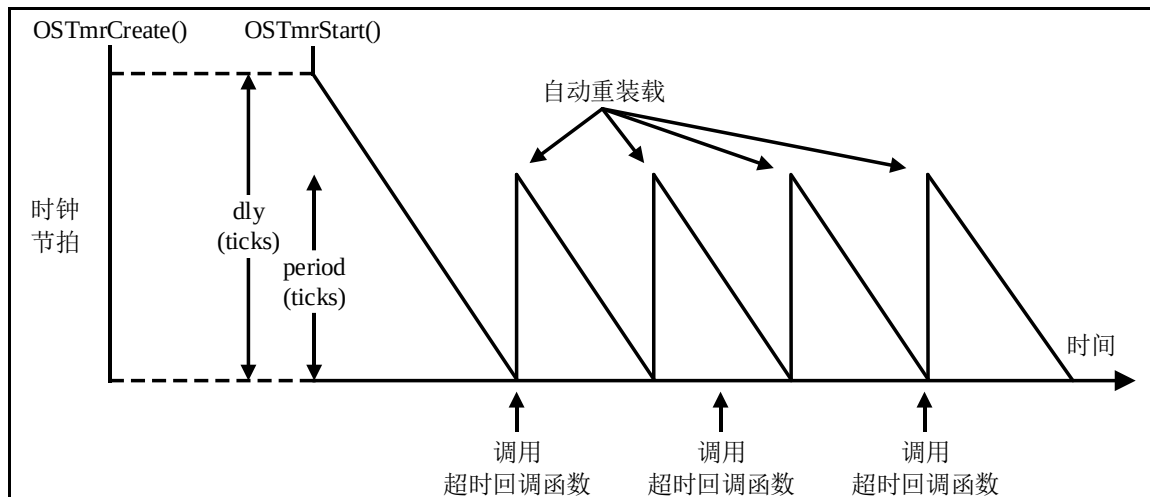


图 17.1.4 周期定时器（dly>0，period>0）

17.2 μC/OS-III 软件定时器相关 API 函数

μC/OS-III 提供了一些列操作软件定时器的 API 函数，这些操作函数如下表所示：

函数	描述
OSTmrCreate()	创建一个软件定时器
OSTmrDel()	删除一个软件定时器
OSTmrRemainGet()	获取软件定时器的剩余超时时间
OSTmrSet()	设置软件定时器的延时时间、周期、超时回调函数及其参数等
OSTmrStart()	开启软件定时器定时
OSTmrStateGet()	获取软件定时器的状态
OSTmrStop()	停止软件定时器定时

表 17.2.1 μC/OS-III 软件定时器相关 API 函数描述

1. 函数 OSTmrCreate()

该函数用于创建一个软件定时器。

函数 OSTmrCreate() 的函数原型，如下所示：

```
void OSTmrCreate( OS_TMR*      p_tmr,
                  CPU_CHAR*    p_name,
                  OS_TICK      dly,
                  OS_TICK      period,
                  OS_OPT        opt,
                  OS_TMR_CALLBACK_PTR p_callback,
                  void*         p_callback_arg,
                  OS_ERR*      p_err)
```

函数 OSTmrCreate() 的形参描述，如下表所示：

形参	描述
p_tmr	指向软件定时器结构体的指针
p_name	指向作为软件定时器名的 ASCII 字符串的指针
dly	软件定时器的开启延时时间
period	周期定时器的定时周期时间

opt	函数操作选项（创建的是单次定时器还是周期定时器）
p_callback	指向软件定时器超时回调函数的指针
p_callback_arg	指向软件定时器超时回调函数参数的指针
p_err	指向接收错误代码变量的指针

表 17.2.2 函数 OSTmrCreate()形参相关描述

函数 OSTmrCreate()的返回值描述，如下表所示：

返回值	描述
无	无

表 17.2.3 函数 OSTmrCreate()返回值相关描述

函数 OSTmrCreate()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功创建一个软件定时器
OS_ERR_ILLEGAL_CREATE_RUN_TIME	在系统运行过程中非法创建内核对象
OS_ERR_OBJ_PTR_NULL	指向软件定时器结构体的指针为空
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_TMR_INVALID_CALLBACK	无效的软件定时器超时处理函数
OS_ERR_TMR_INVALID_DLY	无效的软件定时器开启延时时间
OS_ERR_TMR_INVALID_PERIOD	无效的周期定时器定时周期时间
OS_ERR_TMR_ISR	在中断中非法调用该函数

表 17.2.4 函数 OSTmrCreate()错误代码相关描述

函数 OSTmrCreate()定义在文件 os_tmr.c 中，具体的代码如下所示：

```
void OSTmrCreate( OS_TMR*      p_tmr,
                  CPU_CHAR*    p_name,
                  OS_TICK      dly,
                  OS_TICK      period,
                  OS_OPT        opt,
                  OS_TMR_CALLBACK_PTR p_callback,
                  void*         p_callback_arg,
                  OS_ERR*       p_err)
{
    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        *p_err = OS_ERR_ILLEGAL_CREATE_RUN_TIME;
        return;
    }
#endif
}
```

```
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_TMR_ISR;
        return;
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_tmr == (OS_TMR *)0) {
        *p_err = OS_ERR_OBJ_PTR_NULL;
        return;
    }

    switch (opt) {
        case OS_OPT_TMR_PERIODIC:
            if (period == 0u) {
                *p_err = OS_ERR_TMR_INVALID_PERIOD;
                return;
            }

            if (p_callback == (OS_TMR_CALLBACK_PTR)0) {
                *p_err = OS_ERR_TMR_INVALID_CALLBACK;
                return;
            }
            break;

        case OS_OPT_TMR_ONE_SHOT:
            if (dly == 0u) {
                *p_err = OS_ERR_TMR_INVALID_DLY;
                return;
            }
            break;

        default:
            *p_err = OS_ERR_OPT_INVALID;
            return;
    }
#endif
```

```

/* 判断 μC/OS-III 系统是否运行 */
if (OSRunning == OS_STATE_OS_RUNNING) {
    /* 如果 μC/OS-III 系统已经运行,
     * 那么就要获取软件定时器的互斥信号量
     * 以此保护相关的共享资源
     */
    OS_TmrLock();
}

/* 此宏用于使能检查内核对象类型 */
#if (OS_OBJ_TYPE_REQ > 0u)
#if (OS_CFG_OBJ_CREATED_CHK_EN > 0u)
/* 通过对象类型是否已经被设置
 * 判断软件定时器是否已经被创建
 */
if (p_tmr->Type == OS_OBJ_TYPE_TMR) {
    if (OSRunning == OS_STATE_OS_RUNNING) {
        OS_TmrUnlock();
    }
    *p_err = OS_ERR_OBJ_CREATED;
    return;
}
#endif
#endif

/* 对象类型设置为 OS_OBJ_TYPE_TMR */
p_tmr->Type = OS_OBJ_TYPE_TMR;
#endif

/* 刚创建的软件定时器还没有被启动
 * 因此处于停止态
 */
p_tmr->State = OS_TMR_STATE_STOPPED;

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
/* 如果使能了代码调试功能,
 * 则需要设置软件定时器的名称,
 * 方便调试
 */
p_tmr->NamePtr = p_name;
#else
(void)p_name;
#endif
#endif

```

```

/* 启动延时时间（需要乘上相应的系数） */
p_tmr->Dly          = dly * OSTmrToTicksMult;
/* 剩余超时时间（还未启动） */
p_tmr->Remain       = 0u;
/* 定时周期时间（需要乘上相应的系数） */
p_tmr->Period       = period * OSTmrToTicksMult;
/* 操作选项（单次定时器还是周期定时器） */
p_tmr->Opt          = opt;
/* 超时回调函数 */
p_tmr->CallbackPtr   = p_callback;
/* 超时回调函数参数 */
p_tmr->CallbackPtrArg = p_callback_arg;
/* 指向软件定时器链表中的下一个软件定时器 */
p_tmr->NextPtr       = (OS_TMR *) 0;
/* 指向软件定时器链表中的上一个软件定时器 */
p_tmr->PrevPtr       = (OS_TMR *) 0;

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
    OS_TmrDbgListAdd(p_tmr);
#endif
#if (OS_CFG_DBG_EN > 0u)
    OSTmrQty++;
#endif

/* 判断 μC/OS-III 系统是否运行 */
if (OSRunning == OS_STATE_OS_RUNNING) {
    /* 前面在 μC/OS-III 系统运行的情况下获取了互斥信号量，
     * 这里创建好软件定时器后要释放互斥信号量
     */
    OS_TmrUnlock();
}

/* 无错误 */
*p_err = OS_ERR_NONE;
}

```

2. 函数 OSTmrDel()

该函数用于删除一个软件定时器。

函数 OSTmrDel() 的函数原型，如下所示：

```

CPU_BOOLEAN OSTmrDel( OS_TMR* p_tmr,
                     OS_ERR* p_err)

```

函数 OSTmrDel() 的形参描述，如下表所示：

形参	描述
p_tmr	指向软件定时器结构体的指针
p_err	指向接收错误代码变量的指针

表 17.2.5 函数 OSTmrDel()形参相关描述

函数 OSTmrDel()的返回值描述，如下表所示：

返回值	描述
CPU_BOOLEAN 类型返回值	软件定时器删除是否成功

表 17.2.6 函数 OSTmrDel()返回值相关描述

函数 OSTmrDel()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功创删除一个软件定时器
OS_ERR_ILLEGAL_DEL_RUN_TIME	在系统运行过程中非法删除内核对象
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是软件定时器
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_TMR_INACTIVE	该软件定时器已被删除
OS_ERR_TMR_INVALID	指向软件定时器结构体的指针为空
OS_ERR_TMR_INVALID_STATE	软件定时器处于无效状态
OS_ERR_TMR_ISR	在中断中非法调用该函数

表 17.2.7 函数 OSTmrDel()错误代码相关描述

函数 OSTmrDel()定义在文件 os_tmr.c 中，具体的代码如下所示：

```
CPU_BOOLEAN OSTmrDel( OS_TMR* p_tmr,
                      OS_ERR* p_err)
{
    CPU_BOOLEAN success;
    OS_TICK      time;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        *p_err = OS_ERR_ILLEGAL_DEL_RUN_TIME;
        return (OS_FALSE);
    }
#endif
}
```

```

/* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_TMR_ISR;
        return (OS_FALSE);
    }
#endif

/* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (OS_FALSE);
    }
#endif

/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_tmr == (OS_TMR *)0) {
        *p_err = OS_ERR_TMR_INVALID;
        return (OS_FALSE);
    }
#endif

/* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_tmr->Type != OS_OBJ_TYPE_TMR) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (OS_FALSE);
    }
#endif

/* 获取软件定时器功能的互斥信号量 */
OS_TmrLock();

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 如果软件定时器任务正在处理软件定时器，
 * 则获取软件定时器任务开始处理软件定时器时的时间戳
 * 否则获取当前的时间戳
 * 这个时间戳用于更新软件定时器链表中其他软件定时器的成员变量 Remain
 */

```



```

    if (OSTCBCurPtr == &OSTmrTaskTCB) {
        time = OSTmrTaskTickBase;
    } else {
#if (OS_CFG_DYN_TICK_EN > 0u)
        time = OSTickCtr + OS_DynTickGet();
#else
        time = OSTickCtr;
#endif
    }

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();

    /* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
    OS_TmrDbgListRemove(p_tmr);
#endif

    /* 根据软件定时器的不同状态进行处理 */
    switch (p_tmr->State) {
        /* 运行态或超时态 */
        case OS_TMR_STATE_RUNNING:
        case OS_TMR_STATE_TIMEOUT:
            /* 将该软件定时器从软件定时器链表中移除 */
            OS_TmrUnlink(p_tmr, time);

            /* 去初始化软件定时器 */
            OS_TmrClr(p_tmr);

#if (OS_CFG_DBG_EN > 0u)
            OSTmrQty--;
#endif

            *p_err = OS_ERR_NONE;
            success = OS_TRUE;
            break;

        /* 停止态或完成态 */
        case OS_TMR_STATE_STOPPED:
        case OS_TMR_STATE_COMPLETED:
            /* 去初始化软件定时器 */
            OS_TmrClr(p_tmr);
    }

#if (OS_CFG_DBG_EN > 0u)

```

```

        OSTmrQty--;
#endif

        *p_err = OS_ERR_NONE;
        success = OS_TRUE;
        break;

        /* 未使用态 */
        case OS_TMR_STATE_UNUSED:
            *p_err = OS_ERR_TMR_INACTIVE;
            success = OS_FALSE;
            break;

        /* 无效的软件定时器状态 */
        default:
            *p_err = OS_ERR_TMR_INVALID_STATE;
            success = OS_FALSE;
            break;
    }

    /* 释放软件定时器功能的互斥信号量 */
    OS_TmrUnlock();

    return (success);
}

```

3. 函数 OSTmrRemainGet()

该函数用于获取软件定时器的剩余超时时间。

函数 OSTmrRemainGet()的函数原型，如下所示：

```

OS_TICK OSTmrRemainGet(OS_TMR* p_tmr,
                       OS_ERR* p_err)

```

函数 OSTmrRemainGet()的形参描述，如下表所示：

形参	描述
p_tmr	指向软件定时器结构体的指针
p_err	指向接收错误代码变量的指针

表 17.2.8 函数 OSTmrRemainGet()形参相关描述

函数 OSTmrRemainGet()的返回值描述，如下表所示：

返回值	描述
OS_TICK 类型返回值	软件定时器的剩余超时时钟节拍数

表 17.2.9 函数 OSTmrRemainGet()返回值相关描述

函数 OSTmrRemainGet()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功获取软件定时器的剩余超时时间
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是软件定时器
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行

OS_ERR_TMR_INACTIVE	该软件定时器已被删除
OS_ERR_TMR_INVALID	指向软件定时器结构体的指针为空
OS_ERR_TMR_INVALID_STATE	无效的软件定时器状态
OS_ERR_TMR_ISR	在中断中非法调用该函数

表 17.2.10 函数 OSTmrRemainGet()错误代码相关描述

函数 OSTmrRemainGet()定义在文件 os_tmr.c 中，具体的代码如下所示：

```
OS_TICK OSTmrRemainGet(OS_TMR* p_tmr,
                        OS_ERR* p_err)
{
    OS_TMR* p_tmrl;
    OS_TICK remain;

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (0u);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_TMR_ISR;
            return (0u);
        }
    #endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
        if (OSRunning != OS_STATE_OS_RUNNING) {
            *p_err = OS_ERR_OS_NOT_RUNNING;
            return (0u);
        }
    #endif

    /* 此宏用于使能参数检查 */
    if (OS_CFG_ARG_CHK_EN > 0u)
        if (p_tmr == (OS_TMR *)0) {
            *p_err = OS_ERR_TMR_INVALID;
            return (0u);
        }
    #endif
}
```

```

/* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_tmr->Type != OS_OBJ_TYPE_TMR) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (0u);
    }
#endif

/* 获取软件定时器功能的互斥信号量 */
OS_TmrLock();

/* 根据软件定时器的不同状态进行处理 */
switch (p_tmr->State) {
    /* 运行态 */
    case OS_TMR_STATE_RUNNING:
        /* 获取软件定时器链表 */
        p_tmrl = OSTmrListPtr;
        remain = 0u;
        /* 遍历软件定时器链表中的软件定时器
         * 将指定软件定时器包括之前的所有软件定时器的成员变量 Remain 求和,
         * 便可得到指定软件定时器的实际剩余超时时间
         */
        while (p_tmrl != (OS_TMR *)0) {
            remain += p_tmrl->Remain;
            if (p_tmrl == p_tmr) {
                break;
            }
            p_tmrl = p_tmrl->NextPtr;
        }
        /* 将剩余超时时间转为系统时钟节拍为单位 */
        remain /= OSTmrToTicksMult;
        *p_err = OS_ERR_NONE;
        break;

    /* 停止态 */
    case OS_TMR_STATE_STOPPED:
        if (p_tmr->Opt == OS_OPT_TMR_PERIODIC) {
            /* 周期定时器的总超时时间需要成员变量 Dly 决定
             * 如果设置了 Dly,
             * 那么周期定时器第一次超时的总时间就为 Dly 的值,
             * 否则就是成员变量 Period 的值,
             * 但是要转换成以系统时钟节拍为单位
            */

```

```

        */
        if (p_tmr->Dly == 0u) {
            remain = p_tmr->Period / OSTmrToTicksMult;
        } else {
            remain = p_tmr->Dly / OSTmrToTicksMult;
        }
    } else {
        /* 单次定时器的总超时时间就是成员变量 Dly 的值
        * 但是要转换成以系统时钟节拍为单位
        */
        remain = p_tmr->Dly / OSTmrToTicksMult;
    }
    *p_err = OS_ERR_NONE;
    break;

/* 超时态或完成态 */
case OS_TMR_STATE_TIMEOUT:
case OS_TMR_STATE_COMPLETED:
    /* 剩余超时时间为 0 */
    *p_err = OS_ERR_NONE;
    remain = 0u;
    break;

/* 未使用态 */
case OS_TMR_STATE_UNUSED:
    *p_err = OS_ERR_TMR_INACTIVE;
    remain = 0u;
    break;

/* 无效的软件定时器状态 */
default:
    *p_err = OS_ERR_TMR_INVALID_STATE;
    remain = 0u;
    break;
}

/* 释放软件定时器功能的互斥信号量 */
OSTmrUnlock();

return (remain);
}

```

4. 函数 OSTmrSet()

该函数用于设置软件定时器的延时时间、周期、超时回调函数及其参数等。

函数 OSTmrSet()的函数原型，如下所示：

```
void OSTmrSet( OS_TMR*      p_tmr,
               OS_TICK      dly,
               OS_TICK      period,
               OS_TMR_CALLBACK_PTR p_callback,
               void*         p_callback_arg,
               OS_ERR*       p_err)
```

函数 OSTmrSet()的形参描述，如下表所示：

形参	描述
p_tmr	指向软件定时器结构体的指针
dly	软件定时器的延时启动时间
period	周期定时器的定时周期时间
p_callback	指向软件定时器超时回调函数的指针
p_callback_arg	指向软件定时器超时回调函数参数的指针
p_err	指向接收错误代码变量的指针

表 17.2.11 函数 OSTmrSet()形参相关描述

函数 OSTmrSet()的返回值描述，如下表所示：

返回值	描述
无	无

表 17.2.12 函数 OSTmrSet()返回值相关描述

函数 OSTmrSet()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	设置软件定时器参数成功
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是软件定时器
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_TMR_INVALID	指向软件定时器结构体的指针为空
OS_ERR_TMR_INVALID_CALLBACK	无效的软件定时器超时回调函数指针
OS_ERR_TMR_INVALID_DLY	无效的软件定时器延时启动时间
OS_ERR_TMR_INVALID_PERIOD	无效的周期定时器定时周期时间
OS_ERR_TMR_ISR	在中断中非法调用该函数

表 17.2.13 函数 OSTmrSet()错误代码相关描述

函数 OSTmrSet()定义在文件 os_tmr.c 中，具体的代码如下所示：

```
void OSTmrSet( OS_TMR*      p_tmr,
               OS_TICK      dly,
               OS_TICK      period,
               OS_TMR_CALLBACK_PTR p_callback,
               void*         p_callback_arg,
               OS_ERR*       p_err)
{
    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
    }
#endif
}
```

```
        return;
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_TMR_ISR;
        return;
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return;
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_tmr == (OS_TMR *)0) {
        *p_err = OS_ERR_TMR_INVALID;
        return;
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_tmr->Type != OS_OBJ_TYPE_TMR) {
        *p_err = OS_ERR_OBJ_TYPE;
        return;
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    switch (p_tmr->Opt) {
        case OS_OPT_TMR_PERIODIC:
            if (period == 0u) {
                *p_err = OS_ERR_TMR_INVALID_PERIOD;
                return;
            }
    }
```

```

    }

    if (p_callback == (OS_TMR_CALLBACK_PTR)0) {
        *p_err = OS_ERR_TMR_INVALID_CALLBACK;
        return;
    }
    break;

case OS_OPT_TMR_ONE_SHOT:
    if (dly == 0u) {
        *p_err = OS_ERR_TMR_INVALID_DLY;
        return;
    }
    break;

default:
    *p_err = OS_ERR_TMR_INVALID;
    return;
}
#endif

/* 获取软件定时器功能的互斥信号量 */
OS_TmrLock();

/* 启动延时时间（需要乘上相应的系数） */
p_tmr->Dly = dly * OSTmrToTicksMult;
/* 定时周期时间（需要乘上相应的系数） */
p_tmr->Period = period * OSTmrToTicksMult;
/* 超时回调函数 */
p_tmr->CallbackPtr = p_callback;
/* 超时回调函数参数 */
p_tmr->CallbackPtrArg = p_callback_arg;

*p_err = OS_ERR_NONE;

/* 释放软件定时器功能的互斥信号量 */
OS_TmrUnlock();
}

```

5. 函数 OSTmrStart()

该函数用于开启软件定时器定时。

函数 OSTmrStart() 的函数原型，如下所示：

```

CPU_BOOLEAN OSTmrStart(OS_TMR* p_tmr,
                       OS_ERR* p_err)

```


函数 OSTmrSet() 的形参描述，如下表所示：

形参	描述
p_tmr	指向软件定时器结构体的指针
p_err	指向接收错误代码变量的指针

表 17.2.14 函数 OSTmrStart() 形参相关描述

函数 OSTmrStart() 的返回值描述，如下表所示：

返回值	描述
CPU_BOOLEAN 类型返回值	软件定时器开启是否成功

表 17.2.15 函数 OSTmrStart() 返回值相关描述

函数 OSTmrStart() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	软件定时器开启成功
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是软件定时器
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_TMR_INACTIVE	该软件定时器已被删除
OS_ERR_TMR_INVALID	指向软件定时器结构体的指针为空
OS_ERR_TMR_INVALID_STATE	无效的软件定时器状态
OS_ERR_TMR_ISR	在中断中非法调用该函数

表 17.2.16 函数 OSTmrStart() 错误代码相关描述

函数 OSTmrStart() 定义在文件 os_tmr.c 中，具体的代码如下所示：

```
CPU_BOOLEAN OSTmrStart(OS_TMR* p_tmr,
                        OS_ERR* p_err)
{
    CPU_BOOLEAN success;
    OS_TICK      time;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_TMR_ISR;
            return (OS_FALSE);
        }
    #endif
}
```

```

/* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (OS_FALSE);
    }
#endif

/* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_tmr == (OS_TMR *)0) {
        *p_err = OS_ERR_TMR_INVALID;
        return (OS_FALSE);
    }
#endif

/* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_tmr->Type != OS_OBJ_TYPE_TMR) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (OS_FALSE);
    }
#endif

/* 获取软件定时器功能的互斥信号量 */
OSTmrLock();

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 如果软件定时器任务正在处理软件定时器，
 * 则获取软件定时器任务开始处理软件定时器时的时间戳
 * 否则获取当前的时间戳
 * 这个时间戳用于更新软件定时器链表中其他软件定时器的成员变量 Remain
 */
if (OSTCBCurPtr == &OSTmrTaskTCB) {
    time = OSTmrTaskTickBase;
} else {
#if (OS_CFG_DYN_TICK_EN > 0u)
    time = OSTickCtr + OS_DynTickGet();
#else
    time = OSTickCtr;
#endif
}

```

```
}

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 根据软件定时器的不同状态进行处理 */
switch (p_tmr->State) {
    /* 运行态或超时态 */
    case OS_TMR_STATE_RUNNING:
    case OS_TMR_STATE_TIMEOUT:
        /* 设置软件定时器为运行态 */
        p_tmr->State = OS_TMR_STATE_RUNNING;

        /* 重新开启软件定时器的定时 */
        OS_TmrUnlink(p_tmr, time);

        if (p_tmr->Dly == 0u) {
            p_tmr->Remain = p_tmr->Period;
        } else {
            p_tmr->Remain = p_tmr->Dly;
        }
        OS_TmrLink(p_tmr, time);
        *p_err = OS_ERR_NONE;
        success = OS_TRUE;
        break;

    /* 停止态或完成态 */
    case OS_TMR_STATE_STOPPED:
    case OS_TMR_STATE_COMPLETED:
        /* 设置软件定时器为运行态 */
        p_tmr->State = OS_TMR_STATE_RUNNING;

        /* 开启软件定时器的定时 */
        if (p_tmr->Dly == 0u) {
            p_tmr->Remain = p_tmr->Period;
        } else {
            p_tmr->Remain = p_tmr->Dly;
        }
        OS_TmrLink(p_tmr, time);
        *p_err = OS_ERR_NONE;
        success = OS_TRUE;
        break;
}
```

```

    /* 未使用态 */
    case OS_TMR_STATE_UNUSED:
        *p_err = OS_ERR_TMR_INACTIVE;
        success = OS_FALSE;
        break;

    /* 无效的软件定时器状态 */
    default:
        *p_err = OS_ERR_TMR_INVALID_STATE;
        success = OS_FALSE;
        break;
}

/* 释放软件定时器功能的互斥信号量 */
OSTmrUnlock();

return (success);
}

```

6. 函数 OSTmrStateGet()

该函数用于获取软件定时器的状态。

函数 OSTmrStateGet()的函数原型，如下所示：

```

OS_STATE OSTmrStateGet(OS_TMR* p_tmr,
                       OS_ERR* p_err)

```

函数 OSTmrStateGet()的形参描述，如下表所示：

形参	描述
p_tmr	指向软件定时器结构体的指针
p_err	指向接收错误代码变量的指针

表 17.2.17 函数 OSTmrStateGet()形参相关描述

函数 OSTmrStateGet()的返回值描述，如下表所示：

返回值	描述
OS_STATE 类型返回值	软件定时器的状态

表 17.2.18 函数 OSTmrStateGet()返回值相关描述

函数 OSTmrStateGet()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	获取软件定时器的状态成功
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是软件定时器
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_TMR_INVALID	指向软件定时器结构体的指针为空
OS_ERR_TMR_INVALID_STATE	无效的软件定时器状态
OS_ERR_TMR_ISR	在中断中非法调用该函数

表 17.2.19 函数 OSTmrStateGet()错误代码相关描述

函数 OSTmrStateGet()定义在文件 os_tmr.c 中，具体的代码如下所示：

```

OS_STATE OSTmrStateGet(OS_TMR* p_tmr,

```

```

        OS_ERR* p_err)
{
    OS_STATE state;

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (OS_TMR_STATE_UNUSED);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_TMR_ISR;
        return (OS_TMR_STATE_UNUSED);
    }
#endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
#if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (OS_TMR_STATE_UNUSED);
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_tmr == (OS_TMR *)0) {
        *p_err = OS_ERR_TMR_INVALID;
        return (OS_TMR_STATE_UNUSED);
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_tmr->Type != OS_OBJ_TYPE_TMR) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (OS_TMR_STATE_UNUSED);
    }
#endif
}

```

```

/* 获取软件定时器功能的互斥信号量 */
OS_TmrLock();

/* 获取软件定时器的状态 */
state = p_tmr->State;

/* 判断软件定时器的状态是否有效 */
switch (state) {
    case OS_TMR_STATE_UNUSED:
    case OS_TMR_STATE_STOPPED:
    case OS_TMR_STATE_COMPLETED:
    case OS_TMR_STATE_RUNNING:
    case OS_TMR_STATE_TIMEOUT:
        *p_err = OS_ERR_NONE;
        break;

    default:
        *p_err = OS_ERR_TMR_INVALID_STATE;
        break;
}

/* 释放软件定时器功能的互斥信号量 */
OS_TmrUnlock();

return (state);
}

```

7. 函数 OSTmrStop()

该函数用于停止软件定时器定时。

函数 OSTmrStop()的函数原型，如下所示：

```

CPU_BOOLEAN OSTmrStop( OS_TMR* p_tmr,
                        OS_OPT  opt,
                        void*    p_callback_arg,
                        OS_ERR* p_err)

```

函数 OSTmrStop()的形参描述，如下表所示：

形参	描述
p_tmr	指向软件定时器结构体的指针
opt	函数操作选项
p_callback_arg	指向传给软件定时器超时回调函数的参数的指针
p_err	指向接收错误代码变量的指针

表 17.2.20 函数 OSTmrStop()形参相关描述

函数 OSTmrStop()的返回值描述，如下表所示：

返回值	描述
-----	----

CPU_BOOLEAN 类型返回值	停止软件定时器是否成功
-------------------	-------------

表 17.2.21 函数 OSTmrStop()返回值相关描述

函数 OSTmrStop()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	停止软件定时器成功
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是软件定时器
OS_ERR_OPT_INVALID	无效的函数操作选项
OS_ERR_OS_NOT_RUNNING	μC/OS-III 内核还未运行
OS_ERR_TMR_INVALID	指向软件定时器结构体的指针为空
OS_ERR_TMR_INVALID_STATE	无效的软件定时器状态
OS_ERR_TMR_ISR	在中断中非法调用该函数
OS_ERR_TMR_NO_CALLBACK	软件定时器没有超时回调函数
OS_ERR_TMR_STOPPED	软件定时器当前已经停止

表 17.2.22 函数 OSTmrStop()错误代码相关描述

函数 OSTmrStop()定义在文件 os_tmr.c 中，具体的代码如下所示：

```

CPU_BOOLEAN OSTmrStop( OS_TMR* p_tmr,
                       OS_OPT opt,
                       void* p_callback_arg,
                       OS_ERR* p_err)
{
    OS_TMR_CALLBACK_PTR p_fnct;
    CPU_BOOLEAN          success;
    OS_TICK              time;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
    if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
        if (OSIntNestingCtr > 0u) {
            *p_err = OS_ERR_TMR_ISR;
            return (OS_FALSE);
        }
    endif

    /* 此宏用于使能检查 μC/OS-III 系统是否运行 */
    if (OS_CFG_INVALID_OS_CALLS_CHK_EN > 0u)
    
```

```

    if (OSRunning != OS_STATE_OS_RUNNING) {
        *p_err = OS_ERR_OS_NOT_RUNNING;
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_tmr == (OS_TMR *)0) {
        *p_err = OS_ERR_TMR_INVALID;
        return (OS_FALSE);
    }
#endif

    /* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_tmr->Type != OS_OBJ_TYPE_TMR) {
        *p_err = OS_ERR_OBJ_TYPE;
        return (OS_FALSE);
    }
#endif

    /* 获取软件定时器功能的互斥信号量 */
    OSTmrLock();

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 如果软件定时器任务正在处理软件定时器，
     * 则获取软件定时器任务开始处理软件定时器时的时间戳
     * 否则获取当前的时间戳
     * 这个时间戳用于更新软件定时器链表中其他软件定时器的成员变量 Remain
     */
    if (OSTCBCurPtr == &OSTmrTaskTCB) {
        time = OSTmrTaskTickBase;
    } else {
#if (OS_CFG_DYN_TICK_EN > 0u)
        time = OSTickCtr + OSDynTickGet();
#else
        time = OSTickCtr;
#endif
    }
}

```



```
/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 根据软件定时器的不同状态进行处理 */
switch (p_tmr->State) {
    /* 运行态或超时态 */
    case OS_TMR_STATE_RUNNING:
    case OS_TMR_STATE_TIMEOUT:
        /* 将软件定时器设置为停止态 */
        p_tmr->State = OS_TMR_STATE_STOPPED;

/* 柑橘函数操作选项进行处理 */
switch (opt) {
    /* 执行软件定时器的超时回调函数，
     * 并传入该软件定时器的超时回调函数参数
     */
    case OS_OPT_TMR_CALLBACK:
        OS_TmrUnlink(p_tmr, time);
        p_fnct = p_tmr->CallbackPtr;
        if (p_fnct != (OS_TMR_CALLBACK_PTR)0) {
            (*p_fnct)(p_tmr, p_tmr->CallbackPtrArg);
        } else {
            *p_err = OS_ERR_TMR_NO_CALLBACK;
        }
        break;

    /* 执行软件定时器的超时回调函数，
     * 并传入指定的软件定时器超时回调函数参数
     */
    case OS_OPT_TMR_CALLBACK_ARG:
        OS_TmrUnlink(p_tmr, time);
        p_fnct = p_tmr->CallbackPtr;
        if (p_fnct != (OS_TMR_CALLBACK_PTR)0) {
            (*p_fnct)(p_tmr, p_callback_arg);
        } else {
            *p_err = OS_ERR_TMR_NO_CALLBACK;
        }
        break;

    /* 仅停止软件定时器 */
    case OS_OPT_TMR_NONE:
        OS_TmrUnlink(p_tmr, time);
        break;
```

```

        /* 无效的函数操作选项 */
        default:
            OS_TmrUnlock();
            *p_err = OS_ERR_OPT_INVALID;
            return (OS_FALSE);
    }
    *p_err = OS_ERR_NONE;
    success = OS_TRUE;
    break;

/* 停止态或完成态 */
case OS_TMR_STATE_COMPLETED:
case OS_TMR_STATE_STOPPED:
    /* 将软件定时器设置为停止态 */
    p_tmr->State = OS_TMR_STATE_STOPPED;
    *p_err = OS_ERR_TMR_STOPPED;
    success = OS_TRUE;
    break;

/* 未使用态 */
case OS_TMR_STATE_UNUSED:
    *p_err = OS_ERR_TMR_INACTIVE;
    success = OS_FALSE;
    break;

/* 无效的软件定时器状态 */
default:
    *p_err = OS_ERR_TMR_INVALID_STATE;
    success = OS_FALSE;
    break;
}

/* 释放软件定时器功能的互斥信号量 */
OS_TmrUnlock();

return (success);
}

```

17.3 μC/OS-III 软件定时器实验

17.3.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数使用软件定时器，本实验设计了两个任务，这两个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建软件定时器和其他任务
task1	用于扫描按键，并开启和停止软件定时器

表 17.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 17 UCOS-III 软件定时器实验》。

17.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

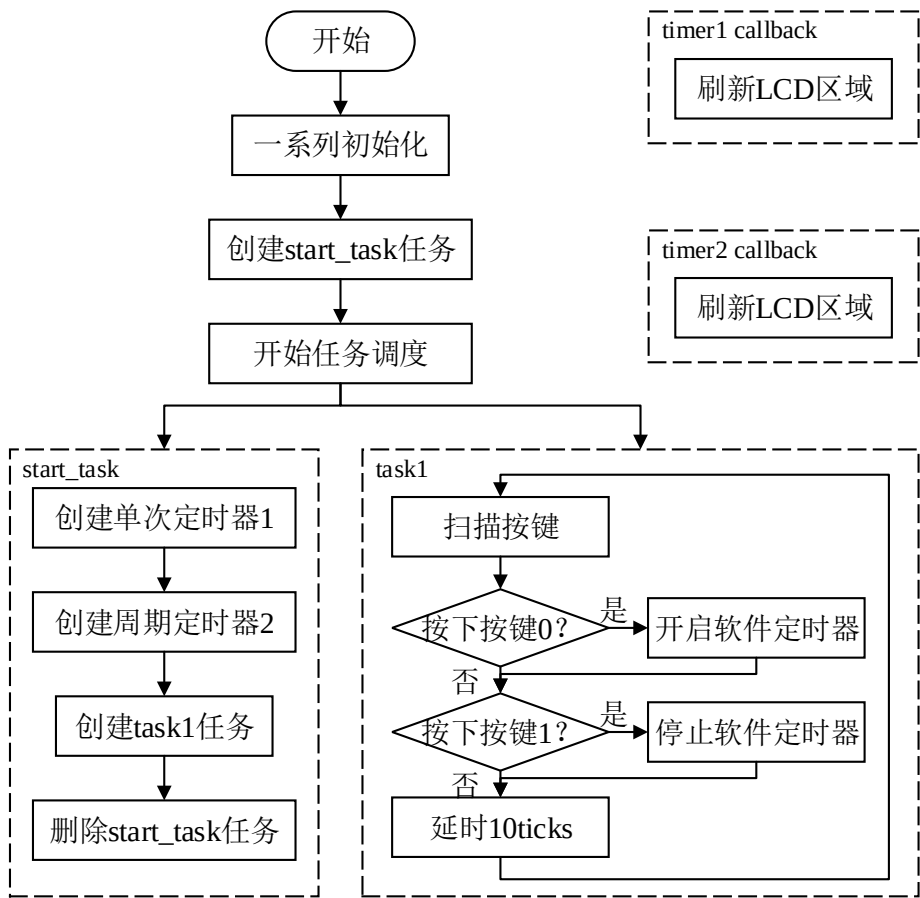


图 17.3.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSTmrCreate()

此函数用于创建软件定时器，请参考第 17.2 小节《μC/OS-III 软件定时器相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_TMR_EN 配置为 1。

(2) 函数 OSTmrStart()

此函数用于开启软件定时器，请参考第 17.2 小节《μC/OS-III 软件定时器相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_TMR_EN 配置为 1。

(3) 函数 OSTmrStop()

此函数用于停止软件定时器，请参考第 17.2 小节《μC/OS-III 软件定时器相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_TMR_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief   start_task
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建软件定时器 1 */
    OSTmrCreate((OS_TMR* )&Timer1,
                (CPU_CHAR* )"Timer1",
                (OS_TICK )10,
                (OS_TICK )0,
                (OS_OPT )OS_OPT_TMR_ONE_SHOT,
                (OS_TMR_CALLBACK_PTR )timer_cb,
                (void* )0,
                (OS_ERR* )&err);

    /* 创建软件定时器 2 */
    OSTmrCreate((OS_TMR* )&Timer2,
                (CPU_CHAR* )"Timer2",
                (OS_TICK )0,
                (OS_TICK )10,
```

```

        (OS_OPT                ) OS_OPT_TMR_PERIODIC,
        (OS_TMR_CALLBACK_PTR    ) timer_cb,
        (void*                  ) 0,
        (OS_ERR*                ) &err);

/* 创建 Task1 */
Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                       TASK1_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*        ) &Task1Task_TCB,
                 (CPU_CHAR*      ) "task1",
                 (OS_TASK_PTR    ) task1,
                 (void*          ) 0,
                 (OS_PRIO        ) TASK1_PRIO,
                 (CPU_STK        *) Task1Task_STK,
                 (CPU_STK_SIZE   ) TASK1_STK_SIZE / 10,
                 (CPU_STK_SIZE   ) TASK1_STK_SIZE,
                 (OS_MSG_QTY     ) 0,
                 (OS_TICK        ) 0,
                 (void*          ) 0,
                 (OS_OPT         ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR*        ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务，并且 task1 任务的栈都是动态分配的，当然，也可以静态地定任务栈，就像定义一个数组一样。同时还创建一个单次定时器 1 和一个周期定时器 2。

(2) task1 任务

```

/**
 * @brief   task1
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void task1(void *p_arg)
{
    uint8_t key;
    OS_ERR  err;

    while (1)
    {
        key = key_scan(0);
    }
}

```

```

switch (key)
{
    case KEY0_PRES:                /* 启动软件定时器 */
    {
        OSTmrStart(&Timer1, &err);
        OSTmrStart(&Timer2, &err);
        break;
    }
    case KEY1_PRES:                /* 停止软件定时器 */
    {
        OSTmrStop(&Timer1, OS_OPT_TMR_NONE, NULL, &err);
        OSTmrStop(&Timer2, OS_OPT_TMR_NONE, NULL, &err);
        break;
    }
    default:
    {
        break;
    }
}

OSTimeDly(10, OS_OPT_TIME_DLY, &err);
}
}

```

可以看到 task1 任务就是用于扫描按键，当检测到按键 0 被按下时，同时开启单次定时器 1 和周期定时器 2，当检测到按键 1 被按下时，同时停止单次定时器 1 和周期定时器 2。

(3) 软件定时器超时回调函数

```

/**
 * @brief   timer_cb
 * @param   p_tmr : 传入参数(超时的软件定时器)
 * @param   p_arg : 传入参数(未用到)
 * @retval  无
 */
void timer_cb(void *p_tmr, void *p_arg)
{
    static uint32_t timer1_num = 0;
    static uint32_t timer2_num = 0;

    if (p_tmr == &Timer1)
    {
        /* LCD 区域刷新 */
        lcd_fill(6, 131, 114, 313, lcd_discolor[++timer1_num % 11]);
        /* 显示定时器 1 超时次数 */
        lcd_show_xnum(79, 111, timer1_num, 3, 16, 0x80, BLUE);
    }
}

```

```

}
else if (p_tmr == &Timer2)
{
    /* LCD 区域刷新 */
    lcd_fill(126, 131, 233, 313, lcd_discolor[++timer2_num % 11]);
    /* 显示定时器 2 超时次数 */
    lcd_show_xnum(199, 111, timer2_num, 3, 16, 0x80, BLUE);
}
else
{
    /* 什么都不做 */
}
}

```

可以看到两个软件定时器共用一个软件定时器超时回调函数，并且都是对 LCD 屏幕进行区域刷新，并且会在 LCD 屏幕上显示对应软件定时器的超时次数。

17.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：

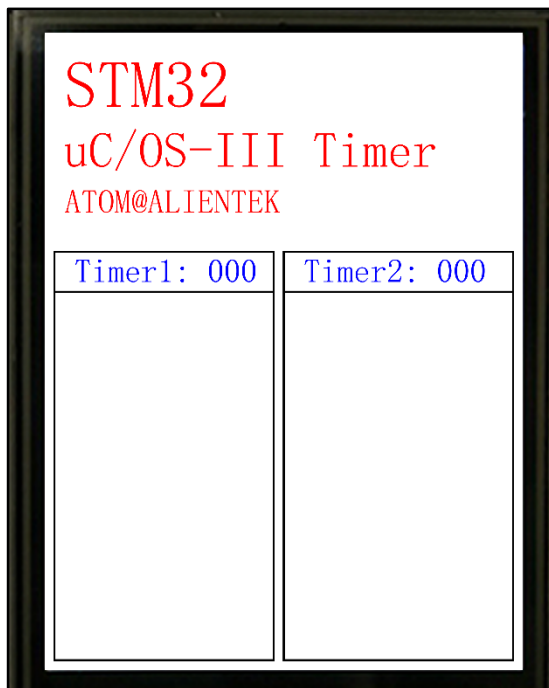


图 17.3.2 LCD 显示内容一

接着按一下按键 0 来启动软件定时器 1 和软件定时器 2，由于软件定时器 1 是单次定时器，软件定时器 2 是周期定时器，因此，在开启软件定时器后，软件定时器 1 只会超时一次，而软件定时器 2 会在定时超时后自动重启，这么一来就能看到 LCD 屏幕上软件定时器 1 对应的显示区域只会刷新一次，并且软件定时器的超时次数一直为 1，而软件定时器 2 对应的显示区域不断刷新，并且软件定时器 2 的超时次数也在一直增加。如下图所示：



图 17.3.3 LCD 显示内容二

多次按下按键 0，所有软件定时器均会在，按下按键的那一刻重新开始定时。

接着按下按键 1 来停止所有软件定时器，不论是周期定时器还是单次定时器，只要被停止了，就不会再继续定时，也就不会再定时超时，从按下按键 1 的那一刻起，软件定时器 1 和软件定时器 2 便不在继续定时，LCD 屏幕也不再刷新，因此所有软件定时器都不再超时了。

第十八章 μC/OS-III 时间戳

在前面的章节中，经常会发现在 μC/OS-III 的代码中经常出现时间戳的相关应用代码，其大多数的作用都是用来测量时间的，例如测量中断被屏蔽的时长或者是某段代码的执行时长，通过时间戳测量出来的时长，就能够判断代码执行的效率，从而为代码的优化提供依据。因此利用好时间戳功能能够极大地方便用户的开发。本章就来学习 μC/OS-III 中时间戳相关的内容。

本章分为如下几部分：

18.1 μC/OS-III 时间戳简介

18.2 μC/OS-III 时间戳相关 API 函数

18.3 μC/OS-III 时间戳实验

18.1 μC/OS-III 时间戳简介

在 μC/OS-III 的源代码中大量地使用到了使用宏定义开关的时间戳相关的代码，这些代码主要用于测量时长，例如测量中断被屏蔽的时长（这是一个对实时操作系统很重要的指标）、测量某段代码执行的时长等。当然，用户也可以在应用代码中使用时间戳的相关代码，以测量某段应用程序代码执行的时间，从而作为代码优化的依据。

在了解了时间戳的作用之后，便该考虑时间戳是一个什么东西呢？实际上，时间戳可以简单的理解为是一个时间点，例如要测量某段代码的执行时长，那么便要在这段代码的开始前获取一个时间点，命名为 Start，然后，同样的在代码段的末尾再获取一个时间点，命名为 End，那么这里的时间点 Start 和时间点 End 就都是一个时间戳，要测量代码的执行时长，是无需考虑 Start 和 End 这两个时间戳具体值的含义的，仅需要计算出 Start 和 End 之间的差值，便可以得到这段代码的执行时长。

时间戳就是一个数，这个数从时间戳功能初始化以来，就在时间戳功能时基定时器的驱动下不断增加，因此在不同的时间点获取到的时间戳都是不一样的。

18.1.1 μC/OS-III 时间戳功能时基定时器

既然是与时间相关的功能，那么肯定是需要由一个时基定时器驱动，μC/OS-III 的时间戳功能可以使用任意位数的硬件或软件定时器模拟 32 位或 64 位定时器作为时基定时器（时间戳的位数与时基定时器的位数是相同的，因为获取时间戳实际上就是获取时基定时器的计数值），但是最好使用与 32 位或 64 位时间戳相同或较大的位数的硬件或软件定时器，因为这样在获取时间戳的时候不会产生额外的计算开销。

在 ARM Cortex-M 内核中有一个 DWT（Data Watchpoint and Trace，数据观察点和跟踪）外设，在 DWT 外设中有一个 32 位的寄存器 CYCCNT（周期计数寄存器），这是一个 32 位的向上计数器，用于记录内核是时钟运行的个数，内核每过一个时钟，CYCCNT 寄存器的值就加 1，就算对于 STM32F1 系列 72MHz 的主频而言，其精度相对来说都是较高的了，如果是 STM32H7 这种高性能的 MCU 而言，普遍 400MHz+ 的主频，完全是可以为时间戳功能提供一个该精度的时基定时器，因此本教程的配套例程中，都是使用 ARM Cortex-M 的 DWT 外设为时间戳功能提供时基，本章教程也仅讲解该情况下的时间戳功能相关配置。

18.1.2 μC/OS-III 时间戳功能配置项

在第 3.3 小节《μC/CPU 配置项》中涉及了时间戳功能相关的三个配置项，如下表所示：

时间戳配置项	描述
CPU_CFG_TS_32_EN	使用 32 位的定时器作为时间戳功能的时基定时器
CPU_CFG_TS_64_EN	使用 64 位的定时器作为时间戳功能的时基定时器
CPU_CFG_TS_TMR_SIZE	定义时间戳的位数，不能小于时基定时器的位数

表 18.1.1 时间戳相关配置项

上表中的三个配置项就用于配置时间戳功能，因为本教程使用 ARM Cortex-M 的 DWT 外设为时间戳功能提供时基，因此配置的配置项如下所示：

```
#define CPU_CFG_TS_32_EN      DEF_ENABLED
#define CPU_CFG_TS_64_EN      DEF_DISABLED
#define CPU_CFG_TS_TMR_SIZE   CPU_WORD_SIZE_32
```

使用 32 为的定时器作为时间戳功能的时基定时器，同时使用 32 位的时间戳，这样在获取时间戳的时候，就无需额外的计算开销。

18.1.3 μC/OS-III 时间戳功能初始化

时间戳功能的初始化是在函数 CPU_Init()中完成的，函数 CPU_Init()定义在文件 cpu_core.c 中，具体的代码如下所示：

```
void CPU_Init(void)
{
    #if ((CPU_CFG_TS_EN == DEF_ENABLED) || (CPU_CFG_TS_TMR_EN == DEF_ENABLED))
        CPU_TS_Init();
    #endif

    /* 忽略其他代码 */
}
```

其中，函数 CPU_TS_Init()就是用于初始化时间戳功能的，同样地定义在文件 cpu_core.c 中，具体的代码如下所示：

```
static void CPU_TS_Init(void)
{
    /* 根据 18.1.2 小节的配置，此代码不被编译 */
    #if (((CPU_CFG_TS_32_EN == DEF_ENABLED) && \
        (CPU_CFG_TS_TMR_SIZE < CPU_WORD_SIZE_32)) || \
        ((CPU_CFG_TS_64_EN == DEF_ENABLED) && \
        (CPU_CFG_TS_TMR_SIZE < CPU_WORD_SIZE_64)))
        CPU_TS_TMR ts_tmr_cnts;
    #endif

    #if (CPU_CFG_TS_TMR_EN == DEF_ENABLED)
        /* 时间戳功能时基定时器的频率会在后边进行初始化 */
        CPU_TS_TmrFreq_Hz = 0u;

        /* 初始化时间戳功能时基定时器 */
        CPU_TS_TmrInit();
    #endif

    /* 根据 18.1.2 小节的配置，此代码不被编译 */
}
```

```
#if ((CPU_CFG_TS_32_EN == DEF_ENABLED) && \
    (CPU_CFG_TS_TMR_SIZE < CPU_WORD_SIZE_32)) || \
    ((CPU_CFG_TS_64_EN == DEF_ENABLED) && \
    (CPU_CFG_TS_TMR_SIZE < CPU_WORD_SIZE_64)))
    ts_tmr_cnts = CPU_TS_TmrRd();
#endif

/* 根据 18.1.2 小节的配置，此代码不被编译 */
#if ((CPU_CFG_TS_32_EN == DEF_ENABLED) && \
    (CPU_CFG_TS_TMR_SIZE < CPU_WORD_SIZE_32))
    CPU_TS_32_Accum = 0u;
    CPU_TS_32_TmrPrev = ts_tmr_cnts;
#endif

/* 根据 18.1.2 小节的配置，此代码不被编译 */
#if ((CPU_CFG_TS_64_EN == DEF_ENABLED) && \
    (CPU_CFG_TS_TMR_SIZE < CPU_WORD_SIZE_64))
    CPU_TS_64_Accum = 0u;
    CPU_TS_64_TmrPrev = ts_tmr_cnts;
#endif
}
```

从上面的代码中可以看出，函数 CPU_TS_Init() 主要是初始化了时间戳功能的时基定时器，初始化时间戳功能时基定时器调用了函数 CPU_TS_TmrInit()，该函数定义在文件 bsp_cpu.c 中，因为时间戳功能的时基定时器需要由用户自定配置，因此时间戳功能时基定时器的初始化代码需要根据实际的情况编写。

函数 CPU_TS_TmrInit() 主要完成时间戳功能时基定时器的初始化和启动并设置保存时间戳功能时基定时器频率的变量，本教程主要针对 DWT 为时间戳功能提供时基的情况，CPU_TS_TmrInit() 的完善的代码如下所示：

```
void CPU_TS_TmrInit(void)
{
    /* 使能跟踪系统，若要使用 DWT 就必须使能跟踪系统 */
    DEM_CR      |= (CPU_INT32U) DEM_CR_TRCENA;

    /* 清零 CYCCNT 寄存器 */
    DWT_CYCCNT  = (CPU_INT32U) 0u;

    /* 使能 CYCCNT 计数器 */
    DWT_CR      |= (CPU_INT32U) DWT_CR_CYCCNTENA;

    /* 设置时间戳功能时基定时器的频率（与内核频率相同） */
    CPU_TS_TmrFreqSet(HAL_RCC_GetSysClockFreq());
}
```

同时为了能够正常使用时间戳功能，还需要额外编写两个相关的函数，分别为函数 CPU_TS_TmrRd() 和函数 CPU_TS32_to_uSec()。

函数 CPU_TS_TmrRd() 用于获取时间戳功能时基定时器的计数值，那也就是或 CYCCNT 寄

寄存器的值，该函数定义在文件 `bsp_cpu.c` 中，具体的代码如下所示：

```
CPU_TS_TMR CPU_TS_TmrRd(void)
{
    return ((CPU_TS_TMR) DWT_CYCCNT);
}
```

函数 `CPU_TS32_to_uSec()` 用于将 32 位的时间戳数值转化为以微秒为单位的时间数值，该函数定义在文件 `bsp_cpu.c` 中，具体的代码如下所示：

```
CPU_INT64U CPU_TS32_to_uSec(CPU_TS32 ts_cnts)
{
    CPU_INT64U ts_us;

    ts_us = ts_cnts / (HAL_RCC_GetSysClockFreq() / DEF_TIME_NBR_uS_PER_SEC);

    return ts_us;
}
```

18.2 μC/OS-III 时间戳相关 API 函数

μC/OS-III 提供了一些与时间戳相关的 API 函数，在使用这些 API 函数之前，请确保正确配置好了时间戳，这些函数中常用的函数如下表所示：

函数	描述
<code>OS_TS_GET()</code>	获取时间戳
<code>CPU_TS32_to_uSec()</code>	将时间戳转换为以微秒为单位的时间数值

表 18.2.1 μC/OS-III 时间戳常用的相关 API 函数描述

1. 函数 `OS_TS_GET()`

该函数用于获取时间戳。

函数 `OS_TS_GET()` 实际上是一个宏定义，定义在文件 `os_cpu.h` 中，具体的代码如下所示：

```
#define OS_TS_GET() (CPU_TS) CPU_TS_TmrRd()
```

函数 `CPU_TS_TmrRd()` 请见第 18.1.3 小节《μC/OS-III 时间戳功能初始化》。

2. 函数 `CPU_TS32_to_uSec()`

该函数用于将时间戳转换为以微秒为单位的时间数值。

函数 `CPU_TS32_to_uSec()` 请见第 18.1.3 小节《μC/OS-III 时间戳功能初始化》。

18.3 μC/OS-III 时间戳实验

18.3.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数使用时间戳功能，本实验设计了两个任务，这两个任务的功能如下表所示：

任务名	任务功能描述
<code>start_task</code>	用于创建其他任务
<code>task1</code>	用于扫描按键，并调用时间戳相关的 API 函数

表 18.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 18 UCOS-III 时间戳实验》。

18.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

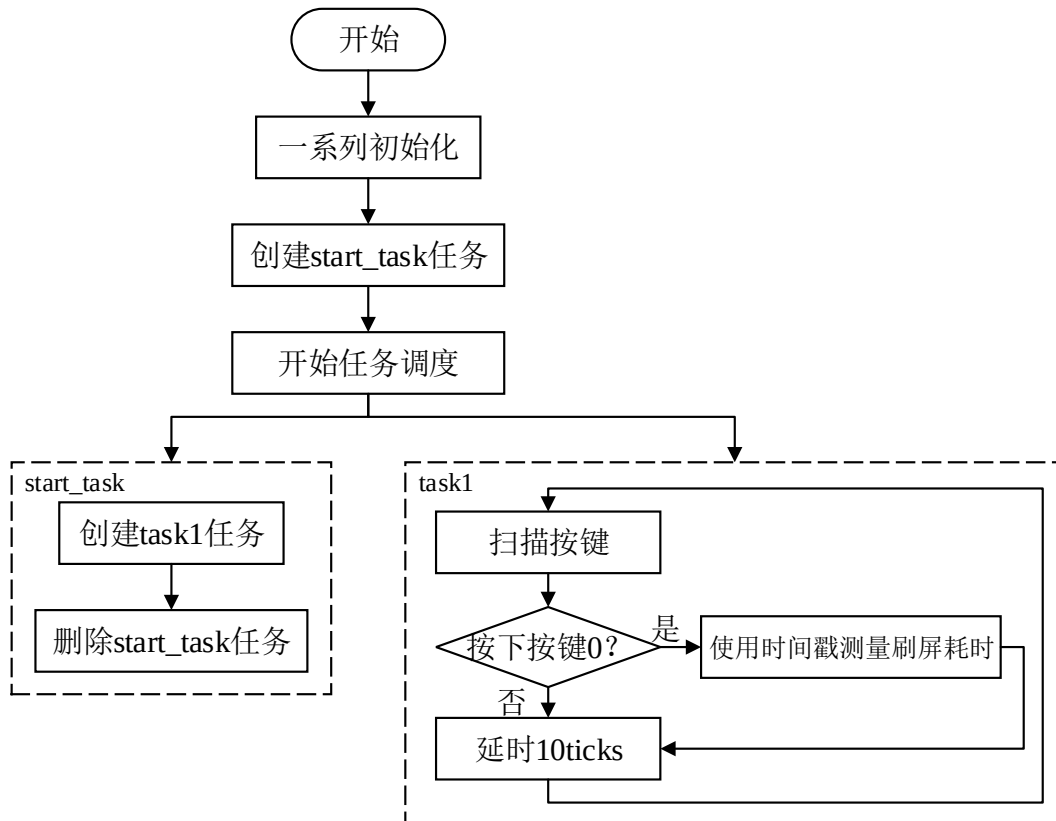


图 18.3.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OS_TS_GET()

此函数用于获取时间戳，请参考第 18.2 小节《μC/OS-III 时间戳相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_TS_EN 配置为 1。

(2) 函数 CPU_TS32_to_uSec()

此函数用于将时间戳转换为以微秒为单位的时间数值，请参考第 18.2 小节《μC/OS-III 时间戳相关 API 函数》。要使用此函数需将配置文件 cpu_cfg.h 文件中的配置项 CPU_CFG_TS_32_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```

/**
 * @brief start_task

```

```

* @param   p_arg : 传入参数(未用到)
* @retval  无
*/
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc(    SRAMIN,
                                           TASK1_STK_SIZE * sizeof(CPU_STK));

    OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                    (CPU_CHAR*    ) "task1",
                    (OS_TASK_PTR  ) task1,
                    (void*        ) 0,
                    (OS_PRIO      ) TASK1_PRIO,
                    (CPU_STK      *) Task1Task_STK,
                    (CPU_STK_SIZE) TASK1_STK_SIZE / 10,
                    (CPU_STK_SIZE) TASK1_STK_SIZE,
                    (OS_MSG_QTY   ) 0,
                    (OS_TICK      ) 0,
                    (void*        ) 0,
                    (OS_OPT       ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                    (OS_ERR*      ) &err);

    /* 删除 Start Task */
    OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务，并且 task1 任务的任务栈都是动态分配的，当然，也可以静态地定任务栈，就像定义一个数组一样。

(2) task1 任务

```

/**
* @brief   task1

```

```

* @param  p_arg : 传入参数(未用到)
* @retval 无
*/
void task1(void *p_arg)
{
    uint8_t      key;
    CPU_TS_TMR   start_ts, end_ts, delta_ts;
    CPU_INT64U   delta_us;
    uint32_t      task1_num = 0;
    OS_ERR       err;

    while (1)
    {
        key = key_scan(0);

        switch (key)
        {
            case KEY0_PRES:
            {
                start_ts = OS_TS_GET();
                lcd_fill(6, 131, 233, 313, lcd_discolor[++task1_num % 11]);
                end_ts = OS_TS_GET();
                delta_ts = end_ts - start_ts;
                delta_us = CPU_TS32_to_uSec(delta_ts);
                lcd_show_string(54, 111, 200, 16, 16, "Delta Us:      us", BLUE);
                lcd_show_num(134, 111, delta_us, 5, 16, BLUE);
            }
            default:
            {
                break;
            }
        }

        OSTimeDly(10, OS_OPT_TIME_DLY, &err);
    }
}

```

可以看到 `task1` 任务就是用于扫描按键，当检测到按键 0 被按下时，使用时间戳功能计算 LCD 刷屏的耗时时间，具体的做法就是在 LCD 刷屏代码的前后获取时间戳，然后计算两次获取到的时间戳的差值，然后就是单位的转换了。

18.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：



图 18.3.2 LCD 显示内容一

接下来按下按键 0，LCD 显示的内容如下所示：



图 18.3.3 LCD 显示内容二

可以看到，LCD 的区域刷新了，并且在 LCD 上显示了刷新 LCD 所消耗的时间为 12036 微秒（这个时间值，在不同的开发环境下，可能会有所差异）。

第十九章 μC/OS-III 内存管理

内存管理主要实现对内存的动态管理，包括内存的申请和释放操作。应用程序可以根据需求申请内存，当申请的内存不再使用时，可以将申请到的内存进行释放，相对于静态的内存分配而言，其大大地提高地对内存地利用率。本章就来学习 μC/OS-III 中内存管理相关的内容。

本章分为如下几部分：

19.1 μC/OS-III 内存管理简介

19.2 μC/OS-III 内存管理相关 API 函数

19.3 μC/OS-III 内存管理实验

19.1 μC/OS-III 内存管理简介

内存管理是指软件运行时对内存资源的分配和使用的一种计数，其最主要的目的就是能够高效且快速地分配，并且在释放的时候释放不再使用的内存空间。例如，电脑上编写 C 程序的时候，会使用标准 C 库中的函数 `malloc()` 和函数 `free()` 动态地进行内存的申请和释放。同样的在嵌入式开发中，编译器也为开发者提供了用于动态申请和释放内存的函数 `malloc()` 和函数 `free()`，但是一般不建议在嵌入式系统中使用函数 `malloc()` 和函数 `free()`，特别是在使用了实时操作系统的嵌入式开发中，因为函数 `malloc()` 和函数 `free()` 在申请和释放内存的时候，会导致产生大量的内存碎片，最后甚至可能出现明明还有大量的空闲内存，但是却无法再申请内存的情况，这是因为大量地内存碎片占据了空闲内存，这些内存碎片数量多、单个占用内存小、地址空间不连续，这将是无法再分配较大块的内存。

为此 μC/OS-III 提供了一个内存管理的方案，μC/OS-III 将一块大内存作为一个内存区，一个内存区中有多个大小均相同的内存块组成，如下图所示：

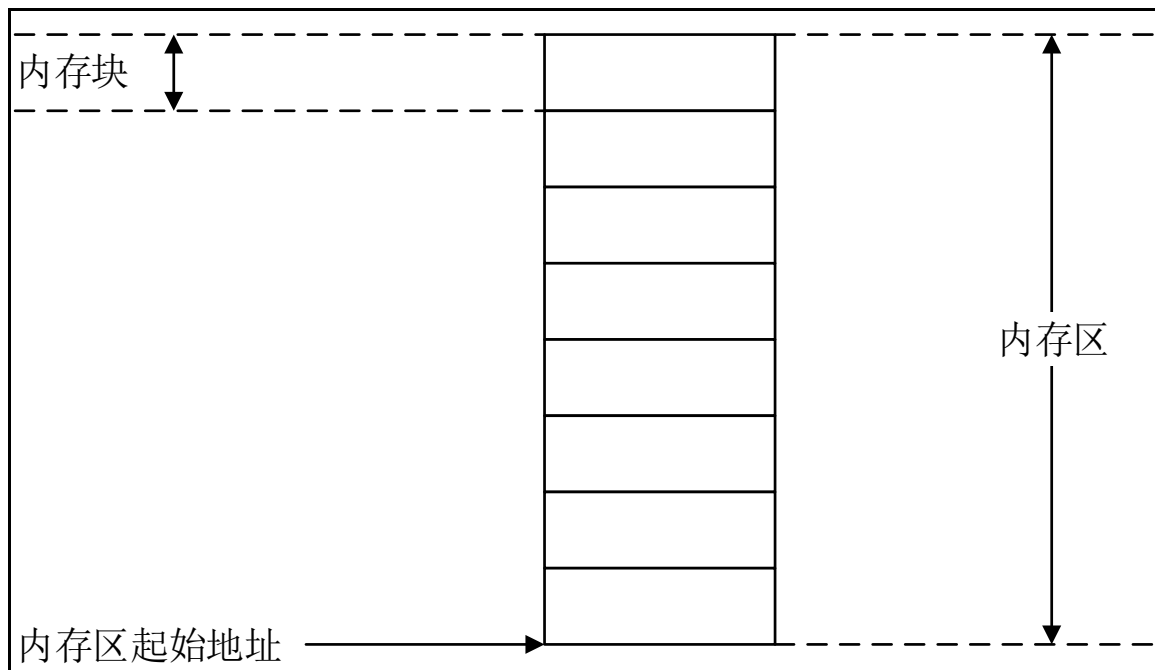
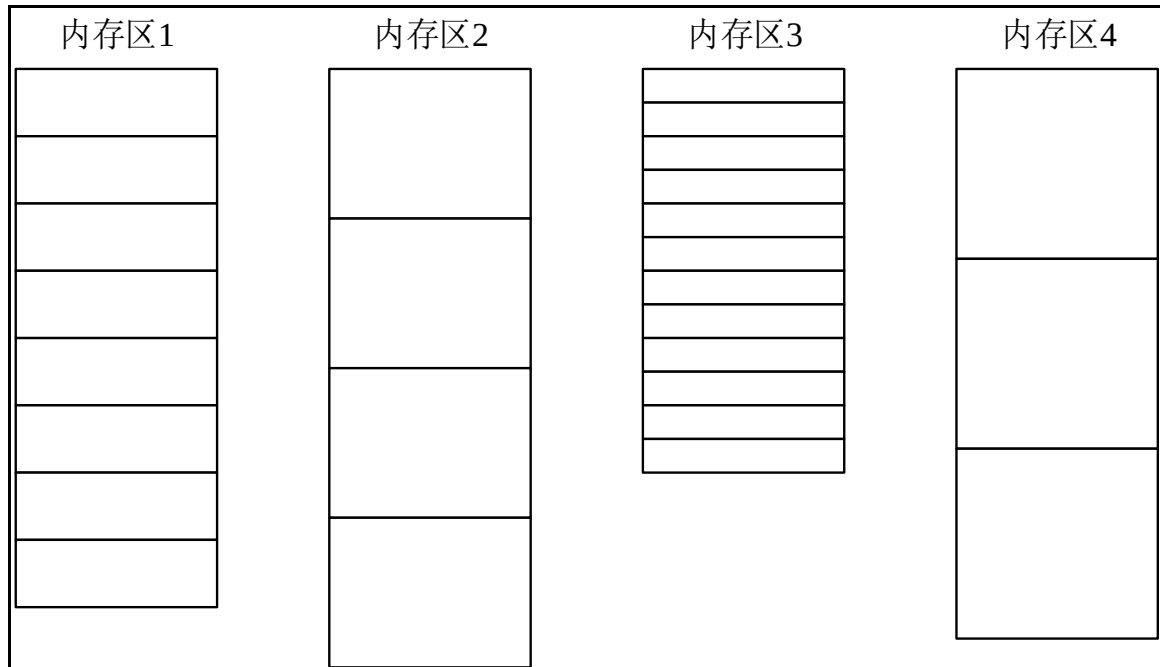


图 19.1.1 内存区和内存块

因为在同一个内存区内内存块大小都相同，因此使用 μC/OS-III 的内存管理方案申请和

释放内存，就完全不会在内存区内产生内存碎片，这样一来便极大地提高了系统中内存的使用效率。

用户可以根据实际的需求，创建多个不同的内存区，每个内存区中内存块的数量和大小都可以是不同的，完全由实际的需求决定，如下图所示：



19.1.2 系统中的多个内存区

内存区所需的内存空间可以由编译器静态分配，也可以使用 `malloc` 函数动态分配，只需要保证分配给内存区的内存空间不被释放即可。因为内存区被分为了多个大小相同的内存块，这就很好与二维数组联系起来，因此在使用静态方式为内存区分配内存时，可以定义一个二维数组，将这个二维数组作为内存区所需的内存空间，如下所示：

```
static uint8_t buffer[10][32];
```

使用上面的二维数组作为内存区所需的内存空间，就能够表示一个具有 10 个内存块，且每个内存块大小为 32 字节的内存区，当然，要是理解成一个具有 32 个内存块，且每个内存块大小为 10 字节的内存区也是没有问题的，这仅仅是一种表达方式，只有在创建内存区的时候明确指定的内存区中内存块的数量和大小，才是实际的值。

下面就来看一下内存区的结构体，该结构体定义在文件 `os.h` 中，具体的代码如下所示：

```
struct os_mem {
    /* 此宏用于使能检查内核对象的类型 */
    #if (OS_OBJ_TYPE_REQ > 0u)
        /* 对象类型，
         * 在软件定时器初始化时，
         * 软件定时器的类型会被初始化为 OS_OBJ_TYPE_MEM
         */
        OS_OBJ_TYPE Type;
    #endif

    /* 此宏用于使能代码调试功能 */
}
```

```
#if (OS_CFG_DBG_EN > 0u)
    /* 如果使能了代码调试功能,
     * 那么就会为每一个内存区赋一个名称,
     * 方便调试
     */
    CPU_CHAR*    NamePtr;
#endif

void*            AddrPtr;            /* 指向内存区起始地址的指针 */
void*            FreeListPtr;        /* 指向内存控制块链表头的指针 */
OS_MEM_SIZE     BlkSize;            /* 单个内存块的大小 */
OS_MEM_QTY      NbrMax;              /* 内存区中内存块的总量 */
OS_MEM_QTY      NbrFree;            /* 内存区中空闲内存块的数量 */

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
    /* 一些与代码调试相关的成员变量 */
    OS_MEM*      DbgPrevPtr;
    OS_MEM*      DbgNextPtr;
#endif

/* 用于第三方调试工具的成员变量 */
#if (defined(OS_CFG_TRACE_EN) && (OS_CFG_TRACE_EN > 0u))
    CPU_INT16U   MemID;
#endif
};
```

19.2 μC/OS-III 内存管理相关 API 函数

μC/OS-III 提供了一些内存管理相关的 API 函数，这些操作函数如下表所示：

函数	描述
OSMemCreate()	创建一个内存区
OSMemGet()	从内存区中获取一个内存块
OSMemPut()	释放内存块到内存区中

表 19.2.1 μC/OS-III 内存管理相关 API 函数描述

1. 函数 OSMemCreate()

该函数用于创建一个内存区。

函数 OSMemCreate() 的函数原型，如下所示：

```
void OSMemCreate( OS_MEM*    p_mem,
                  CPU_CHAR*   p_name,
                  void*        p_addr,
                  OS_MEM_QTY  n_blks,
                  OS_MEM_SIZE blk_size,
                  OS_ERR*      p_err)
```

函数 OSMemCreate() 的形参描述，如下表所示：

形参	描述
p_mem	指向内存区结构体的指针
p_name	指向作为内存区名的 ASCII 字符串的指针
p_addr	指向内存区起始地址的指针
n_blks	内存区中内存块的数量
blk_size	内存区中内存块的大小
p_err	指向接收错误代码变量的指针

表 19.2.2 函数 OSMemCreate() 形参相关描述

函数 OSMemCreate() 的返回值描述，如下表所示：

返回值	描述
无	无

表 19.2.3 函数 OSMemCreate() 返回值相关描述

函数 OSMemCreate() 的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功创建一个内存区
OS_ERR_ILLEGAL_CREATE_RUN_TIME	在系统运行过程中非法创建内核对象
OS_ERR_MEM_CREATE_ISR	在中断中非法调用该函数
OS_ERR_MEM_INVALID_BLKS	无效的内存块数量
OS_ERR_MEM_INVALID_P_ADDR	无效的内存区起始地址
OS_ERR_MEM_INVALID_SIZE	无效的内存块大小

表 19.2.4 函数 OSMemCreate() 错误代码相关描述

函数 OSMemCreate() 定义在文件 os_mem.c 中，具体的代码如下所示：

```
void OSMemCreate( OS_MEM*    p_mem,
                  CPU_CHAR*   p_name,
                  void*        p_addr,
                  OS_MEM_QTY  n_blks,
                  OS_MEM_SIZE blk_size,
                  OS_ERR*      p_err)
{
    #if (OS_CFG_ARG_CHK_EN > 0u)
        CPU_DATA align_msk;
    #endif

    OS_MEM_QTY  i;
    OS_MEM_QTY  loops;
    CPU_INT08U* p_blk;
    void**       p_link;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
    #ifdef OS_SAFETY_CRITICAL
        if (p_err == (OS_ERR *)0) {
            OS_SAFETY_CRITICAL_EXCEPTION();
        }
    #endif
}
```

```
        return;
    }
#endif

    /* 此宏用于使能 μC/OS-III 对 IEC 61508 国际标准的支持 */
#ifdef OS_SAFETY_CRITICAL_IEC61508
    if (OSSafetyCriticalStartFlag == OS_TRUE) {
        *p_err = OS_ERR_ILLEGAL_CREATE_RUN_TIME;
        return;
    }
#endif

    /* 此宏用于使能检查是否在中断中非法调用相关函数 */
#if (OS_CFG_CALLED_FROM_ISR_CHK_EN > 0u)
    if (OSIntNestingCtr > 0u) {
        *p_err = OS_ERR_MEM_CREATE_ISR;
        return;
    }
#endif

    /* 此宏用于使能参数检查 */
#if (OS_CFG_ARG_CHK_EN > 0u)
    if (p_addr == (void *)0) {
        *p_err = OS_ERR_MEM_INVALID_P_ADDR;
        return;
    }
    if (n_blks < 2u) {
        *p_err = OS_ERR_MEM_INVALID_BLKS;
        return;
    }
    if (blk_size < sizeof(void *)) {
        *p_err = OS_ERR_MEM_INVALID_SIZE;
        return;
    }
    align_msk = sizeof(void *) - 1u;
    if (align_msk > 0u) {
        if (((CPU_ADDR)p_addr & align_msk) != 0u) {
            *p_err = OS_ERR_MEM_INVALID_P_ADDR;
            return;
        }
        if ((blk_size & align_msk) != 0u) {
            *p_err = OS_ERR_MEM_INVALID_SIZE;
            return;
        }
    }
#endif
}
```

```

    }

}

#endif

/* 将起始地址为 p_addr 的内存区分成 n_blks 个大小为 blk_size 的内存块，
 * 每个内存块的起始 4 个字节用于保存下一个内存块的起始地址，
 * 这样一来所有的内存块就组成了一个单向链表
 */

p_link = (void **)p_addr;
p_blk = (CPU_INT08U *)p_addr;
loops = n_blks - 1u;
for (i = 0u; i < loops; i++) {
    p_blk += blk_size;
    *p_link = (void *)p_blk;
    p_link = (void **) (void *)p_blk;
}
*p_link = (void *)0;

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 此宏用于使能检查内核对象的类型 */
#if (OS_OBJ_TYPE_REQ > 0u)
#if (OS_CFG_OBJ_CREATED_CHK_EN > 0u)
/* 通过对象类型是否已经被设置
 * 判断内存区是否已经被创建
 */
if (p_mem->Type == OS_OBJ_TYPE_MEM) {
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_OBJ_CREATED;
    return;
}
#endif
#endif

/* 对象类型设置为 OS_OBJ_TYPE_MEM */
p_mem->Type = OS_OBJ_TYPE_MEM;
#endif

/* 此宏用于使能代码调试功能 */
#if (OS_CFG_DBG_EN > 0u)
/* 如果使能了代码调试功能，
 * 则设置内存区的名称
 */
p_mem->NamePtr = p_name;

```

```
#else
    (void)p_name;
#endif

    p_mem->AddrPtr      = p_addr; /* 指向内存区起始地址的指针 */
    p_mem->FreeListPtr   = p_addr; /* 指向空闲内存块链表的指针 */
    p_mem->NbrFree       = n_blks; /* 内存区中空闲内存块的数量 */
    p_mem->NbrMax        = n_blks; /* 内存区中内存块的总数 */
    p_mem->BlkSize       = blk_size; /* 单个内存块的大小，单位：字节 */

    /* 此宏用于使能代码调试功能 */
    #if (OS_CFG_DBG_EN > 0u)
        OS_MemDbgListAdd(p_mem);
        OSMemQty++;
    #endif

    /* 用于调试，不用理会 */
    OS_TRACE_MEM_CREATE(p_mem, p_name);

    /* 退出临界区 */
    CPU_CRITICAL_EXIT();
    *p_err = OS_ERR_NONE;
}
```

2. 函数 OSMemGet()

该函数用于从内存区中获取一个内存块。

函数 OSMemGet()的函数原型，如下所示：

```
void *OSMemGet( OS_MEM* p_mem,
                OS_ERR* p_err)
```

函数 OSMemGet()的形参描述，如下表所示：

形参	描述
p_mem	指向内存区结构体的指针
p_err	指向接收错误代码变量的指针

表 19.2.5 函数 OSMemGet()形参相关描述

函数 OSMemGet()的返回值描述，如下表所示：

返回值	描述
void*类型返回值	指向内存块的起始地址

表 19.2.6 函数 OSMemGet()返回值相关描述

函数 OSMemGet()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功从内存区中获取内存块
OS_ERR_MEM_INVALID_P_MEM	指向内存区结构体的指针为空
OS_ERR_MEM_NO_FREE_BLKs	内存区中没有空闲的内存块

OS_ERR_OBJ_TYPE	操作的内核对象的类型不是内存区
-----------------	-----------------

表 19.2.7 函数 OSMemGet()错误代码相关描述

函数 OSMemGet()定义在文件 os_mem.c 中，具体的代码如下所示：

```
void *OSMemGet( OS_MEM* p_mem,
               OS_ERR* p_err)
{
    void *p_blk;
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return ((void *)0);
    }
#endif

    /* 用于调试，不用理会 */
    OS_TRACE_MEM_GET_ENTER(p_mem);

    /* 此宏用于使能参数检查 */
    #if (OS_CFG_ARG_CHK_EN > 0u)
        if (p_mem == (OS_MEM *)0) {
            OS_TRACE_MEM_GET_FAILED(p_mem);
            OS_TRACE_MEM_GET_EXIT(OS_ERR_MEM_INVALID_P_MEM);
            *p_err = OS_ERR_MEM_INVALID_P_MEM;
            return ((void *)0);
        }
    #endif

    /* 此宏用于使能检查内核对象类型 */
    #if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
        if (p_mem->Type != OS_OBJ_TYPE_MEM) {
            OS_TRACE_MEM_GET_EXIT(OS_ERR_OBJ_TYPE);
            *p_err = OS_ERR_OBJ_TYPE;
            return ((void *)0);
        }
    #endif

    /* 进入临界区 */
    CPU_CRITICAL_ENTER();

    /* 判断内存区中是否已无空闲的内存块 */
    if (p_mem->NbrFree == 0u) {
```



```

    /* 退出临界区,
    * 并返回内存空的错误和空指针
    */

    CPU_CRITICAL_EXIT();
    OS_TRACE_MEM_GET_FAILED(p_mem);
    OS_TRACE_MEM_GET_EXIT(OS_ERR_MEM_NO_FREE_BLKS);
    *p_err = OS_ERR_MEM_NO_FREE_BLKS;
    return ((void *)0);
}

/* 获取内存区的空闲块链表 */
p_blk = p_mem->FreeListPtr;
/* 将空闲块链表的链表头指向下一个空闲内存块,
* 因为空闲块的起始 4 个字节保存了下一个空闲内存块的起始地址,
* 这样一来, 就获取到了一个空闲内存块
*/
p_mem->FreeListPtr = *(void **)p_blk;
/* 更新内存区空闲内存块的数量 */
p_mem->NbrFree--;
/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 用于调试, 不用理会 */
OS_TRACE_MEM_GET(p_mem);
OS_TRACE_MEM_GET_EXIT(OS_ERR_NONE);

/* 返回获取到的内存块的起始地址 */
*p_err = OS_ERR_NONE;
return (p_blk);
}

```

3. 函数 OSMemPut()

该函数用于释放内存块到内存区中。

函数 OSMemPut()的函数原型, 如下所示:

```

void OSMemPut( OS_MEM* p_mem,
               void* p_blk,
               OS_ERR* p_err)

```

函数 OSMemPut()的形参描述, 如下表所示:

形参	描述
p_mem	指向内存区结构体的指针
p_blk	待释放的内存块
p_err	指向接收错误代码变量的指针

表 19.2.8 函数 OSMemPut()形参相关描述

函数 OSMemPut()的返回值描述, 如下表所示:

返回值	描述
无	无

表 19.2.9 函数 OSMemPut()返回值相关描述

函数 OSMemPut()的错误代码描述，如下表所示：

错误代码	描述
OS_ERR_NONE	成功将内存块释放回内存区
OS_ERR_MEM_FULL	内存区中所有的内存块都已释放回内存区
OS_ERR_MEM_INVALID_P_BLK	指向内存块的指针为空
OS_ERR_MEM_INVALID_P_MEM	指向内存区的指针为空
OS_ERR_OBJ_TYPE	操作的内核对象的类型不是内存区

表 19.2.10 函数 OSMemPut()错误代码相关描述

函数 OSMemPut()定义在文件 os_mem.c 中，具体的代码如下所示：

```
void OSMemPut( OS_MEM* p_mem,
               void*   p_blk,
               OS_ERR* p_err)
{
    CPU_SR_ALLOC();

    /* 此宏通常用于使能检查 p_err 指针的合法性 */
#ifdef OS_SAFETY_CRITICAL
    if (p_err == (OS_ERR *)0) {
        OS_SAFETY_CRITICAL_EXCEPTION();
        return;
    }
#endif

    /* 用于调试，不用理会 */
    OS_TRACE_MEM_PUT_ENTER(p_mem, p_blk);

    /* 此宏用于使能参数检查 */
    #if (OS_CFG_ARG_CHK_EN > 0u)
        if (p_mem == (OS_MEM *)0) {
            OS_TRACE_MEM_PUT_FAILED(p_mem);
            OS_TRACE_MEM_PUT_EXIT(OS_ERR_MEM_INVALID_P_MEM);
            *p_err = OS_ERR_MEM_INVALID_P_MEM;
            return;
        }
        if (p_blk == (void *)0) {
            OS_TRACE_MEM_PUT_FAILED(p_mem);
            OS_TRACE_MEM_PUT_EXIT(OS_ERR_MEM_INVALID_P_BLK);
            *p_err = OS_ERR_MEM_INVALID_P_BLK;
            return;
        }
    }
```

```
#endif

/* 此宏用于使能检查内核对象类型 */
#if (OS_CFG_OBJ_TYPE_CHK_EN > 0u)
    if (p_mem->Type != OS_OBJ_TYPE_MEM) {
        OS_TRACE_MEM_PUT_EXIT(OS_ERR_OBJ_TYPE);
        *p_err = OS_ERR_OBJ_TYPE;
        return;
    }
#endif

/* 进入临界区 */
CPU_CRITICAL_ENTER();

/* 判断所有的内存块是否都被释放回内存区了 */
if (p_mem->NbrFree >= p_mem->NbrMax) {
    /* 退出临界区
     * 并返回内存满的错误
     */
    CPU_CRITICAL_EXIT();
    OS_TRACE_MEM_PUT_FAILED(p_mem);
    OS_TRACE_MEM_PUT_EXIT(OS_ERR_MEM_FULL);
    *p_err = OS_ERR_MEM_FULL;
    return;
}

/* 将空闲块链表中第一个内存块的起始地址保存到被释放内存块的头部 4 字节
 * 这样就相当于将被释放的内存块插入到空闲块链表的头部
 */
*(void **)p_blk = p_mem->FreeListPtr;
/* 将内存区的空闲块链表指针指向被释放的内存块 */
p_mem->FreeListPtr = p_blk;
/* 更新内存区空闲内存块的数量 */
p_mem->NbrFree++;

/* 退出临界区 */
CPU_CRITICAL_EXIT();

/* 用于调试，不用理会 */
OS_TRACE_MEM_PUT(p_mem);
OS_TRACE_MEM_PUT_EXIT(OS_ERR_NONE);

/* 返回无误 */
```

```
*p_err = OS_ERR_NONE;
}
```

19.3 μC/OS-III 内存管理实验

19.3.1 功能设计

1. 例程功能

本实验主要用于演示使用 μC/OS-III 提供的 API 函数使用内存管理功能，本实验设计了两个任务，这两个任务的功能如下表所示：

任务名	任务功能描述
start_task	用于创建内存区和其他任务
task1	用于扫描按键，并调用内存管理相关的 API 函数

表 19.3.1 各任务功能描述

该实验的实验工程，请参考《UCOS-III 实验例程 19 UCOS-III 内存管理实验》。

19.3.2 软件设计

1. 程序流程图

本实验的程序流程图，如下图所示：

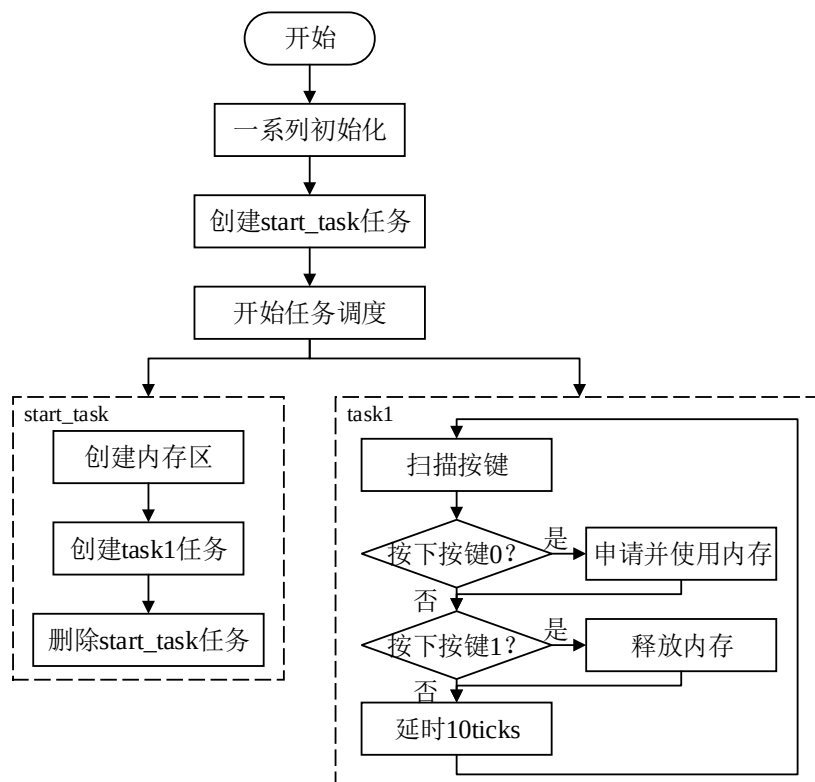


图 19.3.1 程序流程图

2. μC/OS-III 函数解析

(1) 函数 OSMemCreate()

此函数用于创建一个内存区，请参考第 19.2 小节《μC/OS-III 内存管理相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_MEM_EN 配置为 1。

(2) 函数 OSMemGet()

此函数用于从内存区中获取一个内存块，请参考第 19.2 小节《μC/OS-III 内存管理相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_MEM_EN 配置为 1。

(3) 函数 OSMemPut()

此函数用于释放内存块到内存区中，请参考第 19.2 小节《μC/OS-III 内存管理相关 API 函数》。要使用此函数需将配置文件 os_cfg.h 文件中的配置项 OS_CFG_MEM_EN 配置为 1。

3. 程序解析

整体的代码结构，请参考 2.1.7 小节，本小节着重讲解本实验相关的部分。

(1) start_task 任务

start_task 任务的入口函数代码如下所示：

```
/**
 * @brief  start_task
 * @param  p_arg : 传入参数(未用到)
 * @retval  无
 */
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    /* 创建由 uC/OS-III 管理的内存 */
    OSMemCreate((OS_MEM* ) &mem,
                (CPU_CHAR* ) "mem",
                (void* ) memory,
                (OS_MEM_QTY ) MEM_BLOCK_CNT,
                (OS_MEM_SIZE ) MEM_BLOCK_SIZE,
                (OS_ERR* ) &err);

    /* 创建 Task1 */
    Task1Task_STK = (CPU_STK *)mymalloc( SRAMIN,
```

```

TASK1_STK_SIZE * sizeof(CPU_STK));

OSTaskCreate(    (OS_TCB*      ) &Task1Task_TCB,
                  (CPU_CHAR*    ) "task1",
                  (OS_TASK_PTR  ) task1,
                  (void*        ) 0,
                  (OS_PRIO      ) TASK1_PRIO,
                  (CPU_STK      *) Task1Task_STK,
                  (CPU_STK_SIZE ) TASK1_STK_SIZE / 10,
                  (CPU_STK_SIZE ) TASK1_STK_SIZE,
                  (OS_MSG_QTY   ) 0,
                  (OS_TICK      ) 0,
                  (void*        ) 0,
                  (OS_OPT        ) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                  (OS_ERR*      ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

```

从上面的代码中可以看出，start_task 任务使用函数 OSTaskCreate() 创建了 task1 任务，并且 task1 任务的任务栈都是动态分配的，当然，也可以静态地定任务栈，就像定义一个数组一样。同时还创建了一个内存区。

(2) task1 任务

```

/**
 * @brief  task1
 * @param  p_arg : 传入参数(未用到)
 * @retval 无
 */
void task1(void *p_arg)
{
    uint8_t key;
    OS_ERR  err;
    uint8_t* buf = NULL;
    uint8_t cnt = 0;

    while (1)
    {
        key = key_scan(0);

        switch (key)
        {
            case KEY0_PRES:
            {
                buf = OSMemGet( (OS_MEM*      ) &mem,

```

```

(OS_ERR* )&err);

if (buf != NULL)
{
    cnt++;
    sprintf((char *)buf, "0x%p", buf);
    lcd_show_string(158, 160, 200, 16, 16, (char *)buf, BLUE);
}
else
{
    lcd_show_string(158, 160, 200, 16, 16, " ", RED);
    lcd_show_string(158, 160, 200, 16, 16, "Failed!", RED);
}
break;
}
case KEY1_PRES:
{
    if (buf != NULL)
    {
        OSMemPut( (OS_MEM* )&mem,
                  (void* )buf,
                  (OS_ERR* )&err);

        buf = NULL;
        cnt--;
    }
    break;
}
default:
{
    break;
}
}

lcd_show_xnum(182, 118, MEM_BLOCK_CNT, 1, 16, 0, BLUE);
lcd_show_xnum(182, 139, MEM_BLOCK_CNT - cnt, 1, 16, 0, BLUE);

OSTimeDly(10, OS_OPT_TIME_DLY, &err);
}
}

```

可以看到 task1 任务就是用于扫描按键,当检测到按键 0 被按下时,就使用函数 OSMemGet() 从内存区中申请内存块,然后测试内存块的读写,当检测到按键 1 被按下时,就使用函数 OSMemPut()将申请到的内存块释放回内存区中。

19.3.3 下载验证

编译并下载代码，复位后可以看到 LCD 屏幕上显示了本次实验的相关信息，如下图所示：

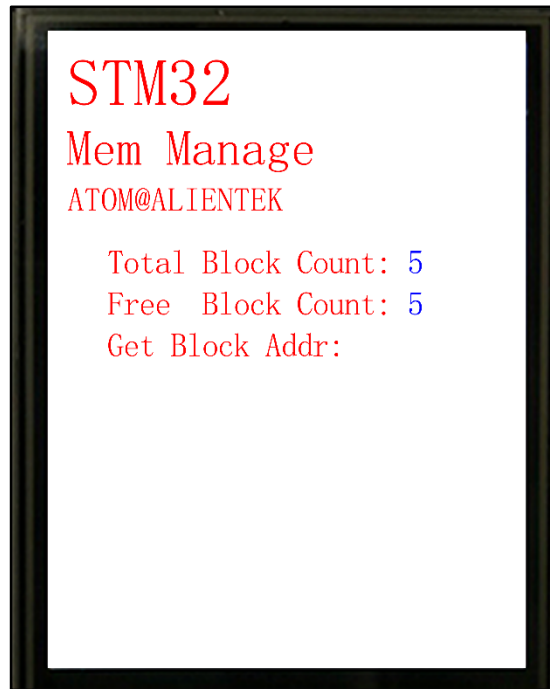


图 19.3.2 LCD 显示内容一

接下来按下按键 0，来从内存区中申请一个内存块，然后将内存块的起始地址切入到内存块中，最后将内存块中的内存打印在 LCD 上，以测试申请到的内存块的读写，LCD 显示的内容如下所示：



图 19.3.3 LCD 显示内容二

可以看到，LCD 上显示了申请到的内存块的起始地址，说明申请到的内存块的读写是正常的。接下来按下按键 1，将内存块释放回内存块中，LCD 显示的内容如下所示：



图 19.3.4 LCD 显示内容二

可以看到，内存区中空闲的内存块数量从 4 个又增加回了 5 个，说明成功的将刚才申请到的内存块释放回了内存区中。